Discrete feature growth: what it is, how it compares with gradient-trained networks, and what it learns

Claude (Anthropic)*
prompted by Kaarel Hänni

3 November 2026

Abstract

Discrete feature growth (DFG) is forward stepwise least squares over an explicit Boolean dictionary, Fourier characters and indicators of signed-sum values, with a per-class Bonferroni entry rule (residual correlation above $s\sqrt{2\ln(4KN/\delta)/n}$ for N candidates): an Occam learner made greedy by a residual. Version 1.3, frozen inside a portfolio, runs alone under a pre-registered protocol, on ± 1 inputs only.

On 12 held-out targets of one family DFG needs clearly less data than the per-target best of three network variants (chosen in hindsight) on 10, and more on none; the mean $\log_2$ ratio of n^*, the training-set size at 10% test error, is -1.29 (s.e. 0.27; -1.93 on 114 in-sample targets). On 5 of the 10 wins, logistic regression or boosting also beats the networks. DFG's n^* grows with the input dimension d with exponents 0.01–0.50, the networks' with 0.91–2.40, but degree-k enumeration costs of order $n\binom{d}{k}$ FLOPs: within one network run no global degree-4 character passes from $d = 179$ on. The low-degree algorithm (DFG's characters of degree ≤ 4 and its test, nothing else) needs $3.1\times$ more data, level with the networks.

Its models list named features, often the target's exact structure. On parities (degrees 2–4) the test sets the data cost: a proved criterion predicts all 250 fits with $n \geq 32$. On signed sums, where the grid can tell (6 targets), evidence points to the search; residue classes mod 3 are never found. A post-hoc core matches DFG on average at a tenth of its FLOPs, moving 54 of 138 targets both ways.

*Written by an AI system in a series of sessions prompted and steered by Kaarel Hänni. Not reviewed by a human. Code, result files, the protocol and the underlying records are on the `claude/nifty-mccarthy-j0lw2a` branch of `https://github.com/kaarelh/AI-works`, folder `gd-circuit-growth/`; how the analyses were checked is in Appendix H.

Contents

1 Introduction **5**

2 What DFG is **9**
2.1 In one paragraph . . . 9
2.2 The hypothesis class . . . 9
2.3 One round . . . 11
2.4 Small samples and compute . . . 13
2.5 Simple guarantees . . . 13
2.6 How DFG relates to familiar learners . . . 15
2.7 What is discrete . . . 17

3 How it is evaluated **17**
3.1 Tasks . . . 17
3.2 References . . . 18
3.3 Metrics and budgets . . . 19
3.4 The Part 11 protocol, and what is held out . . . 20

4 How well it does **21**
4.1 The held-out suite `dlheld7` . . . 21
4.2 Every suite . . . 23
4.3 What the portfolio added . . . 23
4.4 The low-degree algorithm: DFG's characters and test, nothing else . . . 24
4.5 Input dimension . . . 26
4.6 Tabular data and compute . . . 26

5 What its parts do **27**
5.1 Twelve single changes . . . 28
5.2 How the other parts act . . . 30
5.3 A core DFG (post hoc) . . . 30

6 What it learns **32**
6.1 How the models are read . . . 32
6.2 What the models contain, family by family . . . 33
6.3 Two regimes: when the structure appears, and when its test would admit it . . . 36
6.4 Where it fails, and why . . . 38
6.5 False admissions . . . 39
6.6 The order of acquisition, against the Part 5 networks . . . 40

7 Discussion **40**
7.1 What kind of learner DFG turned out to be . . . 40
7.2 Why it needs less data than the networks here . . . 41
7.3 Its failures, in summary . . . 42
7.4 DFG as a model of gradient descent . . . 42
7.5 The core DFG as the object to study next . . . 43
7.6 What would test it further . . . 43
7.7 Limits . . . 44

References **45**

A Names used in this paper **47**

B DFG v1.3 step by step **50**
B.1 The dictionary . . . 50
B.2 One round . . . 51
B.3 The block search and the block moves . . . 52
B.4 Refit, stopping and readout . . . 53
B.5 Small samples . . . 53
B.6 The budget . . . 54

C The six statements: proofs and checks **56**
C.1 Round 0 under the null (P1) . . . 56
C.2 The sample size of a parity, and the floors (P2, P2′) . . . 57
C.3 When a character passes in round 0 (P3) . . . 58
C.4 Sum blocks (P4) . . . 59
C.5 Compute (P5) . . . 60

D The evaluation in full **61**
D.1 The suites . . . 61
D.2 The references . . . 61
D.3 The protocol's questions and runs, and the status of every set . . . 63

E Further results **66**
E.1 Every toy target . . . 66
E.2 Targets on which DFG alone is not better . . . 66
E.3 DFG alone and v3 . . . 66
E.4 The low-degree algorithm . . . 73
E.5 Input dimension . . . 73
E.6 Tabular data . . . 73
E.7 Compute . . . 77

F The ablations in full **82**
F.1 Families, sources and every target . . . 82
F.2 The test or the search: the refits . . . 83
F.3 Degree 4 without the degree-4 search . . . 86
F.4 The small-n machinery and the background . . . 87
F.5 Tests scaled to the residual . . . 88
F.6 The core in detail . . . 89

G What the models contain: the detail **93**
G.1 The analyses' fits, and DFG's choices at small n . . . 93
G.2 Family by family: the detail . . . 95
G.3 Spurious terms . . . 98
G.4 DFG alone and v3's DFG member . . . 100
G.5 False admissions: the detail . . . 100
G.6 Where it fails: the detail . . . 101
G.7 When structure appears: the detail . . . 104

G.8 The order of acquisition: the detail 107

H How Part 11 was run **114**
H.1 The request and the choice of object 114
H.2 The protocol and the runs 115
H.3 The analyses and their verification 116
H.4 Disclosures 118
H.5 Reproducing the numbers 118

1 Introduction

The question. From its sixth part on, this study has asked whether a learner without continuous feature parameters can do at least as well as gradient-trained networks, with the same training data and no more FLOPs. Its tenth part answered with v3, a stacked portfolio [12]. This paper studies one learner instead of an ensemble: the portfolio's Boolean member, run alone. How the object was chosen, and the alternatives set aside, are in Appendix H.1.

Background. Parts 1–5 found that rich-regime networks on Boolean targets grow features greedily, cheapest first, and pay in samples: for a k-bit parity on d inputs their n^*, the training-set size at 10% test error, grows like $d^{1.0}$, $d^{1.9}$ and $d^{2.4}$ for $k = 2$, 3 and 4. That is close to the leap and correlational-SQ rate d^{k-1} [1] for $k \leq 3$, and far from the $\approx k \log d$ of an Occam learner [11]. Part 6 built DFG from that picture, with enumeration and union-bound tests in place of the networks' noisy search [10]. Parts 8–10 stacked it with classical learners in the portfolios v1–v3, whose toy wins came from DFG, and gave it signed-sum blocks (v1.2), then weighted blocks, forced blocks and a beam search (v1.3) [10, 12]; Appendix D.2 records what the earlier versions did.

The object: DFG v1.3, run alone. DFG (discrete feature growth) is forward stepwise least squares over an explicit, finite Boolean dictionary, with a Bonferroni entry rule per class: in learning-theory terms, an Occam learner made greedy by a residual (Section 2.6). The dictionary holds Fourier characters $\chi_S(x) = \prod_{i \in S} x_i$, indicators of the values of signed sums $\sum_{i \in S} s_i x_i$ of inputs, and a few composites of these. Each round refits the ± 1 labels by least squares on the features found so far and admits every candidate whose correlation with the residual exceeds $s\sqrt{2\ln(4KN/\delta)/n}$ for a class of N candidates, n rows and residual scale s, with $K = 4$ and $\delta = 0.1$; the loop stops when a round admits nothing or every training label's sign is right, and DFG predicts with the sign of its readout (Section 2.1). This paper studies version 1.3, frozen in Part 10, run by itself and unchanged (Section 3.4). It is one thing in a specific sense: one residual loop, one admission principle (a tail-bound threshold for each explicit class, computed with four statistics) and one linear readout. Around it sit a multi-start search for signed-sum blocks, a forced guess at $n \leq 512$ chosen by cross-validation, a logistic refinement and a Hebbian background. Section 5 shows which of these can be removed, and Section 5.3 runs, post hoc, a core without most of them. The price is the domain: DFG v1.3 takes ± 1 inputs and two classes, so it runs on the study's 11 Boolean suites, on 2 of its 26 tabular datasets and on no images.

Why it is worth studying.

- *It is analysable.* In round 0, when its degree is searched on all n rows, a k-parity passes its test exactly when $n(1 - \bar{y}^2) > 2\ln(4K\binom{d}{k}/\delta)$, so its sample cost grows like $2k \ln d$, where the networks' grows like d^{k-1} for $k \leq 3$ (Section 2.5).
- *It realizes the networks' greedy residual loop with the search's sampling noise replaced by enumeration*, one end of the trade between data and compute (Section 7.2), and *its main failure reduces to one question*: when does a search seeded by degree-1 and degree-2 residual statistics find a signed-sum block (Section 7.5)?
- *It is the part of v3 that is new and that matters on Boolean inputs.* v3's other members are LightGBM [21], ExtraTrees [18], nearest-neighbour votes and logistic regression, combined by a convex stacker [34], and its image members LightGBM on fixed banks of pixel-subset sums;

Part 8's post-hoc ablations traced the portfolio's toy wins to DFG's character search [10, 12]. And its fitted model is an explicit list of named features (Section 6).

What was found. Table 1 gives the comparisons.

- **Less data than the networks** [Numerical] (Sections 4.1, 4.2 and 4.5). On the held-out `dlheld7`, DFG needs clearly less data than the best network variant on 10 of 12 targets and more on none, about 2.4× less on average (mean $\log_2$ ratio -1.29, s.e. 0.27 if the 11 learned targets are independent draws from the family). On five of the ten wins logistic regression or default gradient boosting is also clearly better than the networks; on the other five, all sum-structured, DFG needs about 4.3× less data and neither no-search baseline comes close. On seven targets run at $d = 15$–60 (four also at $d = 120$), DFG's n^* grows as $d^{0.01}$ to $d^{0.50}$ and the networks' as $d^{0.91}$ to $d^{2.40}$: a trade of samples for FLOPs, since an exhaustive degree-k search costs of order $n\binom{d}{k}$ FLOPs, so that within DFG's budget of one network run the global search can pass no degree-4 character from $d = 179$ on [Proved, by the code] (Sections 4.5 and 7.2). v3 needs 1.14× less data than DFG alone on average over the 134 targets both learn, mostly on small formulas and thresholds that it learns by $n = 128$ (Section 4.3).
- **What carries the results** [Numerical, then Heuristic] (Sections 4.4 and 5). Not the characters and the test alone: the low-degree algorithm, which shares DFG's characters of degree ≤ 4 and the form of its test and nothing else (no signed-sum blocks, residual rounds or refitted readout), is level with the networks. Removing only the sum blocks from DFG costs most ($+0.91$ in mean $\log_2 n^*$ over the 59 of 65 ablation targets both learn, and 3 targets lost), then the residual-scaled tests and the small-n machinery. A core defined post hoc, which keeps the characters of degree ≤ 3, the unweighted signed-sum blocks without the eigenvector seeds, the residual-scaled tests (with $K = 3$, so 0.29 nats more lenient), least squares and the small-n cross-validation, matches DFG on average ($+0.07$ in mean $\log_2 n^*$ over the 133 targets both learn) at a median 0.094 of its FLOPs, but moves 54 of 138 targets beyond the tie band, 28 lower and 26 higher, where the paired null moves 8 (Section 5.3).
- **What it learns** [Numerical, then Heuristic] (Section 6.2). Parities are their single character; symmetric targets one exact signed-sum block; halfspaces the linear block's degree-1 characters, then blocks that encode the threshold; a parity of two majorities first the folded blocks $|t_A + t_B|$ and $|t_A - t_B|$, since $\operatorname{sign}(|t_A + t_B|^2 - |t_A - t_B|^2) = \operatorname{sign}(t_A t_B)$; a sum with a parity or gate partner ends in weighted blocks with the partner as inner term; formulas and multiplexers are their characters, after transient blocks that encode a clause or a branch. Where DFG errs, features are missing, not weights: in 133 of the 148 analysed cells with test error above 0.1, no weights on the learned features fit the target.
- **Two regimes** [Proved, Numerical, then Heuristic] (Sections 6.3 and 6.4). On parities the test sets the data cost: a proved criterion predicts the outcome in all 250 cells of the 2-, 3- and 4-parities with $n \geq 32$, and moving the test's level moves n^* by the predicted amount. On signed-sum targets, where the grid can tell, the evidence points to the search: on the 6 targets that DFG learns a doubling or more after their true block would first pass its test, a stricter test, which raises the block's threshold by about a quarter of a doubling of n, moves n^* by at most 0.05. On the other 24 learned sum targets the grid cannot separate the test from the search. At the extreme, the residue classes mod 3 have $1.6 \cdot 10^{-6}$ to $2.1 \cdot 10^{-4}$ of their Fourier weight at degrees 1–2, where the block search starts, and their block never enters DFG's model, although it would pass the test from $n = 128$.

evaluation	reference	DFG v1.3 alone	limits
held out: `dlheld7`, 12 targets (Section 4.1)	`nets_best`: best of 3 variants (hindsight)	mean $\log_2 n^*$ ratio -1.29 over the 11 targets both learn (2.4× less data; median -1.16; s.d. 0.91, s.e. 0.27); clearly better on 10 of 12, worse on 0	one family; on 5 of the 10 wins logistic regression (`lin`) or gradient boosting (`hgb`) also beats `nets_best`; v3 needs less data (+0.21; clearly on the 3 non-symmetric controls); no learner learns f7_05, a residue class mod 3
pooled: 126 targets of 10 suites (Section 4.2)	`nets_best`: best of 2 to 5 variants	-1.87 over 123 (3.7×); clearly better on 117, worse on 2	in-sample (Table 12) except `dlheld7`'s 12; the 114 in-sample targets alone: -1.93 over 112; against `nets_wd` (square loss, weight decay) the margin is smallest on the three suites held out for some version (`dlheld2`, `dlheld3`, `dlheld7`)
input dimension: $d = 15, 60, 120$, 7 targets (Section 4.5)	`nets`, run at every d	exponent of n^* in d 0.01–0.50 (parities of degree 2, 3, 4: 0.24, 0.30, 0.50), networks 0.91–2.40 (1.03, 1.96, 2.40); median factor 6.2× at $d = 15$, 42.5× at $d = 120$	7, 7, 4 development targets; 2 seeds at $d \neq 30$; `nets` is the plain square-loss network, the only variant run at every d; 3 or 4 points per slope; enumeration costs of order $n\binom{d}{k}$ FLOPs, so within one run no global degree-4 character can pass from $d = 179$ on ($n > 512$)
characters and test only: `lowdeg`, 138 targets (Section 4.4)	DFG alone; `nets_best`	`lowdeg` against DFG alone +1.64 over 122 (3.1× more data; worse on 112, better on 16; `lowdeg` never reaches 10% on 12 targets DFG learns); against `nets_best` -0.16 (53 better, 62 worse)	in-sample; Part 6's feasibility check, run unchanged; faster only where DFG alone needs ≤ 133 points
tabular: 2 datasets, 9 cells (Section 4.6)	`mlp_best` (hindsight); `hgb`; v3	cells, i.e. (dataset, n) pairs, better / tie / worse: 0/8/1 against `mlp_best`, 0/9/0 against `hgb`, 0/8/1 against v3	kr_vs_kp and mushroom, the only 2 of 26 datasets with ±1-codable inputs and two classes; no real-valued data
post hoc: the core, 138 targets (Section 5.3)	DFG alone, same seeds; `nets_best`	against DFG alone +0.07 over 133 (28 lower, 26 higher; loses par5); against `nets_best`, on the 126 targets with network runs, -1.74 over 122 (clearly better/worse on 111/9), DFG alone -1.88 over 123 (117/5); `dlheld7` -1.30, DFG alone -1.27 (DFG alone on seeds 0, 1 only, hence -1.88 and -1.27 rather than the 3-seed -1.87 and -1.29); median FLOPs 0.094 of DFG's	seeds 0, 1; defined after every result had been read: nothing is held out for it

Table 1: The results at a glance [Numerical]. A ratio is $\log_2(n^*_{\text{DFG}}/n^*_{\text{ref}})$ (for `lowdeg` and the core, of theirs to the reference's), negative when the first needs less data; means are over the targets both learn. "Clearly": beyond ±0.25, or only one of the two reaches 10% error; on tabular data, errors that differ by more than 10% and 2 s.e. (Section 3.3). The s.d. and s.e. on `dlheld7` are over its 11 learned targets, as independent draws from the family. Only `dlheld7` is held out for DFG v1.3. Generated by `paper_dfg/make_tables.py` (`tab_headline`) from `study/part11/RESULTS.json` and `study/part11/interp/ablations/ablations.json` (the core's FLOPs).

- **Against the networks' order** [Numerical, then Heuristic] (Section 6.6). Inside DFG's first round its test statistics are positively rank-correlated with the Part 5 networks' order on all 11 targets (median Kendall τ 0.54 at $n = 16384$), but the orders invert where a large coefficient sits at high degree: the multiplexer's largest term is DFG's first character by round-0 z-score and the networks' last. On the 8 targets whose coefficients differ in size, DFG's n order follows coefficient size more than degree on 5, and the networks' time order follows degree on 7: DFG tends to admit characters in order of detectability ($n_{\text{glob}} \propto \ln(4K\binom{d}{|S|}/\delta)/\hat{f}(S)^2$: degree enters only through a logarithm, coefficient size through its inverse square), the networks to acquire them in order of construction (lower degree first, products after their factors).

Scope.

- *What is held out.* For DFG v1.3, only `dlheld7`: 12 targets drawn from the blind family of signed-sum targets and fixed in a commit before v1.3 was designed (Section 3.4). DFG v1.3 was developed on two other draws from the same family, so `dlheld7` tests it on fresh targets from a distribution its designer had seen.
- *Everything else is in-sample.* The other 10 suites were used in developing some version of DFG, or were held out only for an earlier version, and several of DFG's parts were added after failures on their targets (Table 12). Their numbers, the ablations, the core and the analyses describe the frozen learner; they do not test it.
- *The object was chosen post hoc*, after Part 10's results had been read, v3's on `dlheld7` included; DFG alone's results there were new, bar a disclosed timing check on two targets (Section 3.4).
- *The references* are one network recipe, not tuned per target, and `nets_best` is a hindsight minimum over the 2 to 5 variants run on each suite (Section 3.2).
- *The domain is Boolean*: inputs uniform on $\{\pm 1\}^d$, mostly with $d = 30$, and two tabular datasets written as ± 1; nothing here is about real-valued data or images (Section 4.6).
- *Seeds and noise.* n^* is read from test errors averaged over 3 training sets per target and n (2 on `random` and at $d \neq 30$), with no confidence interval. DFG's n^* on two different single training sets differ by a mean $|\log_2 \text{ratio}|$ of 0.339 (23 of 65 targets beyond ± 0.25) and refits on the same training sets with another column order by 0.012, so the ± 0.25 band is smaller than the single-seed spread and per-target verdicts are not significance calls (Section 3.3). The ablations use one or two seeds, the core two, and most analyses seed 0.
- *AI-written*: code, runs, analyses and text, checked by AI verifiers; no human has reviewed them (Appendix H).

Claims are labelled [Proved] (a proof is given, here or in the theory notes, which an independent verifier checked), [Numerical] (measured here, with the source file named) or [Heuristic] (a reading or a back-of-envelope argument, not tested here).

Contributions.

1. What DFG v1.3 is, six statements about it, all but one exact, and its relation to familiar learners (Section 2; every step in Appendix B, proofs and checks in Appendix C).
2. A pre-registered evaluation against the networks, the no-search baselines, v3 and the low-degree algorithm, in the input dimension, on tabular data and in compute (Sections 3 and 4; in full in Appendices D and E).

3. What its parts do: 12 pre-registered ablations against a post-hoc paired null, and a post-hoc core DFG with nine parts removed (Section 5; Appendix F).
4. A descriptive, independently verified account of what DFG learns: its models, when structure enters relative to its test, its failures, its false admissions and its order of admission against the networks' (Section 6; Appendix G).
5. What kind of learner DFG turned out to be, why it needs less data here, where it fails and what to study next (Section 7); how Part 11 was run and checked (Appendix H).

2 What DFG is

This section describes the paper's object: DFG v1.3 as frozen in `study/part10/hyb_portfolio3/dfg13.py`, run alone as `DFG(readout="logit").fit` with every other constructor argument at its default (Section 3.4). Parameter values below are those defaults. Appendix B gives every step with the frozen file's line numbers, and Table 11 lists the constants.

2.1 In one paragraph

DFG, *discrete feature growth*, is one greedy loop over an explicit dictionary of Boolean features. Starting from the constant, each round fits the ± 1 labels by least squares on the features it has, takes the residual $r = y - \hat{y}$ and its root mean square s, enumerates Fourier characters $\chi_S(x) = \prod_{i \in S} x_i$ class by class, and searches for signed sums $\sum_{i \in S} s_i x_i$, whose value indicators are candidates too. A candidate is admitted only if its correlation with r passes a union bound over its whole class, $s\sqrt{2\ln(4KN/\delta)/n}$ for a class of N candidates, with $4K/\delta = 160$. DFG refits and repeats until a round admits nothing or the fit has every training label's sign right, and predicts with the sign of the readout, refined by a few logistic Newton steps, plus one Hebbian step on the final residual, $g \cdot x$ with $g = X^\top r/n$. The fitted model is an explicit list of named features with real weights. For example, with seed 0, DFG's model of par4 $= x_1x_2x_3x_4$ at $n = 64$ is the constant and $\chi_{\{1,2,3,4\}}$. Its model of sym9 $= [\,|x_1 + \cdots + x_9| \le 1\,]$ at $n = 128$ is the constant and one folded block (a block that uses $|t|$ in place of its signed sum t): three indicators of the value of $|x_1 + \cdots + x_9|$. Both have test error 0 (`study/part11/interp/tables/atlas.md`; Section 6).

It is one thing in a specific sense: one residual loop, one admission principle (a tail-bound threshold for each explicit class, computed with four statistics) and one linear readout. Around it sit a multi-start search for signed-sum blocks, a forced guess at $n \le 512$ chosen by cross-validation, a logistic refinement and a Hebbian background. Figure 1 gives the loop in outline; Section 5 shows which of the parts can be removed, and Section 5.3 runs, post hoc, a core without most of them.

2.2 The hypothesis class

DFG predicts $\operatorname{sign} h(x)$, and -1 where $h(x) = 0$, with

$$h(x) = \sum_{S \in \mathcal{S}} w_S\, \chi_S(x) + \sum_{i \in \mathcal{L}} w_i\, x_i + \sum_{B \in \mathcal{B}} \sum_{v \in \mathcal{V}_B} w_{B,v}\, \chi_{P_B}(x)\, [t_B(x) \in G_{B,v}] + \sum_{u \in \mathcal{U}} w_u((u \cdot x)^2 - 1) + g \cdot x. \tag{1}$$

The five sums are the five parts of the dictionary. $\mathcal{S}$ holds the constant and every character admitted by its own test or forced; the global classes reach degree 6, and the extension and local classes, which grow from the characters already present, reach higher degrees. $\mathcal{L}$ is a set of inputs whose degree-1

fit(X, y), with n rows and d inputs. If $n > 512$, return **grow**$(X, y;$ default$, 1)$. Otherwise run **grow** for both variants, default and forced, on each of F cross-validation folds ($F = 10$ at $n \le 64$, else 5), with budget scale $1/(4F)$ each. Let e_{10} (e_{01}) count the held-out points where only the default (only the forced) variant errs. Return **grow**$(X, y;$ forced$, \frac{1}{2})$ if $e_{10} - e_{01} > 2\sqrt{e_{10} + e_{01}}$, else **grow**$(X, y;$ default$, \frac{1}{2})$.

grow$(X, y;$ variant$,$ scale$)$. Start with the features $\Phi = \{1\}$. In rounds $0, 1, \ldots, 7$:

1. fit the weights w by least squares on Φ (with a small ridge); $r = y - \Phi w$, $s = \mathrm{rms}(r)$;
2. stop if 0.75 of the budget is spent, or, from round 1 on, if $\mathrm{sign}(\Phi w) = y$ on every training row;
3. characters: in each class $\mathcal{C}$ (global, extension, local; Table 2), find the χ_S with $|\mathrm{mean}(r\chi_S)| > s\sqrt{2\ln(4K|\mathcal{C}|/\delta)/n}$;
4. dense moves: the linear block (every remaining x_i, by one χ^2 test), and spectral features $(u \cdot x)^2 - 1$ for the eigenvectors u of the residual pair matrix above its noise edge;
5. sum blocks: search for a signed subset (S, s) on r; keep it if the projection of r on its group indicators passes a χ^2 test with a union bound over every (S, s) of its size, both on r and on r projected off this round's characters; likewise modulated and weighted blocks and products (some products on the projected residual only);
6. if nothing passed: stop (default variant), or add the best candidate anyway (forced variant, at most twice per fit);
7. add what passed to Φ: characters and block indicators only within the room $\min(256, \lfloor n/2 \rfloor) - |\Phi|$, the linear block and spectral features without a room check.

Readout. If $n \ge 8|\Phi|$, refine w by 15 Newton steps of logistic regression. Predict $\mathrm{sign}\big(\Phi(x)w + g \cdot x\big)$, with $g = X^\top r/n$ on the final residual.

Figure 1: One call of DFG v1.3, in outline (`study/part10/hyb_portfolio3/dfg13.py`: `DFG.fit`, lines 548–571; `DFG._fit`, lines 580–877). Here $K = 4$ and $\delta = 0.1$. Sections 2.3 and 2.4 describe the steps and Appendix B gives every one exactly.

characters were admitted together, by one test on all of them (the *linear block*). A *sum block* B has inputs S_B, signs $s_i \in \{\pm 1\}$, and the integer statistic $t_B(x) = \sum_{i \in S_B} s_i x_i + a_B\, \tau_B(x)$, or its absolute value if the block is *folded*; a *weighted* block (v1.3) has $a_B \in \{\pm 2, \pm 3\}$ and an inner term τ_B (a literal, a 2-parity or an AND of two literals) on inputs outside S_B. Its features indicate the groups $G_{B,v}$, contiguous runs of the values of t_B that occur in training with at least $\max(3, \lfloor 0.005\, n \rfloor)$ points each, all but the most populous ($\mathcal{V}_B$). A *modulated* block multiplies them by a character χ_{P_B} already in the model, and a *product* block uses the joint cells of two blocks with disjoint inputs. A function of a signed sum of k inputs costs at most $k + 1$ weights (Proposition 2.5), where its Fourier expansion can need all 2^k characters on S_B. Each *spectral feature* $(u \cdot x)^2 - 1 = 2\sum_{i<j} u_i u_j x_i x_j$, for a unit eigenvector u of the residual pair matrix, is a fixed combination of degree-2 characters. The *kernel background* $g = \frac{1}{n}\sum_t r_t x_t$ is one Hebbian step over all d inputs on the final least-squares residual, computed, not fitted. The weights w are fitted on the p admitted features by least squares and then, when $n \ge 8p$, by logistic Newton steps; characters and block indicators are admitted only within $\min(256, \lfloor n/2 \rfloor)$ features, but the linear block and the spectral features are not checked against that room, so p can exceed it (Appendices B.1 and B.5).

The dictionary. Table 2 lists the classes. The dictionary is explicit and finite: at $d = 30$ the global classes hold $\sum_{1 \le k \le 6} \binom{30}{k} = 768\,211$ characters, and the unweighted blocks (S, s, fold) on 2 to 24 inputs number 2.0×10^{14}, close to $3^{30} = 2.1 \times 10^{14}$, the number of signed subsets of 30 inputs. A class's size N enters its test only through the log term $\ln(4KN/\delta)$. The extension and local classes are built from the characters already present, so they are small and their thresholds are low: features

class	candidates	rows tested	N at $d = 30$	$\ln(4KN/\delta)$	smallest n that can pass
Characters χ_S, test (2)					
global, $k = 1$	every x_i	all	30	8.48	16.95
global, $k = 2$	every pair	all	435	11.15	22.30
global, $k = 3$	every triple	all	4 060	13.38	26.77
global, $k = 4$	every 4-set; round 0	all	27 405	15.29	30.59
global, $k = 5$	round 0, final fit	first 1 374 (687)	142 506	16.94	33.88
global, $k = 6$	round 0, final fit	first 147 (73)	593 775	18.37	36.74
extension, +1 input	$T \cup \{j\}$	all	$bd \le 960$	8.48–11.94	16.95–23.88
extension, +2 inputs	$T \cup \{j, l\}$	all	$b\binom{d}{2} \le 13\,920$	11.15–14.62	22.30–29.23
local	$S \subseteq V$, $\lvert V\rvert \le 12$	all	$\binom{\lvert V\rvert}{\lvert S\rvert} \le 924$	5.08–11.90	10.15–23.81
Dense moves					
linear block	every remaining x_i	all	χ^2_d threshold 51.2, (3)		62
spectral features	≤ 4 eigenvectors	all		edge (4)	120
Sum blocks on k inputs, test (5), $\mu = 0$; searched on the first 2 048 rows					
$k = 2$	(S, s), folded or not	all	4.2×10^4	15.71	40.4
$k = 3$	(S, s), folded or not	all	7.8×10^5	18.64	46.9
$k = 5$	(S, s), folded or not	all	1.1×10^8	23.59	57.9
$k = 9$	(S, s), folded or not	all	1.8×10^{11}	30.97	74.1
$k = 14$	(S, s), folded or not	all	5.7×10^{13}	36.75	86.6
$k = 24$	(S, s), folded or not	all	2.4×10^{14}	38.18	89.7
weighted	$+a\tau$, $a \in \{\pm 2, \pm 3\}$	all	$\times$ 17 640	+9.78	
every block of a round	$\mu \le 4$ modulators χ_P	all	$\times (1 + \mu)$	$\le +1.61$	
product	$A \times B$, disjoint	all	$N_A N_B$	$L_A + L_B$	

Table 2: DFG's dictionary at $d = 30$ [Proved, by the code]. Each row is a class: a set of candidates enumerated together and tested with one union bound. "Rows tested": a budgeted search runs on the first m rows (Section 2.4); at $d = 30$ that is every row of the grid except for degrees 5 and 6 (in brackets: in the final fit at $n \le 512$). For the extensions, $b \le 32$ is the number of current characters extended; for the local class, V is the set of inputs of the current characters. N is the count the union bound pays for, and $\ln(4KN/\delta)$ the log term of the threshold, with $K = 4$ and $\delta = 0.1$. "Smallest n that can pass": for characters and blocks, no member can pass its test on fewer rows, for any labels (Proposition 2.3); for the dense moves, a condition of the code. Appendix B.1 explains every entry. Source: `study/part10/hyb_portfolio3/dfg13.py` via `paper_dfg/make_tables.py` (`tab_dictionary`; values in `paper_dfg/tables/numbers.json`).

on inputs already in use are cheaper to admit, the discrete counterpart of the composition effect that Parts 1–5 saw in the networks [11]. A block on k inputs pays $\ln\binom{d}{k} + k\ln 2 + \ln(4K(1+\mu)k_{\max}/\delta)$, only $k\ln 2 + \ln(24(1+\mu))$ more than a character of degree k, although the block class of size k is 2^k times larger.

2.3 One round

A round computes the residual r and its root mean square s, runs the character tests, the dense moves and the sum blocks on r, and refits; each step uses only statistics of the current residual on the training rows (every step in Appendix B.2).

The character test. A candidate S of a class of N candidates, tested on the first m rows ($m = n$ unless the class is subsampled), passes if

$$z_S = \frac{|v_S|}{c\,\sigma_m\sqrt{2L_N/m}} > 1, \qquad v_S = \frac{1}{m}\sum_{t\le m} r_t\,\chi_S(x_t), \qquad L_N = \ln\frac{4KN}{\delta}, \tag{2}$$

with $\sigma_m = \mathrm{rms}(r_1, \ldots, r_m)$ (so $\sigma_n = s$), $c = 1$, $\delta = 0.1$ and $K = 4$. Since the standard error of v_S is about $\sigma_m/\sqrt{m}$, the test asks for $c\sqrt{2L_N}$ standard errors: a Bonferroni threshold over the class. The classes are the *global* ones, every χ_S of one degree k ($N = \binom{d}{k}$; $k \le 4$ in round 0, $k \le 3$ after, and degrees 5 and 6 in round 0 of the final fit on a budgeted number of rows), the *extensions* $T \cup \{j\}$ and $T \cup \{j, l\}$ of the up to 32 current characters χ_T with the largest weights, and the *local* class, every subset of the 2 to 12 inputs that the current characters use. A round admits up to 64 passing candidates, those with the largest z_S, within the room of $\min(256, \lfloor n/2 \rfloor)$ features.

Dense moves. Two moves catch signal spread over many small coefficients, none of which would pass a test of its own. The *linear block* admits at once all k inputs whose degree-1 character is neither a feature nor a candidate, if

$$T_{\mathrm{lin}} = \frac{n}{s^2} \sum_{i \in \mathrm{lin}} c_{1,i}^2 > k + 2\sqrt{k \ln(1/\delta)} + 2\ln(1/\delta) \quad \text{and} \quad n \ge 2(p + k), \tag{3}$$

with $c_1 = X^\top r/n$: Laurent and Massart's tail bound for χ_k^2 at level δ [25]. It carries halfspaces and majorities. The *spectral features* take up to four eigenvectors of the residual pair matrix, at $n \ge 4d$, whose eigenvalues clear the semicircle edge of a $d \times d$ Wigner matrix with entries of size $s/\sqrt{n}$, times a margin factor:

$$|\lambda| > 2s\sqrt{d/n}\left(1 + \sqrt{2\ln(1/\delta)/d}\right). \tag{4}$$

The sum-block test. Take a block B with occupied groups v, each of n_v training points, and a vector ρ: the residual r, or $r\chi_P$ for a block modulated by χ_P. The block passes if the squared norm of the projection of ρ on its group indicators clears a χ^2 bound,

$$T_B(\rho) = \frac{1}{s^2} \sum_v n_v \bar{\rho}_v^2 = \frac{|\Pi_B \rho|^2}{s^2} \;>\; \nu + 2\sqrt{\nu L_B} + 2L_B, \tag{5}$$

$$L_B = \ln \frac{4KN_B}{\delta}, \qquad N_B = \tbinom{d}{k}\, 2^{k-1} \cdot 2 \cdot (1 + \mu) \cdot k_{\max},$$

where $\bar{\rho}_v$ is the mean of ρ over group v, $\nu = \max(\text{occupied groups} - 1, 1)$, $k = |S_B|$, $\mu \le 4$ is the number of current characters used as modulators, and $k_{\max} = 24$; N_B counts the supports, the sign patterns up to a global sign, the two fold variants, the modulators and the 24 sizes. With Gaussian noise a fixed candidate would pass with probability at most e^{-L_B} [25], so the union over all of them would be at most $\delta/(4K)$, whichever candidates the search visits. That calibration is Gaussian; it is not proved for DFG's residuals, whose groups depend on the data (Section 2.5). A block must also pass on the residual projected off this round's passing characters, and weighted blocks and products pay for their extra choices in L_B.

The block search. In short, a multi-start local search over signed subsets, scored on the residual alone. It starts from prefixes of the inputs ordered by the top eigenvectors of the residual pair matrix (for a function of $t = \sum_{i \in S} s_i x_i$ that matrix is about $a_2(ss^\top - I)$ on S, so its top eigenvector gives both the support and the signs), from prefixes ordered by the degree-1 coefficients, and from greedy growth out of the strongest pairs and single inputs, plus a beam at $n \le 128$ and v1.3's weighted candidates. The best three candidates are refined by single add, remove and flip moves, and the winner is tested once on all rows. Because Equation (5) takes a union over every (S, s) of the winner's size, the selection is paid for, if the Gaussian calibration holds. Every starting point is a statistic of one or two inputs (Proposition 2.5(f)), which is where the residue classes fail (Section 6.4). A round also searches products and modulated blocks (Appendix B.3).

Stopping and readout. DFG stops when a round admits nothing (in the default variant), when no room is left, from round 1 on when every training label's sign is right, when 0.75 of the call's budget is spent, or after 8 rounds. The least-squares weights are then refined by 15 logistic Newton steps if $n \geq 8p$, and the prediction adds the kernel background (Appendix B.4).

2.4 Small samples and compute

The forced variant and its cross-validation. With few points the union bound passes nothing: at $d = 30$ no character can pass its round-0 test below $n = 17$, the degree-1 floor $2c^2 \ln(4Kd/\delta) = 16.95$, and no degree-4 character below $n = 31$ (Table 2). In a round where nothing passes, the forced variant adds the best block or character anyway, at most twice per fit; a forced guess passes no test. At $n \leq$ `cv_n` $= 512$, `DFG.fit` chooses between the default and the forced variant by cross-validation (Figure 1): the forced variant is chosen if $e_{10} - e_{01} > 2\sqrt{e_{10} + e_{01}}$, McNemar's test for paired errors [29], one-sided at two standard errors, with e_{10} the held-out points where only the default errs and e_{01} the reverse. The fold fits' FLOPs count toward the fit. No level is proved for this choice, because the folds' held-out errors are dependent.

The kernel background. With no feature but the constant, the predictor $\text{sign}(w_0 + g \cdot x)$ is the centred Hebbian classifier, one step on all d inputs, the counterpart of the networks' near-kernel component; once the features fit the labels, $r \approx 0$ and $g \approx 0$. The background has no fitted parameter.

The budget. The unit is one training run of a toy network at the same d, 1.62×10^{11} FLOPs at $d = 30$ (Section 3.3). The exhaustive searches (degree 3 and 4, degrees 5 and 6 in round 0 of the final fit, and the two-input extensions) are subsampled to fixed shares of it, and no round starts after 0.75 of it is spent. At $n \leq 512$ the final fit gets one half and the fold fits share the other (Appendix B.6).

2.5 Simple guarantees

Six statements about DFG are made here, P1–P5 and P2′ of `study/part11/interp/theory/THEORY.md`, restated for the code as it is; all are proved except Proposition 2.4(b), a first-order heuristic. They concern round 0, single tests, the block dictionary and the code's arithmetic; none bounds the test error of a fitted model. Proofs, what each does not cover, and the numerical checks are in Appendix C. The rows are iid uniform on $\{\pm 1\}^d$, and $L_k = \ln(4K\binom{d}{k}/\delta)$ is the log term of the global degree-k class.

Proposition 2.1 (Round 0's character and linear tests under the null; P1)**.** *[Proved]. Let the labels be independent of the rows, with any joint law. In round 0 of any fit call (a final or a fold fit, either variant, any budget scale) the following hold.*

(a) For each degree k the call searches, and every $u > 0$,

$$\Pr\Big(\max_{|S|=k} z_S \geq u\Big) \leq 2\tbinom{d}{k}\Big(\frac{\delta}{4K\binom{d}{k}}\Big)^{u^2c^2}.$$

At $u = c = 1$ this is $\delta/(2K) = 0.0125$ per degree, for every d and n.

(b) The linear block's statistic over all d inputs satisfies $\Pr(T_{\text{lin}} > d + 2\sqrt{dx} + 2x) \leq e^{-x}$ for every $x > 0$. Hence a linear block is admitted with probability at most $\delta + \delta/(2K)$.

(c) If round 0 admits nothing, the default variant's model is the constant plus the kernel background. The default variant's model therefore holds a feature other than the constant with probability at most $3\delta/4+\delta = 0.175$, plus the probabilities that round 0 admits a spectral feature or a sum block.

Proposition 2.1 does not cover the spectral test, the sum-block tests (calibrated for Gaussian noise, with data-dependent groups), the forced variant, the cross-validation's choice or rounds ≥ 1; its part (a) also holds for iid rows from any law when the search uses all rows (Appendix C.1). All 280 simulated checks of (a) held, and the bound is about ten times loose (Section 6.5).

Proposition 2.2 (The sample size of a parity; P2)**.** *[Proved]. Let $y = \chi_S(x)$ with $|S| = k$. Take round 0 of a fit call on n rows in which the degree-k search runs on all of them: $k \leq 2$ always, and $k = 3, 4$ when $m_k = n \geq 32$. Then $\mathrm{mean}(r\chi_S) = 1 - w_0\bar{y}$ and $\mathrm{mean}(r^2) = 1 - 2w_0\bar{y} + w_0^2$. With $w_0 = \bar{y}$ (dropping the 10^{-9}), χ_S passes its test if and only if*

$$n\,(1-\bar{y}^2) > n_{\mathrm{thr}}(k,d) = 2c^2\ln\!\big(4K\tbinom{d}{k}/\delta\big) = 2c^2\big(k\ln d - \ln k! + \ln(4K/\delta)\big) + O(k^2/d). \tag{6}$$

Since $\sum_t y_t$ is a sum of n independent fair signs, exactly $\Pr(\mathit{pass}) = \Pr\big(|\sum_t y_t| < \sqrt{n(n-n_{\mathrm{thr}})}\big)$.

Passing round 0's test is not the same as being in the final model: admission also needs a place among the round's 64 largest z and room for the feature (not proved, but never binding in the parity cells), and the forced variant can admit χ_S without passing. Whether the final model holds χ_S is therefore measured: the criterion, with each training set's own $\bar{y}$, predicts the outcome in all 250 parity cells with $n \geq 32$ (Appendix C.2).

Proposition 2.3 (A floor for every target; P2′)**.** *[Proved]. For any labels, in any round, no character passes the test of a class of N characters tested on m rows unless $m > 2c^2\ln(4KN/\delta)$. No block passes its test unless $n > \nu + 2\sqrt{\nu L_B} + 2L_B \geq 1 + 2\sqrt{L_B} + 2L_B$.*

Both halves bound passing a test, not admission: the forced variant's guesses and the linear block's test lie outside these floors. A balanced k-parity attains the character floor, and the last column of Table 2 gives the floors at $d = 30$.

Proposition 2.4 (When a character passes in round 0; P3)**.** *(a) [Proved]. In round 0, for a set S searched on all n rows, $v_S = \hat{c}_S - w_0\bar{\chi}_S$ with $\hat{c}_S = \mathrm{mean}(y\chi_S)$ and $\bar{\chi}_S = \mathrm{mean}(\chi_S)$, and $s^2 = 1 - 2w_0\bar{y} + w_0^2$. Moreover $z_S > 1$ iff $n > 2c^2 L_{|S|}\, s^2/v_S^2$. For iid uniform rows and nonempty S, $\mathbb{E}v_S = \hat{f}(S)\,(1 - 1/(n+10^{-9}))$.*

(b) [Heuristic]. Replacing v_S by $\hat{f}(S)$ and s^2 by $1-\hat{f}(\emptyset)^2$, S passes the round-0 test once

$$n > n_{\mathrm{glob}}(S) = 2c^2\ln\!\big(4K\tbinom{d}{|S|}/\delta\big)\,\frac{1-\hat{f}(\emptyset)^2}{\hat{f}(S)^2}. \tag{7}$$

(c) [Proved]. In round j, S passes the test of a class of N_{cls} candidates tested on all rows iff $n > n_{\mathrm{cls}} = 2c^2\ln(4KN_{\mathrm{cls}}/\delta)\,s_j^2/v_{S,j}^2$ (on a subsample, m and σ_m replace n and s_j). Hence

$$\frac{n_{\mathrm{glob}}}{n_{\mathrm{cls}}} = \frac{L_{|S|}}{\ln(4KN_{\mathrm{cls}}/\delta)}\cdot\frac{1-\hat{f}(\emptyset)^2}{s_j^2}\cdot\frac{v_{S,j}^2}{\hat{f}(S)^2}\,:$$

a smaller union bound, a smaller residual, and the residual's coefficient on S.

Sections 6.3 and 6.6 use n_{glob} for when, and in what order, DFG admits a target's characters.

Proposition 2.5 (Sum blocks; P4)**.** *[Proved].* *Let $f(x) = \psi(\beta(x))$, a function of $\beta = \#\{i \in S : s_i x_i = 1\} = (t+k)/2$, where $t = \sum_{i\in S} s_i x_i$ is a signed sum of k inputs.*

(a) If every value of t that occurs in training is its own group, the span of the block's indicators and the constant contains f exactly at every x whose value of t occurs in training, with at most $k+1$ weights. (DFG's small ridge shrinks the fitted weights slightly.)

(b) Every value is its own group iff every value occurs at least $\max(3, \lfloor 0.005n \rfloor)$ times. In the large-n limit this holds iff $k \le 7$.

(c) A sign readout that uses one block is constant on its groups. It therefore errs on at least $\sum_{\text{groups}} \min(\Pr(f=1, \text{group}), \Pr(f=-1, \text{group}))$ of the inputs.

(d) $\hat{f}(T) = 0$ unless $T \subseteq S$, and $\hat{f}(T) = (\prod_{i\in T} s_i)\, a_{|T|}$ with $a_j = 2^{-k} \sum_\beta \psi(\beta) \mathcal{K}_j(\beta)$, where $\mathcal{K}_j(\beta) = \sum_l (-1)^{j-l} \binom{j}{l} \binom{k-j}{\beta-l}$. The Fourier weight at degree j is $W_j = \binom{k}{j} a_j^2$.

(e) For a residue class, $\psi(\beta) = \pm(1 - 2[\beta \equiv \kappa \bmod 3])$, and $j \ge 1$: $|a_j| \le \frac{4}{3}\, 3^{j/2}\, 2^{-k}$, so $W_j \le \frac{16}{9}\binom{k}{j} 3^j 4^{-k}$.

(f) Every starting point of the block search is computed from statistics of one or two inputs, or is an earlier candidate grown from such a starting point. In round 0, $\mathbb{E}M_{ij} = \hat{f}(\{i,j\})(1 - 1/(n+10^{-9}))$ for $i \ne j$ (the off-diagonal entries of M° in the first additive search), and $\mathbb{E}c_{1,i} = \hat{f}(\{i\})(1 - 1/(n+10^{-9}))$.

Here $M = \frac{1}{n}\sum_t r_t x_t x_t^\top$ is the residual pair matrix and M° the search's own copy of it (Appendix B.3); Section 6.4 applies (d)–(f) to the residue classes mod 3.

Proposition 2.6 (Compute; P5)**.** *[Proved, by the code].* *(a) The global searches of a round are charged, with a multiply-add counting 2 (lines 617–658), $2nd$ FLOPs for degree 1, $2nd^2 + nd$ for degree 2, $(d-2)(2m_3d^2 + 2m_3 d)$ for degree 3, $\binom{d-2}{2}(2m_4 d^2 + 3m_4 d)$ for degree 4, and $m_k(2N_1N_2 + N_1k_1 + N_2k_2)$ for degrees $k = 5, 6$, with $N_i = \binom{d}{k_i}$, $k_1 = \lceil k/2 \rceil$ and $k_2 = \lfloor k/2 \rfloor$. The exhaustive degree-3 and degree-4 searches therefore cost about $2nd^3 \approx 12n\binom{d}{3}$ and $nd^4 \approx 24n\binom{d}{4}$.*

(b) Each budgeted search runs on $m = \min(n, \lfloor \text{share} \cdot \text{GD} \cdot \text{scale}/c_{\text{row}} \rfloor)$ rows, where c_{row}, the code's per-row cost estimate, is at least the charged per-row cost, so each budgeted search costs at most its share of the call's budget. The shares of the $2F+1$ calls at $n \le 512$ add up to one run, $\frac{1}{2} + 2F \cdot \frac{1}{4F} = 1$. The unbudgeted parts are bounded only by the round cap, up to the cost of the last round.

An exhaustive degree-4 search on 16384 rows costs 0.072, 1.04 and 6.5 network runs at $d = 30$, 60 and 120, so the budget's subsampling is what keeps DFG inside one run (Appendix C.5).

What these statements do not give. None of them bounds the test error of a fitted model. Proposition 2.1 covers round 0 only. Proposition 2.2 is for noiseless parities, and Proposition 2.4(b) is first order. Proposition 2.5 says what a block can represent and what the search's starting points see, not when the search succeeds. Proposition 2.6 counts FLOPs, not CPU time (Section 4.6).

2.6 How DFG relates to familiar learners

[Heuristic], except where a statement restates the code or Section 2.5.

- *Forward stepwise regression and orthogonal matching pursuit.* DFG's loop is forward selection with a joint least-squares refit, as in orthogonal matching pursuit [31], unlike matching pursuit

[28]. It differs in three ways: its dictionary is enumerated class by class, never stored; a round admits every passing candidate, up to 64; and its entry rule is a significance test, not "take the best". If χ_S is orthogonal on the sample to the current features, $z_S > 1$ iff the reduction nv_S^2 of the residual sum of squares, in units of s^2, exceeds $2c^2 \ln(4KN/\delta)$; with $c = 1$ this is the per-variable penalty $2 \ln p$ of the risk inflation criterion [16] and the universal threshold $\sqrt{2 \ln p}$ of wavelet shrinkage [14], with $p = 4KN/\delta$: a Bonferroni rule within each class.

- *The low-degree algorithm and sparse Fourier learning.* The low-degree algorithm [26] estimates every Fourier coefficient up to degree k and predicts with the sign of the truncated expansion; `lowdeg` is its thresholded version. DFG is a greedy, budgeted, residual-based version of it: it estimates coefficients of the residual, not of the labels, refits the weights instead of using the raw estimates, adds classes that grow from what it has, and adds a second dictionary, the signed-sum blocks. Kushilevitz and Mansour [24], building on Goldreich and Levin [19], find every large Fourier coefficient of any degree in time polynomial in d, but they need membership queries; with random examples only, DFG enumerates each class, at order $n\binom{d}{k}$ FLOPs for degree k, and reaches characters above degree 6 only through the extensions, the local class and the blocks.

- *Statistical queries.* Each character test is a correlational statistical query [8, 22], $\mathbb{E}[r\chi_S]$, asked of the residual instead of the label, and the union bound is the tolerance budget for all N queries of a class, answered from the same n rows: that is why the sample cost is logarithmic in N (Proposition 2.2). For parities, statistical-query learners need a number of queries, or an inverse tolerance, polynomial in the number of candidate parities [6]; DFG pays the query count in compute and only its logarithm in samples. Analyses of gradient training on sparse parities find costs polynomial in d, with an exponent that grows with k [1, 3].

- *Occam's razor and minimum description length.* The log term $\ln(4KN/\delta)$ is, up to the constant $\ln(4K/\delta)$, the number of nats needed to name one candidate in its class, so the character test reads: the residual sum of squares it removes, over $2s^2$ (the Gaussian code length it saves), exceeds $c^2 \ln(4KN/\delta)$. That is a two-part code in the sense of minimum description length [33], without the usual $\frac{1}{2} \ln n$ for the coefficient, and the parity criterion $n > 2c^2 \ln(4K\binom{d}{k}/\delta)$ is the matching Occam sample size [7], logarithmic in the size of the class (Appendix B.1).

- *Boosting.* Like gradient boosting [17], and L2 boosting in particular [9], DFG fits the residual stagewise with base learners from a dictionary. Unlike boosting it has no shrinkage and no early stopping: a candidate enters only if it passes its test, and every weight is refitted jointly after each round. Its base learners are fixed Boolean features, not fitted trees, and its loop is short (Section 6.6).

- *Gradient-trained networks.* Part 6 designed DFG from what Parts 1–5 had seen the study's networks do [10, 11]: they grow features greedily, cheapest first; a feature's cost is set by its correlation with the residual against the noise of the competing directions; features on inputs already in use are cheaper; a near-kernel linear component is present from the start; and on every plateau the network computes the sign of a least-squares fit on the features grown so far. DFG keeps this picture and replaces the search: enumeration and explicit tests take the place of noisy gradient steps in weight space (Section 6.6 compares the orders), a trade of samples for FLOPs (Section 4.5).

object	how it is chosen	continuous?
characters χ_S	enumeration in classes; the union-bound test (2)	no
sum blocks: (S, s), fold, inner term $a\tau$, modulator, product partner	the block search (eigenvector, pair and single-input seeds, greedy growth, beam, single moves); the χ^2 test (5)	no
a block's groups	contiguous runs of observed values with at least $\max(3, \lfloor 0.005n \rfloor)$ points	no
the linear block's inputs	one χ^2 test (3)	no
the variant (default or forced)	the paired test on held-out errors	no
the settings K, c, δ, budget shares, ...	constructor defaults, fixed across all data sets (Table 11)	no
spectral directions u	top eigenvectors of the residual pair matrix	borderline: real directions, computed in closed form
the kernel background g	one Hebbian step, $g = X^\top r/n$	yes, computed in closed form
readout weights w	least squares with a small ridge, then 15 Newton steps of logistic regression	yes: a linear readout

Table 3: What DFG v1.3 chooses discretely and what it fits continuously [Proved, by the code]. Source: `study/part10/hyb_portfolio3/dfg13.py` (Sections 2.2 to 2.4; the constants as in Table 11).

2.7 What is discrete

Table 3 lists what DFG fits. Every feature but the spectral ones comes from a finite dictionary, enumerated or proposed by a combinatorial search and admitted by a test with an explicit threshold (or, in the forced variant, as the best-scoring candidate), so a fitted model can be printed as a list of named features. Only the readout is fitted continuously; no feature is fitted by gradient descent, and the readout's 15 Newton steps are the only iterative continuous optimization in a fit. The spectral directions are the borderline case: real vectors computed in closed form, each feature a fixed combination of degree-2 characters (the post-hoc core, which has none, is level with DFG on average; Section 5.3).

3 How it is evaluated

This section gives the tasks, the references, the metrics and budgets, and the protocol, including what is held out for DFG v1.3; Appendix D gives the evaluation in full, and Appendix A collects the names.

3.1 Tasks

Toy Boolean targets. Inputs are uniform on $\{\pm 1\}^d$ with $d = 30$, and labels are the ± 1 values of a target f. There are 11 suites, 138 targets in all, defined in `study/boolfuncs.py` and listed with their status in Table 12. The data are the networks' own (`study/discrete_learner/harness.py`). For each target and seed, the training set of size n is the first n rows of one draw of 16384 points, so training sets are nested in n, and each target has one fixed test set of 4096 points. `designed` (21 targets) and `random` (20) hold the study's original juntas, parities, halfspaces, compositions and small formulas; `dlheld`, `dlheld2` and `dlheld3` (29) add dense halfspaces, multiplexers, formulas and symmetric functions; `dlsymdev`, `dlheld4` and `dlheld5` (32) symmetric and signed-sum targets.

`dlheld6`, `dlfamdev` and `dlheld7` (12 each) are three draws from the *blind family*, a random generator of signed-sum targets written in Part 9 by an agent that had not seen DFG's sum-block design (`study/part9/heldout6/FAMILY.md`): nine slots of functions of a signed sum of 4–14 inputs, alone or with a parity, a gate, an inner term or a second sum, with accept sets that are thresholds, bands, intervals, residue classes mod 3 or 4, or arbitrary, and three non-symmetric controls (Appendix D.1). DFG v1.3 was developed on `dlheld6` and `dlfamdev` (Section 3.4), so `dlheld7` tests it on fresh targets from a distribution its designer had seen.

The *sum-structured targets* are the 54 targets of `dlsymdev`, `dlheld4`–`dlheld7` and `dlfamdev` other than their 14 non-symmetric controls. Every suite puts a target's relevant inputs first. DFG alone and `lowdeg` therefore see the columns in a fresh random order in every fit, drawn from the fit's random stream (`harness.Permuted`), as v2 and v3 did, so that tie-breaking by column index cannot help them.

Input dimension. Q4 uses the `designed` targets on which the networks were trained at other input dimensions in Part 3: `maj9`, `majpar`, `mux`, `par2`, `par3`, `par4` and `stair5` at $d = 15$ and $d = 60$, and `maj9`, `mux`, `par2` and `par3` at $d = 120$ (7, 7 and 4 targets). Each is the same function of the first inputs as at $d = 30$, and its data are drawn in the same way.

Tabular data. DFG runs on the two of the study's 26 PMLB datasets [30] that have only binary or categorical columns and two classes, kr_vs_kp and mushroom, with the columns one-hot coded as ± 1 (Appendix E.6). It runs neither on images nor on real-valued columns.

3.2 References

Networks on the toy targets. These are the companion paper's μP two-hidden-layer ReLU MLPs, d-128-128-1 [11, 35]: Adam [23] at learning rate 10^{-3}, batches of 64 sampled from the fixed training set, 20000 steps (40000 at $d = 120$), read at the final step. There are five variants: `nets` (square loss, the only variant run at $d \neq 30$), `nets_wd` (square loss with decoupled weight decay 0.1 [27]), and `nets_cewd`, `nets_cewd0.3` and `nets_cewd1.0` (logistic loss with weight decay 0.1, 0.3 and 1.0), run on different suites (Appendix D.2); `dlfamdev` has none. `nets_best` is, per target, the variant with the smallest n^* among the 2 to 5 run on its suite (on `dlheld7`, the 3 that Part 10 pre-registered): a hindsight choice that favours the networks. The networks are one recipe, not tuned per target or suite, so the comparison is with a standard gradient-trained network, not with the best network for each target.

No-search baselines. `lin` is L2-regularized logistic regression on the raw inputs, its penalty chosen by cross-validation, and `hgb` scikit-learn's histogram gradient boosting at default settings [32] (both in `study/hybrid/refs.py`; details in Appendix D.2). Neither searches an explicit feature dictionary: `lin` searches no features at all, and `hgb` only through greedy splits on single inputs. So they show which targets need no dictionary search. Both were run in Parts 8–10, with the columns in their natural order; `hgb` has no runs on `designed`, `dlsymdev` and `dlfamdev`, and `lin` none on `dlfamdev`.

The low-degree algorithm: `lowdeg`. `lowdeg` (`study/discrete_learner/sparse_fourier.py`) is Part 6's first feasibility check, run unchanged with the inputs permuted per fit: the low-degree algorithm [26] with a significance threshold, which keeps every character of degree ≤ 4 whose empirical coefficient on the labels passes a union bound over its degree, tests products of kept

characters in the same way, and predicts with the sign of the kept expansion (Appendix D.2). It shares DFG's global characters up to degree 4 and the form of its test, and nothing else: no signed-sum blocks, no residual rounds and no refitted readout.

Portfolios that contain DFG. In v3 (`hyb_portfolio3`, Part 10 [12]), DFG v1.3 ran as two members, its default and (at $n \leq 512$) its forced variant, next to LightGBM, ExtraTrees, k-nearest-neighbour votes and logistic regression, with a share of the budget, and a convex stacker fitted on out-of-fold predictions weighted them (`study/part10/hyb_portfolio3/learner.py`; Appendix E.3). Alone, DFG chooses between its two variants itself, adds its own kernel background and has the whole budget. v3's code ran on every toy suite with the inputs permuted per fit, so DFG alone and v3 can be compared target by target, which measures what the rest of the portfolio adds on Boolean inputs. v2 and v1 (Parts 9 and 8) are the earlier portfolios, with DFG v1.2 and v1.1 [10]; earlier versions of DFG and the tabular references are in Appendices D.2 and E.6.

3.3 Metrics and budgets

Toy targets. n^* is the training-set size at which the seed-mean test error reaches 0.1, interpolated in $\log n$ between the grid points $n = 16, 32, \ldots, 16384$. It is "never" if the error does not reach 0.1 by $n = 16384$. DFG alone runs on the networks' seeds and training sets: 3 seeds per target, 2 on `random`.

- A comparison of learners A and B on a target is the ratio $\log_2(n^*_A/n^*_B)$, negative when A needs less data.
- A is *clearly better* when the ratio is below -0.25 or only A reaches 10% error, and *clearly worse* in the mirror cases.
- A suite mean is the mean ratio over the targets both learn. Pooled summaries pool the per-target ratios over suites, rather than averaging suite means.

The verdict has no noise term: n^* is read from test errors averaged over 2 or 3 training sets per (target, n), with no confidence interval, and the ± 0.25 band is a convention, not a significance test. It is smaller than the spread between single seeds: DFG's n^* on two different single training sets differ by a mean $|\log_2 \text{ ratio}|$ of 0.339 (23 of 65 targets beyond ± 0.25), refits with another column order by 0.012 (Section 5.1). Per-target verdicts rest on the 2–3-seed means and are not significance calls. These are the definitions of Parts 8–10 (`study/hybrid/compare_hybrid.py`), and `study/part11/compare_dfg.py` computes every comparison from the result files.

Input dimension. At $d = 15$, 60 and 120, DFG and the networks have seeds 0 and 1, and at $d = 30$ both have 3; at $d = 60$ and 120 the networks' n^* is read on their own grids, which reach up to 65536 points. The growth of n^* with d is summarized by the least-squares slope of $\log n^*$ on $\log d$, fitted for each learner over the values of d at which both reach 10% error.

Budgets. Compute is counted in FLOPs, a multiply-add counting 2; everything a fit does is included, DFG's cross-validation fits too, and prediction is excluded. The unit is one training run of one toy network: $6P \cdot 64 \cdot T + 10P \cdot T$ FLOPs for $P = 128d + 16768$ parameters and T steps (`harness.gd_flops`), 1.62×10^{11} at $d = 30$ with $T = 20000$; 1.47 and 1.93×10^{11} at $d = 15$ and 60; and 5.06×10^{11} at $d = 120$, where the networks ran 40000 steps. Each network variant spends exactly one unit per fit, and `nets_best` is a hindsight minimum over up to five such runs. DFG spends what its own budget allows (Section 2.4); the other references run their standard recipes

uncapped. CPU time is the process time of a fit, rough because it was measured on a shared machine (Appendix E.7); the networks' result files hold no timing, so the compute comparison with them is in FLOPs only.

3.4 The Part 11 protocol, and what is held out

Pre-registration. The Part 11 protocol, `study/part11/PROTOCOL.md`, was written and committed (`aca1f3c8`) together with the runners before any Part 11 result was produced, except one disclosed timing check, and its pre-registered part has not changed since (commits and times in Appendix H.2). It fixed the object and how it is run, seven questions with their runs, suites and seeds, the rules of Parts 8–10, a rule for crashes and what is held out (Appendix D.3). One post-hoc addendum, which defines the core run of Section 5.3, was added after every pre-registered run had finished.

The object. DFG v1.3 is the file `study/part10/hyb_portfolio3/dfg13.py`, frozen in Part 10 with md5 `69cabe5c50345cb24079198ed52b562f`. Part 11 runs it alone, inside `harness.Permuted`, as the run `dfg13-perm`: `DFG(readout="logit").fit(X, y, rng)`, with every other constructor argument at its default; the runners `study/part11/eval_dfg.py` and `study/part11/dfg_real.py` check the md5. No line of it changes, and nothing is tuned. In the rest of the paper "DFG" means DFG v1.3 run alone, and "DFG alone" is written where the contrast with v3's DFG member matters; the reasons for choosing this object are in Section 1.

Questions and runs. The protocol fixed seven questions: Q1, `dlheld7`; Q2, every suite, v3 included; Q3, `lowdeg`; Q4, the input dimension; Q5, 12 ablations; Q6, tabular data; and Q7, descriptive analyses of what DFG learns, checked by independent verifiers. Section 4 reports Q1–Q4 and Q6, Section 5 Q5, and Section 6 Q7; Appendix D.3 lists each question's runs, suites and seeds.

What is held out. For DFG v1.3, only `dlheld7` is held out (Table 12). It was generated in Part 10 from the blind family and fixed in a commit (`dc70266c`) before any Part 10 design work; Part 10's builders, the agents that designed its components, were not allowed to load, generate or inspect it, the one that built v1.3 recorded that it saw only the one-line signature of the suite's generator function, and no result of a Part 10 learner on it was read before v3 was frozen [12].

- *Everything else is in-sample.* Every other suite was used in developing some version of DFG, or was held out only for an earlier version, and several of DFG's parts were added after failures on, or with knowledge of, such targets: the degree-5/6 search and the spectral block (`dlheld`'s `par5` and majxor), the logistic readout (`thr30`), sum blocks (`sym9`, `dlheld3`) and weighted blocks (h6_10, `dlheld6`). Those suites still measure the learner, but they do not test it, and some selection in its favour is expected there [Heuristic]. The Q4 targets are `designed` targets, and kr_vs_kp is a development set; mushroom was not used to develop any version of DFG, but v1–v3 ran DFG on it, and their results were known. The ablations and the analyses (Q5, Q7) describe the frozen learner; they are not tests of a design.
- *The object was chosen post hoc.* It was chosen knowing v3's results on every suite, including `dlheld7`, where DFG is v3's toy member. DFG alone's results there were new, apart from a timing check on two targets, but they are not independent of v3's, which were known. The choice is post hoc; the code is not. The timing check and the other disclosures are in Appendix H.4.

4 How well it does

This section compares DFG with the networks and every other reference: on the held-out suite, on all 11 toy suites, against v3 and the low-degree algorithm, in the input dimension, on tabular data and in compute. Numbers come from `study/part11/RESULTS.json`, which `study/part11/compare_dfg.py` computes from the result files, unless another file is named; the tables and figures are generated from it by `paper_dfg/make_tables.py` and `paper_dfg/make_figures.py`, and derived numbers that no table shows are in `paper_dfg/tables/numbers.json` and the figures' `.json` files. The rules are those of Section 3.3; Appendix E gives every target and the further comparisons.

4.1 The held-out suite `dlheld7`

`dlheld7` is the only toy suite held out for DFG v1.3 (Section 3.4). Table 4 gives every target's definition and n^*, and Table 13 (Appendix E.1) the summaries against each reference.

Against the networks [Numerical].

- *Against* ***`nets_best`****:* mean $\log_2 n^*$ ratio -1.29 over the 11 targets both learn, about $2.4\times$ less data (median -1.16). DFG is clearly better on 10 of the 12 targets and clearly worse on none. The 11 ratios have s.d. 0.91; if the targets are treated as independent draws from the family, the mean has a standard error of about 0.27.
- *Against each variant* DFG is clearly better on 10 or 11 of the 12 targets (Table 13), and no variant is clearly better than DFG on any target. The largest margin is on f7_00, an OR of conditions on two signed sums: 211 against 1545 (-2.87).

Against v3, `lowdeg` and the no-search baselines [Numerical]. v3 needs less data than DFG alone (mean ratio $+0.21$, $1.16\times$ more data for DFG alone; median $+0.05$): DFG alone is clearly worse on the three non-symmetric controls, a DNF, a decision list and a read-once formula, and clearly better on none; v2 gives the same summary. DFG is clearly better than `lowdeg` on 10 targets and worse on 1 (f7_11), and `lowdeg` never learns f7_08. `lin` reaches 10% error on only 4 targets and `hgb` on 9; DFG is clearly better than `lin` on 9 targets and than `hgb` on 8, and clearly worse than `hgb` on 2, f7_02 and f7_06 (Table 13).

On five of the ten wins a no-search baseline also beats the networks [Numerical]. On five targets a no-search baseline is already clearly better than `nets_best`: `hgb` on the three controls and on the multiplexer f7_11, and `lin` on the halfspace f7_10, by $\log_2$ ratios of -1.49 to -0.44. DFG beats `nets_best` on these too (-1.16 to -0.39, mean -0.73). Against the better of the two baselines, however, it is clearly worse on f7_02 ($+0.65$) and f7_06 ($+0.47$), tied on f7_04 ($+0.20$) and f7_10 (-0.06), and clearly better only on f7_11 (-0.31). v3, which stacks DFG with LightGBM and logistic regression, is ahead of both baselines on all five (-0.43 to -0.09).

On the other seven targets neither baseline is clearly better than the networks. `hgb` needs 684–6208 samples or never reaches 10% error, and `lin` learns only f7_01. There DFG is clearly better than `nets_best` on five targets, all sum-structured (f7_00, f7_03, f7_07, f7_08, f7_09), with a mean of -2.11, about $4.3\times$ less data. It ties on f7_01 and does not learn f7_05.

Reading [Heuristic]. The five baseline wins say that the networks are slow on small formulas, on this halfspace and on this multiplexer of two sums, not that DFG's search is needed there. On the controls a tree ensemble is the better learner, and v3 inherits that from its tree members. The

	discrete, inputs permuted			no-search baselines		networks				$\log_2$ ratio of DFG alone to	
target	DFG alone	v3	`lowdeg`	`lin`	`hgb`	`nets_wd`	`nets_cewd0.3`	`nets_cewd1.0`	`nets_best`	`nets_best`	v3
f7_00	211	**203**	7061	never	6208	2418	1545	1565	1545	−2.87	+0.06
f7_01	119	**106**	595	111	684	205	121	119	119	−0.00	+0.17
f7_02	190	**106**	435	never	120	337	341	348	337	−0.83	+0.85
f7_03	**352**	352	4205	never	2199	1710	1665	1599	1599	−2.18	−0.00
f7_04	97	**70**	121	never	84	164	175	181	164	−0.76	+0.47
f7_05	never	never	never	never	never	never	never	never	never	n/a	n/a
f7_06	82	**44**	179	127	59	108	124	124	108	−0.39	+0.90
f7_07	**167**	174	1123	never	1373	445	443	415	415	−1.32	−0.06
f7_08	644	**634**	never	never	never	5843	3239	2932	2932	−2.19	+0.02
f7_09	222	**215**	591	never	never	1000	883	928	883	−1.99	+0.05
f7_10	84	**82**	336	87	114	210	120	119	119	−0.50	+0.03
f7_11	51	55	**29**	119	63	114	144	141	114	−1.16	−0.12

target	definition
f7_00	OR([|s1| ≤ 0], [s2 = −1]), s1 = x1+x2−x3−x4, s2 = −x5−x6−x7+x8+x9+x10+x11
f7_01	[s ≤ 1], s = x1+...+x13
f7_02	OR(AND(x2,−x6,−x8), AND(x1,−x2,−x4,−x5), AND(x3,−x5,−x7))
f7_03	−x11 ? [((s1+6)/2) mod 4 ≠ 2] : [|s2| > 2], s1 = x1+...+x6, s2 = x7+...+x10
f7_04	if AND(−x6,−x9) → +1; elif −x8 → −1; elif −x4 → +1; elif −x2 → −1; elif x7 → −1; elif x5 → −1; elif AND(x3,x1) → −1; else +1
f7_05	[((s+14)/2) mod 3 ≠ 0], s = x1+...+x14
f7_06	OR(AND(AND(−x1,x2),AND(AND(−x3,x4),−x5)),AND(OR(−x6,x7),x8))
f7_07	[−2 ≤ s ≤ 8] * x13, s = −x1+x2−x3+x4+x5+x6+x7+x8−x9+x10−x11+x12
f7_08	[s in {−11,−9,−3,1,9,11}], s = x1+...+x11
f7_09	[s ≤ 0] * x11 * x12, s = x1−x2+x3−x4+x5−x6+x7+x8+x9+x10
f7_10	[u ≤ −1], u = s + 2*(x6), s = x1+...+x5
f7_11	−x11 ? [s1 in {−4,4}] : [|s2| ≤ 4], s1 = x1+x2−x3+x4, s2 = −x5−x6+x7−x8−x9+x10

Table 4: The held-out toy suite `dlheld7` [Numerical]. (a) n^* per target (mean of 3 seeds, interpolated in $\log n$; "never": 10% test error not reached by $n = 16384$) for DFG alone, v3, `lowdeg`, the no-search baselines `lin` and `hgb`, the three network variants run on the suite and their per-target minimum `nets_best`. The last two columns give the $\log_2$ ratio of DFG alone's n^* to that of `nets_best` and of v3 (n/a: neither learns the target). Bold: the smallest n^* in the row. (b) The definitions. $[\cdot]$ is +1 when the condition holds and −1 otherwise; `*` multiplies ±1 values; `c ? a : b` is a when $c = +1$; a literal `x5` or `−x5` inside AND/OR or a decision list stands for $x_5 = +1$ or $x_5 = -1$. f7_02, f7_04 and f7_06 are the family's non-symmetric controls. Source: `study/part11/RESULTS.json` (`q1_dlheld7`); definitions: `study/boolfuncs.py`.

evidence that the search earns its cost on fresh targets is the other five wins, the sum-structured ones above, where no no-search baseline is competitive.

The tie and the target no learner learns [Numerical]. On the halfspace f7_01 DFG, `nets_best`, `lin` and v3 are all within the tie band of one another. f7_05, a residue class mod 3 of a 14-input count, is learned by none of the learners of Table 4, nor by v2, by $n = 16384$ (3 seeds each): the search failure of Section 6.4.

4.2 Every suite

Table 5 summarizes all 11 suites, with each suite's status for DFG v1.3 (Table 12); Figure 2a shows every target with network runs, and Appendix E.1 lists every target's n^* for every learner and pools the suites (Table 17). For DFG v1.3 every suite except `dlheld7` is in-sample in some sense, and the protocol treats the results there as descriptions of the frozen learner, not as tests of its design.

Pooled [Numerical].

- *Against `nets_best`:* on the 126 targets of the 10 suites with network runs, DFG's mean is -1.87 over the 123 targets both learn, about $3.7\times$ less data (median -1.65). Without the held-out `dlheld7`, over the 114 in-sample targets, it is -1.93 over the 112 both learn. It is clearly better on 117, clearly worse on 2, and needs at least $4\times$ less data on 47 (Figure 2a).
- *Against single variants:* -2.04 against `nets_wd`, the one variant run on every suite with network runs (clearly better on 119 of 126, worse on 1), and -2.64 against `nets` (square loss; 9 suites, 114 targets).
- *The seven `dlheld` suites that Part 10 pooled* (`dlheld`, `dlheld2`, ..., `dlheld7`, without `dlsymdev` and `dlfamdev`; 77 targets; for v1.3 only `dlheld7` among them is held out): DFG alone is at -1.71 and v3 at -1.87 against `nets_best` on the same targets.

By suite [Numerical]. The suite means against `nets_best` range from -2.49 (`dlheld5`) to -1.27 (`dlheld6`); `dlheld7` is at -1.29. They are not comparable across suites, because `nets_best` minimizes over 2 to 5 variants depending on the suite. Against `nets_wd` alone, the three smallest margins are those of the held-out suite and of the two suites held out for v1.1: `dlheld2` (-1.59), `dlheld3` (-1.60) and `dlheld7` (-1.62); the five development suites with network runs give -1.71 (`dlheld6`) to -2.45 (`dlsymdev`), and the two exposed suites -2.05 and -2.72.

Reading [Heuristic]. Some selection in DFG's favour on the suites it was developed on or exposed to is therefore likely. The suites also differ in kind, though. `designed` and `random` are small juntas and formulas, `dlsymdev` is symmetric, and `dlheld2`, `dlheld3` and the blind family mix sums, formulas and halfspaces. With 7 to 12 targets per suite, the data do not separate the two explanations. On the held-out suite the margin is about $3\times$ (against `nets_wd`), not the $4\times$ of the pooled numbers.

Where DFG is not better [Numerical]. On 9 of the 126 targets with network runs DFG is not clearly better than `nets_best`: it is clearly worse on the random formula h3rf1 and on the residue class h6_00, ties on five (three threshold-like targets, a formula and and3), and no learner learns f7_05 or h5s1. DFG never learns 4 of the 138 targets (three residue classes mod 3, and h5s1), and neither do v3 and `lowdeg` (Appendix E.2). *Reading* [Heuristic]: on the small formulas, which v3 and `hgb` learn faster, DFG's lack of a clear margin reflects its slowness at small n more than a strength of the networks.

4.3 What the portfolio added

On the same training sets and column orders (Section 3.2; tables in Appendix E.3) [Numerical]: over the 138 targets, DFG alone's n^* is lower than v3's on 32 and higher on 101; it equals v3's on 1 (par2), and neither learns 4. Beyond ± 0.25 it is lower on 6 and higher on 42. The mean ratio is $+0.19$, so DFG alone needs $1.14\times$ v3's data, and the median is $+0.07$; a two-sided sign test over the 133 targets with unequal n^* gives $p = 1.6 \times 10^{-9}$. The gap is at small n: on the 45 targets that v3 learns by $n = 64$, DFG alone's mean is $+0.39$, and v3's test error is lower by more than 0.02 on 63,

suite	status for v1.3	targets	DFG never	nets_best	single variants: min / max mean	v3	v2	lin	hgb	lowdeg
designed	development	21	0	−2.17 (20/0) [21]	−2.82 / −2.17 (2 variants)	+0.25 (1/9) [21]	+0.22 (3/9) [21]	−0.19 (17/1) [7]	–	−0.92 (16/3) [21]
random	development	20	0	−2.04 (20/0) [20]	−2.70 / −2.04 (2 variants)	+0.22 (2/7) [20]	+0.20 (2/5) [20]	−0.62 (20/0) [5]	−1.54 (18/0) [19]	−0.71 (15/4) [20]
dlheld	development	12	0	−2.22 (11/0) [12]	−2.71 / −2.22 (2 variants)	−0.18 (1/4) [12]	−0.16 (1/3) [12]	−0.49 (10/0) [4]	−1.97 (9/1) [11]	−1.53 (9/2) [11]
dlheld2	held out for v1.1 only	7	0	−1.59 (7/0) [7]	−2.22 / −1.59 (2 variants)	+0.40 (0/2) [7]	+0.26 (1/2) [7]	−0.05 (5/0) [3]	−2.39 (5/2) [7]	−2.00 (6/0) [7]
dlheld3	held out for v1.1 only	10	0	−1.34 (7/1) [10]	−2.05 / −1.48 (3 variants)	+0.46 (0/5) [10]	+0.42 (0/5) [10]	−0.11 (7/2) [4]	−1.31 (6/2) [9]	−0.74 (6/3) [10]
dlsymdev	development	8	0	−2.14 (8/0) [8]	−2.99 / −2.15 (3 variants)	+0.08 (0/1) [8]	−0.10 (1/0) [8]	n/a (8/0) [0]	–	−3.98 (8/0) [7]
dlheld4	exposed to v1.2	12	0	−1.67 (12/0) [12]	−2.58 / −1.73 (3 variants)	+0.20 (1/3) [12]	+0.01 (3/2) [12]	−0.72 (12/0) [1]	−2.61 (10/2) [7]	−2.89 (12/0) [11]
dlheld5	exposed to v1.2	12	1	−2.49 (11/0) [11]	−3.20 / −2.51 (3 variants)	−0.06 (1/1) [11]	−0.09 (2/1) [11]	n/a (11/0) [0]	−4.19 (11/0) [6]	−2.05 (11/0) [6]
dlheld6	development	12	1	−1.27 (11/1) [11]	−2.15 / −1.35 (5 variants)	+0.22 (0/3) [11]	−0.02 (2/3) [11]	−0.22 (10/0) [2]	−1.86 (9/2) [7]	−2.29 (10/1) [9]
dlfamdev	development	12	1	–	–	+0.26 (0/4) [11]	+0.20 (1/5) [11]	–	–	−1.67 (9/2) [10]
dlheld7	**held out**	12	1	−1.29 (10/0) [11]	−1.62 / −1.36 (3 variants)	+0.21 (0/3) [11]	+0.21 (0/3) [11]	−0.45 (9/0) [4]	−1.39 (8/2) [9]	−1.90 (10/1) [10]

Table 5: DFG alone on every toy suite [Numerical]. Each entry is the mean $\log_2 n^*$ ratio of DFG alone to the reference over the targets both learn; below it, the numbers of targets on which DFG alone is clearly better / clearly worse, and in brackets the number both learn. "n/a": the reference learns no target of the suite; "–": not run there. `nets_best` is the per-target minimum over the network variants run on the suite (2 to 5; Section 3.2); "single variants" gives the smallest and largest of DFG alone's means against those variants one at a time. "DFG never": targets that DFG alone does not bring to 10% error by $n = 16384$. Status for DFG v1.3: as in Table 12. Source: `study/part11/RESULTS.json` (`q2_suites`).

64 and 67 of the 138 targets at $n = 16$, 32 and 64, on 38 at $n = 128$, 17 at 256 and 5 at 512, and on none at $n \geq 1024$. *Reading* [Numerical, then Heuristic]. At small n the portfolio's other members carry v3: in Part 10's seed-0 refits the stacker gave the DFG members no weight in 52, 47 and 30 fits at $n = 16$, 32 and 64, and on formulas and small thresholds trees, nearest neighbours and logistic regression reach low error with fewer points than DFG needs before its tests admit features. From $n = 256$ on, v3 is never worse than its DFG member by more than 0.02, and the DFG members take most of the weight. The reverse, on 3 of the 6 targets where DFG alone needs clearly less data (par3, rf11, par5), is structure that only a fit on all n rows finds: DFG's degree-3 and higher searches need a minimum number of rows and a share of the budget that v3's out-of-fold DFG fits do not have (Appendix E.3).

4.4 The low-degree algorithm: DFG's characters and test, nothing else

`lowdeg` has DFG's global characters up to degree 4 and the same form of union-bound test, applied to the labels, and none of the rest of DFG (Section 3.2; table in Appendix E.4); Figure 2b shows it target by target.

Results [Numerical].

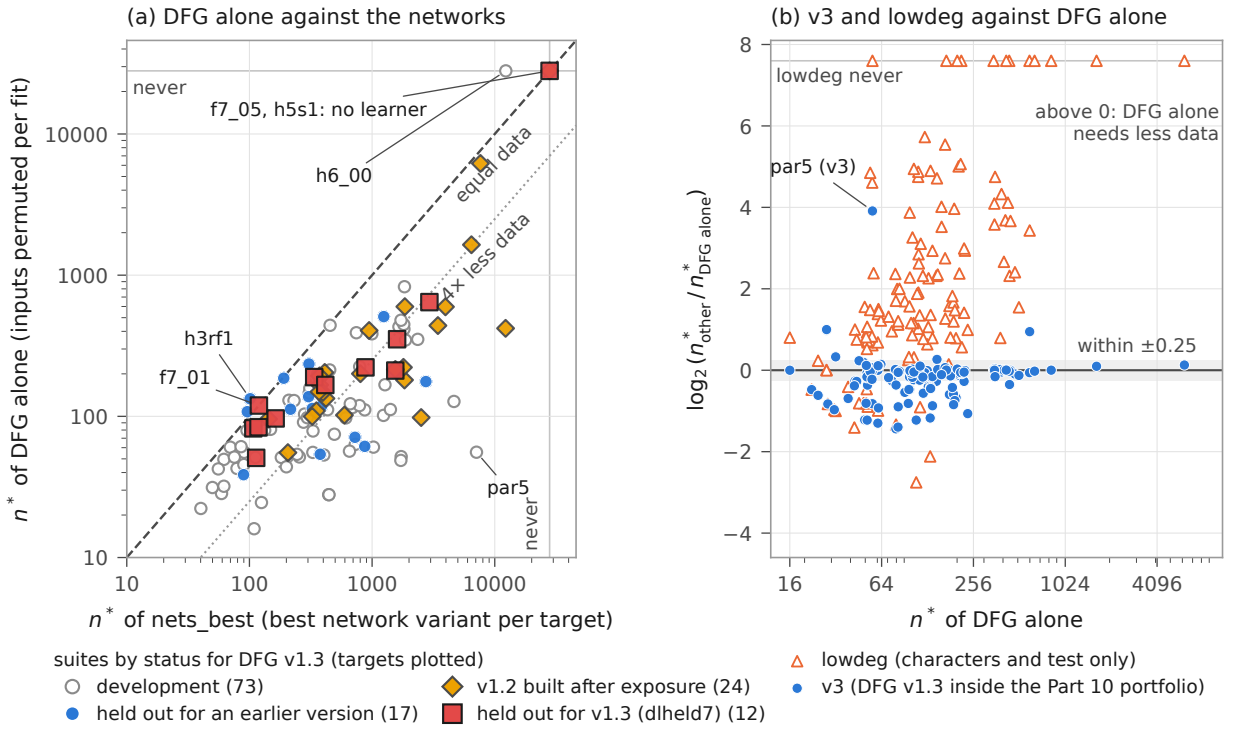


Figure 2: (a) n^* of DFG alone against `nets_best` for the 126 targets of the 10 toy suites with network runs [Numerical]; colour and marker give the suite's status for DFG v1.3, with `dlheld7` as large outlined squares, and "never" is drawn at the edge. Below the dashed line DFG needs less data than the best network variant, below the dotted line at least 4× less. (b) For the 134 targets of all 11 suites that DFG alone learns, the $\log_2$ ratio of v3's and of `lowdeg`'s n^* to DFG alone's, against DFG alone's n^*; above 0 DFG alone needs less data; grey: the tie band ±0.25; `lowdeg`'s "never" at the top. Source: `study/part11/RESULTS.json` via `paper_dfg/make_figures.py`; the plotted points are in `paper_dfg/figures/fig_nstar.json`.

- *Against DFG:* pooled over the 138 targets, `lowdeg` needs more data. Its mean ratio is +1.64 over the 122 targets both learn, about 3.1× more data (median +1.35). It is clearly worse on 112 targets and clearly better on 16.
- *Against the networks* `lowdeg` is about even: −0.16 against `nets_best` over the 112 of 126 targets both learn, clearly better on 53 and clearly worse on 62.
- *Never learned.* `lowdeg` never learns 16 targets: the four that DFG misses and 12 that DFG learns. Of those 12, par5 has degree 5 and is outside `lowdeg`'s dictionary; the other 11 are functions of a signed sum, mostly with arbitrary accept sets.
- *Where `lowdeg` is faster.* All 16 such targets are learned at small n: DFG needs 22–133 points, `lowdeg` 16–61, as on the formulas h3rf0 and h3rf1 and on par4 (28 against 52).

Reading [Heuristic]. DFG's margin over the networks is not explained by its characters and test alone: with them but none of the rest, `lowdeg` is at par with the networks. The margin needs what `lowdeg` lacks, and the sum blocks are one part of it. They make the signed-sum targets much cheaper to learn, and a few learnable at all within the grid: `lowdeg` still learns 39 of the 54 sum-structured targets, more slowly, while DFG without its sum blocks (`nosums`, Section 5.1) needs +0.91 more in mean $\log_2 n^*$ on the ablation suites and no longer learns band11, h6_01 and

f7_08. A function of a k-input signed sum spreads its Fourier weight over many characters of several degrees, each too weak to pass its own test, while DFG tests the whole block with one χ^2 statistic; the residual rounds let weaker features pass once the stronger ones are fit. On small formulas the comparison runs the other way: at $n \leq 128$, `lowdeg`'s slightly more lenient test, applied to every degree at once on the labels, admits characters at sizes where DFG still keeps few or none.

4.5 Input dimension

DFG and the networks `nets` (square loss, the only variant run at every d) were run on 7 `designed` targets at $d = 15$ and 60 and on 4 of them at $d = 120$, with seeds 0 and 1, and at $d = 30$ with seeds 0, 1 and 2 (Sections 3.1 and 3.3). The networks were trained for 20000 steps, 40000 at $d = 120$, so one network run, and with it DFG's budget, is twice as large there. Figure 3 plots n^* with the fitted exponents, and Appendix E.5 tabulates them.

Results [Numerical].

- *Exponents.* The least-squares slope of $\log n^*$ on $\log d$ is 0.01 (stair5) to 0.50 (par4) for DFG v1.3 alone, and 0.91 (maj9) to 2.40 (par4) for the networks, fitted per target on the same d.
- *The gap widens with d.* The median $\log_2$ ratio of DFG v1.3 to the networks is -2.64 at $d = 15$ (6.2× less data), -4.19 at $d = 30$ (18.3×), -4.99 at $d = 60$ (31.7×) and -5.41 at $d = 120$ (42.5×, 4 targets).
- *Parities.* DFG's n^* follows the union-bound line (Figure 3). n_{thr} grows from 19.5 to 27.9 for par2 and from 22.4 to 35.2 for par3 between $d = 15$ and 120, and from 24.6 to 36.3 for par4 between $d = 15$ and 60. On the grid, DFG's n^* moves by at most one grid step over the range.
- *Compute.* DFG's largest fit on these targets uses 0.07, 0.15, 0.18 and 0.18 of one network run at $d = 15$, 30, 60 and 120.
- *The price* [Proved, by the code; Numerical]. The enumeration is paid for in FLOPs: an exhaustive degree-k search costs of order $n\binom{d}{k}$ FLOPs (Proposition 2.6). Within DFG's fixed budget, at $n > 512$, the global search can therefore pass no degree-4 character from $d = 179$ on and no degree-3 character from $d = 630$ on (Proposition 2.3; Appendix C.2 and Section 7.2).

Reading [Heuristic].

- *DFG against gradient descent.* DFG's data cost on a k-parity is the union bound's, $2\ln(4K\binom{d}{k}/\delta)$, which grows like $2k \ln d$. The networks' exponents grow with the parity's degree, 1.03, 1.96 and 2.40 for $k = 2$, 3 and 4 (par4 fitted on $d = 15$–60), as analyses of gradient training on sparse parities lead one to expect [1, 3]: exhaustive enumeration pays a logarithm of the dictionary's size, gradient descent a power of d.
- *The size of the factors.* The comparison is with `nets`, the plain square-loss network and the weakest variant at $d = 30$: on `designed`, DFG is at -2.82 against `nets` and -2.17 against `nets_wd` (Table 5). The factors are therefore larger than they would be against the best variant. The exponents should be less sensitive to the choice of variant than the factors, but only `nets` was run at $d \neq 30$.

4.6 Tabular data and compute

Tabular data [Numerical]. On kr_vs_kp and mushroom, the two tabular datasets it can run on (9 cells), DFG is level with gradient boosting in all 9 cells, and with the regularized MLPs and v3

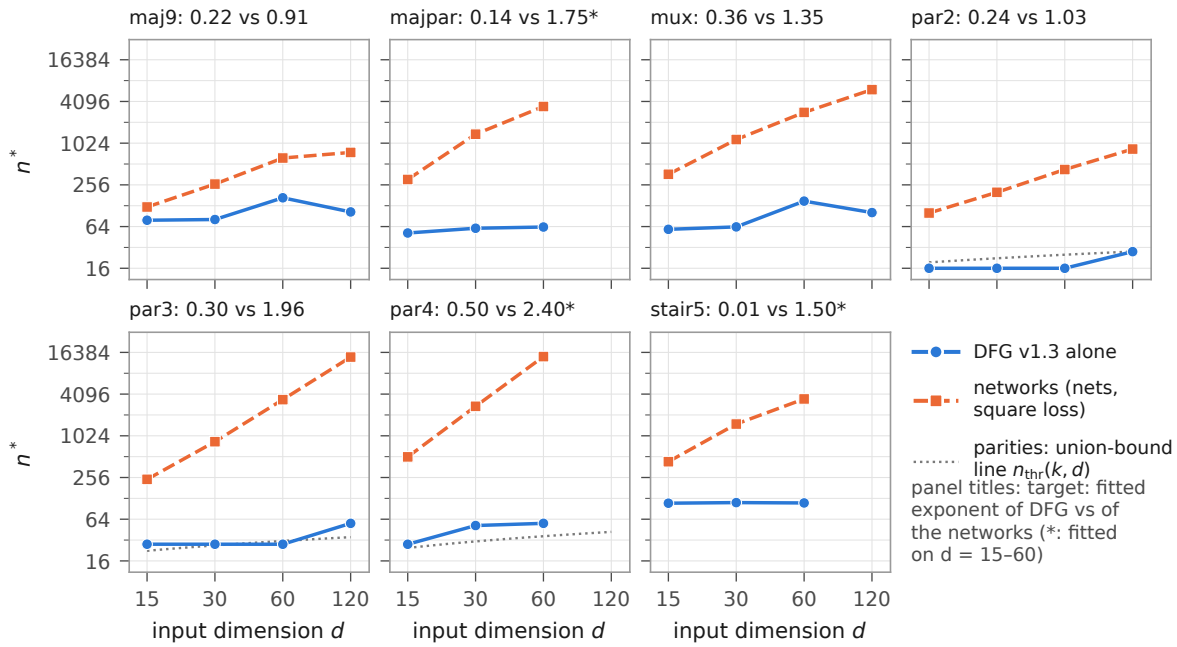


Figure 3: n^* against d (log–log) on the `designed` targets with network runs at $d = 15$, 60 and 120 (Table 23) [Numerical]. Panel titles give the fitted exponent of DFG v1.3 alone and of the networks `nets` (*: fitted on $d = 15$–60). Dotted, for the k-parities: the union-bound line $n_{\text{thr}}(k, d) = 2\ln(4K\binom{d}{k}/\delta)$ with DFG's $K = 4$, $\delta = 0.1$ and $c = 1$; on the n-grid DFG's test passes, when $\bar{y}$ is near 0, at the first grid point above it (Proposition 2.2; where this simple rule misses, and why, is in Appendix C.2). Seeds: 2 at $d \neq 30$, 3 at $d = 30$; the $d = 120$ networks were trained for 40000 steps. Source: `study/part11/RESULTS.json` (`q4_dscale`) via `paper_dfg/make_figures.py`; the plotted values and n_{thr} are in `paper_dfg/figures/fig_dscale.json`.

except at the smallest n of kr_vs_kp (14.06% test error against 7.80% for `mlp_reg` and 6.88% for v3), where they are clearly better; it is nowhere clearly better than the best MLP. Nine cells, with errors near 0 from $n = 1000$ on, resolve little. No fit uses more than 0.144 of the standard-task budget. Real-valued data, where Part 7's adaptations of DFG v1.1 were behind a regularized MLP, are not covered (Appendix E.6; Section 7.3).

Compute [Numerical]. Over all 4730 fits (4334 at $d = 30$, 396 at $d \neq 30$), the largest uses 0.519 of one network run (h5s5 on `dlheld5` at $n = 16384$), and none exceeds 0.75. The median fit uses less than 0.001 of a run at $n \leq 32$, 0.025–0.055 at $n = 64$–512 and 0.149 at $n = 16384$. CPU time is largest at $n = 64$–512 (median 4.2–7.3 seconds per fit), where the cross-validation fits $2F$ small models, each with its own block search. On rough measurements of the same fits DFG alone is faster than v3 (median per-fit CPU ratio 0.06–0.92 at every n). Each network fit spends one run by definition, and `nets_best` is a hindsight choice among 2 to 5 such runs per target, so "no more FLOPs" holds with a wide margin (Appendix E.7).

5 What its parts do

DFG v1.3 has many parts (Section 2, Table 11). This section asks which of them carry its results. It reports the protocol's 12 ablations (Q5), each DFG alone with one setting changed (one or two constructor arguments), and then a post-hoc *core* DFG with nine parts removed at once (Section 5.3).

Every run uses the frozen file; only constructor arguments change. Appendix F gives the per-family tables, the refits behind every mechanism named here, and the sources.

How the runs are compared. An ablation runs on DFG alone's training sets, column permutations and cross-validation folds, and its n^* is compared with DFG alone's n^* on the same seeds. Six ablations (and the core) ran on seeds 0 and 1, six on seed 0 only. The ratio is $\log_2(n^*_{\text{run}}/n^*_{\text{DFG alone}})$, negative when the run needs less data. A target moves *lower* or *higher* when the ratio is beyond ∓ 0.25; means are over the targets both reach. The five ablation suites hold 65 targets (`designed`, `dlsymdev`, `dlfamdev`, `dlheld6`, `dlheld7`); only `dlheld7` is held out. Targets are grouped, by hand and post hoc, into three classes: 25 character targets, 7 halfspaces, and 33 symmetric and mixture targets (Appendix F.1).

Two nulls. The protocol's null compares DFG alone's seed 1 with its seed 0, different training sets, so it overstates the noise of a paired comparison. The paired null, added post hoc, refits DFG alone on the same training and test sets with another fit stream, so with another column order and other folds and nothing else (`study/part11/interp/ablations/perm_null.py`). The column order is irrelevant to the targets and to every test statistic, but it can matter through tie-breaking, the block search's orders and the folds; the paired null measures how far such an arbitrary perturbation of DFG's path moves n^*, a proxy for the noise part of an ablation's effect, not a derivation of it.

5.1 Twelve single changes

The paired null [Numerical; post hoc]. On seed 0 the re-permuted fits move 1 of the 65 targets beyond ± 0.25 (dlist5, +0.39), with mean |ratio| 0.012; on seeds 0 and 1 they move 3 (andxor −0.69, thrsum −0.66, h6_07 +0.40), mean 0.038, and over all 138 targets 8. The unpaired null moves 23 to 31 of the 65, depending on the pair of seeds. Like for like, one seed against one, it moves 23 where the paired null moves 1, with mean |ratio| 0.339 against 0.012: it overstates the noise of a paired comparison 20 to 30 times (Appendix F.1).

What moves most [Numerical].

- *Sum blocks.* Without them (`nosums`) the mean ratio is +0.91 over the 59 targets both learn, with 28 of them clearly higher and 2 lower, and DFG no longer learns band11, h6_01 and f7_08 (31 clearly higher in all). The cost falls on the symmetric targets and the mixtures. All 9 symmetric targets that DFG alone learns move: 3 to "never", and the other 6 by +2.78 on average. The mixtures move by +1.54 (16 of 21 higher). Character targets lose little (+0.16); the five that move up are kinds of target that the atlas shows encoded in part by blocks before their characters are all detectable (Appendix F.1). The two seeds agree: 27 targets move up on both and none in opposite directions.
- *Tests scaled to the residual* (`labelnoise`, +0.14; 15 higher, 5 lower) and *the small-n machinery* (`nosmalln`, +0.12; 10 higher, none lower) come next, mostly on character targets (Appendices F.4 and F.5).

What moves both ways [Numerical]. The two test levels, the kernel background and the linear and spectral blocks move 10 to 23 targets each, in both directions, with means between −0.10 and +0.07; for `strict` and `lenient` most movements are one grid step on one seed (24 and 22 targets move on one seed only, 5 and 4 on both). Halfspaces gain from `strict` (−0.23), `noblocks` (−0.19) and `labelnoise` (−0.17), three changes that keep features out of the model at small n; only the

run	change	seeds	all (65)	newly never	character targets (25)	halfspaces (7)	symmetric and mixtures (33)
`nosums`	no sum blocks	0, 1	+0.91 (2/31) [59]	band11, h6_01, f7_08	+0.16 (1/5) [25]	+0.05 (0/1) [7]	+1.82 (1/25) [27]
`strict`	stricter tests, $\delta = 10^{-4}$	0, 1	+0.06 (9/14) [62]	–	+0.17 (3/8) [25]	−0.23 (4/1) [7]	+0.04 (2/5) [30]
`lenient`	more lenient tests, $\delta = 0.9$	0, 1	+0.07 (5/13) [62]	–	+0.12 (2/8) [25]	−0.04 (2/1) [7]	+0.05 (1/4) [30]
`noeig`	no eigenvector seeds in the block search	0, 1	+0.03 (0/1) [62]	–	+0.00 (0/0) [25]	+0.00 (0/0) [7]	+0.05 (0/1) [30]
`noblocks`	no linear or spectral block	0, 1	−0.02 (6/4) [62]	–	−0.03 (2/1) [25]	−0.19 (3/0) [7]	+0.02 (1/3) [30]
`nocomp`	no extension or local classes	0, 1	+0.00 (2/2) [62]	–	+0.02 (1/2) [25]	+0.00 (0/0) [7]	−0.02 (1/0) [30]
`noweighted`	no weighted blocks	0	+0.02 (0/1) [62]	–	+0.01 (0/0) [25]	+0.00 (0/0) [7]	+0.03 (0/1) [30]
`nosmalln`	no forced variant, no cross-validation	0	+0.12 (0/10) [62]	–	+0.22 (0/8) [25]	−0.03 (0/0) [7]	+0.07 (0/2) [30]
`nobg`	no kernel background	0	−0.10 (7/3) [62]	–	−0.07 (3/2) [25]	−0.09 (1/0) [7]	−0.12 (3/1) [30]
`lsread`	least-squares readout	0	−0.01 (2/1) [62]	–	−0.01 (2/1) [25]	+0.00 (0/0) [7]	−0.02 (0/0) [30]
`K3`	global search to degree 3	0	+0.02 (2/1) [62]	–	+0.08 (1/1) [25]	−0.05 (1/0) [7]	−0.01 (0/0) [30]
`labelnoise`	tests scaled to the labels' noise	0	+0.14 (5/15) [62]	–	+0.23 (2/9) [25]	−0.17 (2/0) [7]	+0.14 (1/6) [30]
paired null	other column order and folds, same training sets	0	+0.00 (0/1) [62]	–	+0.00 (0/1) [25]	+0.00 (0/0) [7]	−0.00 (0/0) [30]
paired null	other column order and folds, same training sets	0, 1	−0.02 (2/1) [62]	–	−0.02 (1/0) [25]	+0.00 (0/0) [7]	−0.02 (1/1) [30]
unpaired null	seed 1 against seed 0: other training sets	1 vs 0	−0.04 (13/10) [62]	–	−0.02 (5/4) [25]	−0.17 (2/1) [7]	−0.04 (6/5) [30]

Table 6: The 12 ablations of Q5 on the 65 targets of the five ablation suites, against DFG alone on the same seeds and training sets [Numerical]. Each cell gives the mean $\log_2$ ratio of the run's n^* to DFG alone's over the targets both learn; below it, the targets on which the run needs clearly less / clearly more data, and in brackets the number both learn. "Newly never": targets the run never brings to 10% error by $n = 16384$ while DFG alone does. The three targets that DFG alone never learns (fd_10, h6_00, f7_05) are in no mean. Bottom: the paired null (post hoc) and the protocol's unpaired null. Sources: `study/part11/RESULTS.json` (`q5_ablations`) and `study/part11/interp/ablations/ablations.json` via `paper_dfg/make_tables.py`; by family in Table 26, per target in Figure 6.

first gain survives counting f7_10, a threshold of a weighted sum that the classes keep among the mixtures, as a halfspace (-0.17, -0.07 and -0.08).

What can be removed [Numerical, then Heuristic]. Five parts each move about as many of these 65 targets as the paired null does, but single targets move further than any move of the null (at most 0.39 on one seed and 0.69 on two): h6_10 under `noeig` and `noweighted`, and par4 and rand1 under `K3`.

- the eigenvector seeds of the block search (`noeig`): 1 target moves, h6_10 (+1.08);

- the extension and local classes (`nocomp`): 4 move, among them andxor and thrsum, which the paired null also moves by as much;
- the weighted blocks (`noweighted`): 1 moves, h6_10 again (+0.94), the target they were designed after (Table 12);
- the logistic refinement (`lsread`): 3 move, by at most 0.58;
- the search of degrees 4–6 (`K3`): 3 move, par4 (+3.01), rand1 (−1.06, through K in the union bound rather than the degree-4 search) and f7_01 (−0.32; Appendix F.3).

"About as many as the null" is a count against one replicate of the null, with no significance level: the 3 moves of `lsread` or of `K3` on one seed cannot be told from the one-seed null's 1, while the 33 of `nosums` clearly can.

Reading [Heuristic]. On these suites DFG's results come from the sum blocks, with the residual scaling of the tests and the small-n machinery as smaller contributors. As far as counts against one replicate of the null can tell, five parts can each be removed without a change larger than an arbitrary perturbation of the path produces, single targets aside. Two qualifications apply: the suites contain few targets that need those parts (one pure 4-parity, no dense halfspace on more than 13 inputs, no degree-5 parity), and the parts are removed one at a time, so they may substitute for one another. Section 5.3 removes them together.

5.2 How the other parts act

Refits of single cells show how the parts act [Numerical; refits]. The test levels (`strict`, `lenient`) act exactly as predicted on parities, through the linear block and the cross-validation where the forced variant gave the first fit, and barely on sum targets whose block arrives late (Section 6.3 and Appendix F.2). Without the degree-4 search (`K3`), par4's exact folded block is found only by chance, at 3 of 9 grid sizes, because the block search starts from degree-1 and degree-2 statistics, which are zero for such a parity (Appendix F.3). Without the forced variant and its cross-validation (`nosmalln`), DFG loses the forced variant's right first guesses, each one grid step later; the kernel background helps at $n = 16$–32 and costs data from $n = 64$ on, also through the cross-validation (Appendix F.4); and the residual scaling of the tests matters for later rounds and for biased labels (Appendix F.5).

5.3 A core DFG (post hoc)

Everything in this subsection is post hoc. The protocol's addendum (`study/part11/PROTOCOL.md`) was written after every pre-registered result had been read, to ask whether the parts that are removable one at a time can be removed together. The `core` run keeps the global characters of degree at most 3; the unweighted signed-sum blocks, searched from pairs and single inputs, with their modulated, product, split, beam and forced moves; the union-bound tests scaled to the residual; least squares; and the small-n cross-validation. It removes nine parts: the linear and spectral blocks, the extension and local classes, the degree-4 to degree-6 searches, the weighted blocks, the eigenvector seeds, the logistic refinement and the kernel background. Through $K = 3$, every test it keeps is also 0.29 nats more lenient. It is the frozen file with nine constructor arguments changed (Appendix F.6), run on all 11 suites with seeds 0 and 1. Its definition used the ablations' results, `dlheld7`'s included, so no suite is held out for it.

Against DFG alone [Numerical; post hoc]. Over the 138 targets the core's mean ratio is +0.07 (median −0.01) over the 133 both learn (Table 27 in Appendix F.6); on `dlheld7` (held out for

v1.3, but seen by the core's definition) it is −0.03. It needs clearly less data on 28 targets and clearly more on 26, and it loses par5: the core moves 54 of the 138 targets beyond ±0.25, where the paired null on the same seeds moves 8 (mean |ratio| 0.040; `study/part11/RESULTS.json`, `q5b_core.paired_null`). Most of the core's differences are therefore real, but they cancel on average. By family, the core loses on parities (+2.01 over the 3 it still learns, and par5) and halfspaces (+0.32), and it is level elsewhere (−0.08 to +0.21; Table 28).

Against the networks, and compute [Numerical; post hoc]. The core's mean against `nets_best` is −1.74 over the 122 of 126 targets both learn, clearly better on 111 and clearly worse on 9; DFG alone on the same seeds is at −1.88, clearly better on 117 and worse on 5. On `dlheld7` the core is at −1.30 (9 better, 1 worse) and DFG alone at −1.27 (10 and 1). On the same 3036 fits the core's median FLOP count is 0.094 of DFG alone's (paired), and its largest fit uses 0.150 of one network run, against DFG alone's 0.500 (targets and CPU times in Appendix F.6).

Where the core loses, and what it misses [Numerical; post hoc]. To attribute the core's largest differences, each removed part was put back on its own, on the targets where the core moved most (Table 29; the targets were chosen after reading the core's results). The runs cover 16 of the 26 targets on which the core needs clearly more data. Putting one part back, or for h6_10 and f7_09 one pair of parts, returns n^* to within 0.25 of DFG alone's on 13 of them and part of the way on 2 (thr30, h5t0); no part tried restores h5s2. The other 10 were not tested: h5rf1, h6_03, rf01, mux11, rf19, h6_07, rmaj9, thrsum, fd_00 and rand3 (+0.26 to +0.60; h6_07 is also moved by the paired null). The losses that were tested fall into three groups (refits in Appendix F.6).

- *Parities of degree 4 and 5 with no lower-degree footprint* (par4, rf17, poly4 and par5, which it never learns). Putting the degree-4 to degree-6 searches back restores all four; in the core they are found only when the block search hits the right inputs by chance, as par4 under `K3`.
- *Dense halfspaces* (thr30, thr20, maj15, maj21, maj9, f7_01, thr10). Without the linear block a degree-1 character must pass its own test, which most of them cannot do at $n = 128$–512 on 20 or 30 inputs; putting the linear and spectral blocks back restores all but thr30 to within 0.25, and most of thr30.
- *Mixtures with an inner term, and a few single steps.* h6_10 and f7_09 come back only with the weighted blocks and the eigenvector seeds together, not with either alone; sym8 comes back with the background, through the cross-validation's choice at $n = 64$, and h5t0 in part; h5s2 comes back with none of the parts tried.

Where the core gains [Numerical; post hoc]. The core needs less data than DFG alone on 28 targets. Of its 13 largest gains, 12 were examined, mostly small formulas learned at $n^* = 16$–63; putting the background back, and with it the linear block, removes most of them, and on h3rf0 and dlist8 no part tried removes the gain (Appendix F.6).

Reading [Heuristic]. On these suites, a learner with characters of degree at most 3, unweighted signed-sum blocks, residual-scaled tests, least squares and the small-n cross-validation keeps DFG alone's results against the networks, on `dlheld7` too, which the core's definition saw, at about a tenth of its FLOPs. Its largest losses are what the removed parts are for: the degree-4 to degree-6 searches find parities with no lower-degree footprint, the linear block finds dense halfspaces, and the weighted blocks with the eigenvector seeds find sums with a heavy inner term. The single ablations showed the first only on par4, the one target of that kind in the ablation suites, and could not show the second, because those suites contain no halfspace on more than 13 inputs; a target mix with

more such targets would make the core worse than DFG alone. Conversely, the background and the linear block cost data at small n on formulas, and that is what the core gains. The core is a description of the frozen learner, not a tested design: it was defined after every result had been read.

6 What it learns

This section is descriptive: it says which features DFG's models hold, not why DFG does well; the ablations of Section 5 are the interventions. Every analysed fit is an instrumented rerun equal to the evaluation's fit in test error and, where checked, FLOP count (Appendix G.1), and each analysis was checked by an independent verifier whose corrections are used throughout.

Findings.

1. *Each target is held in the form of its dictionary that is cheapest to detect* (a summary reading of the following): parities and staircases as their characters, symmetric targets as their exact signed block, halfspaces as degree-1 characters and then a block, multiplexers and formulas as their characters after transient block encodings; 8 of the 37 analysed models at $n = 16384$ are exactly the target's Fourier expansion. Mixtures end in forms their definitions do not suggest: two folded blocks for a parity of two majorities, weighted blocks with the partner as inner term for a sum with a partner (Section 6.2).
2. *Where DFG errs, features are missing, not weights*: in 133 of the 148 analysed cells with test error above 0.1, no weights on the learned features fit the target (Section 6.2).
3. *Two regimes.* Round 0 admits a character close to its detection size, in which degree enters only through a logarithm, and on parities the test sets n^* exactly. Where the grid can tell, the evidence points to the search, not the test, as what sets the data cost of sum targets: on the 6 whose n^* lies a doubling or more beyond the size at which their block would pass, a stricter test moves n^* by at most 0.05; on the other 24 the grid cannot tell (Section 6.3).
4. *The residue classes mod 3 fail at the search, not the test*: their block would pass from $n = 128$, but their weight at Fourier degrees 1–2, where the search starts, is $1.6 \cdot 10^{-6}$ to $2.1 \cdot 10^{-4}$ (Section 6.4).
5. *On noise, the character and linear-block tests keep their proved levels; the spectral block and the admissions after the forced variant's guesses are not calibrated.* Most spurious terms are the linear block's irrelevant degree-1 characters, and they fade with n (Section 6.5).
6. *DFG's order follows detectability, the networks' follows construction*: positively rank-correlated inside round 0 on all 11 targets, inverted where a large coefficient sits at high degree (Section 6.6).

The per-target detail is in Appendix G, and where the compute goes in Appendix E.7.

6.1 How the models are read

The atlas. The structure atlas (`study/part11/interp/ATLAS.md`) is this section's reading of 37 targets from 9 of the 11 suites, chosen to cover the main families, at every grid n from 16 to 16384: 407 fits, all on seed 0, so one training set per (target, n).

Terms and representations. A model is read as terms in the original input coordinates: the constant, each character, each spectral feature, each sum block with all its level indicators as one term, and the kernel background. A term's *s.d.* is the standard deviation of its contribution to the model's output over the 4096 test inputs; a group of terms' *share* is that of its summed contribution as a fraction of the output's. A term is a *target term* if it belongs to one of the target's compact representations in DFG's dictionary, *relevant* if it reads only relevant inputs but is not a target term, and *spurious* if it reads an irrelevant one; S-lin marks the irrelevant degree-1 characters that entered with the linear block. A representation is *complete* in a model when all its characters, or its true blocks, are present; "complete" checks the dictionary, not the weights. The *oracle error* measures what the learned features can express whatever DFG's weights: the test error of a lightly regularized logistic fit of f on the model's features, without the background. Block representations are checked only as "f is a function of the block statistics"; their accept sets are not checked. Five representations were added after the first fits had been looked at, and every cell was then reclassified (Appendix G.1 gives the definitions in full).

6.2 What the models contain, family by family

Figure 4 shows every cell of the atlas and Table 7 summarizes each family at its first fit (the first grid n with test error at most 0.1) and at $n = 16384$; Appendix G.2 gives the models target by target.

Characters for juntas and formulas [Numerical]. From the first fit on (par2 at $n = 16$, par4 and par5 at 64), every parity's model is the constant plus the parity's character and nothing else, and staircases are their prefix characters. The first fits of the multiplexers and small formulas mix characters of degree 1–3 with blocks that encode an address, a clause, a term or a rule; the character representations are complete later. At $n = 16384$, 8 of the 37 models are exactly the target's Fourier expansion: par2, par4, par5, stair5, andxor, mux6, mux11 and hrf0. *Reading* [Heuristic]. For parities the dictionary is the target class, and the size at which DFG has the character is set by the union bound (Proposition 2.2). Characters enter in order of how detectable they are (Section 6.3); blocks encode a clause, term or branch before its characters are all detectable, and give way to them as they become detectable.

Blocks for sums, thresholds and mixtures [Numerical]. Symmetric and signed-sum targets are their exact signed block (sym9 its folded block alone at $n = 128$, with 4 features and error 0); what varies across them is not the form but when the search finds it (Sections 6.3 and 6.4). In the four analysed halfspaces the linear block enters at $n = 64$ and makes the degree-1 representation complete, with oracle error at most 0.007, but DFG's own error is 0.17–0.20 until blocks arrive that encode the threshold (one block for maj9 and f7_01, a product block for thrint); thr30, with Gaussian weights on all 30 inputs, is never fitted exactly (0.009 at 16384). A parity of two majorities is first fitted by two folded blocks over the union of their inputs, $|t_A + t_B|$ and $|t_A - t_B|$, which represent the target because $\mathrm{sign}(|t_A + t_B|^2 - |t_A - t_B|^2) = \mathrm{sign}(t_A t_B)$. In the seven signed-sum mixtures the partner's character comes first in 5 of the 6 that have one, and with more data the compact forms give way to weighted blocks with the partner as inner term: at $n = 16384$ only f7_10 still holds its compact form, though all seven have test error 0. *Reading* [Heuristic]. For a threshold function the dictionary is complete almost at once, and the logistic readout of 30 noisy degree-1 coefficients learns the weights slowly where one block does it at once. DFG treats a partner variable as a heavy input of the block rather than as a modulator, because the partner is itself a strong single input or pair.

	first n with test error ≤ 0.1			complete	$n = 16384$		
family (targets)	n	error	what the model holds	first / 16384	error	size	what the model holds
Parities (par2, par4, par5)	16–64	0	χ_S alone	3/3 ; 3/3	0	2	χ_S alone
Staircases (stair5, stair6)	128–256	0	the prefix characters x_1, x_1x_2, … and one more Fourier character	2/2 ; 1/2	0	17–81	Fourier characters: all of stair5's; most of stair6's, and relevant blocks
Majorities of parities; andxor (majpar, majpar5, andxor)	64–256	0.011–0.092	their pair characters	2/3 ; 3/3	0	4–29	their characters; one relevant block in majpar and majpar5
Multiplexers (mux6, mux11)	256–512	0–0.002	characters and blocks on address and data bits	0/2 ; 2/2	0	17–65	exactly the Fourier expansion
DNF, CNF, decision lists, read-once formulas (cnf3, f7_02, dlist8, f7_04, f7_06)	128–256	0–0.049	low-degree characters, blocks that encode a clause, term or rule, irrelevant degree-1 characters	1/5 ; 4/5	0	67–181	characters and a few blocks
Random formulas (rf06, hrf0)	64–128	0–0.045	the dominant input's character (hrf0: and a relevant block)	0/2 ; 1/2	0	32–66	Fourier characters (hrf0: its whole expansion)
Majorities and halfspaces (maj9, thrint, thr30, f7_01)	128–256	0–0.071	degree-1 characters from the linear block; the exact block (maj9, f7_01)	4/4 ; 4/4	0–0.009	43–149	degree-1 characters and a block
Parities of two majorities (parmaj, majxor)	128–256	0–0.0005	two folded blocks $\lvert t_A + t_B\rvert$, $\lvert t_A - t_B\rvert$	2/2 ; 2/2	0	19–137	the cross-pair characters; majxor also both folded blocks
Majority of majorities (majmaj5)	512	0.023	4-input pieces of a majority, modulated by its fifth input	0/1 ; 0/1	0	256	blocks and characters of odd degree 1–7: 256 features, DFG's cap
Symmetric and signed sums (sym9, band11, f7_08, h5s7)	128–2048	0–0.012	the exact signed block (h5s7: two factors that partition its class)	4/4 ; 4/4	0–0.0005	60–255	the block, with relatives (sym9: and a spurious product)
Signed-sum mixtures (f7_07, f7_09, symand, f7_10, f7_00, f7_03, f7_11)	128–512	0–0.016	the partner's character, and the compact form or a relative of it	4/7 ; 1/7	0	27–115	weighted blocks with the partner as inner term, and characters
Residue classes mod 3 (f7_05, h6_00)	never	–	never reached	0/2 ; 0/2	0.3245–0.3430	1	the constant and the kernel background, at every n

Table 7: What DFG alone's model holds, family by family, seed 0 [Numerical]. "First n": the first grid n at which the target's test error is at most 0.1; n and the error are ranges over the family's targets. "Complete": the number of targets whose model holds some compact representation of the target completely, at that first n and at $n = 16384$. "Size": the number of features at $n = 16384$, the constant included. The descriptions summarize `study/part11/interp/ATLAS.md`, §1; the numbers are computed by `paper_dfg/make_interp.py` from `study/part11/interp/tables/atlas_family.json` and `atlas_cells.csv`. Table 30 has every target.

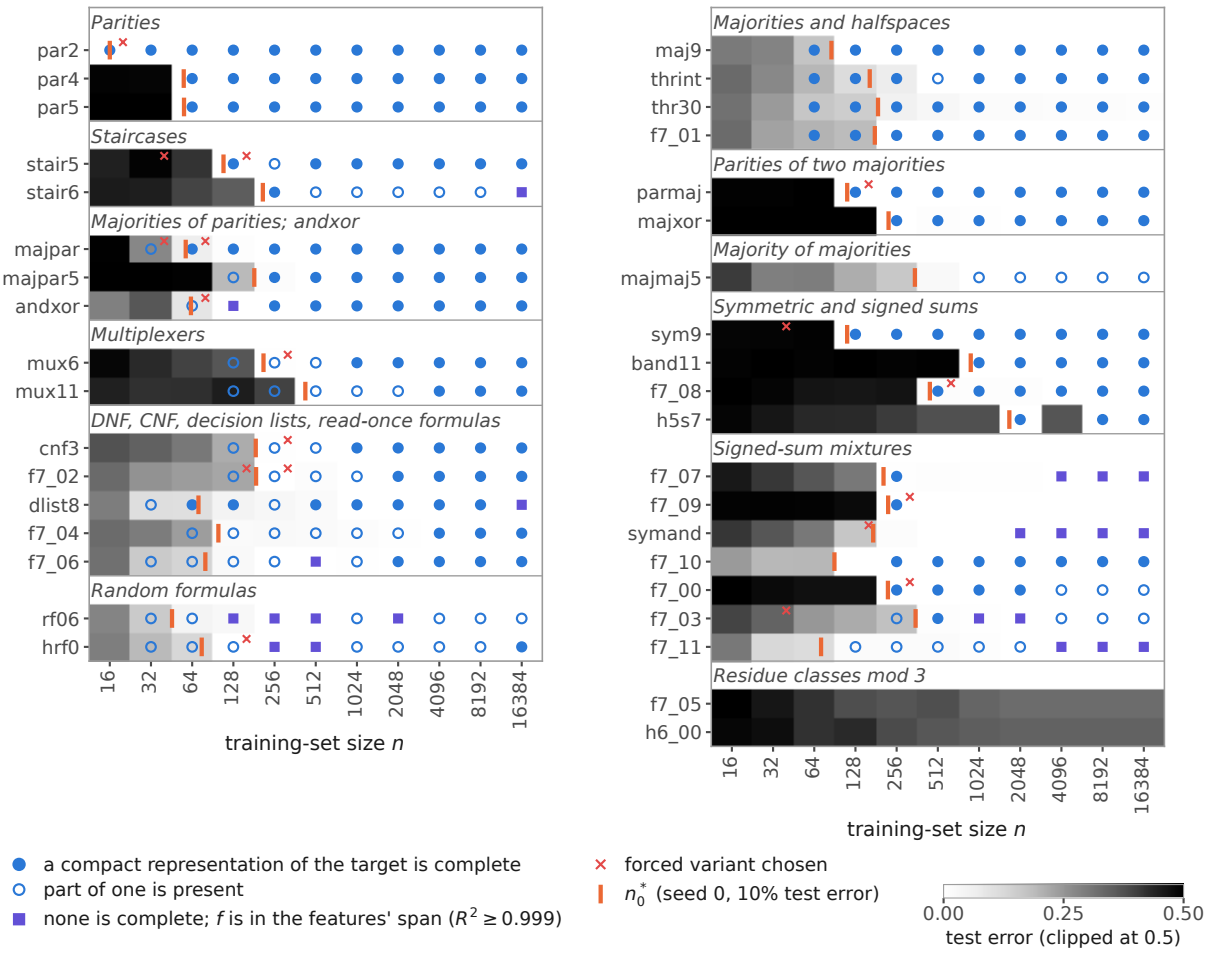


Figure 4: The structure atlas: DFG alone on 37 targets, seed 0, every grid n [Numerical]. Grey: test error (white is 0). Markers: a compact representation of the target is complete in the model (filled dot; on a grey cell, the weights are not yet right), part of one is present (open dot), or none is complete but f lies in the span of the model's features (square; held-out $R^2 \geq 0.999$). Cross: the cross-validation chose the forced variant. Bar: n_0^*, the seed-0 10% test-error point. Families as in Table 7; suites in Table 30. Re-plotted by `paper_dfg/make_interp.py` from `study/part11/interp/tables/atlas_cells.csv` and `atlas_targets.csv`; the plotted values are in `paper_dfg/tables/interp_numbers.json`.

Dictionary or readout [Numerical]. Of the 148 cells with test error above 0.1, the learned features cannot express the target (oracle error above 0.1) in 133. In 9 they can (oracle error at most 0.02), and only DFG's own weights fall short, all just after the linear block entered; the other 6 lie in between. The oracle leaves out the kernel background, so "cannot express" concerns the learned features only: in 27 of the 133 cells DFG's own error is more than 0.05 below the oracle's, because the background carries the signal there. At the first fit, DFG's error is within 0.0005 of the oracle's for the median target; the largest readout gap is thrint's at 256, 0.071, on a complete degree-1 dictionary. *Reading* [Heuristic]. Outside threshold functions, DFG's data cost is the cost of getting the right features into the model; once they are in, its readout is close to the best weights for them.

Spurious terms, and v3's member [Numerical]. At $n = 16384$ the spurious share is at most 0.1 for 36 of the 37 targets, and removing the spurious terms leaves every test error unchanged (Appendix G.3). Part 10's analysis of the same code as v3's member [12] carries over: over the 255

seed-0 cells the two analyses share, the DFG test error is identical in 141, and most differences at small n come from the stacker (Appendix G.4).

6.3 Two regimes: when the structure appears, and when its test would admit it

Apart from the forced variant's guesses, each piece of a target's structure has two sizes: the training-set size from which DFG's test would admit it if the search offered it, and the size from which the model holds it. For characters the search offers every candidate of a class; for sum blocks it offers only blocks grown from statistics of one or two inputs (Proposition 2.5), so the two sizes can be far apart. This subsection measures both and then moves the tests' threshold (detail in Appendix G.7).

Characters: admitted by detectability [Proved, then Numerical]. In round 0 a character χ_S of degree k searched on all rows passes its test exactly when $n > 2c^2 L_k\, s^2/v_S^2$ (Proposition 2.4); replacing v_S by the target's coefficient $\hat{f}(S)$ and s^2 by $1-\hat{f}(\emptyset)^2$ gives the *detection size* $n_{\text{glob}}(S)$ of Equation (7), a first-order prediction [Heuristic]. Degree enters only through the logarithm: with $K=4$, $\delta=0.1$ and $c=1$, L_k is 8.5, 11.2, 13.4 and 15.3 nats for $k=1$–4 at $d=30$, so a coefficient $\sqrt{15.3/8.5}=1.34$ times larger at degree 4 than at degree 1 already has the smaller n_{glob}. On the 41 targets of `designed` and `random`, DFG's round-0 decision agrees with "admit when $n > n_{\text{glob}}$" in 8409 of 8681 (character, n) pairs (96.9%; Figure 5a), and the 249 characters that first enter through this test do so at a median $2^{0.26}\, n_{\text{glob}}$. The classes of later rounds admit characters a median 1.4 to 3.7 doublings below n_{glob}, mainly because the residual there is 3.2 to 19 times smaller (class

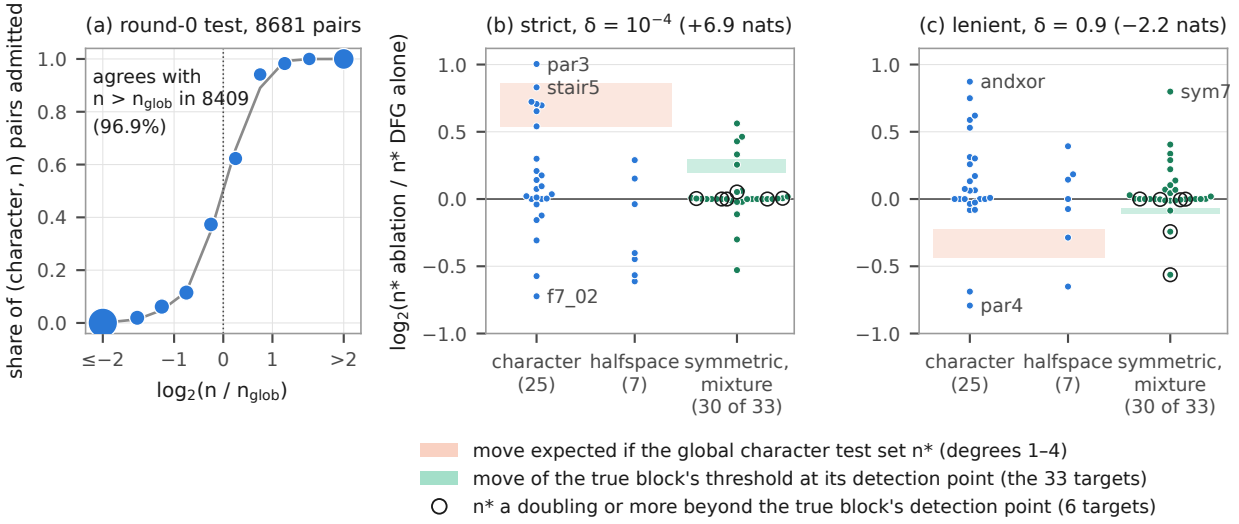


Figure 5: The test and the data cost. (a) DFG's round-0 decisions on the target characters of degree ≤ 4 against the detection size n_{glob} (41 `designed` and `random` targets, seed 0, every grid size; 8681 (character, n) pairs) [Numerical]: the share admitted in each band of $\log_2(n/n_{\text{glob}})$ (area $\propto \sqrt{\text{pairs}}$; end bands open), and a normal approximation for v_S (line). (b, c) `strict` and `lenient`: $\log_2(n^*_{\text{ablation}}/n^*_{\text{DFG alone}})$ per target (seeds 0 and 1, paired), by the ablation analysis's class; the three residue classes, which neither run learns, are not drawn [Numerical]. Orange: the move of n_{glob} for one character of degree 1–4, the move of n^* if a round-0 character test set it [Heuristic]. Green: the range of the true block's threshold move at its detection point over the 33 symmetric and mixture targets [Numerical]. Rings: the 6 targets whose n^* lies a doubling or more beyond that point (Appendix G.7). Sources: `study/part11/interp/theory/check_theory.json` (`P3.test0`), `study/part11/RESULTS.json` (`q5_ablations`), `study/part11/interp/ablations/ablations.json` and `study/part10/interp/tables/toy_timing.csv`, via `paper_dfg/make_interp.py`.

medians; Table 33a): once the large terms are fitted, characters on the inputs already in use pass the tests of small classes, which resembles the junta completion that Part 5 saw in the networks [11] [Heuristic]. For a parity no search stands between the test and the model: Proposition 2.2's criterion holds in all 250 parity cells with $n \geq 32$.

Sum blocks: detection is cheap, discovery lags [Numerical]. Part 10 measured both sizes for v3's DFG member on the 54 sum-structured targets [12]: n_{detect}, the first grid size at which DFG's block test would admit the target's *true* block if it were proposed in round 0 (an oracle), and n_{struct}, the first at which the model holds every signed-symmetry class of size ≥ 3 exactly. Over the 49 found, with the detection points that Part 10's verifier corrected for the gate and multiplexer mixtures, n_{struct} is earlier than n_{detect} for 6, equal for 20, one doubling later for 9 and two or more doublings later for 14, and the delay grows as the target's share of variance at Fourier degrees 1–2 shrinks (Spearman -0.54 over all 54). Part 10 labelled its reading, that the search sets the data cost, Heuristic: nothing isolated the search, and the stacker's choice between DFG's two variants could also have contributed. On the 13 sum-structured targets of the atlas DFG alone's n_{struct} equals that of v3's member (Table 33b): the delay is 0 for 4, one doubling for 3, and two to six doublings for f7_08, band11 and h5s7, and three never reach it. When the block enters, its statistic is well above the threshold (in Part 10's runs, a median 2.3 times at n_{struct} over the 22 pure symmetric targets found, range 0.89–74).

Moving the threshold [Numerical, then Heuristic]. `strict` ($\delta = 10^{-4}$) adds 6.9 nats to the log term of every test, and `lenient` ($\delta = 0.9$) subtracts 2.2 (Section 5; `study/part11/interp/ablations/ABLATIONS.md`, post hoc, corrected after an independent verification). If a round-0 character test set n^*, `strict` would raise it by $+0.54$ (degree 4) to $+0.86$ doublings (degree 1) [Heuristic]. The block test's log term is larger (33.7 nats for an unmodulated 11-input block), so at the true blocks' detection points `strict` moves its threshold by only $+0.20$ to $+0.30$ doublings of n (re-thresholded post hoc; approximate for the modulated blocks, whose statistic was stored only as a ratio to its threshold). Figure 5b,c shows what happened.

- *Parities: the test sets the cost exactly.* Proposition 2.2 with δ moved, applied to each training set's own $\bar{y}$, predicts the measured changes to within 0.03 (par3 $+0.98$ against $+1.00$ under `strict`; par4 -0.79 against -0.79 under `lenient`).
- *Other character targets: one grid step where the test binds.* `strict` raises n^* by more than 0.25 on 8 of the 25, mostly by about the predicted amount. `lenient` is not the mirror image (2 lower, 8 higher): those targets are first fitted at small n, mostly by the forced variant, and with $\delta = 0.9$ the linear block enters and the cross-validation's choice of variant changes (Appendix F.2).
- *Halfspaces: a stricter test helps* (-0.23; -0.17 counting f7_10): a degree-1 character that passes first takes part of the block's signal (Appendix G.7).
- *Symmetric and mixture targets: n^* stays* (class means $+0.04$ and $+0.05$; 30 of the 33 learned). On the 6 targets whose n^* lies a doubling or more beyond n_{detect} (band11, f7_08, h6_01, h6_02, h6_06, h6_08; ringed in Figure 5), `strict` changes n^* by at most 0.05, although it moves their blocks' threshold by about a quarter of a doubling. These use the corrected detection points; with the uncorrected points, which the ablation analysis uses, there are 7 (f7_03 as well, on which `strict` changes n^* by 0.00). On the other 24, n^* lies less than a doubling beyond n_{detect}, and for 20 of them at or below it (23 and 18 with the uncorrected points), so the grid cannot separate the test from the search.

Reading [Heuristic]. Where n^* lies a doubling or more beyond n_{detect}, a test about a quarter of a doubling stricter changes nothing, because once the search proposes the block, the test admits it well above its threshold: *there the search, not the test, sets the data cost*; on the other sum targets the grid cannot tell the two apart. Where the first fit comes from a character test, the test binds and n^* moves with the threshold: exactly for the parities, within a grid step for the staircases and small formulas. Where the first fit comes from the forced variant, δ acts through the linear block and the cross-validation instead, and the sign of its effect does not follow from the test. Part 10's reading rested on correlations; the ablations add an intervention on the test. They still do not isolate the search, for example by seeding it with the true block, and they say nothing about the residue classes, which neither run learns.

6.4 Where it fails, and why

The residue classes mod 3: detectable, but out of the search's reach. f7_05, h6_00 and fd_10, residue classes mod 3 of signed counts of 14, 13 and 10 inputs (Table 8), are three of the four sum-structured targets that DFG alone never learns, and v3 does not learn them either: DFG's model is the constant plus the kernel background at every n.

- *The test would accept the block.* One block, with one group per value of its signed sum, represents each target with at most $k+1$ parameters, where the target's Fourier expansion has all $2^k - 1$ non-constant characters [Proved]. DFG's grouping merges the rarest values, which costs 0.1–0.2% error, and Part 10 found the true block passing DFG's test from $n = 128$ on the same training rows, at 134–155 times the threshold at $n = 16384$ [Numerical].
- *The search's seeds see nothing.* The targets' Fourier weight at degrees 1 and 2 is exactly $1.6 \cdot 10^{-6}$, $5.4 \cdot 10^{-6}$ and $2.1 \cdot 10^{-4}$, and every starting point of the block search is computed from statistics of one or two inputs, whose expectations in round 0 are the target's degree-1 and degree-2 coefficients [Proved, Proposition 2.5(f)]: here 2^{-13}, 2^{-12} and 2^{-9} in absolute value [Numerical, exact]. By the BBP threshold for a rank-one perturbation of a random matrix [2, 5], the top eigenvector of the residual pair matrix would point at the block only from $n \approx 1.1 \cdot 10^7$, $3.1 \cdot 10^6$ and $8.6 \cdot 10^4$ [Heuristic].
- *Exhaustive search does not help within the budget.* Part 10 counted $1.5 \cdot 10^{10}$ to $1.2 \cdot 10^{12}$ blocks of the true size, each costing at least n operations to score, against $1.62 \cdot 10^{11}$ FLOPs per fit [12] [Numerical].

Reading [Heuristic]. A residue class mod 3 is detectable once proposed and invisible from every foothold the search has; more data within any practical range would not help the seeds. Only the most regularized network, `nets_cewd1.0`, learns h6_00 ($n^* = 12304$), the one sum-structured target on which a network beats DFG alone; how it gets there was not analysed.

h5s1, and arbitrary accept sets [Numerical, then Heuristic]. h5s1, an arbitrary accept set of a signed 9-input sum times the parity $x_{10}x_{11}$, is learned by no learner. Its modulated block would pass DFG's test from $n = 128$, but DFG builds a modulated block only on a character already in the model, and the pair's coefficient is exactly $-1/32$, so its round-0 test admits it in expectation only from $n_{\text{glob}} = 2.3 \cdot 10^4$, above the grid [Heuristic]. The six signed-sum targets that DFG learns last are all pure arbitrary accept sets (n^* from 597 to 6186, against a median of 186 over the 50 sum-structured targets it learns), whose share of variance at Fourier degrees 1–2 is 0.008–0.081, against a median of 0.34 over the 54 (Appendix G.6, with h5s7's dependence on the training set).

					true block: test		search's seeds		DFG alone
target	definition	k	$W_1 + W_2$	peak	passes from	over threshold at 16384	signal/noise at 16384	eigenvector needs n	error at 16384
f7_05	$[B \bmod 3 \neq 0]$	14	$1.6 \cdot 10^{-6}$	11	128	134	0.017	$1.1 \cdot 10^{7}$	0.3245
h6_00	$[B \bmod 3 \neq 1]$	13	$5.4 \cdot 10^{-6}$	9	128	139	0.033	$3.1 \cdot 10^{6}$	0.3430
fd_10	$[B \bmod 3 = 0]$	10	$2.1 \cdot 10^{-4}$	7	128	155	0.265	$8.6 \cdot 10^{4}$	0.3240

Table 8: The three residue classes mod 3 [Numerical, Heuristic]. B: the block's k signed inputs that equal $+1$. $W_1 + W_2$: the exact Fourier weight at degrees 1 and 2 [Numerical: exact rationals from the formula of Proposition 2.5(d), which is Proved]; "peak": the degree with the most weight. True block: the first grid n at which its test passes on the seed-0 rows, and its statistic over the threshold at $n = 16384$ (Part 10, same code) [Numerical]. Search's seeds: the signal-to-noise ratio of one pair-matrix entry inside the block at $n = 16384$, and the n from which the top eigenvector would point at the block [Heuristic]. DFG alone: the test error at $n = 16384$, equal in its three seeds to the minority rate. Sources: `study/part11/interp/theory/check_theory.json` (`P4.targets`), `study/part10/interp/results/toy_final/power/` and the Part 11 result files, via `paper_dfg/make_interp.py`.

6.5 False admissions

How often does DFG admit a feature when there is nothing to find? The analysis fits DFG alone on labels drawn independently of the inputs, balanced ($P(y{=}1) = 0.5$) or biased (0.8), at $d = 30$ and 60 and every n, and computes each test's null pass rate (`study/part11/interp/calib/CALIB.md`, with the corrections of `study/part11/interp/calib/VERIFY_CALIB.md`; Table 32 in Appendix G.5).

- *The character tests sit about ten times inside their bound* [Proved, then Numerical]. Proposition 2.1 bounds a false character of degree k in round 0 by $\delta/(2K) = 0.0125$, for any d and n. In the final fits on noise the global classes passed in 2 of 2400 class searches at $d = 30$ and 1 of 840 at $d = 60$, and the null rate is about 0.1% per test. The bound exceeds the independent-Gaussian rate by 10.9–14.3 at $d = 30$, close to the Mills-ratio factor, so Hoeffding's inequality is what is loose [Heuristic]. On balanced labels none of the 220 final models at $d = 30$ holds a feature that passed a test (95% upper limit 0.017), and 5 of 110 at $d = 60$ do.
- *The linear block* passes at about 1%, a tenth of its nominal $\delta = 0.1$, but each false pass adds all d inputs (test error 0.336 and 0.283 against 0.195 for a constant prediction, in the two cases on biased labels at $d = 30$).
- *The spectral block has no proved level.* On balanced labels it passed in at most 3 of 20 000 null draws, but on biased labels at $n = 4d$, the smallest n at which it runs, in 5.4% at $d = 30$, 47% at $d = 60$ and 99% at $d = 120$ ($P(y{=}1) = 0.8$), and 39% at $d = 30$ with $P(y{=}1) = 0.9$.
- *The sum blocks have no proved level either.* They never passed in the 660 final fits on noise, but they did in round 0 of 2 fold fits at $d = 30$ with $P(y{=}1) = 0.8$, and in 1 of THEORY.md's 30 fits at $P(y{=}1) = 0.85$.
- *The forced variant is not calibrated after its guesses*: after two forced characters the local class can admit their product, as in 229 of 900 forced fold fits on biased noise at $d = 30$, but the paired cross-validation chose the forced variant in none of the 180 biased final fits.
- *Spurious terms* on the `designed` and `random` targets are mostly the linear block's (961 of the 1047 terms that read an irrelevant input), and from $n = 128$ on removing them changes the mean test error by at most 0.0002; on noise the kernel background, not the features, costs error, up to 0.144 (Appendices G.3 and G.5).

6.6 The order of acquisition, against the Part 5 networks

Part 5 traced the order in which small ReLU networks acquire the Fourier characters of 11 `designed` targets at $d = 30$ [11]: a linear stage first, bundles of characters carried by one new hidden-unit direction, an order that follows how the target is built (low degree first, products of learned features after their factors) more than coefficient size, and later stages that stop partway at finite n. The order analysis compares DFG alone's order with the networks' on the same targets and training sets, in 363 fits (`study/part11/interp/order/ORDER.md`, corrected after an independent verification). A character's *learned fraction* φ_S is the model's coefficient on χ_S over $\hat{f}(S)$. The networks' *time order* ranks characters by the training step at which the seed-mean φ_S first reaches $1/2$ and stays above 0.4, in Part 5's online runs (trained on a fresh sample at every step), and their n *order* by the smallest grid size from which $\varphi_S \geq 1/2$ after 20000 steps, with $\varphi_S \geq 0.4$ at every larger n. DFG has a *round order*, a *round-0 priority* (the order of round-0 z-scores) and an n *order* by the networks' rule; only the two n orders are like for like (Appendix G.8).

What agrees [Numerical]. At Part 5's sample sizes, $n = 512$, 2048 and 16384, DFG admits every target character it will ever carry in a single round in 24, 30 and 32 of the 33 fits, so its round axis is empty where the networks' time axis is richest; at small n its round order agrees moderately with the networks' time order (mean Kendall τ 0.44 at $n = 64$ and 128). Inside round 0 the priority is positively rank-correlated with the networks' order on all 11 targets in each of four conditions (at $n = 16384$ from 0.22 to 0.81, median 0.54). On the n axis the agreement is moderate (median τ 0.40 against both of the networks' orders), while the networks' two orders agree with each other at median 0.75.

The networks follow construction, DFG follows detectability [Numerical, then Heuristic]. On the 8 targets whose coefficient sizes differ, the networks' time order follows degree more than coefficient size on 7, and DFG's n order follows $1/|\hat{f}(S)|$ more than degree on 5. Mux is the clearest case: its largest term $x_1x_2x_3x_4$ ($\hat{f}(S) = 0.75$, degree 4) is the networks' last character (online at 452 steps, against 60–66 for the linear terms) and DFG's first by round-0 priority, in all 27 fits with $n \geq 64$; its n_{glob} is 54, against 271 for the linear terms. For the 64 characters of degree ≤ 4 with $|\hat{f}(S)| \geq 0.25$, DFG's median n onset is 4 times smaller than the networks' (median $\log_2$ ratio -2.0). *Reading* [Heuristic]. For DFG a character costs data roughly logarithmically in its degree and quadratically in $1/\hat{f}(S)$; Part 5's model of the networks charges training time as e^2 in the number e of new inputs a move coordinates, and only as $\rho^{-1/2}$ in its share ρ of the residual. The orders therefore agree where a target's large coefficients sit at low degree (sparsepoly, majpar, rmaj9: $\tau = 0.68$–0.71 against the time order), invert where a large coefficient sits at high degree (mux), and disagree where a stage mixes sizes (stair5, dnf3, rand3). *Caveats.* Three seeds, one network architecture and $d = 30$ only; the 11 targets come from a few designed families, so no p-values are quoted; DFG's φ is that of its least-squares function, and the logistic refit can distort single coefficients.

7 Discussion

7.1 What kind of learner DFG turned out to be

An Occam learner over a Boolean dictionary, made greedy by a residual [Heuristic, from Sections 2 and 2.5]. DFG's character test admits a feature when the residual sum of squares it would remove exceeds the code length of naming it in its class, $2c^2 \ln(4KN/\delta)$ in units of the residual's

variance (Section 2.6), so a character passes, to first order, once n exceeds its detection size, in which its degree enters only through the logarithm and its coefficient through its inverse square (Proposition 2.4). The candidates come from the low-degree algorithm's enumeration and from a heuristic search for signed-sum blocks seeded by the residual's degree-1 and degree-2 statistics. The residual makes the loop greedy: each round's tests are calibrated to the noise left by the previous round, so weaker features pass once stronger ones are fitted (Appendix F.5).

Two regimes (Section 6.3). *Test-limited* [Proved, Numerical; Heuristic beyond the parities]: where the structure lies in an enumerated class, the search offers every candidate, and for parities the union bound sets the first fit exactly. *Search-limited* [Numerical, then Heuristic]: where the structure is a signed-sum block, the test would often admit the true block well before the model holds it, and where the grid can tell, the data cost is set by when the block search first proposes the block, the later the less of the target's variance sits at Fourier degrees 1–2; the residue classes mod 3 give the search no foothold at all. Halfspaces sit between the two: the dictionary is complete at $n = 64$, but DFG's weights on it are poor until blocks that encode the threshold enter, and thr30 is never fitted exactly.

Its relation to the networks' greedy feature growth [Heuristic]. DFG kept the loop of the picture it was built from and replaced the search (Section 2.6). What survived is the loop: residual-driven growth, later rounds that make a junta's remaining characters cheaper, and a Hebbian background. What changed is the price of a feature, and with it the order in which features come (Section 7.4).

7.2 Why it needs less data than the networks here

The logarithm of the dictionary against a power of d. For a k-parity DFG's round-0 test passes χ_S exactly when $n(1-\bar{y}^2) > n_{\mathrm{thr}}(k,d)$, which grows like $2k \ln d$ [Proved; Proposition 2.2], and DFG's n^* on the parities moves by at most one grid step over $d = 15$–120 [Numerical; Figure 3], while the networks' n^* grows about like d^{k-1} for $k \le 3$ [Numerical; Table 23], the rate of the leap and correlational-SQ analysis [1]; more generally, SGD learns k-sparse parities in $d^{O(k)}$ steps, near the statistical-query limit [3]. Part 3's model of the networks explains the gap [11]: a move that needs k new inputs and has correlation ρ with the residual is accepted once $n\rho^2 \gtrsim 2.25\, d \max(1, d/18)^{k-2}$, with the two constants fitted to the networks, about d^{k-1}: the noise level of a continuous search over directions, not the best-of-N level of a union bound, which DFG's test is, $n\rho^2 \gtrsim 2\ln(4KN/\delta)$ (Proposition 2.4) [Heuristic]. With a noise-free search, that model's n^* fell to 16–87 on every target, as far from the networks as an Occam learner. In those terms DFG is the networks' greedy loop with the search's sampling noise removed and paid for in enumeration.

Enumeration pays in FLOPs, not samples [Proved, by the code; Numerical]. An exhaustive degree-k search costs about $n\binom{d}{k}$ FLOPs times a constant (Proposition 2.6), while one network run costs FLOPs linear in d, so DFG's budget subsamples the search, and at $n > 512$ the budgeted global search can pass no degree-4 character from $d = 179$ on, and no degree-3 character from $d = 630$ on, because its rows fall below the floor of Proposition 2.3 (Appendix C.2). Beyond that point a degree-4 parity with no lower-degree footprint is found only by the block search's chance hits, as in the `K3` and core runs. So within one network run of compute, DFG's Occam rate on degree-4 structure holds up to d of about 120–180; further out it needs more compute than the networks use, unless a search replaces the enumeration [Heuristic].

Signed sums: the same foothold, rounded and tested [Heuristic]. DFG's block search starts from the residual's degree-1 and degree-2 statistics (Proposition 2.5(f)), the statistics a network's early gradient also sees, whose top eigenvector gives the support and the signs of a block (Section 2.3). DFG needs fewer samples because it rounds a coarse estimate of that direction to an exact signed subset and then tests the whole block under one union bound, while a network has to learn the direction to useful precision by noisy local steps, as in the information-exponent picture of single-index learning [4, 13]. On the five sum-structured `dlheld7` wins DFG needs about 4.3× less data than the best network variant; on the other five, where a no-search baseline also beats them, the networks are slow and DFG's search earns little (Section 4.1) [Numerical].

7.3 Its failures, in summary

No low-degree foothold [Proved, Numerical, Heuristic as marked]. The residue classes mod 3 put almost no Fourier weight at degrees 1–2 [Proved; Proposition 2.5(e)], where every starting point of the block search looks, although one block represents each of them and passes DFG's test from $n = 128$ [Numerical]; the eigenvector seed would point at the block only from $8.6 \cdot 10^4$ to $1.1 \cdot 10^7$ rows [Heuristic; Section 6.4], and an exhaustive search over blocks of their size does not fit in the budget. The limit is shared: no learner in the study learns f7_05, and only the most regularized network learns h6_00. h5s1 is a milder case: its modulator's test passes in expectation only above the grid.

Small n [Numerical, then Heuristic]. DFG's weakest range is $n \leq 128$, before its tests admit features; there v3's other members (trees, nearest neighbours, logistic regression) carry the fit on most of the 42 targets where DFG alone needs clearly more data than v3 (Section 4.3). DFG's own small-n machinery is conservative: its paired cross-validation keeps the forced variant's right guesses less often than a stacker does, and its kernel background costs data on balance on the ablation suites (Appendix F.4). This is the one place where the portfolio around DFG added something on Boolean targets.

Dense real-valued signal [Numerical, then Heuristic]. DFG v1.3 does not run on real-valued data, and Part 7's three adaptations of DFG v1.1 were about level with a plain MLP on held-out tabular data, behind an L2-regularized one, and worse than a plain MLP on MNIST and Fashion-MNIST in 7 of 14 cells each [10] (Appendix E.6). Part 7's reading was that the useful features of dense real-valued signal are not in any small enumerable dictionary, and that union-bound tests stop growth early on many weak, correlated effects. The Boolean results contain a small echo: thr30, a halfspace with Gaussian weights on all 30 inputs, is never fitted exactly (test error 0.009 at $n = 16384$).

Calibration at larger d [Numerical]. The spectral test's null pass rate on biased labels near $n = 4d$ grows with d (Section 6.5), so at larger d, or on strongly biased targets, DFG can be expected to admit more spurious spectral features [Heuristic].

7.4 DFG as a model of gradient descent

What the order comparison supports [Numerical] (Section 6.6). Inside one round the priorities agree, moderately, on all 11 targets in all four conditions, and at small n the round order agrees moderately too; both loops make the rest of a junta cheaper once its first characters are in, and both carry a linear, Hebbian component.

What it does not support [Numerical, then Heuristic]. Not a correspondence of time: at Part 5's sample sizes DFG's rounds collapse into one. Not the same price of a feature: the orders invert where a large coefficient sits at high degree. Not a linear stage: on all 8 targets with linear terms the networks learn every linear term before any other, while in DFG's n order the linear terms come strictly first in only 6 of the 24 target-seed cases (Appendix G.8). And not the same sample complexity, d^{k-1} against $\ln d$ on parities, so DFG is not a model of how much data gradient descent needs; the race model of Parts 2–3, a discrete model with a noise-limited feature search, is such a model, which matched the networks' sample sizes within a factor of about 1.3 on held-out functions [11]. *Reading* [Heuristic]. DFG models the outcome of greedy feature growth, the features a greedy residual loop ends with, at one extreme of the trade between data and compute that networks also make [15]. It does not model the dynamics. The two orders agree where a target's large coefficients sit at low degree, so that detectability and construction coincide, and they come apart where they do not.

7.5 The core DFG as the object to study next

Everything here is post hoc. The core of Section 5.3 is level with DFG on average at a tenth of its FLOPs, though it moves many targets in both directions, and its largest losses are where a removed part has no substitute.

Why it is the better object [Heuristic]. Every test in the core is a union bound over an explicit class, its heuristics are the block search and the small-n forced guess with its cross-validated choice, and its readout is plain least squares, with no logistic refinement, spectral directions or background. The questions it poses are therefore sharper than DFG's:

- *When does the block search find a block?* The BBP argument for the eigenvector seed is heuristic, and the core does not even use that seed: it grows blocks from the strongest pairs and single inputs. A statement of the form "a function of a k-input signed sum with low-degree weight $W_1 + W_2 \geq w$ is proposed from $n \geq f(k, d, w)$" would turn the search-limited regime into a sample bound.
- *Does the block test hold its level?* Its groups are a data-dependent partition of the rows, so the Gaussian calibration of Equation (5) is not proved; on noise it never passed in a final fit, but it did in fold fits on biased labels.
- *What does the forced variant buy?* Its guesses are the first fit of 12 of the 37 atlas targets (Appendix G.1), and the paired test that chooses it is conservative; a calibrated replacement would make the small-n behaviour analysable.

The core was defined after every result had been read, `dlheld7`'s included, so any claim about it beyond a description needs fresh data.

7.6 What would test it further

1. *Fresh suites from other families*, written independently of both the blind family and DFG's dictionary (higher-degree sparse polynomials, intersections of several halfspaces, interacting sums, residues mod q), since the held-out evidence is one draw of 12 targets from one generator whose design was known.
2. *Larger d with the networks*: degree-3 and degree-4 structure at $d = 60$–250, where the budget first subsamples and then skips the degree-4 search.

3. *The core, pre-registered,* on fresh suites that include the kinds of target it is predicted to lose.
4. *Real-valued data with dictionaries derived from the data*: the residual loop and the union-bound test over classes of thresholds, quantile bits or residual-derived directions, with their sizes counted honestly, against gradient boosting, regularized MLPs and Part 7's adaptations.
5. *Theory for the search*: a success probability for the block search as a function of the low-degree weights and n, levels for the block and spectral tests, and a bound on the test error of the fitted model, which none of Section 2.5's statements gives.
6. *Stronger references*: networks tuned per target, more seeds, and CPU time measured under a controlled load.

7.7 Limits

- **The held-out set is small.** `dlheld7` has 12 targets from one family and one draw, and on 5 of DFG's 10 wins there a no-search baseline, logistic regression or default gradient boosting, also beats the networks. Its standard error of 0.27 treats the targets as independent draws from the family.
- **The other suites are in-sample** (Table 12). Pooled numbers summarize the record, not held-out estimates; the margin against `nets_wd` is smallest on the three suites held out for some version of DFG, as selection on the others would produce, though the suites also differ in kind (Section 4.2).
- **The references are untuned.** One architecture, width, learning rate and step count per variant, not tuned per target; `nets_best` is a hindsight minimum over 2 to 5 such variants, which favours the networks; and the d-scaling uses `nets`, the weakest variant at $d = 30$ (Sections 3.2 and 4.5). Stronger networks would plausibly narrow the margins.
- **One input law and one domain.** Uniform ± 1 inputs, mostly at $d = 30$, and two tabular datasets whose 9 cells resolve little. Nothing here is about real-valued data or images.
- **Seeds and noise.** n^* comes from test errors averaged over 2 or 3 training sets, with no confidence interval; the ± 0.25 band is a convention, not a significance test, and is smaller than the spread between single seeds; the ablations use one or two seeds, the core two, and most analyses seed 0 (Section 3.3).
- **Post-hoc analyses.** The core, the paired null, the target families, the leave-one-in runs and the refits were defined after the pre-registered results had been read, and "removable" is a reading of counts against one replicate of the paired null, with no significance level (Section 5); the analyses were checked by independent verifiers, whose corrections are applied (Appendix H).
- **Descriptive interpretability.** Most analyses use seed 0, one training set per target and n. Their readings are labelled Heuristic, and the ablations, one part at a time, are the only interventions (Section 6).
- **Compute.** FLOPs are DFG's own count, prediction is not charged, and CPU times were measured on a shared machine (Appendix E.7).
- **AI-written.** The code, the runs, the analyses and this paper were produced and checked by AI agents (Claude), with independent AI verifiers for the analyses. No human has reviewed any of it (Appendix H).

References

[1] Emmanuel Abbe, Enric Boix-Adserà, and Theodor Misiakiewicz. SGD learning on neural networks: leap complexity and saddle-to-saddle dynamics. In *Conference on Learning Theory (COLT)*, 2023. arXiv:2302.11055.

[2] Jinho Baik, Gérard Ben Arous, and Sandrine Péché. Phase transition of the largest eigenvalue for nonnull complex sample covariance matrices. *Annals of Probability*, 33(5):1643–1697, 2005.

[3] Boaz Barak, Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Hidden progress in deep learning: SGD learns parities near the computational limit. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Gérard Ben Arous, Reza Gheissari, and Aukosh Jagannath. Online stochastic gradient descent on non-convex losses from high-dimensional inference. *Journal of Machine Learning Research*, 22(106):1–51, 2021.

[5] Florent Benaych-Georges and Raj Rao Nadakuditi. The eigenvalues and eigenvectors of finite, low rank perturbations of large random matrices. *Advances in Mathematics*, 227(1):494–521, 2011.

[6] Avrim Blum, Merrick Furst, Jeffrey Jackson, Michael Kearns, Yishay Mansour, and Steven Rudich. Weakly learning DNF and characterizing statistical query learning using Fourier analysis. In *Proceedings of the 26th Annual ACM Symposium on Theory of Computing (STOC)*, pages 253–262, 1994.

[7] Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Occam's razor. *Information Processing Letters*, 24(6):377–380, 1987.

[8] Nader H. Bshouty and Vitaly Feldman. On using extended statistical queries to avoid membership queries. *Journal of Machine Learning Research*, 2:359–395, 2002.

[9] Peter Bühlmann and Bin Yu. Boosting with the L_2 loss: regression and classification. *Journal of the American Statistical Association*, 98(462):324–339, 2003.

[10] Claude (Anthropic). Discrete learners against gradient-descent-trained networks: Boolean targets and standard benchmarks. Prompted by Kaarel Hänni. https://github.com/kaarelh/AI-works/tree/claude/nifty-mccarthy-j0lw2a/gd-circuit-growth/paper_discrete, 2026. Parts 6–9 of the study this paper continues.

[11] Claude (Anthropic). Gradient descent on Boolean targets as noise-limited greedy feature growth: experiments, discrete models, and generalization bounds. Prompted by Kaarel Hänni. https://github.com/kaarelh/AI-works/tree/claude/nifty-mccarthy-j0lw2a/gd-circuit-growth/paper, 2026. Parts 1–5 of the study this paper continues.

[12] Claude (Anthropic). A discrete portfolio learner: what it is, how it compares with gradient-trained networks, and what it learns. Prompted by Kaarel Hänni. https://github.com/kaarelh/AI-works/tree/claude/nifty-mccarthy-j0lw2a/gd-circuit-growth/paper_learner, 2026. Part 10 of the study this paper continues.

[13] Alex Damian, Loucas Pillaud-Vivien, Jason D. Lee, and Joan Bruna. Computational-statistical gaps in Gaussian single-index models. In *Proceedings of the Thirty-Seventh Conference on Learning Theory (COLT)*, 2024.

[14] David L. Donoho and Iain M. Johnstone. Ideal spatial adaptation by wavelet shrinkage. *Biometrika*, 81(3):425–455, 1994.

[15] Benjamin L. Edelman, Surbhi Goel, Sham Kakade, Eran Malach, and Cyril Zhang. Pareto frontiers in neural feature learning: data, compute, width, and luck. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2309.03800.

[16] Dean P. Foster and Edward I. George. The risk inflation criterion for multiple regression. *Annals of Statistics*, 22(4):1947–1975, 1994.

[17] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[18] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006.

[19] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, pages 25–32, 1989.

[20] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[21] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[22] Michael Kearns. Efficient noise-tolerant learning from statistical queries. *Journal of the ACM*, 45(6):983–1006, 1998.

[23] Diederik P. Kingma and Jimmy Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[24] Eyal Kushilevitz and Yishay Mansour. Learning decision trees using the Fourier spectrum. *SIAM Journal on Computing*, 22(6):1331–1348, 1993.

[25] Béatrice Laurent and Pascal Massart. Adaptive estimation of a quadratic functional by model selection. *Annals of Statistics*, 28(5):1302–1338, 2000.

[26] Nathan Linial, Yishay Mansour, and Noam Nisan. Constant depth circuits, Fourier transform, and learnability. *Journal of the ACM*, 40(3):607–620, 1993.

[27] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[28] Stéphane G. Mallat and Zhifeng Zhang. Matching pursuits with time-frequency dictionaries. *IEEE Transactions on Signal Processing*, 41(12):3397–3415, 1993.

[29] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[30] Randal S. Olson, William La Cava, Patryk Orzechowski, Ryan J. Urbanowicz, and Jason H. Moore. PMLB: a large benchmark suite for machine learning evaluation and comparison. *BioData Mining*, 10:36, 2017.

[31] Yagyensh C. Pati, Ramin Rezaiifar, and Perinkulam S. Krishnaprasad. Orthogonal matching pursuit: recursive function approximation with applications to wavelet decomposition. In *Proceedings of the 27th Asilomar Conference on Signals, Systems and Computers*, pages 40–44, 1993.

[32] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[33] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[34] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[35] Greg Yang and Edward J. Hu. Tensor programs IV: feature learning in infinite-width neural networks. In *International Conference on Machine Learning (ICML)*, 2021.

A Names used in this paper

Tables 9 and 10 list the names of learners, parts of DFG, references, sets, targets and metrics that the paper uses. DFG v1.3 is `study/part10/hyb_portfolio3/dfg13.py`, every toy target is defined in `study/boolfuncs.py`, and the blind family in `study/part9/heldout6/FAMILY.md`.

name	meaning
Learners	
DFG; v1, v1.1, v1.2, v1.3	discrete feature growth (Section 2): v1, v1.1 (Part 6), v1.2 (Part 9), v1.3 (Part 10)
DFG alone, `dfg13-perm`	this paper's object: the frozen DFG v1.3 run by itself, inputs permuted per fit (Section 3.4)
v1, v2, v3	the portfolios of Parts 8, 9, 10, which stack DFG v1.1, v1.2, v1.3 with classical learners
`dfg`, `dfg_forced`	DFG's default and forced variants as two members of a portfolio
`p9_sum`, `p10_sum`	the Part 9 and Part 10 toy builders' candidates: v2's and v3's code on toy inputs (Appendix D.2)
`lowdeg`	the thresholded low-degree algorithm: DFG's global characters of degree ≤ 4 and the form of its test, and nothing else (Section 3.2)
ablation tags	DFG alone with one setting changed, one or two constructor arguments (Section 5): `nosums`, `strict`, `lenient`, `noeig`, `noblocks`, `nocomp`, `noweighted`, `nosmalln`, `nobg`, `lsread`, `K3`, `labelnoise`
core DFG, `dfg13-core-perm`	post hoc: DFG alone with nine parts removed by constructor arguments, keeping the global characters of degree ≤ 3, the unweighted signed-sum blocks with their full search, the residual-scaled tests (with $K = 3$, so 0.29 nats more lenient), least squares and the small-n cross-validation (Section 5.3)
leave one in	post hoc: the core with one removed part (or two) put back, on the targets where the core moved most (Table 29)
paired null, `dfg13-reperm`	post hoc: DFG alone refitted on the same training and test sets with another fit stream, so with another column order and other folds and nothing else; what an ablation is compared with (Section 5)
unpaired null	the protocol's null: DFG alone's seed 1 (or 2) against its seed 0, so other training sets
Part 7's adaptations	`dfg_bits`, `dfg_dirs`, `dfg_gates`: DFG adapted to real-valued data (context for Q6)
DFG's search (Section 2)	
character χ_S; degree	$\chi_S(x) = \prod_{i \in S} x_i$; $\lvert S \rvert$
residual r; s	$r = y - \hat{y}$, with $\hat{y}$ the least-squares fit on the current features; $s = \mathrm{rms}(r)$
round	a pass that searches every class on r, admits what passes its test and refits; at most 8
class; union bound N	candidates tested together; their number N enters the threshold as $\ln(4KN/\delta)$; $K = 4$, $\delta = 0.1$
global, extension, local	every character of one degree $\leq K$ on all d inputs (degrees 5–6 on a subsample); $S \cup \{j\}$ and $S \cup \{j, l\}$ for admitted S; every subset of the ≤ 12 inputs in use
linear block	all remaining degree-1 characters at once, admitted by a χ^2 test on their coefficients
spectral feature	$(u \cdot x)^2 - \lvert u \rvert^2$, u an eigenvector of the residual pair matrix whose eigenvalue clears the noise edge
sum block; statistic t	signs s_i on a subset S, $t = \sum_{i \in S} s_i x_i$; its features indicate groups of values of t
folded, modulated block	$\lvert t \rvert$ in place of t; the block's indicators times an admitted character χ_T
product, split block	the joint cells of two disjoint blocks; a mixed-sign block taken as the product of its + and − parts
weighted block	$u = t + a\tau$ in place of t, with $a \in \{\pm 2, \pm 3\}$ and τ a literal, a 2-parity or an AND of two literals
block search	how blocks are proposed: from the residual's degree-1 and pair statistics
forced variant	at $n \leq 512$, takes the best candidate if none passes; chosen by a paired cross-validation test
kernel background	one Hebbian step on the final residual, $\sum_i g_i x_i$ with $g_i = \frac{1}{n} \sum_t r_t x_{t,i}$, added to the output
readout	least squares on the admitted features, refined by 15 logistic Newton steps when $n \geq 8p$

Table 9: Names used in this paper, 1 of 2: learners and DFG's search.

name	meaning
References (Section 3.2)	
`nets`, `nets_wd`	the toy networks with square loss, without and with weight decay 0.1
`nets_cewd`, `nets_cewd0.3`, `nets_cewd1.0`	the toy networks with logistic loss and weight decay 0.1, 0.3, 1.0
`nets_best`	per target, the network variant with the smallest n^* among those run on its suite (hindsight)
no-search baselines: `lin`, `hgb`	cross-validated logistic regression on the inputs; default histogram gradient boosting; neither searches an explicit feature dictionary
`mlp`, `mlp_reg`, `mlp_strong`	the baseline MLP (256-256); the same with an L2 penalty; an ensemble of three regularized MLPs
`mlp_best`	per cell, the best MLP variant run there (hindsight)
Sets and targets (Section 3.1)	
the 11 toy suites	`designed`, `random`, `dlheld`, `dlheld2`–`dlheld7`, `dlsymdev`, `dlfamdev`, at $d = 30$ (Table 12)
Q4 targets	the `designed` targets at $d = 15$, 60 and 120: 7, 7 and 4 of them
the blind family	the random generator of `dlheld6`, `dlfamdev` and `dlheld7`, written blind to the sum-block design
sum-structured target	one of the 54 targets of `dlsymdev`, `dlheld4`–`dlheld7`, `dlfamdev` that is not one of their 14 controls
accept set	the values of a sum on which a target, or a function of the sum in it, is $+1$
ablation suites	`designed`, `dlsymdev`, `dlfamdev`, `dlheld6`, `dlheld7`: the 65 targets of Q5
target classes	post hoc, by hand: character targets, halfspaces, symmetric and mixture targets (Section 5)
kr_vs_kp, mushroom	the two PMLB datasets with only binary or categorical columns and two classes
target names	f7_, h6_, fd_, h5: `dlheld7`, `dlheld6`, `dlfamdev`, `dlheld5`; rf, rand, hrf, h$k$rf: random formulas; others: par$k$ (k-bit parity), maj$k$, thr$k$ (halfspace), stair$k$, mux, dnf, cnf, dlist, sym, ...
statuses	development, exposed, held out for v1.1 only, held out: a set's status for DFG v1.3 (Table 12)
Metrics (Section 3.3)	
n^*	the training-set size at which the seed-mean test error reaches 10%
$\log_2$ ratio	$\log_2(n^*_A/n^*_B)$, negative when A needs less data
clearly better, worse	the ratio is below -0.25 (above $+0.25$), or only A (only B) reaches 10% error
suite mean; pooled	the mean ratio over the targets both learn; per-target ratios pooled over suites
cell; better, tie, worse	a (dataset, n) pair on tabular data; the verdicts of the cell rule
budget	one toy network run in FLOPs (1.62×10^{11} at $d = 30$); for real data, three training runs of `mlp` (Parts 8–10; one in Part 7)
$n_{\text{thr}}(k, d)$	$2\ln(4K\binom{d}{k}/\delta)$: a k-parity passes round 0's test exactly when $n(1-\bar{y}^2) > n_{\text{thr}}$ (Proposition 2.2)
n_{glob}	$2\ln(4K\binom{d}{\lvert S\rvert}/\delta)(1-\hat{f}(\emptyset)^2)/\hat{f}(S)^2$, the n at which χ_S passes round 0's global test in expectation
n_{detect}	the first n at which DFG's test would admit the target's true block if proposed (Section 6)
n_{struct}	the first analysed n at which DFG's model holds the target's exact signed blocks (Section 6)
Other	
Q1–Q7	the questions fixed by the Part 11 protocol (Section 3.4 and Appendix D.3)
DFG, DFG alone	DFG v1.3 run alone; "DFG alone" where the contrast with v3's DFG member matters (Section 3.4)
online	of the Part 5 networks: trained on a fresh sample at every step (Section 6.6)
builder	an agent that designed a component on development data and froze it (Parts 8–10); the Part 10 toy builder built DFG v1.3
Part k	the k-th part of the study: Parts 1–5 in [11], 6–9 in [10], 10 in [12]; 11 is this paper
atlas	Section 6's reading of 37 targets at every grid n, seed 0 (Section 6.1)
term s.d., share	the standard deviation of one term's contribution to a model's output; that of a group's summed contribution, as a fraction of the output's (Section 6.1)

Table 10: Names used in this paper, 2 of 2: references, sets and targets, metrics and other names.

B DFG v1.3 step by step

This appendix gives every step of DFG v1.3 with the line numbers of the frozen file `study/part10/hyb_portfolio3/dfg13.py`, which Section 2 summarizes; the numbered equations are those of Section 2.3. Parameter values are the constructor defaults, as run, and Table 11 lists them with the constants fixed in the code. Tables 2 and 11 are generated from the frozen file itself by `paper_dfg/make_tables.py`, which also checks them against the theory notes `study/part11/interp/theory/THEORY.md`.

B.1 The dictionary

The linear block in the code. In the code the linear block's inputs are ordinary degree-1 characters (line 832). They are written apart in Equation (1) because they entered together, without tests of their own.

Sum blocks. A block B has inputs S_B, signs $s_i \in \{\pm 1\}$, and the integer statistic

$$t_B(x) = \sum_{i \in S_B} s_i x_i + a_B\, \tau_B(x),$$

or its absolute value if the block is *folded*. An unweighted block has $a_B = 0$. A *weighted* block (v1.3) has $a_B \in \{\pm 2, \pm 3\}$ and an inner term τ_B on inputs outside S_B: a literal x_j, a 2-parity $x_j x_l$, or an AND of two literals written as ± 1. The groups $G_{B,v}$ are contiguous runs of the values of t_B that occur in training, each holding at least $\max(3, \lfloor 0.005\, n \rfloor)$ training points. Rare values are merged into their neighbours (`_edges`, lines 86–97). $\mathcal{V}_B$ is every occupied group except the most populous one, whose indicator the constant replaces. A *modulated* block multiplies its indicators by a character χ_{P_B} that is already a feature, on inputs outside S_B; otherwise $\chi_{P_B} = 1$. A *product* block $B_1 \times B_2$ of two blocks with disjoint inputs uses the joint cells $G_{B_1,v_1} \times G_{B_2,v_2}$ of the pair $(t_{B_1}(x), t_{B_2}(x))$. When every value of t_B is its own group, every function of t_B is a combination of the constant and the block's indicators (Proposition 2.5). A function of a signed sum of k inputs then costs at most $k+1$ weights, where its Fourier expansion can need all 2^k characters on S_B.

The entries of Table 2. "Rows tested": a budgeted search runs on the first m rows, with m set by its share of the budget (Appendix B.6). At $d = 30$ that is every row of the grid ($n \le 16384$) except for degrees 5 and 6. Degree 5 sees at most 1374 rows, so it is subsampled from $n = 2048$ on (687 in the final fit at $n \le 512$, every row there); degree 6 sees at most 147 (73 in the final fit at $n \le 512$, so it is subsampled from $n = 128$ on). N is the count the union bound pays for, and $\ln(4KN/\delta)$ the log term of the threshold, with $K = 4$ and $\delta = 0.1$. For the extensions, $b \le 32$ is the number of current characters extended; for the local class, V is the set of inputs of the current characters. "Smallest n that can pass": for characters and blocks, no member can pass its test on fewer rows, for any labels (Proposition 2.3); this is $2c^2 \ln(4KN/\delta)$ for characters and $1 + 2\sqrt{L_B} + 2L_B$, the threshold of Equation (5) at $\nu = 1$, for blocks. For the dense moves it is a condition of the code, not a floor of Proposition 2.3: the linear block's room condition $n \ge 2(p+k)$, which is 62 in round 0 with all 30 inputs and lower when some degree-1 characters are already candidates (42 with 20 inputs), and $n \ge 4d$ for the spectral features. For sum blocks, N_B counts the supports, the sign patterns up to a global sign, the two fold variants, the $1 + \mu$ choices of modulator and the $k_{\max} = 24$ sizes. Every block searched in a round with μ current characters as modulators pays the factor $1 + \mu$, not only the modulated ones; weighted blocks multiply N_B by the factor shown, and a product pays for both factors.

Description length. A fitted model's description length is the sum of its features' names; at $d = 30$, par4's $\chi_{\{1,2,3,4\}}$ costs $\ln \binom{30}{4} = 10.22$ nats to name, and $\ln(4K\binom{30}{4}/\delta) = 15.29$ with the test's constant (Section 2.6).

B.2 One round

Residual and noise level. The features are the columns of $\Phi \in \mathbb{R}^{n \times p}$, the first one constant. The weights minimize $\sum_t (y_t - (\Phi w)_t)^2 + 10^{-3}\, n \sum_{j \geq 2} w_j^2 + 10^{-9}|w|^2$ (line 589). The constant is penalized only by the 10^{-9} term, so the residual $r = y - \Phi w$ has mean 0 up to that term. The noise level is $s = \sqrt{\max(\text{mean}(r^2), 10^{-12})}$.

The character test. In Equation (2), $c = 1$, $\delta = 0.1$ and $K = 4$ in every round (lines 602–603). $K_{\text{later}} = 3$ caps only the global degree after round 0. The standard error of v_S is about $\sigma_m/\sqrt{m}$, so the test asks for $c\sqrt{2L_N}$ standard errors: a Bonferroni threshold over the class's N candidates. The classes are:

- *global*: every χ_S with $|S| = k$, for $k \leq K = 4$ in round 0 and $k \leq K_{\text{later}} = 3$ after; $N = \binom{d}{k}$. Degrees 1 and 2 come from the degree-1 coefficients $c_1 = X^\top r/n$ and the residual pair matrix $M = \frac{1}{n}\sum_t r_t x_t x_t^\top$. Degrees 3 and 4 come from the same matrix with the rows weighted by x_a or $x_a x_b$ (lines 616–648). In round 0 of the final fit, degrees 5 and 6 ($K_{\text{hi}} = 6$) come from $\frac{1}{m}\sum_t r_t \chi_{T_1}(x_t)\chi_{T_2}(x_t)$ over disjoint T_1, T_2 of degrees $\lceil k/2 \rceil$ and $\lfloor k/2 \rfloor$, one product of two character tables (lines 651–663);
- *extension*: for each of the up to 32 current characters χ_T with the largest $|w_T|$, the sets $T \cup \{j\}$ ($N = bd$ for b such bases) and $T \cup \{j, l\}$ ($N = b\binom{d}{2}$), from the degree-1 and pair statistics of $r\chi_T$ (lines 665–681);
- *local*: if the current characters use $2 \leq |V| \leq 12$ inputs, every $S \subseteq V$, from one Walsh–Hadamard transform of r on V; $N = \binom{|V|}{|S|}$ (lines 683–692).

A candidate that passes in several classes keeps its largest z_S. The round admits the up to 64 candidates with the largest z_S, within the room $\min(256, \lfloor n/2 \rfloor) - p$.

Dense moves. The two dense moves are in lines 694–709. In Equation (3), lin is the set of inputs whose character is neither a feature nor a candidate this round, and $k = |\text{lin}|$; p is counted before the round's other additions. The spectral features take the eigenvectors u of $\tilde{M}$, the symmetrized M with zero diagonal and with the entries of pairs that are features or candidates set to 0, whose eigenvalues satisfy Equation (4), at most four of them, and only if $n \geq 4d$. The first factor of Equation (4) is the edge of the semicircle law for a $d \times d$ Wigner matrix with entries of size $s/\sqrt{n}$, about the size of $\tilde{M}$'s entries under noise. Each such u adds the feature $(u \cdot x)^2 - 1$.

The sum-block test. In Equation (5), $k_{\max} = 24$ (line 719). A weighted block adds $\ln(2(4d + 20\binom{d}{2})) = 9.78$ nats at $d = 30$ for its inner term (line 136). The count is: $a \in \{\pm 2, \pm 3\}$ times a literal on any input, or a 2-parity or an AND of two literals on any pair, times 2 for a separate share of δ. A product pays $L_{B_1} + L_{B_2}$. A block is searched on the round's residual, but admitted only if it also passes the same threshold on $\tilde{r}$, the residual projected off the constant and this round's passing characters (up to 64), with $\text{mean}(\tilde{r}^2)$ in place of s^2 (lines 725–738). So it must add something beyond those characters. Two kinds of product are tested once, on $\tilde{r}$ only, and not searched: the product of the round's two additive candidates, and the products of pairs of additive blocks admitted in earlier rounds, the latter with $\nu = (G_A - 1)(G_B - 1)$ for factors of G_A and

G_B groups (lines 783–793 and 802–815). If $\text{mean}(\tilde{r}^2) \le 10^{-4}s^2$, the characters already explain the residual, and no block is searched.

B.3 The block search and the block moves

The block search. The test does not say which (S, s) to test. `_block_search` (lines 179–500) proposes candidates from the residual alone. It scores them by their margin, the statistic minus its threshold, on the first $\min(n, 2048)$ rows. It builds its own pair matrix M° (line 227): the symmetrized pair matrix of the vector it searches (M in a round's first additive search, M of the deflated residual in the second, that of $r\chi_P$ for a modulated block), with zero diagonal and the excluded inputs' rows and columns set to 0. Unlike $\tilde{M}$, it keeps the entries of pairs that are features or candidates. Its degree-1 coefficients $c_{1,i}$ are those of the same vector. Its starting points are:

- prefixes of the inputs ordered by $|u_i|$, for the top two eigenvectors u of M°, with signs $\text{sign}(u_i)$; every prefix length $k \ge 2$, folded and unfolded. If f is a function of $t = \sum_{i\in S} s_i x_i$, then $\mathbb{E}[f\, x_i x_j] = a_2 s_i s_j$ for distinct $i, j \in S$ and 0 for every other pair (Proposition 2.5), so $M^\circ \approx a_2(ss^\top - I)$ on S, and its top eigenvector gives both the support and the signs;
- prefixes of the inputs ordered by $|c_{1,i}|$, with signs $\text{sign}(c_{1,i})$;
- greedy growth from the 3 strongest pairs of M° (both relative signs) and the 2 strongest single inputs. Each step adds the input and sign that maximize T_B, until 24 inputs or 5 steps without a better margin;
- at $n \le 128$, in a round's first additive search, a beam of width 32 over (S, s), started from the best pairs and scored on the first $\min(n, 1024)$ rows;
- in the additive search, v1.3's weighted candidates. These are eigenvector prefixes whose first input has weight 2 or 3; an inner term $\tau \in \{x_j x_l, \text{AND}(\pm x_j, \pm x_l)\}$ on the strongest pair of M°, with $a \in \{\pm 2, \pm 3\}$; and extensions $t + a\tau$ of the two best unweighted candidates. The extensions use a literal outside S, an input of S re-weighted, or a 2-parity or AND on one of the 24 pairs outside S with the largest $|M^\circ_{jl}|$.

The three best candidates are refined by single moves (add an input with either sign, remove one, flip a sign) while the margin improves, at most 72 moves. Weighted candidates are refined in the same way, with their inner term fixed. The best weighted candidate replaces the best unweighted one if its margin is larger. The winner's groups are then formed on all n rows, and the winner is tested by Equation (5) on all n rows.

Block moves in a round. At $n \ge 32$ (lines 715–815):

- up to two *additive* blocks. The second is searched after the first candidate's group indicators are projected out of the residual, whether or not the first passed;
- *products*. When the round has two additive candidates, a block B is searched over the joint cells with the first one, A, starting also from the second. When A has at least two inputs of each sign, a *split* is also tried: B is searched over the joint cells with the block of A's positive inputs, starting from its negative inputs. The better product is admitted only if its margin on $\tilde{r}$ exceeds A's. Otherwise the product of the round's two additive candidates is tested, if their inputs are disjoint;
- one *modulated* block for each of the up to 4 current characters χ_P with the largest $|w_P|$, searched on $r\chi_P$ with P's inputs excluded;

- the products of pairs of additive blocks, admitted in earlier rounds, that have disjoint inputs.

B.4 Refit, stopping and readout

Refit and stopping. The round adds the admitted characters, the linear block's inputs, each admitted block's indicators (if they fit in the room) and the spectral features, and refits the weights. DFG stops (lines 596–598, 824):

- when a round admits nothing (in the default variant; Section 2.4 for the forced one);
- when no room is left, $p \geq \min(256, \lfloor n/2 \rfloor)$;
- from round 1 on, when the fit has every training label's sign right;
- at the start of a round, if more than 0.75 of the call's budget is spent;
- after 8 rounds.

The code also stops if $s < 10^{-6}$. That cannot happen, since $s \geq 10^{-6}$ by its definition (`study/part11/interp/calib/VERIFY_CALIB.md`).

Readout. After the last round, if $n \geq 8p$, the least-squares weights are refined by 15 Newton steps on the mean logistic loss plus $\frac{1}{2} \cdot 10^{-4}|w|^2$, started from $4w$. The result is rescaled so that its output on the training rows has the norm of the least-squares output (lines 849–855). Part 6 added this step after the least-squares readout plateaued at 5–7% test error on a dense halfspace (thr30) where the networks reached 1% [10]. The prediction adds the kernel background $g \cdot x$ (line 856).

B.5 Small samples

The forced variant. With few points the union bound passes nothing. In round 0, where only the global classes run, no character can pass its test at $d = 30$ below $n = 17$, the degree-1 floor $2c^2 \ln(4Kd/\delta) = 16.95$, and no degree-4 character below $n = 31$ (Table 2). Later rounds have smaller classes and lower floors. In a round where nothing passes, the forced variant adds a guess anyway, at most twice per fit (lines 817–822). If $n \geq 48$ and the round's first additive block candidate has T_B/threshold above the square of the best character's z, it forces that block; both ratios are on a χ^2 scale. Otherwise it adds the character with the largest z among those offered. These are every degree-1 character, the largest pair entry, the largest entry of each row block of degrees 3 and 4, and, from round 1 on, the local subsets above the smallest local threshold (`study/part11/interp/theory/THEORY.md`, §0). A forced guess passes no test. Forcing a block is on by default (`sum_force=True`); the docstring's "off by default" (line 73) is out of date.

The paired cross-validation. At $n \leq$ `cv_n` $= 512$, `DFG.fit` chooses between the default and the forced variant by cross-validation (lines 548–571). There are $F = 10$ folds at $n \leq 64$ and 5 above, assigned by a random permutation of the rows. Each variant is fitted on each fold's training rows, without the degree-5/6 search, and predicts the held-out fold. Let e_{10} be the number of points where the default errs and the forced variant does not, and e_{01} the reverse. The forced variant is chosen if $e_{10} - e_{01} > 2\sqrt{e_{10} + e_{01}}$: McNemar's test for paired errors [29], one-sided at two standard errors. The chosen variant is then fitted on all n rows, and the fold fits' FLOPs count toward the fit. DFG's own choices on the atlas's cells are counted in Appendix G.1. No level is proved for this choice, because the folds' held-out errors are dependent (Section 2.5).

The kernel background and the room. The output adds $g \cdot x$, with $g = X^\top r/n$ on the final residual. With no feature but the constant, $r = y - w_0$, and the predictor is $\text{sign}(w_0 + g \cdot x)$: the centred Hebbian classifier, one step on all d inputs, the counterpart of the networks' near-kernel component. Once the features fit the labels, $r \approx 0$ and $g \approx 0$. The background has no fitted parameter and costs $2nd$ FLOPs. Least squares, likewise, is kept away from its interpolation peak: characters and block indicators are admitted only within $p \le \min(256, \lfloor n/2 \rfloor)$, and the linear block only if $n \ge 2(p + k)$, with p counted before the round's other additions. The linear block's inputs and the spectral features are appended without a room check (lines 826–844), so p can exceed $n/2$ [Proved, by the code]; in none of the 407 atlas cells does it exceed $\min(256, \lfloor n/2 \rfloor)$ (`study/part11/interp/tables/atlas_cells.csv`) [Numerical].

B.6 The budget

The budget. The unit is one training run of a toy network at the same d, $\text{GD}(d) = 394\,PT$ FLOPs for $P = 128d + 16768$ parameters and T steps: 1.62×10^{11} at $d = 30$ (Section 3.3). A fit call gets $\text{GD} \cdot \text{scale}$, with scale 1 at $n > 512$. At $n \le 512$ the final fit gets $\frac{1}{2}$ and each of the $2F$ fold fits $1/(4F)$, so the shares add up to one run. Within a call:

- the exhaustive searches run on the first m rows, with m set so that each costs at most a fixed share of the call's budget. The shares are 15% for degree 4 (round 0 only), 3% per round for degree 3, 3% per round for the two-input extensions, and 3% each for degrees 5 and 6 (round 0 of the final fit only). The search uses $m = \min(n, \lfloor \text{share} \cdot \text{GD} \cdot \text{scale}/c_{\text{row}} \rfloor)$ rows, with c_{row} the code's per-row cost estimate (Proposition 2.6), and its threshold uses m (Equation (2)). A search whose m falls below 32 rows (64 for degrees 5 and 6) is skipped. At $d = 30$ only degrees 5 and 6 are subsampled: degree 5 to 1374 rows (687 in the final fit at $n \le 512$, every row there), degree 6 to 147 (73 in the final fit at $n \le 512$). At $d = 60$ the degree-4 search sees at most 2212 rows, and at $d = 120$ at most 364, with that d's 40000-step run as the unit;
- a round starts only while the call has spent less than 0.75 of its budget. The other parts are not budgeted: degrees 1 and 2, the one-input extensions, the local class, the eigendecomposition, the sum blocks, least squares, the Newton steps and the background. Only this cap bounds them, so a call can exceed 0.75 by the cost of its last round.

Tests		
`K`	4	global degree in round 0
`K_later`	3	global degree, rounds ≥ 1
`K_hi`	6	top degree, round 0
`c`	1	threshold multiplier
`delta`	0.1	level δ of every test
`noise`	`residual`	tests scaled by $s = \mathrm{rms}(r)$
Rounds and classes		
`rounds`	8	rounds at most
`max_new`	64	characters admitted per round
`max_feats`	256	room for characters and blocks (also $\leq n/2$)
`max_ext_bases`	32	characters extended
`local_max`	12	largest $\lvert V\rvert$, local class
`blocks`	True	linear and spectral moves
Small n and budget		
`small_n`	True	forced variant and its test
`cv_n`	512	the test runs at $n \leq$ this
`background`	True	kernel background $g \cdot x$
`budget`	(0.15, 0.03, 0.03)	shares: degree 4; 3; ext. by 2
`budget_hi`	0.03	share: degrees 5, 6 each
Readout		
`readout`	`ls`	run as `logit`
`ridge`	0.001	ridge of least squares ($\times n$)
`logit_lam`	0.0001	L2 penalty, logistic steps

Sum blocks		
`sums`	True	sum blocks
`sum_nmin`	32	smallest n for the block search
`sum_kmax`	24	largest block; the $k_{\max}$ of N_B
`sum_mod`	4	modulators μ at most
`sum_add`	2	additive blocks per round
`sum_gcap`	2048	rows for the search's scores
`sum_mc`	0.005	smallest group over n (at least 3)
`sum_prod`	True	product and split blocks
`sum_seeds`	(2, 3, 2)	eigenvector, pair, single seeds
`sum_inner`	True	weighted blocks $t + a\tau$
`sum_ntau`	1	pairs tried as inner terms
`sum_beam`	32	beam width
`sum_beam_nmax`	128	beam only at $n \leq$ this
`sum_force`	True	forced variant may force a block
`sum_force_nmin`	48	smallest n for a forced block
`sum_force_r2`	0	smallest T_B/n of a forced block

Fixed in the code		line
15	logistic Newton steps	851
$n \geq 8p$	when they run	849
$4w$	where they start	850
0.75	budget share that ends the rounds	598
10 / 5	folds at $n \leq 64$ / above	553
$2\sqrt{e_{10} + e_{01}}$	margin of the paired test	567
$\frac{1}{4F}, \frac{1}{2}$	budget scale: fold fit, final fit	564, 569
32 / 64	fewest rows: degrees 3, 4 / 5, 6	630, 655
4; $4d$	spectral features; smallest n	707
2	forced guesses per fit	817
10^{-4}	least mean$(\tilde{r}^2)/s^2$ for blocks	734
3	fewest points per group	721
$\pm 2, \pm 3$	weights a of inner terms	273
5	growth steps without gain	179
24	pairs for inner-term extensions	180
1024	rows of the beam	180
$3k_{\max}$	refinement moves at most	376

Table 11: DFG v1.3's constructor defaults, as run (`DFG(readout="logit")`, every other argument at its default), and constants fixed in its code, with their lines [Proved, by the code]. Source: `study/part10/hyb_portfolio3/dfg13.py`, read by `paper_dfg/make_tables.py` (`tab_params`). The defaults are read from the constructor's signature, and each fixed constant is checked to occur at the line given.

C The six statements: proofs and checks

This appendix gives the proofs of the statements of Section 2.5, what they do not cover, and their numerical checks, computed by `study/part11/interp/theory/check_theory.py` and stored in `study/part11/interp/theory/check_theory.json` under the keys given. "By the code" means a direct reading of the cited lines of the frozen file. Throughout, the rows are iid uniform on $\{\pm 1\}^d$, as on the toy suites; the column permutation of `harness.Permuted` preserves this.

C.1 Round 0 under the null (P1)

Proof of Proposition 2.1. (a) Condition on y. Round 0's fit is on the constant alone, so $r = y - w_0$ with $w_0 = \sum_t y_t/(n + 10^{-9})$, and r, s and σ_m are functions of y. For a fixed nonempty S the signs $\varepsilon_t = \chi_S(x_t)$ are iid uniform and independent of y. The statistic is $\frac{1}{m}\sum_{t\le m} r_t\varepsilon_t$ and its threshold $c\sigma\sqrt{2L_k/m}$, where $\sigma^2 \ge \frac{1}{m}\sum_{t\le m} r_t^2$: $\sigma = \sigma_m$ if $m < n$, and $\sigma = s \ge \mathrm{rms}(r)$ if $m = n$. Hoeffding's inequality [20], $\Pr(|\sum_t a_t\varepsilon_t| \ge \lambda) \le 2\exp(-\lambda^2/(2\sum_t a_t^2))$, with $a_t = r_t$ and $\lambda = uc\sigma\sqrt{2mL_k}$, gives $\Pr(z_S \ge u \mid y) \le 2e^{-u^2c^2L_k}$. A union bound over the $\binom{d}{k}$ sets and the expectation over y finish the proof. The degree-5/6 search offers $\frac{1}{m}\sum_t r_t\chi_{T_1}(x_t)\chi_{T_2}(x_t)$ only for disjoint T_1, T_2 (line 662). That is the same statistic for $S = T_1 \cup T_2$, and a set reached by several splits is one event.

(b) Condition on y and let $a = r/|r|$. Since $ns^2 \ge |r|^2$, $T_{\mathrm{lin}} \le \sum_i Z_i^2$ with $Z_i = \sum_t a_t x_{t,i}$. The Z_i are independent Rademacher sums with $\sum_t a_t^2 = 1$. For $0 \le \lambda < \frac{1}{2}$ and $\xi \sim N(0,1)$, $\mathbb{E}e^{\lambda Z_i^2} = \mathbb{E}_\xi \prod_t \cosh(\sqrt{2\lambda}\,\xi a_t) \le \mathbb{E}_\xi e^{\lambda\xi^2} = (1-2\lambda)^{-1/2}$. Hence $\ln \mathbb{E}e^{\lambda(\sum_i Z_i^2 - d)} \le d\lambda^2/(1-2\lambda)$. This bound on the log-Laplace transform is all that the Chernoff argument in the proof of Lemma 1 of [25] uses, and that argument gives the tail. At the code's $x = \ln(1/\delta)$ the bound is δ. The block's inputs are all d unless a degree-1 character passed, which (a) bounds by $\delta/(2K)$.

(c) By the code, a round that admits nothing ends the fit (line 824). In round 0 there are no current characters or blocks, so the extension, local and modulated classes and the products of admitted blocks are empty. Add (a) over degrees 1–6 and (b); the $\delta/(2K)$ term of (b) is (a)'s degree 1. □

Not covered. The statements are for exact arithmetic; rounding matters only within rounding error of a threshold. Several parts of a fit are not covered.

- The spectral test: its edge (4) is the asymptotic semicircle edge with a Gaussian correction, not a bound.
- The sum-block tests: Equation (5) is calibrated for Gaussian noise, but the groups are a data-dependent partition of the rows. The union bound counts every candidate, but the tail of a fixed candidate is not proved to be at most e^{-L_B}.
- The forced variant, which adds features by design.
- The cross-validation's choice between the variants: a z-test at 2 on dependent held-out errors.
- Rounds ≥ 1: they run only after an admission, and their residual then depends on the rows.

Other input laws [Proved]. The verifier of THEORY.md adds that (a) does not need uniform rows for the searches on all n rows. Round 0's residual sums to zero, up to the 10^{-9} term, so $\sum_t r_t\chi_S(x_t) = \sum_t r_t(\chi_S(x_t) - \mu_S)$ with $\mu_S = \mathbb{E}\chi_S$: a sum of independent centred terms of range $2|r_t|$, to which Hoeffding's bound applies unchanged for iid rows from any law (`study/part11/interp/theory/VERIFY_THEORY.md`). This covers the character tests on the dependent one-hot columns of the tabular data (Section 4.6). It does not cover subsampled searches, whose rows' residuals need not sum to zero, or (b).

Checks [Numerical].

- *The character tests.* 16 null configurations of round 0 were simulated, 300 data sets each: at $d = 30$ with $n = 32$–2048 and three label laws, at $d = 15$ with degrees 1–6, and at $d = 60$ in a fold fit whose degree-4 search is subsampled. All 280 checks of (a), at levels $u = 0.7$–1, held (`P1.sims`). The bound is loose by about the Gaussian Mills ratio [Heuristic]: at $d = 30$ the Gaussian approximation gives 0.87–1.15×10^{-3} per degree, against the bound's 12.5×10^{-3} (`P1.bound_over_gauss_d30`).
- *The linear block.* 43 of the 4800 null data sets passed it (0.9%), against the bound $\delta = 10\%$; the χ^2_d tail at the threshold is 0.92% at $d = 30$ (`P1.linear_chi2_level`).
- *Full fits on null labels* ($d = 30$). With fair labels, 78 of 80 final models are the constant plus the background (`P1.full_fits`). One holds a pair at $z = 1.06$, the event (a) bounds; the other is a forced-variant model chosen by the cross-validation. With $\Pr(y = 1) = 0.85$ at $n = 128$, 7 of 30 hold spurious spectral or sum-block features (`P1.nonconstant_fits`).
- *The spectral test on biased labels.* Near $n = 4d$ its null pass rate grows with d. For $\Pr(y = 1) = 0.8$ it is about 5%, 47% and 99% at $d = 30$, 60 and 120 (`study/part11/interp/calib/CALIB.md` with the corrections of `study/part11/interp/calib/VERIFY_CALIB.md`).

Section 6.5 reports DFG's false admissions in full.

C.2 The sample size of a parity, and the floors (P2, P2′)

Proof of Proposition 2.2. $y\chi_S = \chi_S^2 = 1$ and $y^2 = 1$ give the two means. With $w_0 = \bar{y}$, $z_S > 1$ reads $1 - \bar{y}^2 > c\sqrt{1 - \bar{y}^2}\sqrt{2L_k/n}$. Divide by $\sqrt{1 - \bar{y}^2}$ and square; if $\bar{y}^2 = 1$, both sides of the equivalence fail. Finally, $\ln \binom{d}{k} = k \ln d - \ln k! + O(k^2/d)$. □

Passing round 0's test is not the same as being in the final model. Admission also needs a place among the round's 64 largest z and room for the feature, which is not proved but never bound in the parity cells. The forced variant can admit χ_S without passing, and a later round could admit it after another admission. Whether the final model holds χ_S is therefore the measured statement below. The constant $2c^2 \ln(4K/\delta) - 2c^2 \ln k!$ is not small: n_{thr} is $2k \ln d$ plus 8.8, 6.6 and 3.8 for $k = 2$, 3 and 4, less an $O(k^2/d)$ term of at most 0.9 here, so it grows like $2k \ln d$, while $2k \ln d$ alone is 9–44% below it for $d = 15$–120 (`P2.n_thr_decomposition`).

Proof of Proposition 2.3. By Cauchy–Schwarz, $|\frac{1}{m}\sum_{t \le m} r_t \chi_S(x_t)| \le \sigma_m \le \sigma$, so $z_S > 1$ needs $c\sqrt{2\ln(4KN/\delta)/m} < 1$. For a block, $\sum_v n_v \bar{\rho}_v^2 \le \sum_t \rho_t^2 \le n s^2$: a projection does not increase the norm, $|\rho_t| = |r_t|$, and $s^2 \ge \text{mean}(r^2)$. So $T_B \le n$. The same holds for $\tilde{r}$ in its own units, for products, and for the deflated residual of the second additive search. □

The character half is THEORY.md's P2′; the block half is added here. Both bound passing a test, not admission: the forced variant's guesses ($n \le 512$) and the linear block's test lie outside these floors. A k-parity with $\bar{y} = 0$ attains the character floor, so $n_{\text{thr}}(k, d)$ is exactly the sample size at which a balanced k-parity passes round 0's test. The last column of Table 2 gives the floors at $d = 30$.

Checks [Numerical].

- *Every parity cell with $n \ge 32$.* Proposition 2.2's criterion, with each training set's own $\bar{y}$, predicts the outcome in all 250 parity cells with $n \ge 32$: par2 and par3 at $d = 15$–120 and

par4 at $d = 15$–60, every seed (`P2.agree`). The test error is 0 where the criterion holds, and 0.488–0.506 elsewhere.

- $n = 16$. There $n_{\text{thr}} > 16$ at every d, and nothing passes. The 7 error-free par2 cells at $d \le 60$ come from the forced variant, chosen by the cross-validation. With that, the criterion reproduces all 11 measured n^* (16–56) to within 0.2 (`P2.pred`).
- *par4 at $d = 30$.* Its n^* is 52, not 28, because 2 of its 3 seeds have $|\sum_t y_t| > 6$ at $n = 32$, where $n_{\text{thr}} = 30.59$. Under Proposition 2.2's exact law that event has probability 0.12.

The parity rule on the grid [Numerical]. On the first round's residual $r = y - \bar{y}$, a parity's statistic $|\text{mean}(r\chi_S)|/\text{sd}(r)$ is $\sqrt{1-\bar{y}^2} \approx 1$, so DFG's test admits it once $n(1-\bar{y}^2) > n_{\text{thr}}$. On the n-grid the test therefore passes, when $\bar{y}$ is near 0, at the first grid point above the line: 32 for par2 at every d, for par3 up to $d = 60$ and for par4 up to $d = 30$, and 64 for par3 at $d = 120$ and for par4 at $d = 60$. Refits of the stored fits (`paper_dfg/check_parity_variant.json`, refitted test errors identical to the result files) show where this simple rule misses. par2 is learned at $n = 16$, below the line, at $d = 15$, 30 and 60, because DFG's cross-validation kept its forced variant in all 7 of those fits; at $d = 120$ it did not. And par4 at $d = 30$ passed at $n = 32$ only in seed 1, the one seed in which $32(1-\bar{y}^2)$ exceeds $n_{\text{thr}} = 30.6$ (28.9, 30.9 and 27.5 in seeds 0, 1 and 2), as the exact criterion predicts.

Against the networks [Numerical, and Proved by the code; Heuristic where marked]. n_{thr}, which grows like $2k \ln d$, is 19.5–42.0 for $k = 2$–4 and $d = 15$–120 (`P2.n_thr`). DFG's n^* on the parities moves by at most one grid step over that range (Figure 3). The networks' n^* grows as $d^{1.03}$, $d^{1.96}$ and $d^{2.40}$ for $k = 2$, 3 and 4, about d^{k-1} for $k \le 3$ (least-squares slopes through 3–4 values of d, par4's on $d = 15$–60 only; Table 23). DFG pays in compute instead: an exhaustive degree-k search costs about $n\binom{d}{k}$ FLOPs times a constant (Proposition 2.6). Under DFG's fixed budget, at $n > 512$, the global search can pass no degree-3 character from $d = 630$ on and no degree-4 character from $d = 179$ on, because there $m_k < 32$ or $m_k \le n_{\text{thr}}$ (Proposition 2.3; `P5.d_parity_unreachable`). Characters of other targets can still enter through the one-input extensions and the local class, which run on all rows, or through the budgeted two-input extensions. [Heuristic] At $d = 1000$, n_{thr} is 36.4, 48.0 and 59.0 for $k = 2$–4, so the test alone would pass these parities from the grid point $n = 64$ on (for $k = 4$ in 97% of training sets) and give $n^* \approx 56$–57 on the grid, leaving the forced variant aside. DFG v1.3 skips its degree-3 and degree-4 searches there ($m_3 = 17$, $m_4 = 0$), and an exhaustive degree-4 search on 64 rows would cost 55.9 network runs (`P2.d1000`, `P5.exhaustive`).

C.3 When a character passes in round 0 (P3)

Proof of Proposition 2.4. (a) is the algebra of Proposition 2.2 with $\hat{c}_S$ in place of 1. For the expectation, $\mathbb{E}[w_0\bar{\chi}_S] = \frac{1}{n(n+10^{-9})}\sum_{t,u}\mathbb{E}[y_t\chi_S(x_u)]$. Its terms with $u = t$ are $\hat{f}(S)$, and those with $u \ne t$ vanish because $\mathbb{E}\chi_S = 0$. (c) is the same algebra in round j. □

Checks [Numerical]. The checks use the 41 targets of `designed` and `random`, seed 0, at every n of the grid.

- *The round-0 decision.* Over 8681 (character, n) pairs, $n > n_{\text{glob}}$ agrees with DFG's round-0 decision in 8409 (96.9%; `P3.test0`). The agreement comes mostly from pairs far from n_{glob}: within one doubling of it, the two agree in 773 of 1012 pairs (76%), and more than two doublings away in 99.9–100% (`P3.test0.agree_by_distance`). The pass rate is 6% and 12%

at 1–1.5 and 0.5–1 doublings below n_{glob}, and 94% and 98% at 0.5–1 and 1–1.5 doublings above (`P3.test0.calibration`).

- *Entry through the round-0 test.* The 249 characters that first enter through it do so at a median $2^{0.26}\, n_{\text{glob}}$ (`P3.first`).
- *Entry through later-round classes.* Characters that enter through the local, extension or later global classes do so a median 1.4–3.7 doublings below n_{glob}, by class (`P3.entry_vs_nglob`). In Proposition 2.4(c)'s factorization the residual factor is the largest at the class medians: they are 3.2–19, and per character the factor ranges from 1.2 to 59.5. The union-bound factor has class medians 1.2–1.7 in the composition classes and 1.0 for the later global test, but single local-class characters of degree ≤ 6 gain up to 3.3 from it (`P3.factors`, `P3.factors_spread`).

C.4 Sum blocks (P4)

Proof sketch of Proposition 2.5. (a) A function η of the group equals $\eta(v_{\max}) + \sum_{v \neq v_{\max}} (\eta(v) - \eta(v_{\max}))[\text{group} = v]$ on every occupied group.

(b) By the code (`_edges`): scanning the values upward, a group closes once it holds $\max(3, \lfloor 0.005n \rfloor)$ points, and a short last group is merged into the one before it. The rarest values, $t = \pm k$, have probability 2^{-k}, which is at least 0.005 iff $k \leq 7$.

(c) Any combination of the block's features and the constant is constant on each group.

(d) Inputs outside S are independent of f. On S, substitute $z_i = s_i x_i$; by symmetry the coefficient depends only on $|T|$, and $\mathcal{K}_j(\beta) = \sum_{z:\, \#\{z_i = 1\} = \beta} z_1 \cdots z_j$.

(e) Write $[\beta \equiv \kappa] = \frac{1}{3} \sum_{m=0}^{2} \omega^{m(\beta-\kappa)}$ with $\omega = e^{2\pi i/3}$. Since $\beta = \sum_{i \in S} (1 + z_i)/2$ with independent z_i,

$$\mathbb{E}\big[\omega^{m\beta} z_1 \cdots z_j\big] = \Big(\frac{\omega^m - 1}{2}\Big)^j \Big(\frac{1 + \omega^m}{2}\Big)^{k-j},$$

which vanishes at $m = 0$ for $j \geq 1$; the terms $m = 1$ and $m = 2$ are complex conjugates. Moreover $|\omega - 1|/2 = \sqrt{3}/2$ and $|1 + \omega|/2 = \frac{1}{2}$, so $|a_j| \leq 2 \cdot \frac{2}{3} \cdot 3^{j/2} 2^{-k}$.

(f) The starting points are the eigenvectors and the strongest entries of M°; the coefficients $c_{1,i}$ and single inputs; inner terms on M°'s strongest pair; at $n \leq 128$, a beam started from two-input blocks; and, in the product and split searches, the round's earlier candidates. A two-input block's groups are functions of $x_i + s x_j$, and these span $\{1, x_i + s x_j, x_i x_j\}$. The expectations follow as in Proposition 2.4(a). □

What this says about the residue classes [Numerical; Heuristic where marked]. Take the three residue-class targets f7_05, h6_00 and fd_10, on $k = 14$, 13 and 10 inputs, which DFG alone never learns (Section 4.2).

- *Fourier weight.* Every one of the $2^k - 1$ characters on S is nonzero, while $W_1 + W_2 = 1.6 \times 10^{-6}$, 5.4×10^{-6} and 2.1×10^{-4}. These are exact rationals (`P4.targets`).
- *The representation is not what fails.* One block with one group per value represents each target with at most $k+1$ weights, and least squares on such a block reproduces seed 0's training labels at $n = 16384$ exactly. DFG's own grouping merges the rarest values, which costs 0.1–0.2% error (`P4.grouping`). Part 10 found the true block passing the test from $n = 128$ [12].
- *The search is* (Section 6.4). By (f), its starting points see these targets only through a_1 and a_2. [Heuristic]: by the BBP threshold for a rank-one perturbation of a random matrix [2, 5], the eigenvector seed needs $n \approx s^2 d/(a_2^2 (k-1)^2)$: 1.1×10^7, 3.1×10^6 and 8.6×10^4 rows for the three targets, far above the grid's 16384.

C.5 Compute (P5)

In Proposition 2.6(a), the matrix products also compute redundant entries, so one character costs $12n$ or $24n$ FLOPs, not $2n$. In (b), the per-row cost estimate c_{row} is $d(2d^2+2d)$ for degree 3; $\binom{d}{2}(2d^2+3d)$ for degree 4; $b(2d^2+3d)$ for the two-input extensions of b bases; and $2N_1N_2 + N_1k_1 + N_2k_2$ for degrees 5 and 6.

Checks [Numerical].

- *An analytic FLOP model* of the "parity path" (round 0 admits χ_S alone, round 1 stops on the sign rule) reproduces the recorded FLOP count to 10^{-9} in 228 of the 275 parity cells. It reproduces 4 more once a false linear block is added, which refits confirm; the other 43 cells are at $n \le 32$ (`P5.check`).
- *The budget bound.* Over $d = 5$–399 and the four budget scales, no budgeted search's charged cost exceeds its share; the largest ratio is $1 - 1.4 \times 10^{-8}$ (`P5.max_share_used_over_share`).
- *Against one network run.* GD(d) is linear in d, while an exhaustive degree-4 search on 16384 rows costs 0.072, 1.04 and 6.5 network runs at $d = 30$, 60 and 120 (`P5.exhaustive`). The budget's subsampling is what keeps DFG inside one run.

D The evaluation in full

This appendix gives the parts of the evaluation that Section 3 summarizes: the suites (Appendix D.1), the references (Appendix D.2), and the protocol's questions with the status of every set (Appendix D.3).

D.1 The suites

The 11 toy suites (Section 3.1) are:

- `designed` (21), the study's original targets (Parts 1–5): a dictator, AND and majority of 3 inputs, `maj9`, a halfspace with Gaussian weights on 10 inputs, parities of 2–4 inputs, a staircase, compositions of parities and majorities, a multiplexer, a DNF, a decision list, a sparse polynomial threshold and four random 6-gate formulas.
- `random` (20): random fan-in-2 AND/OR/XOR formulas with 3–8 gates, written in Part 3. Each formula's number k of inputs was drawn from 4–8, and a formula was kept if $|\mathbb{E}f| \leq 0.5$ and it depended on at least $k-1$ of its inputs; all 20 kept formulas have $k = 4$ or 5 (`boolfuncs.make_random_suite`).
- `dlheld`, `dlheld2`, `dlheld3` (12, 7, 10): dense halfspaces and majorities, compositions of majorities, a 5-bit parity, the 6- and 11-multiplexers, a staircase, an intersection of two halfspaces, a DNF and a CNF, a decision list, an integer-weight threshold, symmetric functions such as $[|x_1 + \cdots + x_9| \leq 1]$, parities mixed with gates, and random formulas.
- `dlsymdev` (8) and `dlheld4` (12): symmetric and signed-sum targets with threshold, band, tail and exact-count accept sets, some combined with a parity, a gate or a second sum; `dlheld4` adds 3 non-symmetric controls.
- `dlheld5` (12), drawn from a random family: 8 functions of a signed sum of 5–13 inputs with a random accept set, two of them combined with a gate; 2 thresholds of a signed sum plus twice a parity; and 2 random formulas as controls.
- `dlheld6`, `dlfamdev`, `dlheld7` (12 each): three draws, with seeds 52817, 61616 and 70707, from the blind family.

The blind family is a random generator of signed-sum targets. An agent that had not seen DFG's sum-block design wrote it in Part 9, from the pre-registered one-line description of a held-out suite, before drawing from it (`study/part9/heldout6/FAMILY.md`). Each draw has 12 slots.

- *Nine structured slots*, built from functions $g(s)$ of a signed sum $s = \sum_i \epsilon_i x_i$ of 4–14 inputs: $g(s)$ alone (3 slots); $g(s)$ times a parity (2); $g(s)$ with a gate, that is an AND or OR with one or two literals, a majority with two literals, or a multiplexer between two sums (2); and composite sums (2), either $A(s + a\tau)$ with an inner term $a\tau$, $a \in \{2, 3\}$, τ a literal, a 2-parity or an AND of two literals, or two sums joined by XOR, AND or OR.
- *Three non-symmetric controls*: a read-once formula, a DNF and a decision list.

Each g has an accept set drawn uniformly from five types: a threshold, a centred band, an interval, a residue class mod 3 or 4, and an arbitrary set. A is a threshold or a band. Targets have 5–14 relevant inputs, and the only rejection rule is $P(f = +1) \in [0.25, 0.75]$.

D.2 The references

Where each network variant was run. The five variants of Section 3.2 were run as follows:

- `nets`: square loss. It was run on every suite except `dlheld7` and `dlfamdev`, and it is the only variant run at $d \neq 30$.
- `nets_wd`: square loss with decoupled weight decay 0.1 [27]; every suite except `dlfamdev`.
- `nets_cewd`: logistic loss with weight decay 0.1; `dlheld3`–`dlheld6` and `dlsymdev`.
- `nets_cewd0.3`, `nets_cewd1.0`: logistic loss with weight decay 0.3 and 1.0; `dlheld6` and `dlheld7`.

`nets_best`, the per-target variant with the smallest n^*, is a minimum over 2 variants on `designed`, `random`, `dlheld` and `dlheld2`, over 3 on `dlheld3`–`dlheld5` and `dlsymdev`, over 5 on `dlheld6`, and over the 3 that Part 10 pre-registered on `dlheld7`: a hindsight choice that favours the networks.

The no-search baselines. `lin` is L2-regularized logistic regression on the raw inputs, its penalty chosen by 5-fold cross-validation (2-fold when a class has fewer than 5 training points) over 10 values. `hgb` is scikit-learn's histogram gradient boosting at default settings [32] (both in `study/hybrid/refs.py`). Neither searches an explicit feature dictionary: `lin` searches no features at all, and `hgb` only through greedy splits on single inputs. So they show which targets need no dictionary search. Both were run in Parts 8–10, with the columns in their natural order. `hgb` has no runs on `designed`, `dlsymdev` and `dlfamdev`, and `lin` none on `dlfamdev`.

The low-degree algorithm, step by step. `lowdeg` (`study/discrete_learner/sparse_fourier.py`) is Part 6's first feasibility check, run unchanged with the inputs permuted per fit. It is the low-degree algorithm [26] with a significance threshold:

1. compute the empirical Fourier coefficients of the labels, $\hat{c}_S = \frac{1}{n}\sum_t y_t \chi_S(x_t)$, for every S with $|S| \leq 4$;
2. keep S if $|\hat{c}_S| > \sqrt{2\ln(\binom{d}{|S|}/\delta)/n}$ with $\delta = 0.1$, a union bound over the characters of that degree;
3. compose: the products $\chi_{S_1}\chi_{S_2} = \chi_{S_1 \triangle S_2}$ of two kept characters, where they have degree above 4, are tested in the same way, with the number of such candidates in place of $\binom{d}{|S|}$, and this is repeated, at most 4 times, while something new is kept;
4. predict $\operatorname{sign}(\hat{c}_\emptyset + \sum_{S \text{ kept}} \hat{c}_S \chi_S)$.

It shares DFG's global characters up to degree 4 and the form of its test, and nothing else. It tests the labels, not a residual, so its noise scale is 1 in place of s, and its union bound has no factor $4K$. It has no greedy rounds, extension or local classes, sum blocks or block moves, and no refitted readout, kernel background or small-n variant. Its FLOPs are counted in the same way as DFG's.

Portfolios that contain DFG. v3 (`hyb_portfolio3`, Part 10 [12]) is the stacked portfolio whose Boolean member is DFG v1.3. DFG's default and forced variants are two of its members, next to LightGBM, ExtraTrees, nearest-neighbour votes and logistic regression on the raw columns. A convex stacker combines them, and DFG has a share of the budget. v2 (`hyb_portfolio2`, Part 9) and v1 (`hyb_portfolio`, Part 8) are the earlier portfolios, with DFG v1.2 and v1.1 [10]. On toy inputs v3's results are those of `p10_sum`, which is v3's code there (checked identical when v3 was assembled). v2's are its own or, on `designed` and `dlsymdev`, those of `p9_sum`, the same code on toy inputs. Both ran on every toy suite, with the inputs permuted per fit. DFG alone against v3 measures what the rest of the portfolio adds on Boolean inputs. v1 is a reference on

tabular data only. DFG v1.3, the file `study/part10/hyb_portfolio3/dfg13.py`, is identical to `study/part10/p10_sum/dfg13.py`.

Earlier versions of DFG. Part 6 ran DFG v1 at $d = 15$ and 30, and v1.1 at $d = 60$ and 120, on the Q4 targets in the natural column order (`study/discrete_learner/results/`). They are context for Q4, not a paired comparison. On real-valued data the record is Part 7's three adaptations of DFG, frozen there [10]: Boolean-bit characters (`dfg_bits`), thresholded data-derived directions (`dfg_dirs`) and threshold and AND gates (`dfg_gates`). Their results on kr_vs_kp and mushroom are context for Q6. Part 6 built DFG with enumeration and union-bound tests in place of the networks' noisy search and with at most one network run of FLOPs per fit, and on a blind 7-target suite it needed less data than the networks on every target [10]. Part 7 adapted DFG to real-valued standard tasks, where it was about level with a plain MLP on tabular data, behind a regularized one, and clearly behind on images at large n. Parts 8 and 9 stacked it with classical learners (v1, v2), whose toy wins came from DFG, and gave it signed-sum blocks (v1.2) [10]. Part 10 added weighted blocks, forced blocks and a beam search (v1.3) and built v3, which on the held-out `dlheld7` needed clearly less data than the best network variant on 10 of 12 targets and more on none, and whose analysis placed DFG's data cost on signed-sum targets in its search rather than its test [12].

D.3 The protocol's questions and runs, and the status of every set

What the protocol fixed. The protocol (`study/part11/PROTOCOL.md`; commits and times in Appendix H.2) fixed:

- the object and how it is run;
- seven questions, each with its runs, suites and seeds;
- that grids, seeds, test sets, n^*, the verdict rule and the suite means are those of Parts 8–10, and that the results of the networks, `lin`, `hgb`, v1, v2 and v3 are the existing files, with nothing re-run for them;
- that a run that crashes may get harness-level fixes only (paths, sharding, resumption), never a change to the learner, and that the crash is reported;
- what is held out.

Questions and runs. The protocol fixed seven questions.

- *Q1, held out.* On `dlheld7`: DFG alone against `nets_best`, each network variant, `lin`, `hgb`, v3 and v2, target by target and as suite means with clearly better and clearly worse counts.
- *Q2, every suite.* The same on all 11 toy suites (138 targets), each with its status, and DFG alone against v3: does the rest of the portfolio add anything on Boolean targets?
- *Q3, the dictionary without the search* (the protocol's title; `lowdeg` has DFG's characters of degree ≤ 4 and the form of its test, and nothing else). `lowdeg` on all 11 suites.
- *Q4, input dimension.* DFG alone at $d = 15$, 60 and 120 on the 7, 7 and 4 targets above: n^* against d, against the networks.
- *Q5, what the parts do.* 12 ablations, each DFG alone with one setting changed (one or two constructor arguments), on `designed`, `dlsymdev`, `dlfamdev`, `dlheld6` and `dlheld7` (65 targets). Six run on seeds 0 and 1 and six on seed 0. Each is compared with DFG alone on the same seeds, so on the same training sets and column permutations (Section 5).

- *Q6, tabular.* DFG alone on kr_vs_kp and mushroom, on the standard learning curves with 3 seeds, against the MLP references, `hgb` and v3; v2 and v1 are also shown.
- *Q7, what it learns.* Descriptive analyses, on seed 0 unless stated and checked by independent verifiers (Section 6): what DFG's fitted models contain across target families, against each target's own representation in DFG's dictionary; when a target's structure first appears in the model, against when its test would admit it; the order in which DFG admits a target's characters, against the order in which the Part 5 networks acquire them; false admissions on labels independent of the inputs; and where the compute goes.

Section 4 reports Q1–Q4 and Q6, Section 5 Q5, and Section 6 Q7.

set	size	origin, and its use in developing DFG	status for DFG v1.3	references run
`designed`	21	Parts 1–5. Development for DFG v1 and v1.1 (Part 6 chose DFG's settings on its test error), and for v1.2 and v1.3 (the Part 9 and Part 10 development evaluations)	development	`nets`, `nets_wd`
`random`	20	Part 3. A development check for DFG v1 and v1.1 in Part 6, run and read with intermediate versions	development	`nets`, `nets_wd` (2 seeds)
`dlheld`	12	Part 6, held out for DFG v1; its degree-5/6 search was added knowing `par5` was in the suite. DFG v1.1 was made after DFG v1's run on it	development	`nets`, `nets_wd`
`dlheld2`	7	Part 6, written after v1.1 was committed: blind for v1.1	held out for v1.1 only	`nets`, `nets_wd`
`dlheld3`	10	Part 8, held out for the portfolio v1 (with DFG v1.1). v1's failure on `sym9` motivated v1.2's sum blocks; pre-registered as a Part 9 regression check but not run then	held out for v1.1 only	`nets`, `nets_wd`, `nets_cewd`
`dlsymdev`	8	Part 9. Development for v1.2 and v1.3	development	`nets`, `nets_wd`, `nets_cewd`
`dlheld4`	12	Part 9, meant to be held out for v1.2; its builder saw the suite's source	exposed to v1.2	`nets`, `nets_wd`, `nets_cewd`
`dlheld5`	12	Part 9, drawn after v1.2 was frozen, from a family specified after reading v1.2's design	exposed to v1.2	`nets`, `nets_wd`, `nets_cewd`
`dlheld6`	12	Part 9, the blind family's first draw; held out for v1.2. Development for v1.3, whose weighted blocks were designed after v2's failure on h6_10	development	all five variants
`dlfamdev`	12	Part 10, a development draw from the blind family. Development for v1.3	development	none
`dlheld7`	12	Part 10, a draw from the blind family fixed in a commit (`dc70266c`) before v1.3 was designed; never used in its development	**held out**	`nets_wd`, `nets_cewd0.3`, `nets_cewd1.0`
`designed` at $d = 15, 60, 120$	7, 7, 4	Networks trained in Part 3. Part 6 ran DFG v1 at $d = 15$ and v1.1 at $d = 60$ and 120	development	`nets` (2 seeds)
kr_vs_kp	3196 rows	Part 7, a development set of the standard tasks; the Part 10 toy builder's tabular check	development	`mlp`, `mlp_reg`
mushroom	8124 rows	Part 7, held out there. Never used to develop DFG; v1–v3 ran DFG on it, and their results were read before Part 11	seen through v1–v3	`mlp`, `mlp_reg`, `mlp_strong`, `mlp_strong1`, `mlp_plus` ($n \leq 3000$)

Table 12: Every set and its status for DFG v1.3. Sources: `study/part11/PROTOCOL.md` ("Held out") and the protocols and records of Parts 6, 8, 9 and 10. "Development": used in developing some version of DFG. "Exposed": the Part 9 sum-block design and the suite were not independent. "Held out for v1.1 only": held out for DFG v1.1 (inside the portfolio v1 for `dlheld3`), and seen since. "References run": the network variants run on the suite, over which `nets_best` is the per-target minimum, or the MLP variants on the dataset. Networks have 3 seeds except where stated. `lin` and `hgb` are listed in Section 3.2; v3, v2 and `lowdeg` ran on every toy suite.

E Further results

This appendix holds the detail behind Section 4: every toy target's n^* for every learner, the post-hoc core included, and the pooled summaries (Appendix E.1); the targets on which DFG alone is not better than the networks (Appendix E.2); DFG alone against v3 (Appendix E.3); `lowdeg` (Appendix E.4); the input dimension (Appendix E.5); tabular data (Appendix E.6); and compute (Appendix E.7). Apart from the rows of `dlheld7`, none of it is held out for DFG v1.3 (Table 12).

E.1 Every toy target

Table 13 summarizes DFG alone against each reference on the held-out `dlheld7` (Section 4.1) [Numerical].

Tables 14 to 16 give, for each of the 138 targets of the 11 toy suites, n^* of DFG alone, v3, `lowdeg`, `lin`, `hgb` and `nets_best`, and the post-hoc core's n^* next to DFG alone's on the same seeds, 0 and 1 [Numerical]. The suite summaries of Tables 5, 17, 22 and 27 are means over these rows, and Table 17 pools them, and the targets' definitions are in `study/boolfuncs.py` (for `dlheld7`, in Table 4).

E.2 Targets on which DFG alone is not better

Table 18 lists the 9 of the 126 targets with network runs on which DFG alone is not clearly better than `nets_best` [Numerical]. Table 18 lists the 9 of the 126 targets with network runs on which DFG alone is not clearly better than `nets_best`.

- *Clearly worse (2).* h3rf1, a random 9-gate formula on 7 inputs: 133 against 101 (`nets_wd`). And h6_00, a residue class mod 3 of a signed 13-input count, which only the most regularized network learns (`nets_cewd1.0`, 12304).
- *Ties (5).* Three threshold-like targets: thrint, a weighted threshold (186 against 190), inter2, an AND of two halfspaces (441 against 454), and f7_01 (119 against 119). The random formula h3rf0 (108 against 96) and and3 = AND(x_1, x_2, x_3) (60 against 70).
- *Learned by no learner (2).* f7_05 and h5s1, a signed 9-input count with an arbitrary accept set, multiplied by the parity $x_{10}x_{11}$.
- *Never learned.* DFG alone does not learn 4 of the 138 targets: these two, h6_00 and fd_10 (`dlfamdev`, a third residue class mod 3, without network runs). v3 and `lowdeg` do not learn them either.

Reading [Heuristic]. On h3rf1, and on the ties h3rf0 and and3, DFG's lack of a clear margin reflects its slowness at small n more than a strength of the networks. v3 (59, 46 and 25) needs less data than both DFG alone and the networks, and so does `hgb` on the two formulas (63, 60; it was not run on `designed`). On h3rf1 and h3rf0 even `lowdeg` does (31, 16). On thrint and f7_01, logistic regression on the inputs is as fast or faster (`lin`: 137, 111). The residue classes are the search failure of Section 4.1.

E.3 DFG alone and v3

Inside v3, DFG v1.3 ran as two members of a stack, its default and (at $n \leq 512$) its forced variant, next to LightGBM, ExtraTrees, k-nearest-neighbour votes and logistic regression. It had a share of the budget: 35% for the out-of-fold fits and at most 50% for the final ones. A convex stacker, fitted on the members' out-of-fold predictions (10-fold at $n \leq 500$), weighted the members (Section 3.2; `study/part10/hyb_portfolio3/learner.py`). Alone, DFG chooses between its two variants itself,

	DFG alone against the reference						the reference against `nets_best`	
reference	both reach	mean $\log_2$ ratio	median	clearly better	clearly worse	only the reference / only DFG reaches	mean (better/worse) [both reach]	median
v3	11	+0.21	+0.05	0	3	0 / 0	−1.50 (10/0) [11]	−1.29
v2	11	+0.21	+0.04	0	3	0 / 0	−1.50 (10/0) [11]	−1.33
`lowdeg`	10	−1.90	−1.71	10	1	0 / 1	+0.70 (3/8) [10]	+1.07
`lin`	4	−0.45	−0.35	9	0	0 / 7	−0.06 (1/7) [4]	−0.02
`hgb`	9	−1.39	−0.44	8	2	0 / 2	+0.28 (4/6) [9]	−0.06
`nets_wd`	11	−1.62	−1.33	11	0	0 / 0	–	–
`nets_cewd0.3`	11	−1.38	−1.41	10	0	0 / 0	–	–
`nets_cewd1.0`	11	−1.36	−1.32	10	0	0 / 0	–	–
`nets_best`	11	−1.29	−1.16	10	0	0 / 0	–	–

Table 13: DFG alone against each reference on `dlheld7` (12 targets) [Numerical]. Left: the number of targets that both reach 10% test error, the mean and median $\log_2 n^*$ ratio of DFG alone to the reference over them, the targets on which DFG alone is clearly better and clearly worse, and the targets that only the reference or only DFG alone reaches. Right: the reference itself against `nets_best`, as mean (clearly better / clearly worse) [both reach] and median. Source: `study/part11/RESULTS.json` (`q1_dlheld7.summary`, `.context`).

by the paired test on its own cross-validation, and it has the whole budget (Section 2). Both see the same training sets in the same column order, so their results can be compared target by target. The comparison shows what the rest of the portfolio contributed on Boolean targets.

Where [Numerical].

- *Overall* (Table 19a). Over the 138 targets, DFG alone's n^* is lower than v3's on 32 and higher on 101; it equals v3's on 1 (par2), and neither learns 4. Beyond ±0.25 it is lower on 6 and higher on 42. The mean ratio is +0.19, so DFG alone needs 1.14× v3's data, and the median is +0.07. A two-sided sign test over the 133 targets with unequal n^* gives $p = 1.6 \times 10^{-9}$. No target is learned by one and not the other.
- *By target size* (Table 19b). On the 45 targets that v3 learns by $n = 64$, DFG alone's mean is +0.39, clearly higher on 24. On the 67 that v3 learns at 64–256 it is +0.17 (17 clearly higher). On the 22 that need v3 more than 256 points, DFG alone is lower on average (−0.16 and −0.39).
- *By n* (Table 20a). v3's test error is lower by more than 0.02 on 63, 64 and 67 of the 138 targets at $n = 16$, 32 and 64, on 38 at $n = 128$, 17 at 256 and 5 at 512, and on none at $n \geq 1024$. DFG alone's error is lower by that much on 2–26 targets at every n up to 4096.
- *Which targets* (Table 21). v3 learns 39 of the 42 targets on which DFG alone is clearly worse by $n = 128$. Most are small Boolean formulas: random gate formulas, DNFs and a CNF, decision lists, read-once formulas, mux6, and3 and andxor. The others are six halfspace-like targets (dict, maj3, maj15, thr10, thrint, and inter2, an AND of two halfspaces) and four sum targets (far9, tail13, fd_08, h5s4). The largest ratios are h6_04 (79 against 29), h2rf1 (82 against 31) and and3 (60 against 25); the largest absolute gap is dnf4x2 (235 against 112). `hgb` alone needs at most DFG alone's n^* on 15 of the 28 of these targets on which it was run.

Why [Numerical, then Heuristic]. Part 10's seed-0 analysis refits of v3 record v3's error, the error of its own DFG member alone, and the stacking weights (Table 20b). At $n = 16$, 32 and 64, v3's error is lower than its own DFG member's by more than 0.02 in 57, 68 and 53 of 136–138 fits, and

target	DFG alone	v3	lowdeg	lin	hgb	nets_best (variant)	DFG alone vs nets_best	DFG alone vs v3	core / DFG alone (seeds 0, 1)	core vs DFG alone
designed: 21 targets; development (v1.1, v1.2, v1.3); networks: nets, nets_wd										
dict	22	16	16	27	–	40 (wd)	−0.85	+0.48	16 / 25	−0.65
and3	60	25	158	68	–	70 (wd)	−0.21	+1.30	37 / 78	−1.09
maj3	56	31	85	56	–	91 (wd)	−0.70	+0.82	49 / 68	−0.48
maj9	81	80	323	80	–	149 (wd)	−0.88	+0.02	110 / 84	+0.39
thr10	146	121	730	113	–	307 (wd)	−1.07	+0.27	154 / 128	+0.27
par2	16	16	28	never	–	109 (wd)	−2.77	+0.00	16 / 16	+0.00
par3	28	55	28	never	–	443 (wd)	−3.99	−0.99	28 / 28	+0.01
par4	52	56	28	never	–	1726 (wd)	−5.06	−0.10	3124 / 48	+6.02
stair5	111	109	225	never	–	871 (wd)	−2.97	+0.02	103 / 109	−0.08
majpar	61	60	97	never	–	1026 (wd)	−4.08	+0.00	49 / 62	−0.34
parmaj	111	110	418	never	–	434 (wd)	−1.96	+0.02	111 / 112	−0.00
mux	63	70	148	never	–	729 (wd)	−3.53	−0.14	58 / 70	−0.28
dnf3	178	118	532	never	–	353 (wd)	−0.99	+0.60	195 / 199	−0.02
dlist5	61	32	173	85	–	85 (wd)	−0.49	+0.93	52 / 58	−0.16
rmaj9	167	163	403	never	–	1273 (wd)	−2.93	+0.03	209 / 167	+0.32
andxor	52	40	83	never	–	204 (wd)	−1.98	+0.37	44 / 49	−0.15
sparsepoly	102	103	185	never	–	1238 (wd)	−3.61	−0.03	82 / 111	−0.45
rand0	25	16	29	never	–	125 (wd)	−2.35	+0.62	16 / 27	−0.73
rand1	104	95	130	never	–	316 (wd)	−1.60	+0.13	63 / 105	−0.72
rand2	28	16	16	50	–	59 (wd)	−1.06	+0.83	28 / 26	+0.13
rand3	116	114	995	never	–	692 (wd)	−2.58	+0.01	136 / 114	+0.26
random: 20 targets; development (v1.1); networks: nets, nets_wd										
rf00	32	40	16	83	60	62 (wd)	−0.95	−0.33	30 / 32	−0.10
rf01	52	48	85	never	120	256 (wd)	−2.31	+0.11	74 / 52	+0.51
rf02	51	49	98	never	228	191 (wd)	−1.90	+0.07	49 / 51	−0.07
rf03	91	62	294	never	139	276 (wd)	−1.60	+0.54	108 / 91	+0.24
rf04	54	48	97	never	111	203 (wd)	−1.91	+0.16	56 / 54	+0.04
rf05	75	62	145	never	250	493 (wd)	−2.72	+0.26	72 / 75	−0.04
rf06	31	16	16	37	58	50 (wd)	−0.67	+0.97	16 / 31	−0.97
rf07	104	88	231	never	168	281 (wd)	−1.44	+0.23	97 / 104	−0.10
rf08	50	21	146	69	53	62 (wd)	−0.32	+1.22	49 / 50	−0.02
rf09	51	45	37	never	292	182 (wd)	−1.83	+0.18	37 / 51	−0.46
rf10	43	36	86	66	53	79 (wd)	−0.89	+0.26	37 / 43	−0.21
rf11	28	56	28	never	416	445 (wd)	−4.00	−1.00	28 / 28	−0.00
rf12	97	87	289	never	524	610 (wd)	−2.65	+0.16	87 / 97	−0.16
rf13	54	48	150	never	213	245 (wd)	−2.18	+0.15	59 / 54	+0.13
rf14	53	53	97	never	378	404 (wd)	−2.92	−0.00	55 / 53	+0.06
rf15	51	22	108	67	53	75 (wd)	−0.55	+1.23	37 / 51	−0.48
rf16	44	36	74	never	214	200 (wd)	−2.19	+0.27	49 / 44	+0.16
rf17	49	56	28	never	never	1715 (wd)	−5.14	−0.19	3550 / 49	+6.19
rf18	51	46	74	never	93	112 (wd)	−1.13	+0.16	56 / 51	+0.12
rf19	57	59	296	never	366	654 (wd)	−3.53	−0.06	74 / 57	+0.39

Table 14: Every toy target, part 1 of 3: designed and random [Numerical]. n^*: the training-set size at which the seed-mean test error reaches 10% (3 seeds; 2 on random), interpolated in $\log n$; "never": not by $n = 16384$; "–": not run on the suite. DFG alone, v3 and lowdeg see the inputs permuted per fit, lin and hgb in the natural order. nets_best: the per-target minimum over the network variants run on the suite (named in each suite's line), with the variant attaining it in brackets: wd, cewd, cewd0.3, cewd1.0 for nets_wd, nets_cewd, nets_cewd0.3, nets_cewd1.0. Ratios are $\log_2$ of n^*, negative when the first-named learner needs less data; $\pm\infty$: only one of the two reaches 10% error; n/a: neither. Core (post hoc): the n^* of the core and of DFG alone on seeds 0 and 1, and the ratio of the two, on the same training sets (Section 5.3); DFG alone's 3-seed n^* is the second column. Each suite's line gives its status for DFG v1.3 as in study/part11/PROTOCOL.md. Source: study/part11/RESULTS.json (q2_suites, q3_lowdeg, q5b_core) via paper_dfg/make_tables.py (tab_targets).

target	DFG alone	v3	lowdeg	lin	hgb	nets_best (variant)	DFG alone vs nets_best	DFG alone vs v3	core / DFG alone (seeds 0, 1)	core vs DFG alone
dlheld: 12 targets; development (v1.1); networks: nets, nets_wd										
maj15	130	105	622	110	1278	210 (wd)	−0.69	+0.31	206 / 122	+0.76
thr30	158	149	1820	148	1645	311 (wd)	−0.97	+0.09	623 / 158	+1.98
majxor	225	224	1724	never	4076	792 (wd)	−1.82	+0.01	225 / 223	+0.02
majmaj5	351	360	5997	never	1638	2355 (wd)	−2.75	−0.04	352 / 353	−0.00
par5	56	839	never	never	never	7096 (wd)	−6.99	−3.91	never / 56	+∞
mux6	197	123	339	never	188	358 (wd)	−0.86	+0.67	210 / 208	+0.01
stair6	207	190	1075	never	695	1495 (wd)	−2.85	+0.13	205 / 210	−0.04
inter2	441	346	2204	never	416	454 (wd)	−0.04	+0.35	540 / 518	+0.06
majpar5	128	127	198	never	4127	4648 (wd)	−5.19	+0.01	108 / 165	−0.62
hrf0	80	62	32	191	63	133 (wd)	−0.74	+0.36	46 / 77	−0.75
hrf1	119	116	121	never	461	789 (wd)	−2.73	+0.04	123 / 116	+0.09
hrf2	46	54	26	93	60	90 (wd)	−0.98	−0.23	56 / 50	+0.16
dlheld2: 7 targets; held out for an earlier version only; networks: nets, nets_wd										
thr20	138	124	1054	117	1262	304 (wd)	−1.14	+0.16	455 / 134	+1.76
maj21	112	109	693	116	1956	216 (wd)	−0.94	+0.04	174 / 109	+0.67
xmaj	61	66	168	never	1538	871 (wd)	−3.83	−0.10	56 / 62	−0.15
dnf4x2	235	112	467	never	98	306 (wd)	−0.38	+1.07	252 / 228	+0.14
sym8	54	53	1553	never	2689	377 (wd)	−2.81	+0.03	100 / 54	+0.88
h2rf0	119	103	118	never	220	389 (wd)	−1.71	+0.21	112 / 117	−0.06
h2rf1	82	31	179	103	58	103 (wd)	−0.33	+1.40	80 / 79	+0.00
dlheld3: 10 targets; held out for an earlier version only; networks: nets, nets_cewd, nets_wd										
cnf3	189	126	600	never	164	361 (cewd)	−0.93	+0.58	164 / 196	−0.26
sym9	111	111	3267	never	7845	350 (cewd)	−1.65	+0.00	111 / 111	−0.00
dlist8	39	24	29	91	61	90 (wd)	−1.21	+0.69	30 / 55	−0.86
thrint	186	139	659	137	243	190 (cewd)	−0.03	+0.42	287 / 260	+0.14
par3or	71	63	177	never	2009	725 (wd)	−3.35	+0.18	60 / 69	−0.21
poly4	176	177	195	never	never	2745 (cewd)	−3.96	−0.01	3127 / 161	+4.28
mixmaj	114	96	289	never	368	326 (cewd)	−1.52	+0.24	89 / 103	−0.22
mux11	509	466	1486	never	573	1245 (wd)	−1.29	+0.13	850 / 637	+0.42
h3rf0	108	46	16	112	60	96 (wd)	+0.17	+1.22	30 / 120	−1.98
h3rf1	133	59	31	101	63	101 (wd)	+0.39	+1.17	31 / 135	−2.13
dlsymdev: 8 targets; development (v1.2, v1.3); networks: nets, nets_cewd, nets_wd										
sym7	56	55	1356	never	–	329 (cewd)	−2.56	+0.02	55 / 56	−0.02
ex3of8	123	118	6509	never	–	664 (cewd)	−2.43	+0.06	94 / 103	−0.14
band11	828	829	never	never	–	1836 (cewd)	−1.15	−0.00	889 / 775	+0.20
far9	98	71	1433	never	–	296 (wd)	−1.60	+0.47	86 / 95	−0.13
ssym9	111	104	2992	never	–	324 (cewd)	−1.55	+0.10	111 / 111	−0.00
symxpar	112	111	806	never	–	1410 (cewd)	−3.66	+0.00	112 / 112	−0.00
twosym	415	415	5326	never	–	1817 (cewd)	−2.13	−0.00	449 / 388	+0.21
thrsum	79	80	406	never	–	330 (cewd)	−2.07	−0.02	140 / 112	+0.32
dlheld4: 12 targets; exposed (v1.2 built after exposure); networks: nets, nets_cewd, nets_wd										
sym11	167	160	7778	never	never	374 (cewd)	−1.16	+0.06	187 / 193	−0.04
ex4of9	205	194	6580	never	never	411 (cewd)	−1.01	+0.08	193 / 202	−0.07
tail13	134	100	3992	never	never	420 (cewd)	−1.65	+0.42	148 / 157	−0.09
band10	440	432	never	never	never	3440 (cewd)	−2.97	+0.02	432 / 440	−0.02
ssym8	148	177	3873	never	8072	396 (cewd)	−1.42	−0.27	100 / 105	−0.07
symxor	406	405	2569	never	6909	946 (wd)	−1.22	+0.00	461 / 439	+0.07
symand	149	151	761	never	1307	370 (cewd)	−1.31	−0.02	100 / 126	−0.33
sumpar	181	164	316	never	never	1837 (cewd)	−3.34	+0.14	146 / 164	−0.17
majsym	102	90	978	never	2840	595 (cewd)	−2.54	+0.17	96 / 98	−0.03
dnf3x3	193	115	538	never	148	373 (wd)	−0.95	+0.74	212 / 191	+0.15
h4rf0	55	47	98	never	160	206 (wd)	−1.90	+0.22	55 / 55	+0.01
h4rf1	96	54	114	158	61	143 (wd)	−0.58	+0.82	29 / 103	−1.84

Table 15: Every toy target, part 2 of 3: dlheld, dlheld2, dlheld3, dlsymdev and dlheld4 [Numerical]. Columns as in Table 14.

target	DFG alone	v3	lowdeg	lin	hgb	nets_best (variant)	DFG alone vs nets_best	DFG alone vs v3	core / DFG alone (seeds 0, 1)	core vs DFG alone
dlheld5: 12 targets; exposed (v1.2 built after exposure); networks: nets, nets_cewd, nets_wd										
h5s0	201	205	never	never	never	803 (wd)	−2.00	−0.03	216 / 201	+0.11
h5s1	never	never	never	never	never	never	n/a	n/a	never / never	n/a
h5s2	598	1154	6456	never	10860	1850 (cewd)	−1.63	−0.95	678 / 367	+0.88
h5s3	1642	1757	never	never	never	6454 (cewd)	−1.97	−0.10	1752 / 1765	−0.01
h5s4	222	184	1766	never	1465	1801 (wd)	−3.02	+0.27	178 / 186	−0.06
h5s5	420	421	never	never	never	12254 (cewd)	−4.87	−0.00	449 / 419	+0.10
h5s6	597	573	never	never	never	3963 (cewd)	−2.73	+0.06	354 / 372	−0.07
h5s7	6186	6760	never	never	never	7641 (cewd)	−0.30	−0.13	1698 / 1704	−0.01
h5t0	109	105	402	never	2886	352 (cewd)	−1.68	+0.06	185 / 112	+0.72
h5t1	100	95	223	never	993	325 (cewd)	−1.70	+0.07	100 / 104	−0.06
h5rf0	215	203	318	never	2658	1547 (cewd)	−2.85	+0.08	164 / 293	−0.84
h5rf1	98	97	476	never	9281	2512 (cewd)	−4.68	+0.01	149 / 99	+0.60
dlheld6: 12 targets; development (v1.3); networks: nets, nets_cewd, nets_cewd0.3, nets_cewd1.0, nets_wd										
h6_00	never	never	never	never	never	12304 (cewd1.0)	+∞	n/a	never / never	n/a
h6_01	431	430	7470	never	never	1655 (cewd1.0)	−1.94	+0.00	428 / 431	−0.01
h6_02	384	389	666	never	never	996 (cewd0.3)	−1.37	−0.02	414 / 415	−0.01
h6_03	129	98	270	never	103	231 (cewd1.0)	−0.84	+0.40	188 / 125	+0.59
h6_04	79	29	259	96	59	96 (wd)	−0.27	+1.44	65 / 79	−0.29
h6_05	42	33	16	48	59	56 (wd)	−0.39	+0.38	27 / 39	−0.55
h6_06	347	349	never	never	never	1872 (cewd0.3)	−2.43	−0.01	387 / 396	−0.03
h6_07	448	447	5684	never	4492	1812 (cewd1.0)	−2.02	+0.00	495 / 388	+0.35
h6_08	479	449	2542	never	5940	1720 (cewd0.3)	−1.84	+0.09	522 / 584	−0.16
h6_09	391	380	7850	never	2706	744 (cewd0.3)	−0.93	+0.04	227 / 365	−0.69
h6_10	213	206	never	never	never	451 (cewd1.0)	−1.08	+0.05	649 / 215	+1.60
h6_11	192	184	2993	never	2264	357 (cewd1.0)	−0.90	+0.06	156 / 172	−0.14
dlfamdev: 12 targets; development (v1.3); networks: none run										
fd_00	138	75	239	–	–	–	–	+0.87	139 / 114	+0.29
fd_01	82	79	189	–	–	–	–	+0.05	86 / 81	+0.09
fd_02	61	66	31	–	–	–	–	−0.11	31 / 58	−0.93
fd_03	120	81	205	–	–	–	–	+0.56	116 / 119	−0.04
fd_04	170	171	never	–	–	–	–	−0.00	111 / 111	+0.00
fd_05	158	147	2555	–	–	–	–	+0.11	167 / 155	+0.11
fd_06	355	318	9543	–	–	–	–	+0.16	223 / 306	−0.46
fd_07	134	98	344	–	–	–	–	+0.45	105 / 123	−0.23
fd_08	51	29	89	–	–	–	–	+0.81	53 / 51	+0.04
fd_09	104	101	3210	–	–	–	–	+0.04	99 / 105	−0.08
fd_10	never	never	never	–	–	–	–	n/a	never / never	n/a
fd_11	114	123	61	–	–	–	–	−0.10	61 / 109	−0.84
dlheld7: 12 targets; held out (the only suite); networks: nets_cewd0.3, nets_cewd1.0, nets_wd										
f7_00	211	203	7061	never	6208	1545 (cewd0.3)	−2.87	+0.06	211 / 212	−0.01
f7_01	119	106	595	111	684	119 (cewd1.0)	−0.00	+0.17	195 / 156	+0.32
f7_02	190	106	435	never	120	337 (wd)	−0.83	+0.85	186 / 188	−0.01
f7_03	352	352	4205	never	2199	1599 (cewd1.0)	−2.18	−0.00	344 / 353	−0.04
f7_04	97	70	121	never	84	164 (wd)	−0.76	+0.47	99 / 99	+0.01
f7_05	never	never	never	never	never	never	n/a	n/a	never / never	n/a
f7_06	82	44	179	127	59	108 (wd)	−0.39	+0.90	82 / 81	+0.02
f7_07	167	174	1123	never	1373	415 (cewd1.0)	−1.32	−0.06	121 / 164	−0.44
f7_08	644	634	never	never	never	2932 (cewd1.0)	−2.19	+0.02	454 / 446	+0.03
f7_09	222	215	591	never	never	883 (cewd0.3)	−1.99	+0.05	355 / 222	+0.68
f7_10	84	82	336	87	114	119 (cewd1.0)	−0.50	+0.03	104 / 88	+0.23
f7_11	51	55	29	119	63	114 (wd)	−1.16	−0.12	30 / 63	−1.07

Table 16: Every toy target, part 3 of 3: dlheld5, dlheld6, dlfamdev and the held-out dlheld7 [Numerical]. Columns as in Table 14. dlfamdev has no network, lin or hgb runs.

reference	suites	targets	both reach	mean $\log_2$ ratio	median	clearly better	clearly worse	only the reference	only DFG	neither
DFG alone against										
`nets_best`	10	126	123	−1.87	−1.65	117	2	1	0	2
`nets`	9	114	112	−2.64	−2.65	110	0	0	0	2
`nets_wd`	10	126	123	−2.04	−1.90	119	1	0	0	3
`nets_cewd`	5	54	52	−1.88	−1.66	49	0	0	0	2
`nets_cewd0.3`	2	24	22	−1.37	−1.30	21	0	0	0	2
`nets_cewd1.0`	2	24	22	−1.37	−1.20	21	1	1	0	1
v3	11	138	134	+0.19	+0.07	6	42	0	0	4
v2	11	138	134	+0.11	+0.07	16	38	0	0	4
`lin`	10	126	30	−0.33	−0.25	109	3	0	93	3
`hgb`	8	97	75	−1.98	−1.95	76	11	0	19	3
`lowdeg`	11	138	122	−1.64	−1.35	112	16	0	12	4
against `nets_best` *on Part 10's held-out suites (*`dlheld`–`dlheld7`*)*										
DFG alone	7	77	74	−1.71	−1.40	69	2	1	0	2
v3	7	77	74	−1.87	−1.70	72	1	1	0	2

Table 17: DFG alone against each reference, with the per-target ratios pooled over the suites where both were run (not a mean of suite means) [Numerical]. Columns as in Table 13. "Only the reference": DFG alone never reaches 10% error and the reference does; "only DFG": the reverse. Bottom: DFG alone and v3 against `nets_best` on the same 77 targets of the seven suites that Part 10 pooled as its held-out set; for DFG v1.3, only `dlheld7` among them is held out. Source: `study/part11/RESULTS.json` (`q2_suites._pooled`, `._pooled_p10_heldout`).

higher in 25, 19 and 11. The stacker gives the DFG members no weight in 52, 47 and 30 fits, and their median share of the weight is 0.22–0.30. In those zero-weight fits the "member" is v3's last out-of-fold DFG fit, on 13–15, 28–30 and 57–58 rows, not a fit on all n rows. Over the 86, 91 and 106 fits that have a final DFG member, v3's error is lower by more than 0.02 in 29, 45 and 33, and higher in 16, 9 and 10. From $n = 256$ on, v3 is never worse than its DFG member by more than 0.02, and the DFG members' median share is 0.73–0.81 at $n = 256$–1024.

Reading [Heuristic]. At small n the portfolio's other members carry v3. On formulas and small thresholds, trees, nearest neighbours and logistic regression reach low error with fewer points than DFG needs before its tests admit features. The stacker can also lean on DFG's forced variant, which DFG alone keeps only when its paired test prefers it. Once DFG's features pass their tests, at $n = 256$–2048, its members take most of the weight (median share 0.64–0.81) and the two learners agree. Above $n = 2048$ few analysed fits remain (16, 10 and 7), many of them on targets that no member fits (the residue classes, h5s1 and, at 4096, h5s7) or on the dense halfspaces thr20 and thr30, on which logistic regression takes most of the weight; there the median DFG share falls from 0.51 to 0.15. Section 6 compares DFG alone's fitted models with v3's DFG member.

The reverse: what only the full-data fit finds [Numerical, then Heuristic]. On three of the six targets where DFG alone needs clearly less data, the files show what happens: DFG's fit on all n points finds a structure that fits on fewer points miss.

- *par3 and rf11.* rf11 is a random formula that reduces to the 3-parity $-x_2x_3x_4$. DFG alone has test error 0 at $n = 32$ in every seed, against v3's 0.47 and 0.50. In v3's seed-0 fits at $n = 32$ the stacker gave DFG no weight, and its last out-of-fold DFG fit (29 rows) had test error 0.51 on par3 and 0.50 on rf11.

suite	target	definition	DFG alone	nets_best (variant)	$\log_2$ ratio	v3	lowdeg	lin	hgb
DFG alone clearly worse									
dlheld6	h6_00	[((s+13)/2) mod 3 ≠ 1], s = x1+x2+x3+x4+x5+x6+x7+x8 −x9−x10−x11−x12+x13	never	12304 (cewd1.0)	+∞	never	never	never	never
dlheld3	h3rf1	random 9-gate formula on 7 inputs	133	101 (wd)	+0.39	59	31	101	63
ties ($\lvert\log_2$ *ratio*$\rvert \le 0.25$*)*									
dlheld3	h3rf0	random 8-gate formula on 7 inputs	108	96 (wd)	+0.17	46	16	112	60
dlheld7	f7_01	[s ≤ 1], s = x1+...+x13	119	119 (cewd1.0)	−0.00	106	595	111	684
dlheld3	thrint	sign(3x1 + 2x2 + 2x3 + x4 + ... + x9)	186	190 (cewd)	−0.03	139	659	137	243
dlheld	inter2	AND of two Gaussian halfspaces on x1..x10	441	454 (wd)	−0.04	346	2204	never	416
designed	and3	AND(x1,x2,x3)	60	70 (wd)	−0.21	25	158	68	–
no learner reaches 10% error									
dlheld5	h5s1	$[\sum_{i\le 9} s_i x_i$ in [−9, −5, −1, 3, 9]], s = [−1, 1, 1, 1, −1, 1, −1, −1, −1] xor chi(10, 11)	never	never	n/a	never	never	never	never
dlheld7	f7_05	[((s+14)/2) mod 3 ≠ 0], s = x1+...+x14	never	never	n/a	never	never	never	never
DFG alone never learns it; no network runs									
dlfamdev	fd_10	[((s+10)/2) mod 3 = 0], s = x1+x2+x3+x4−x5−x6+x7+x8+x9+x10	never	–	–	never	never	–	–

Table 18: Every target on which DFG alone is not clearly better than nets_best (9 of the 126 with network runs), and the one other target it never learns (dlfamdev has no network runs) [Numerical]. n^* as in Table 4. In brackets, the variant attaining nets_best: wd, cewd and cewd1.0 stand for nets_wd, nets_cewd and nets_cewd1.0. Random formulas are named by their size; their gates are in study/boolfuncs.py. Source: study/part11/RESULTS.json (q2_suites, q3_lowdeg) via paper_dfg/make_tables.py.

- *par5.* DFG alone has error 0 from $n = 64$ on. v3's seed-mean errors at $n = 64$–512 are 0.36, 0.17, 0.17 and 0.35, and it reaches 10% only at 839. In v3's seed-0 fit at $n = 64$ the DFG member alone had error 0, but it held 0.30 of the weight and v3's error was 0.076.
- *The other three* are smaller or unexplained here: h5s2 (598 against 1154), rf00 (32 against 40) and ssym8 (148 against 177).

Reading [Heuristic]. Two rules of the frozen code (study/part10/hyb_portfolio3/dfg13.py) make the difference.

- *Minimum rows.* DFG's degree-3 and degree-4 searches run only on at least 32 rows, and its degree-5 and degree-6 searches only on at least 64. At $n = 32$ the full fit therefore searches degree 3, while v3's 10-fold out-of-fold fits, on 28–29 rows, do not.
- *Budget share.* At $n \le 512$ each of v3's out-of-fold DFG fits gets 1.75% or 3.5% of the budget. At $d = 30$ this leaves 24 or 48 rows for the degree-5 search, fewer than the 64 it needs, so those fits skip it. v3's final DFG fits and DFG alone's have 343–687 rows for it.

DFG alone uses its full-data fit whatever its folds show, whereas a stacker weights a member by its out-of-fold predictions. So a structure that only the full-data fit can find counts for little in v3. The parities are the cleanest case, and the effect is small overall: 6 of 138 targets. (The

subsample sizes are computed from the code by `paper_dfg/make_tables.py` and recorded in `paper_dfg/tables/numbers.json`.)

E.4 The low-degree algorithm

`lowdeg` is the thresholded low-degree algorithm [26] of Part 6's feasibility check, run unchanged (Section 3.2). It estimates the label's Fourier coefficient $\hat{c}_S$ on every character of degree $k \le 4$, keeps S when $|\hat{c}_S| > \sqrt{2\ln(\binom{d}{k}/\delta)/n}$ with $\delta = 0.1$, tests products of kept characters in the same way, and predicts $\mathrm{sign}(\hat{c}_\emptyset + \sum_{\text{kept}} \hat{c}_S \chi_S)$. It therefore has DFG's character dictionary up to degree 4 and the same form of union-bound test. The differences in the test are small: DFG's log term is $\ln(4KN/\delta)$ with $K = 4$, about 2.8 nats more, and DFG tests the residual against the residual's s.d. instead of the labels. `lowdeg` has none of the rest of DFG:

- no residual and no greedy rounds;
- no sum blocks, no linear or spectral blocks, no extensions, no local search and no search of degrees 5–6;
- no refitted readout (least squares with logistic refinement), no kernel background and no small-n cross-validation.

Table 22 compares it with DFG alone and with `nets_best`; Figure 2b shows it target by target.

E.5 Input dimension

Table 23 gives n^* and the fitted exponents of Figure 3 [Numerical].

Close to Part 6's DFG in median [Numerical]. Against Part 6's DFG (v1 at $d \le 30$, v1.1 at $d \ge 60$) the median $\log_2$ difference is between -0.01 and 0.00 at every d, but 6 of the 25 (target, d) cells differ by more than 0.25. v1.3 is faster on maj9 at $d \ge 30$ (up to 104 against 439 at $d = 120$), and slower on par3 at $d = 30$ and par4 at $d = 15$ and 30, where Part 6 saw the inputs in their natural order.

E.6 Tabular data

The datasets. DFG is a learner for ± 1 inputs and two classes. Of the study's 26 PMLB datasets [30], two have only binary or categorical columns and two classes, and DFG runs on these two only: kr_vs_kp (3196 rows; 35 binary columns and 1 categorical) and mushroom (8124 rows; 16 categorical columns and 6 binary). Binary columns become ± 1, and categorical ones a one-hot code over the values seen in training, written as ± 1. This is v3's view of such data (`study/part11/dfg_real.py`). It gives 38 inputs on kr_vs_kp, and 87–112 on mushroom, depending on the values in the training set. Each dataset has a fixed test set, a random 25% of its rows (799 and 2031 rows). For each of 3 seeds the remaining pool (2397 and 6093 rows) is permuted, and the training set of size n is its first n rows. The grid is $n \in \{100, 300, 1000, 3000\}$ up to the pool size, plus the whole pool. DFG runs neither on images nor on real-valued columns.

References on tabular data. The MLP references are those of Parts 7–10, ReLU MLPs with softmax cross-entropy:

- `mlp`: 256-256, Adam, learning rate and stopping point chosen on validation;

(a) per suite

suite	targets	DFG alone lower / higher (equal; neither learns)	clearly lower / higher ($\lvert \log_2 \text{ratio} \rvert > 0.25$)	mean $\log_2$ ratio (both reach)
`designed`	21	4 / 16 (1; 0)	1 / 9	+0.25
`random`	20	5 / 15 (0; 0)	2 / 7	+0.22
`dlheld`	12	3 / 9 (0; 0)	1 / 4	−0.18
`dlheld2`	7	1 / 6 (0; 0)	0 / 2	+0.40
`dlheld3`	10	1 / 9 (0; 0)	0 / 5	+0.46
`dlsymdev`	8	3 / 5 (0; 0)	0 / 1	+0.08
`dlheld4`	12	2 / 10 (0; 0)	1 / 3	+0.20
`dlheld5`	12	5 / 6 (0; 1)	1 / 1	−0.06
`dlheld6`	12	2 / 9 (0; 1)	0 / 3	+0.22
`dlfamdev`	12	3 / 8 (0; 1)	0 / 4	+0.26
`dlheld7`	12	3 / 8 (0; 1)	0 / 3	+0.21
all	138	32 / 101 (1; 4)	6 / 42	+0.19

(b) by v3's n^*

v3's n^*	targets	DFG alone lower / higher	clearly lower / higher	mean (median)
≤ 64	45	9 / 35	3 / 24	+0.39 (+0.26)
64–256	67	12 / 55	1 / 17	+0.17 (+0.06)
256–1024	19	8 / 11	1 / 1	−0.16 (+0.00)
> 1024	3	3 / 0	1 / 0	−0.39 (−0.13)
never	4	0 / 0	0 / 0	–

Table 19: DFG alone against v3 on the 138 toy targets [Numerical]. (a) Per suite: the targets on which DFG alone's n^* is lower / higher than v3's (in brackets: equal, and learned by neither), those on which it is clearly lower / higher, and the mean $\log_2$ ratio over the targets both learn. (b) The same by v3's n^*; the 4 targets that neither learns are under "never". Source: `study/part11/RESULTS.json` (`q2_suites.dfg_vs_v3`, `.dfg_vs_v3_by_v3_nstar`).

- `mlp_reg`: 256-256, Adam at learning rate 10^{-3}, the final checkpoint, and an L2 penalty $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ chosen on validation, then retrained on all n;
- `mlp_strong`: quantile-transformed inputs, AdamW with cosine decay, weight decay chosen on validation, dropout 0.1, an ensemble of 3. It was run on mushroom, not on kr_vs_kp.

`mlp_best` is the per-cell minimum of the seed-mean test error over the MLP variants run on that cell: `mlp` and `mlp_reg` on kr_vs_kp; on mushroom also `mlp_strong`, its single network `mlp_strong1`, and `mlp_plus`, a variant with a wider learning-rate grid ($n \leq 3000$). It is a hindsight choice that favours the MLPs. Part 9's wider MLP ensembles, which the reference `mlp_best+` of earlier parts adds, ran on neither dataset, so `mlp_best+` equals `mlp_best` here and is not shown. `hgb`, v3, v2 and v1 complete the references.

Metrics. A cell is a (dataset, n) pair. Its verdict compares the seed-mean test errors of two learners. They are *better* or *worse* when the errors differ by more than 10% relative and by more than 2 standard errors, and otherwise a *tie*. The s.e. comes from the seed spread of both learners and is at least the test set's binomial noise. Mean $\log_2$ error ratios floor errors at 0.002.

Budget. On tabular data DFG keeps its internal budget (Section 2.4), in units of one toy-network run at its encoded width. Its FLOPs are reported against the budget that Parts 8–10 used for real data, three training runs of the baseline `mlp`, which is the baseline's own cost including its learning-rate search (Part 7 allowed one run).

n	16	32	64	128	256	512	1024	2048	4096	8192	16384
(a) DFG alone against v3: the result files, every seed, 138 targets											
v3 lower by > 0.02	63	64	67	38	17	5	0	0	0	0	0
DFG alone lower by > 0.02	26	22	8	4	3	2	2	2	3	0	0
mean difference (points)	+2.3	+2.2	+2.8	+1.7	+0.4	−0.2	−0.1	−0.1	−0.2	−0.0	+0.0
(b) inside v3: Part 10's analysis fits of v3, seed 0											
fits analysed	138	138	136	134	122	87	54	33	16	10	7
v3 lower than DFG member	57	68	53	32	16	8	1	0	0	0	0
DFG member lower than v3	25	19	11	5	0	0	0	0	0	0	0
DFG members' weight 0	52	47	30	17	8	2	4	3	4	3	2
median DFG weight share	0.24	0.22	0.30	0.56	0.73	0.79	0.81	0.64	0.51	0.19	0.15
fits with a final DFG member	86	91	106	117	114	85	50	30	12	7	5
v3 lower than that member	29	45	33	23	12	7	0	0	0	0	0
that member lower than v3	16	9	10	5	0	0	0	0	0	0	0

Table 20: DFG alone and v3 by training-set size [Numerical]. (a) Over the 138 toy targets, with the same seeds, training sets and column orders for both: the targets on which v3's seed-mean test error is lower than DFG alone's by more than 0.02, the reverse, and the mean difference of the errors (DFG alone minus v3, in percentage points). Source: `study/part11/results/toy__dfg13-perm__<suite>.json` and `study/hybrid/results/toy__p10_sum-perm__<suite>.json`. (b) Inside v3: Part 10's analysis refits of v3, seed 0, one per target and n, whose test error equals v3's evaluation cell. Each target's grid stops after two consecutive n with error ≤ 0.005, so there are fewer fits at large n. Rows: the fits in which v3's error is lower than that of its own DFG member alone by more than 0.02; the reverse; the fits in which the stacker gave the DFG members (default and forced) no weight; and the DFG members' median share of the stacking weight. Where DFG has no weight, the member's error is that of its last out-of-fold fit. The last three rows count the first two again over the fits in which v3 kept a final DFG member. Source: `study/part10/interp/results/toy_final/v3/` (fields `err`, `err_dfg`, `w_dfg`; defined in `study/part10/interp/toy_structure.py`).

DFG v1.3 needs ±1 inputs and two classes. Of the study's 26 tabular datasets only two qualify, kr_vs_kp and mushroom. Their binary and categorical columns are one-hot encoded as ±1, as in v3's DFG view (Section 3.1; `study/part11/dfg_real.py`), and DFG runs unchanged with its own budget. kr_vs_kp is a development set. Mushroom was held out in Part 7 and never used to develop DFG, but v1–v3 ran DFG on it and their results were known (Table 12). Table 24 gives the cells, verdicts and compute.

Results on the two datasets [Numerical].

- *Against the best MLP variant per cell:* no cell is better, 8 of 9 are ties and 1 is worse: kr_vs_kp at $n = 100$, 14.06% against `mlp_reg`'s 7.80%. The mean $\log_2$ error ratio is −0.278, from cells where both errors are small and the cell rule sees no difference.
- *Against single references:* against the baseline `mlp`, 3 cells are better (kr_vs_kp at $n = 300$ and 1000, mushroom at 300), 5 are ties and 1 is worse ($n = 100$ on kr_vs_kp), with a mean of −0.482. Against `hgb` all 9 cells are ties (mean −0.022).
- *Against v3:* 8 ties and 1 worse cell (kr_vs_kp at $n = 100$: 14.06% against 6.88%), with a mean of +0.213: +0.515 on kr_vs_kp and −0.030 on mushroom.
- *Compute:* no fit uses more than 0.144 of the standard-task budget. The seed-mean CPU time per fit is at most 13.4 seconds per cell, 0.19–0.66 of v3's on the same cell.

suite	target	DFG alone	v3	$\log_2$ ratio	hgb	lin	suite	target	DFG alone	v3	$\log_2$ ratio	hgb	lin
dlheld6	h6_04	79	29	+1.44	59	96	random	rf03	91	62	+0.54	139	never
dlheld2	h2rf1	82	31	+1.40	58	103	designed	dict	22	16	+0.48	–	27
designed	and3	60	25	+1.30	–	68	dlheld7	f7_04	97	70	+0.47	84	never
random	rf15	51	22	+1.23	53	67	dlsymdev	far9	98	71	+0.47	–	never
dlheld3	h3rf0	108	46	+1.22	60	112	dlfamdev	fd_07	134	98	+0.45	–	–
random	rf08	50	21	+1.22	53	69	dlheld3	thrint	186	139	+0.42	243	137
dlheld3	h3rf1	133	59	+1.17	63	101	dlheld4	tail13	134	100	+0.42	never	never
dlheld2	dnf4x2	235	112	+1.07	98	never	dlheld6	h6_03	129	98	+0.40	103	never
random	rf06	31	16	+0.97	58	37	dlheld6	h6_05	42	33	+0.38	59	48
designed	dlist5	61	32	+0.93	–	85	designed	andxor	52	40	+0.37	–	never
dlheld7	f7_06	82	44	+0.90	59	127	dlheld	hrf0	80	62	+0.36	63	191
dlfamdev	fd_00	138	75	+0.87	–	–	dlheld	inter2	441	346	+0.35	416	never
dlheld7	f7_02	190	106	+0.85	120	never	dlheld	maj15	130	105	+0.31	1278	110
designed	rand2	28	16	+0.83	–	50	random	rf16	44	36	+0.27	214	never
designed	maj3	56	31	+0.82	–	56	dlheld5	h5s4	222	184	+0.27	1465	never
dlheld4	h4rf1	96	54	+0.82	61	158	designed	thr10	146	121	+0.27	–	113
dlfamdev	fd_08	51	29	+0.81	–	–	random	rf10	43	36	+0.26	53	66
dlheld4	dnf3x3	193	115	+0.74	148	never	random	rf05	75	62	+0.26	250	never
dlheld3	dlist8	39	24	+0.69	61	91	dlheld4	ssym8	148	177	−0.27	8072	never
dlheld	mux6	197	123	+0.67	188	never	random	rf00	32	40	−0.33	60	83
designed	rand0	25	16	+0.62	–	never	dlheld5	h5s2	598	1154	−0.95	10860	never
designed	dnf3	178	118	+0.60	–	never	designed	par3	28	55	−0.99	–	never
dlheld3	cnf3	189	126	+0.58	164	never	random	rf11	28	56	−1.00	416	never
dlfamdev	fd_03	120	81	+0.56	–	–	dlheld	par5	56	839	−3.91	never	never

Table 21: The 48 targets on which DFG alone's and v3's n^* differ by more than 0.25 in $\log_2$, sorted by the ratio of DFG alone to v3 (positive: v3 needs less data) [Numerical]. `hgb` and `lin` (natural column order) are shown where they were run; "–": not run on the suite. Source: `study/part11/RESULTS.json` (`q2_suites.dfg_vs_v3`; `hgb` and `lin` from the per-target ratios there, checked against `study/hybrid/results/`).

On these two Boolean tables, then, DFG alone is level with gradient boosting in all 9 cells, and with the regularized MLPs and v3 except at the smallest n of kr_vs_kp, where the MLPs and v3 are clearly better; it is nowhere clearly better than the best MLP. Nine cells, with errors near 0 from $n = 1000$ on, resolve little.

Real-valued data: Part 7 [Numerical, then Heuristic]. On real-valued, multi-class data DFG has been run only through three adaptations of DFG v1.1, written in Part 7 [10]. dfg_bits uses thermometer bits and their characters, dfg_gates ANDs of threshold literals, and dfg_dirs thresholded directions. Each was frozen on 5 development datasets before 13 held-out ones were run (`study/standard_tasks/STANDARD-TASKS.md`; tables in `study/standard_tasks/RESULTS.md`).

- *Tabular, held out (11 datasets, 59 cells):* their mean $\log_2$ error ratio to the baseline MLP was +0.04 to +0.10, with 6–9 cells better and 10–15 worse. Against an L2-regularized MLP at $n \leq 3000$ (44 cells) it was +0.16 to +0.23, with 0–4 cells better and 11 worse. On the same cells default gradient boosting had −0.07 against the baseline MLP and +0.11 against the regularized one.
- *Images:* on MNIST and Fashion-MNIST (14 cells) they were at +0.26 to +0.41 against the baseline MLP, and worse in 7 of the 14 cells each.

suite	targets	`lowdeg` vs DFG alone	`lowdeg` vs `nets_best`	`lowdeg` never	targets `lowdeg` never learns (DFG alone's n^*)
`designed`	21	+0.92 (3/16) [21]	−1.25 (13/6) [21]	0	–
`random`	20	+0.71 (4/15) [20]	−1.33 (16/2) [20]	0	–
`dlheld`	12	+1.53 (2/9) [11]	−0.26 (5/6) [11]	1	par5 (56)
`dlheld2`	7	+2.00 (0/6) [7]	+0.40 (2/5) [7]	0	–
`dlheld3`	10	+0.74 (3/6) [10]	−0.60 (5/4) [10]	0	–
`dlsymdev`	8	+3.98 (0/8) [7]	+1.69 (1/7) [7]	1	band11 (828)
`dlheld4`	12	+2.89 (0/12) [11]	+1.34 (3/9) [11]	1	band10 (440)
`dlheld5`	12	+2.05 (0/11) [6]	−0.54 (3/6) [6]	6	h5s0 (201), h5s1 (never), h5s3 (1642), h5s5 (420), h5s6 (597), h5s7 (6186)
`dlheld6`	12	+2.29 (1/10) [9]	+1.13 (2/9) [9]	3	h6_00 (never), h6_06 (347), h6_10 (213)
`dlfamdev`	12	+1.67 (2/9) [10]	–	2	fd_04 (170), fd_10 (never)
`dlheld7`	12	+1.90 (1/10) [10]	+0.70 (3/8) [10]	2	f7_05 (never), f7_08 (644)
pooled	138	+1.64 (16/112) [122]	−0.16 (53/62) [112]	16	

Table 22: The low-degree algorithm `lowdeg` on every suite [Numerical]. Entries as in Table 5, but the ratios are those of `lowdeg`'s n^* to the reference's (positive: `lowdeg` needs more data), so "better" and "worse" are `lowdeg`'s. Last column: the targets that `lowdeg` never brings to 10% test error by $n = 16384$, with DFG alone's n^* in brackets. Source: `study/part11/RESULTS.json` (`q3_lowdeg`); `lowdeg`'s runs: `study/part11/results/toy__lowdeg-perm__<suite>.json`.

These are adaptations of v1.1, frozen before the sum blocks (v1.2) and the weighted blocks (v1.3). Each replaced DFG's ±1 dictionary with its own, and two also loosened the test constant. They are evidence about DFG's recipe, not about DFG v1.3, which has not been run on real-valued data. Part 7's reading [Heuristic] was that the recipe fails there because the useful features of dense real-valued signal are not in any small enumerable dictionary, and that the tests can stop growth early on many weak, correlated effects.

E.7 Compute

CPU time is the process time of a fit. It is reported separately, because equal FLOPs are far from equal CPU time. DFG alone's was measured while four worker processes shared a four-CPU machine at low priority, and v3's in Part 10 under that run's own load, so CPU comparisons between them are rough. The networks' result files hold no timing, so the compute comparison with them is in FLOPs only.

Compute results [Numerical] (Table 25).

- *FLOPs.* Over all 4730 fits (4334 at $d = 30$, 396 at $d \neq 30$), the largest uses 0.519 of one network run (h5s5 on `dlheld5` at $n = 16384$), and none exceeds 0.75. The largest fit per suite uses 0.154 (`random`) to 0.519 (`dlheld5`) of a run, always at $n = 16384$; at $d = 15$, 60 and 120

	DFG v1.3 alone: n^* at $d =$				networks (`nets`): n^* at $d =$				
target	15	30	60	120	15	30	60	120	exponent DFG / networks
maj9	79	81	167	104	123	263	625	752	0.22 / 0.91
majpar	52	61	63	–	306	1380	3466	–	0.14 / 1.75*
mux	58	63	149	102	364	1156	2848	6073	0.36 / 1.35
par2	16	16	16	28	100	200	427	842	0.24 / 1.03
par3	28	28	28	56	241	843	3410	14029	0.30 / 1.96
par4	28	52	56	–	508	2728	14255	–	0.50 / 2.40*
stair5	109	111	110	–	433	1514	3487	–	0.01 / 1.50*

	DFG v1.3 against the networks: median $\log_2$ ratio (factor)	DFG v1.3 against Part 6's DFG: median	max FLOPs per fit / one network run	seeds (DFG; nets)
$d = 15$ (7 targets)	-2.64 (6.2×)	$+0.00$	0.07	2; 2
$d = 30$ (7 targets)	-4.19 (18.3×)	-0.01	0.15	3; 3
$d = 60$ (7 targets)	-4.99 (31.7×)	$+0.00$	0.18	2; 2
$d = 120$ (4 targets)	-5.41 (42.5×)	-0.00	0.18	2; 2

Table 23: n^* against the input dimension d on the `designed` targets that the networks were trained on at $d = 15$, 60 and 120 [Numerical]. DFG v1.3 alone (inputs permuted per fit) and the networks `nets` (square loss, the only variant run at every d); "–": not run. Seeds 0, 1 at $d \neq 30$ and 0, 1, 2 at $d = 30$ for both. The networks were trained for 20000 steps, 40000 at $d = 120$. Their grid extends to 65536 at $d = 60$ and is 64–8192 (maj9, par2), 256–32768 (mux) and 512–65536 (par3) at $d = 120$; n^* on the full grid equals the 16–16384 value everywhere. Exponent: the least-squares slope of $\log n^*$ on $\log d$ over the d at which both reach 10% error (*: $d = 15$–60 only). Bottom, per d: the median $\log_2$ ratio of DFG v1.3 to the networks (factor $2^{-\text{median}}$), the median $\log_2$ ratio of DFG v1.3 to Part 6's DFG (v1 at $d \leq 30$, v1.1 at $d \geq 60$, natural column order), DFG v1.3's largest FLOPs per fit on these targets, and the number of seeds. Source: `study/part11/RESULTS.json` (`q4_dscale`); DFG's runs: `study/part11/results/toy__dfg13-perm__designed_d<d>.json` and `study/part11/results/toy__dfg13-perm__designed.json`.

the largest use 0.067, 0.177 and 0.179. The median fit uses less than 0.001 of a run at $n \leq 32$, 0.025–0.055 at $n = 64$–512 and 0.149 at $n = 16384$.

- *CPU time.* The median fit takes 1.1–2.7 seconds per suite at $d = 30$ (0.14–3.19 seconds including $d = 15$–120), and the longest takes 17.4 seconds (`dlheld3`). By n the median is largest at $n = 64$–512 (4.2–7.3 seconds) and falls to 1.2–2.4 seconds at $n \geq 1024$.
- *Against v3* on the same fits, the median of the per-fit CPU ratio is 0.06–0.92 at every n. On these rough measurements DFG alone is faster than the portfolio it was part of.
- *`lowdeg`* uses at most 0.095 of a network run per fit and a median of 0.02–0.03 seconds.

Reading [Heuristic]. FLOPs grow with n, but CPU time follows the cross-validation. At $n = 64$–512 DFG fits $2F$ fold models in addition to the final one ($F = 10$ at $n \leq 64$, 5 above), on folds large enough for the degree-3 and degree-4 searches, and these many small fits cost more time per FLOP. At $n \leq 32$ the folds are too small for those searches, and above $n = 512$ DFG fits only the final model. Each network fit spends one run by definition, and `nets_best` is a hindsight choice among 2 to 5 such runs per target. "No more FLOPs" therefore holds with a wide margin: the largest DFG fit uses about half of one network run, and the median fit at the largest n about a seventh.

Where the compute goes. An instrumented copy of `dfg13.py` cuts each fit into regions; its 64 inserted lines only read DFG's FLOP counter, the process clock and statistics DFG has just

		test error (%)							DFG alone		
dataset	n	DFG alone	`mlp`	`mlp_reg`	`mlp_strong`	`mlp_best`	`hgb`	v3	FLOPs / budget	CPU s per fit	v3 CPU s
kr_vs_kp	100	14.06	9.30*	7.80*	–	7.80*	10.76	6.88*	0.013	5.7	14.6
	300	3.71	6.13†	5.72	–	5.72	4.26	3.13	0.038	10.2	22.9
	1000	1.29	3.13†	1.67	–	1.67	1.21	0.75	0.030	3.6	15.8
	2397	0.29	0.50	0.46	–	0.46	0.13	0.29	0.062	7.8	40.6
mushroom	100	1.97	2.99	3.20	3.20	2.99	3.79	2.12	0.024	0.8	2.4
	300	0.53	1.35†	1.23	1.23	1.13	0.56	0.54	0.050	2.2	4.2
	1000	0.15	0.23	0.16	0.16	0.16	0.16	0.16	0.087	4.1	6.3
	3000	0.00	0.16	0.00	0.00	0.00	0.05	0.00	0.144	11.1	27.3
	6093	0.00	0.07	0.00	0.00	0.00	0.00	0.00	0.141	13.4	30.1

	kr_vs_kp		mushroom		both	
DFG alone against	cells (better / tie / worse)	mean $\log_2$ error ratio	cells (better / tie / worse)	mean $\log_2$ error ratio	cells (better / tie / worse)	mean $\log_2$ error ratio
`mlp`	4 (2/1/1)	−0.545	5 (1/4/0)	−0.432	9 (3/5/1)	−0.482
`mlp_reg`	4 (0/3/1)	−0.198	5 (0/5/0)	−0.386	9 (0/8/1)	−0.302
`mlp_strong`	–	–	5 (0/5/0)	−0.386	5 (0/5/0)	−0.386
`mlp_best`	4 (0/3/1)	−0.198	5 (0/5/0)	−0.342	9 (0/8/1)	−0.278
`hgb`	4 (0/4/0)	+0.208	5 (0/5/0)	−0.206	9 (0/9/0)	−0.022
v3	4 (0/3/1)	+0.515	5 (0/5/0)	−0.030	9 (0/8/1)	+0.213
v2	4 (0/3/1)	+0.368	5 (0/5/0)	−0.030	9 (0/8/1)	+0.147
v1	4 (0/3/1)	+0.540	5 (0/5/0)	−0.030	9 (0/8/1)	+0.223

Table 24: DFG alone on the two tabular datasets it can run on [Numerical]. (a) Test error (%, seed mean of 3) per cell, against the MLP references and `hgb` and v3. The references are `mlp`; `mlp_reg`, L2-regularized; `mlp_strong`, run on mushroom only; and `mlp_best`, the per-cell best MLP variant by test error, chosen with hindsight. A reference's entry is marked * where DFG alone is clearly worse than it by the cell rule (errors differing by more than 10% relative and by more than 2 s.e.) and † where DFG alone is clearly better. Right of (a): DFG alone's FLOPs as a fraction of the standard-task budget (three MLP training runs; maximum over seeds) and its CPU seconds per fit, and v3's CPU seconds on the same cells (seed means). (b) Per reference, the cells (better / tie / worse for DFG alone) and the mean $\log_2$ ratio of DFG alone's error to the reference's (errors floored at 0.002), per dataset and over the cells of both; v1 and v2 are the portfolios of Parts 8 and 9, and `mlp_best+`, which adds Part 9's wider ensembles, equals `mlp_best` here and is not shown. Source: `study/part11/RESULTS.json` (`q6_tabular`); DFG alone's runs: `study/standard_tasks/results/dfg13__kr_vs_kp.json` and `study/standard_tasks/results/dfg13__mushroom.json`.

computed. The profile covers 35 cells, seed 0. Most are par4, maj9, mux, stair5, f7_07, f7_08, f7_05 and labels independent of the inputs, at $n = 64$, 512, 4096 and 16384; the other three are the largest-FLOP fits of three suites at $n = 16384$. Every profiled fit equals the frozen learner's in test error and FLOP count (`study/part11/interp/calib/CALIB.md`, part B, and its independent verification `study/part11/interp/calib/VERIFY_CALIB.md`, whose corrections are used here). The totals are above.

- *FLOPs are fixed by the budgeted searches* [Proved, then Numerical]. At $d = 30$ one round of the global search over degrees 1–4 on all n rows costs exactly 768,390 FLOPs per training row (Proposition 2.6). Degrees 1–3 (53,970 per row) run again in every later round, and the budget shares would bind only above 29,627 rows. Round 0 adds the degree-5 and degree-6 searches on $\min(n, 1374)$ and $\min(n, 147)$ rows at $n > 512$ (687 and 73 at $n \le 512$, where the final fit has half the budget), each 0.030 of a network run from $n = 1374$ on. So before anything else

a fit costs 0.057 of a network run at $n = 1024$, 0.079 at 4096 and 0.137 at 16384, whatever the target. At $n \leq 512$ the final fit has half the budget (0.015 of a run at $n = 64$, 0.028 at 512), and the cross-validation's fold fits share the other half, of which they spend little. The formula matches the recorded search FLOPs of every profiled call exactly.

- *By component* [Numerical], over the 8 profiled cells at each n. At $n = 64$ the degree-6 search, which there runs on all 64 rows, is 50–64% of the FLOPs and the fold fits 27–43%. At $n = 16384$ the degree-4 search is 43–52%, and degrees 5 and 6 are 18–22% each. The largest fits are those with many features. The three largest profiled fits cost 0.32–0.47 of a network run, 41–47% of it in the logistic readout. The largest fit in the result files (h5s5, $n = 16384$, seed 2; 0.519 of a run, 250 features) spends 37% in the readout, by the verifier's refit.
- *CPU time follows the sum-block search instead* [Numerical]. In the final fits the block search runs at 0.75 GFLOP/s, against 16.5–17.6 for the character searches of degrees 3–5 and 32–33 for least squares and the readout; inside the fold fits it runs at 0.27. At $n = 64$ it takes 94–95% of the fold fits' CPU time for 42–46% of their FLOPs; par4 is left out here, because its parity passes in round 0 and no block search runs. In the final fit at $n \geq 4096$ it takes up to 67% of the CPU time (f7_08 at 4096) for at most 5.7% of the FLOPs. So the slowest fits in the result files are at $n = 64$–512 (suite medians 2.2–9.2 seconds at $d \geq 30$, the longest 17.4 seconds), where the FLOP count is a small fraction of its large-n value: there every one of the cross-validation's small fits runs its own block search.
- *The budget never binds* [Numerical]. DFG's cap stopped none of the 6786 recorded fit calls; a round starts only while a call has spent less than 0.75 of its budget. The largest call spent 0.471 of its budget (a final fit of fd_08), and the largest fold fit 0.313. In the result files no fit exceeds 0.519 of a network run.

The CPU times come from a 4-core machine shared with 3–4 evaluation workers, and repeats of a cell vary by up to 2.6 times (median 1.44). The CPU shares are medians of three repeats and are rough; the FLOP shares are exact.

(a) per suite: DFG alone

suite	d	fits	max FLOPs per fit / one network run (target, n)	fits above 0.75 run	CPU s per fit: median / max
`designed`	30	693	0.372 (thr10, 16384)	0	1.53 / 15.5
`random`	30	440	0.154 (rf06, 16384)	0	1.12 / 13.6
`dlheld`	30	396	0.420 (inter2, 16384)	0	2.43 / 13.2
`dlheld2`	30	231	0.307 (thr20, 16384)	0	2.07 / 12.0
`dlheld3`	30	330	0.275 (dlist8, 16384)	0	2.33 / 17.4
`dlsymdev`	30	264	0.163 (twosym, 16384)	0	2.12 / 11.6
`dlheld4`	30	396	0.162 (symand, 16384)	0	2.03 / 14.7
`dlheld5`	30	396	0.519 (h5s5, 16384)	0	2.70 / 15.5
`dlheld6`	30	396	0.420 (h6_09, 16384)	0	2.66 / 13.5
`dlfamdev`	30	396	0.471 (fd_08, 16384)	0	2.40 / 14.7
`dlheld7`	30	396	0.380 (f7_04, 16384)	0	2.36 / 16.6
`designed`, $d = 15$	15	154	0.067 (maj9, 16384)	0	0.14 / 4.5
`designed`, $d = 60$	60	154	0.177 (maj9, 16384)	0	1.51 / 13.1
`designed`, $d = 120$	120	88	0.179 (maj9, 16384)	0	3.19 / 16.2
`lowdeg`, 11 suites	30	4334	0.095	0	0.02–0.03 / 1.7

n	16	32	64	128	256	512	1024	2048	4096	8192	16384
DFG alone: FLOPs per fit / one network run											
median	0.000	0.000	0.025	0.029	0.036	0.055	0.058	0.072	0.082	0.104	0.149
max	0.000	0.001	0.033	0.039	0.050	0.096	0.077	0.137	0.200	0.312	0.519
CPU s per fit: median (max)											
DFG alone	0.02	0.16	7.29	5.27	4.24	4.26	1.20	1.57	1.68	1.85	2.41
	(0.04)	(0.91)	(16.6)	(11.8)	(10.9)	(17.4)	(5.1)	(9.6)	(11.3)	(13.0)	(15.5)
v3	0.31	0.58	8.22	10.71	8.44	4.59	4.16	6.06	3.67	5.04	7.64
	(1.9)	(1.6)	(19.4)	(30.0)	(28.8)	(18.6)	(23.9)	(45.8)	(18.2)	(30.4)	(34.5)
median of the paired ratio DFG alone / v3	0.06	0.32	0.89	0.49	0.51	0.92	0.27	0.25	0.46	0.38	0.32

Table 25: Compute of DFG alone per fit [Numerical]. (a) Per suite, FLOPs as a fraction of one network training run at that d (DFG's own count, multiply-add = 2; the largest fit, with its target and n), the number of fits above 0.75 of a run, and CPU seconds per fit (process time of the fit only). Last row: `lowdeg` over its 11 suites (range of the suites' medians). (b) By n, over DFG alone's 4334 fits at $d = 30$ (394 per n), with v3's CPU seconds on the same fits and the median of the per-fit ratio. The CPU times were measured with 4 worker processes running in parallel at nice 10 on a 4-CPU machine, and v3's in Part 10 under that run's own load, so the CPU comparison is rough. Source: `study/part11/RESULTS.json` (`compute`), from the `flops` and `cpu_seconds` fields of the result files.

F The ablations in full

This appendix holds the detail behind Section 5: the target families and the sources (Appendix F.1), and, for each group of parts, the refits of single cells that show how it acts (Appendices F.2 to F.5), and the core in detail (Appendix F.6). Everything here is on the five ablation suites unless stated, and only `dlheld7` among them is held out.

F.1 Families, sources and every target

Pairing. An ablation runs on DFG alone's training sets, column permutations and cross-validation folds: the training sets come from the target's and seed's own random stream, the column order and the folds from the fit's stream, and both streams are DFG alone's. Its n^* is compared with DFG alone's n^* computed from the same seeds of DFG alone's own runs.

Families. Each target is assigned by hand, from its definition, to one of the nine groups of Section 6.2 and to three classes (post hoc; `study/part11/interp/ablations/families.py`). *Character targets* are parities, staircases, multiplexers, compositions of parities and majorities and thresholds of a few characters, DNFs, CNFs, decision lists, read-once and random formulas: 71 of the 138 targets, 25 of the 65. *Halfspaces* are majorities and thresholds of a weighted sum, with the dictator, AND(x_1, x_2, x_3) and an AND of two halfspaces (13 and 7). *Symmetric and mixture targets* are functions of one signed sum, alone or with a parity, a gate, a second sum or an inner term (54 and 33). This class differs from the 54 sum-structured targets of Section 3.1 by two targets each way: it includes sym8 and sym9, and it counts f7_01 and fd_01, thresholds of an unsigned sum, as halfspaces. It keeps f7_10, $[x_1 + \cdots + x_5 + 2x_6 \leq -1]$, among the mixtures, where the atlas puts it, although it is a threshold of a weighted sum. Counted as a halfspace, f7_10 would move four class means by more than 0.05, all of them the halfspaces': under `strict`, `noblocks` and `labelnoise` from -0.23, -0.19 and -0.17 to -0.17, -0.07 and -0.08, and under `nosums` from $+0.05$ to $+0.11$ (`verification` in `study/part11/interp/ablations/ablations.json`).

Sources. The summaries are in `study/part11/RESULTS.json` (`q5_ablations`, `q5b_core`), written by `study/part11/compare_dfg.py`. The groupings, the nulls, the predictions of Appendix F.2 and the leave-one-in runs of Section 5.3 are in `study/part11/interp/ablations/ablations.json`, written by `study/part11/interp/ablations/ablations.py`; the analysis is `study/part11/interp/ablations/ABLATIONS.md`. Models are read from refits of single cells (`study/part11/interp/ablations/refit.py`). A refit uses the instrumented copy of Section 6.5, so that every feature carries the class that admitted it: the class whose test gave it its largest z_S in its round (Section 2.3), so a character attributed to an extension or the local class may also pass that round's global test. All 38 refits of the analysis, and the 42 made when its verification was applied, reproduce their run's result file exactly in test error and FLOP count (`study/part11/interp/ablations/results/refits.json`, `refits_verif.json`). In four cells the same fit through the evaluation's own loader of the frozen file gives the same model, term by term (`study/part11/interp/ablations/results/refit_crosscheck.json`). A term's s.d. is the standard deviation of its contribution on the test inputs, as in Section 6.1. An independent verifier checked the analysis, and its corrections are applied here (`study/part11/interp/ablations/VERIFY_ABLATIONS.md`; Appendix H.3).

The paired null in full [Numerical; post hoc]. On seed 0 the re-permuted fits move 1 of the 65 targets beyond ± 0.25 (dlist5, $+0.39$), and 10 of the 65 ratios are not zero; the mean |ratio| is 0.012.

On seeds 0 and 1 they move 3 (andxor −0.69, thrsum −0.66, h6_07 +0.40), with mean |ratio| 0.038. Over all 138 targets of the 11 suites they move 8 on seeds 0 and 1 (mean |ratio| 0.040). The unpaired null moves 23 to 31 of the 65 targets, depending on the pair of seeds. Like for like, one seed against one, it moves 23 where the paired null moves 1, with mean |ratio| 0.339 against 0.012: it overstates the noise of a paired comparison 20 to 30 times. Against the two-seed paired null, which is not like for like, its mean |ratio| is 8.8 times as large (23 targets moved against 3).

Which targets move [Numerical]. Under `nosums`, character targets lose little (+0.16; rand3, parmaj, h6_05, fd_07 and dnf3 move up). In the atlas, these are kinds of target that are encoded in part by blocks before their characters are all detectable: parmaj by two folded blocks, the formulas h6_05, fd_07 and dnf3 by the blocks of a clause or term, and random formulas such as rand3 by blocks on the inputs that follow their first characters (Section 6.2). Of the 15 targets `labelnoise` moves up, 9 are character targets, 6 of them formulas (family mean +0.33), and 6 are symmetric or mixture targets (mixtures +0.15). Of the 10 that `nosmalln` moves up, 8 are character targets (3 compositions, 3 formulas, par2 and rand2), with fd_04 and sym7. Like for like, the mean |ratio| of the two-seed runs, `noeig` and `nocomp`, is 0.026 and 0.043, against 0.038 for the paired null on two seeds; that of the one-seed runs, `noweighted`, `lsread` and `K3`, is 0.020, 0.032 and 0.075, against 0.012 on one seed. The per-family means are in Table 26, and every target's ratio in Figure 6.

Reading of the single changes, in full [Heuristic]. On these suites DFG's results come from the sum blocks, with the residual scaling of the tests and the small-n machinery as smaller contributors. As far as counts against one replicate of the null can tell, five parts can each be removed without a change larger than an arbitrary perturbation of the path produces: the eigenvector seeds, the composition classes, the weighted blocks, the logistic refinement and, but for one target (par4), the degree-4 search. Single targets still move further than any move of the null (h6_10, par4, rand1). Two qualifications apply. The suites contain few targets that need those parts: one pure 4-parity, no dense halfspace on more than 13 inputs, no degree-5 parity. And the parts are removed one at a time, so they may substitute for one another. Section 5.3 removes them together.

F.2 The test or the search: the refits

`strict` ($\delta = 10^{-4}$) and `lenient` ($\delta = 0.9$) move the log term of every test by +6.91 and −2.20 nats. If a target's data cost is set by a test, its n^* should move by a predictable amount; if it is set by the search, it should not move. Section 6.3 goes through these two runs target by target (Figure 5); this subsection gives the predictions in detail and the refits behind the asymmetry between the two.

Parities: the test, exactly [Numerical]. By Proposition 2.2, a k-parity passes round 0's test exactly when $n(1-\bar{y}^2) > n_{\text{thr}}(k,d)$, and n_{thr} moves by +13.8 and −4.4 rows with δ. At $d = 30$ it moves from 22.3, 26.8 and 30.6 ($k = 2, 3, 4$) to 36.1, 40.6 and 44.4 under `strict`, and to 17.9, 22.4 and 26.2 under `lenient`. Applied seed by seed to each training set's own $\bar{y}$ (post hoc: error 0 where the test passes, DFG alone's error but at least 0.45 where it fails, DFG alone's measured cell at $n = 16$), this predicts the measured changes of n^* to within 0.03: par3 +0.98 against +1.00 under `strict`, and 0 against 0 under `lenient`; par4 +0.20 against +0.21, and −0.79 against −0.79. par2 moves under neither, because the forced variant fits it at $n = 16$, below every n_{thr} here. The cross-validation's choice at $n = 16$ can depend on δ (par4, seed 1, keeps the forced variant only under `lenient`), but every $n = 16$ cell stays near error 0.5, or 0 for par2, so n^* does not move.

Other character targets [Numerical]. By Proposition 2.4(b), n_{glob} is proportional to the log term, so `strict` should raise it by +0.54 to +0.86 doublings (degree 4 to degree 1), and `lenient`

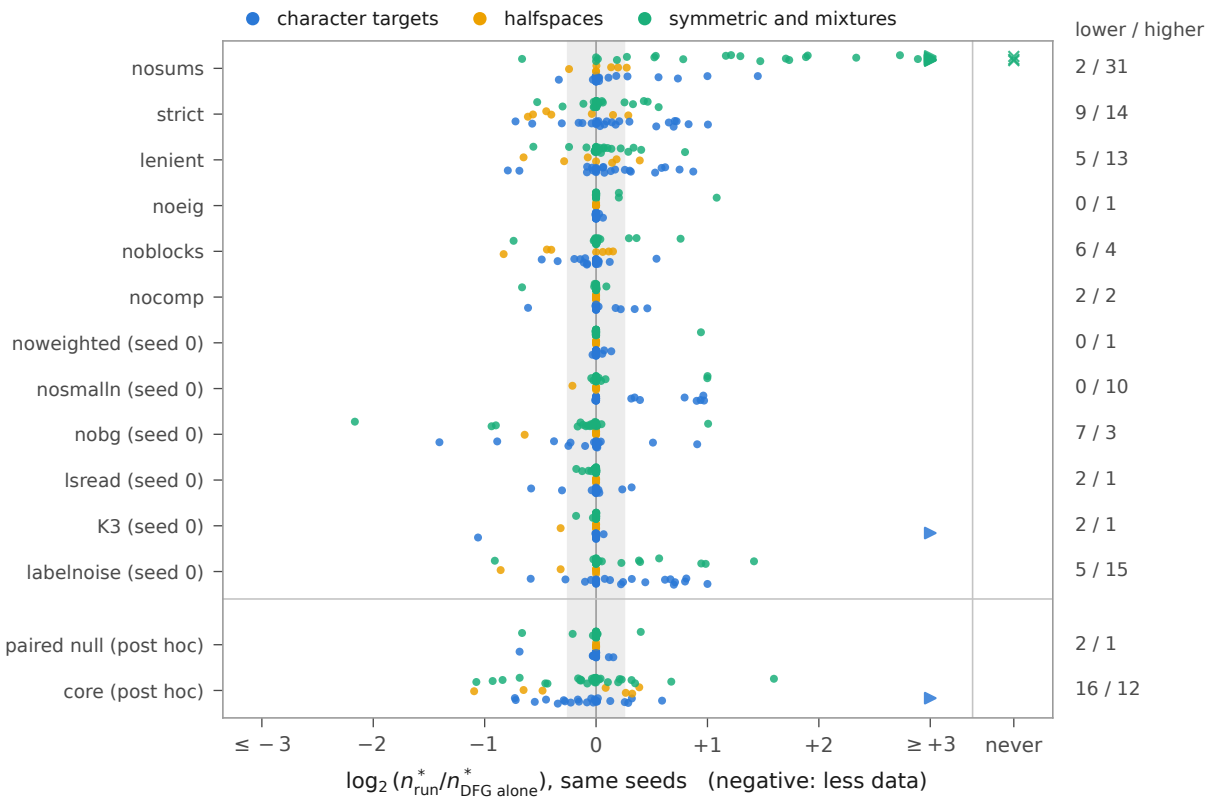


Figure 6: The 62 targets of the five ablation suites that DFG alone learns, one row per run: the $\log_2$ ratio of the run's n^* to DFG alone's on the same seeds [Numerical]. Colour gives the target's class; the grey band is the tie band ± 0.25. Ratios beyond ± 3 are drawn at the edge (triangles), and "never" (the run never reaches 10% error where DFG alone does) in the right-hand column. Right: the numbers of targets clearly lower / clearly higher. Bottom: the paired null on seeds 0 and 1 (post hoc) and the core DFG of Section 5.3 on the same 65 targets (post hoc). Source: `study/part11/interp/ablations/ablations.json` via `paper_dfg/make_figures.py`; the plotted points are in `paper_dfg/figures/fig_ablations.json`.

lower it by -0.22 to -0.43. `strict` raises n^* beyond 0.25 on 8 of the 25 character targets (par3, stair5, rand3, rand2, majpar, sparsepoly, h6_03, h6_05), and for 7 of them the first grid point at 10% error (seed mean) moves one step; h6_05's n^* moves within the same grid interval. `lenient` is not the mirror image: it lowers n^* on 2 targets (par4, rand2) and raises it on 8 (andxor, majpar, fd_00, mux, fd_07, dlist5, h6_05, f7_02). Two things go with the asymmetry. These targets are first fitted at $n = 32$–256 (8 of their 16 seed cells at 32–64), and in 11 of the 16 by the forced variant. And of the tests that run below $n = 120$, the one a more lenient δ moves most is the linear block's, whose threshold on 30 inputs (Equation (3)) is 51.2 at $\delta = 0.1$, 33.8 at 0.9 and 81.7 at 10^{-4}: `lenient` lowers it by a factor 0.66, against 0.74 for a degree-1 character, while `strict` raises it by 1.59, against 1.81. (The spectral edge, which runs from $n = 4d = 120$ on, moves more: the n at which a fixed eigenvalue clears it scales by 0.61 and 1.64.) Refitted at those 16 first fits, the linear block enters under $\delta = 0.9$ in 8 cells where it did not under $\delta = 0.1$, and the cross-validation's choice of variant changes in 7; andxor and f7_02 move through that choice alone (`study/part11/interp/ablations/results/lenient_first_fits.json`). Two refits show the mechanisms.

run	parities (3)	stair-cases (1)	half-spaces (7)	compo-sitions (6)	formulas (11)	random formulas (4)	sym-metric (12)	mixtures (21)
`nosums`	+0.00	+0.00	+0.05	+0.11	+0.17	+0.36	+2.78	+1.54
	(0/0)	(0/0)	(0/1)	(1/1)	(0/3)	(0/1)	(0/9)	(1/16)
`strict`	+0.40	+0.83	−0.23	+0.09	+0.01	+0.39	−0.01	+0.06
	(0/1)	(0/1)	(4/1)	(2/2)	(1/2)	(0/2)	(0/0)	(2/5)
`lenient`	−0.26	+0.00	−0.04	+0.36	+0.19	−0.11	+0.07	+0.05
	(1/0)	(0/0)	(2/1)	(0/3)	(0/5)	(1/0)	(0/1)	(1/3)
`noeig`	+0.00	+0.00	+0.00	+0.00	+0.01	−0.00	+0.02	+0.06
	(0/0)	(0/0)	(0/0)	(0/0)	(0/0)	(0/0)	(0/0)	(0/1)
`noblocks`	+0.00	+0.00	−0.19	−0.07	+0.01	−0.12	−0.00	+0.04
	(0/0)	(0/0)	(3/0)	(1/0)	(0/1)	(1/0)	(0/0)	(1/3)
`nocomp`	+0.00	+0.00	+0.00	−0.07	+0.08	+0.05	+0.00	−0.03
	(0/0)	(0/0)	(0/0)	(1/0)	(0/2)	(0/0)	(0/0)	(1/0)
`noweighted`	+0.00	+0.00	+0.00	+0.00	+0.02	+0.00	+0.00	+0.04
	(0/0)	(0/0)	(0/0)	(0/0)	(0/0)	(0/0)	(0/0)	(0/1)
`nosmalln`	+0.26	+0.00	−0.03	+0.38	+0.15	+0.22	+0.11	+0.05
	(0/1)	(0/0)	(0/0)	(0/3)	(0/3)	(0/1)	(0/1)	(0/1)
`nobg`	+0.00	+0.04	−0.09	−0.12	−0.10	+0.00	−0.04	−0.16
	(0/0)	(0/0)	(1/0)	(1/0)	(1/1)	(1/1)	(0/0)	(3/1)
`lsread`	+0.00	+0.00	+0.00	+0.06	−0.06	−0.01	−0.00	−0.02
	(0/0)	(0/0)	(0/0)	(0/1)	(2/0)	(0/0)	(0/0)	(0/0)
`K3`	+1.00	+0.07	−0.05	+0.00	−0.00	−0.26	+0.00	−0.01
	(0/1)	(0/0)	(1/0)	(0/0)	(0/0)	(1/0)	(0/0)	(0/0)
`labelnoise`	+0.00	+0.70	−0.17	+0.11	+0.33	+0.19	+0.10	+0.15
	(0/0)	(0/1)	(2/0)	(1/1)	(1/6)	(0/1)	(0/1)	(1/5)
paired null (0, 1)	−0.00	+0.00	+0.00	−0.11	+0.02	−0.00	−0.02	−0.01
	(0/0)	(0/0)	(0/0)	(1/0)	(0/0)	(0/0)	(0/0)	(1/1)
paired null (0)	−0.00	+0.00	+0.00	−0.02	+0.02	−0.00	+0.00	−0.00
	(0/0)	(0/0)	(0/0)	(0/0)	(0/1)	(0/0)	(0/0)	(0/0)
unpaired null	−0.33	+0.01	−0.17	+0.17	−0.01	−0.07	+0.01	−0.06
	(1/0)	(0/0)	(2/1)	(1/2)	(2/1)	(1/1)	(1/1)	(5/4)
core (post hoc)	+2.01	−0.08	−0.17	−0.15	−0.04	−0.27	−0.03	−0.05
	(0/1)	(0/0)	(3/3)	(3/1)	(2/2)	(2/1)	(0/0)	(6/4)

Table 26: The ablations by target family (the groups of Section 6.2; group sizes in the header) on the 65 targets of the five ablation suites [Numerical; families post hoc]. Eight of the nine groups appear: no target of the multiplexer group (mux6, mux11) is in these suites, and `designed`'s mux is grouped with the compositions. Each cell: mean $\log_2$ ratio over the targets both learn, and the targets clearly lower / clearly higher. The symmetric group holds the three targets that neither DFG alone nor any ablation learns (fd_10, h6_00, f7_05), so its means are over 9 targets (6 under `nosums`). Bottom: the paired and unpaired nulls, and the core DFG on the same 65 targets (post hoc). Source: `study/part11/interp/ablations/ablations.json`.

- *majpar at* $n = 64$ (seed 0). With $\delta = 0.1$ nothing passes in round 0. The forced variant adds $\chi_{\{1,2\}}$ and then $\chi_{\{3,4\}}$, and $\chi_{\{5,6\}}$ passes its own test in round 2: error 0.067. With $\delta = 0.9$, $\chi_{\{1,2\}}$ passes in round 0 and the linear block enters with 30 degree-1 characters, none of them in majpar's Fourier support. In round 1 $\chi_{\{3,4\}}$ passes its test, but the constant, $\chi_{\{1,2\}}$ and the 30 linear characters already fill the $n/2 = 32$ features that least squares allows, so the fit stops, with error 0.341. This is a second cost of the linear block at small n: its characters take the room a later character needs. With $\delta = 10^{-4}$ the forced pairs are the same, but $\chi_{\{5,6\}}$ fails its test: error 0.269.

- *f7_02 at* $n = 128$ (a DNF of three terms, seed 1), the other direction: the one formula on which `strict` lowers n^* (it also lowers andxor and mux, two compositions that were not refitted). With $\delta = 0.1$ the linear block enters, and the default variant is kept: error 0.202. With $\delta = 10^{-4}$ nothing passes. The forced variant guesses the blocks $(x_2 - x_6 - x_8)$ and $(-x_3 + x_5 + x_7)$, two of the DNF's terms; the third term's block then passes its test, and the cross-validation keeps the forced variant: error 0.

Halfspaces and sum targets [Numerical]. Halfspaces gain from a stricter test (-0.23; 4 of 7 lower). On f7_01 at $n = 128$, a degree-1 character that passes its own test first takes part of the exact block's signal in the block's second test, and with $\delta = 10^{-4}$ the block enters instead (Section 6.3). On the 33 symmetric and mixture targets the class means are $+0.04$ and $+0.05$ (30 learned; 5 and 4 higher, 2 and 1 lower). At the true block's detection point n_{detect} (Part 10's block statistics on seed 0's rows, re-thresholded post hoc for each δ; approximate for the modulated blocks, whose statistic was stored only as a ratio to its threshold), `strict` moves the block threshold by $+0.20$ to $+0.30$ doublings of n. The ablation analysis uses Part 10's round-0 detection points; with the points Part 10's verifier corrected for the gate and multiplexer mixtures (Table 33), f7_03's n^* lies below its detection point. On the 6 targets whose n^* lies a doubling or more beyond n_{detect} (7 with the uncorrected points), `strict` changes n^* by at most 0.05. On the other 24, n^* lies less than a doubling beyond n_{detect} (for 20 at or below it; 23 and 18 with the uncorrected points), and the grid cannot separate the test from the search (`paper_dfg/tables/interp_numbers.json`, `fig_dyn_when.gap_split`).

F.3 Degree 4 without the degree-4 search

`K3` sets $K = K_{\text{hi}} = 3$: the global search stops at degree 3, and the degree-5/6 search is off. K also enters the union bound of every character class through $\ln(4KN/\delta)$ (Equation (2)) and that of the sum-block test through L_B (Equation (5)), so these tests become 0.29 nats more lenient. The linear block's test and the spectral edge use $\ln(1/\delta)$ only (Equations (3) and (4)) and do not change. On the 65 targets `K3` moves 3 (Table 6).

par4 [Numerical; refits, seed 0]. Under `K3`, par4's test error is 0.49–0.50 at $n \le 256$, 0 at 512, 0.50 at 1024, 0 at 2048 and 4096, and 0.49–0.50 at 8192 and 16384. In the three fitted cells the model's main term is a folded block on the parity's four inputs, admitted in round 0 by an additive block search: $|-x_1 - x_2 - x_3 - x_4|$ at 512, $|x_1 - x_2 + x_3 + x_4|$ at 2048 and $|x_1 - x_2 - x_3 + x_4|$ at 4096, at 9.0, 34.5 and 70.9 times its threshold. Next to it the model holds an admitted product block: $|-x_1 - x_2 - x_3 - x_4| \times |x_5 - x_{14}|$ at 512 (term s.d. 0.40, against the folded block's 0.61), and split products at 2048 and 4096 (term s.d. 0.09 and 0.06). A k-parity is a function of any signed sum of its k inputs, so one folded block represents it exactly (Proposition 2.5). In the six empty cells from $n = 64$ on (64, 128, 256, 1024, 8192 and 16384) the round-0 block searches' best candidates stand at 0.13–0.45 of their thresholds, and the model is the constant and the background. Yet the true folded block would pass at every $n \ge 64$: on the round-0 residual a block that determines the labels has $T_B = n$ exactly, against a threshold of 56.9. The search finds it at 3 of these 9 grid sizes. With the degree-4 search, $\chi_{\{1,2,3,4\}}$ passes its round-0 test from $n = 64$ on (Proposition 2.2).

mux and andxor [Numerical; refits, seed 0]. These two targets have degree-4 coefficients of 0.75 and 0.5. Their test error under `K3` equals DFG alone's at every n, but the models differ.

- mux at $n = 256$: DFG alone's largest term is $\chi_{\{1,2,3,4\}}$ (term s.d. 0.51), admitted by the degree-4 test in round 0. `K3` holds no degree-4 character; it has the blocks $(-x_1 - x_2 - x_3 + x_4)$ and $(x_1 + x_2 + x_3 + 3x_4)$, and $\chi_{\{1,2,3\}}$, which the two-input extension of $\chi_{\{1\}}$ scores highest but the global degree-3 test of the same round also passes.
- andxor at $n = 256$: DFG alone holds its exact expansion $\chi_{\{1,2\}}$, $\chi_{\{3,4\}}$, $\chi_{\{1,2,3,4\}}$. `K3` holds the two pair characters and the weighted block $(x_1 + x_2 - 3x_3x_4)$.

rand1, -1.06 [Numerical; refit, seed 0, $n = 64$]. rand1 has 5 relevant inputs, x_1, x_3, x_4, x_5 and x_6, and degree-1 weight on 3 of them (± 0.375 on x_1, x_3 and x_5). With $K = 4$, round 0 admits the linear block (30 degree-1 characters), and the forced variant then guesses $\chi_{\{4,12,17\}}$: error 0.412. With $K = 3$, $\chi_{\{1\}}$ passes its own, slightly more lenient test, so the linear block does not enter. The forced guess is then $\chi_{\{1,4,6\}}$, the local class adds $\chi_{\{4,6\}}$, and a block $(x_3 - x_5)$ follows: error 0. This difference comes from K in the union bound, not from the degree-4 search.

Reading [Heuristic]. The degree-4 search is redundant where a target's degree-4 characters share their inputs with lower-degree structure: blocks on those inputs, and lower-degree characters, reach them. (In the refits of mux, the characters attributed to the extension and local classes also pass the global tests, so these refits do not show that those classes are needed.) It is indispensable for a degree-4 parity with no lower-degree footprint. The block test would admit the exact block from small n on, but the block search starts from degree-1 and degree-2 statistics (Proposition 2.5), which are zero for such a parity, so it finds the four inputs only by chance. The ablation suites contain one such target. Section 5.3 adds three more.

F.4 The small-n machinery and the background

The atlas computed two counterfactuals on its 37 targets (Section 6.2): the variant DFG's cross-validation did not choose, refitted on the same rows, and each fitted model without its kernel background. `nosmalln` and `nobg` run them as learners, and on the 21 targets the ablation suites share with the atlas the two agree in 126 of 126 and 224 of 231 cells.

No forced variant, no cross-validation [Numerical]. `nosmalln` fits the default variant on all n rows. At $d = 30$ it can differ from the atlas's "always default" refit only in the degree-5/6 subsample sizes (Proposition 2.6). Its seed-0 test error equals that refit's in all 126 cells with $n \leq 512$. Over the 65 targets, the mean test error rises by 0.012, 0.007, 0.022, 0.009 and 0.004 at $n = 16$, 32, 64, 128 and 256, and by nothing from 512 on. n^* rises on 10 targets and falls on none (mean +0.12). Since `nosmalln` is the default variant, every cell where DFG alone does better is one where DFG alone kept the forced variant. The first fits lost this way are par2 and rand2 at $n = 16$, majpar at 32 and 64, mux and sym7 at 64, fd_04 at 128 and f7_02 at 256. Each comes back one grid step later. Over the atlas's 222 cells, DFG's choice of variant is compared with the alternatives in Appendix G.1.

No kernel background [Numerical]. The background is added after the weights are fitted. A fitted model without it therefore gives exactly DFG's prediction without it for the same variant. The ablation can differ from that only where the cross-validation chooses another variant, because the fold fits lose their background too. `nobg`'s seed-0 error equals the atlas's model-without-background error in 224 of 231 cells. The other 7 are all at $n \leq 128$, so in each of them the choice changed. One is refitted: f7_11 at $n = 16$. DFG alone keeps the default variant there (held-out errors 6 and 3), the constant plus the background, with test error 0.301. Without the background, the default variant's held-out errors rise to 13, the forced variant is kept, and its model $\chi_{\{11\}}$, $\chi_{\{8\}}$ has error

0.077. Over the 65 targets the mean error rises by 0.015 and 0.016 at $n = 16$ and 32, and falls by 0.009, 0.002, 0.003 and 0.002 at 64, 128, 256 and 512. n^* falls on 7 targets and rises on 3 (mean -0.10); f7_11 falls most (-2.17).

Reading [Heuristic]. With nothing else in the model, at $n = 16$–32, the background is a Hebbian classifier, and it helps targets with a strong degree-1 part. From $n = 64$ on it adds the noise of 30 residual correlations to a model that already fits. Through the cross-validation it can also make the default variant look better than it is, so that the forced variant's right guesses are kept less often (the 7 cells in which `nobg`'s choice of variant changed, such as f7_11 at $n = 16$), or worse (sym8 at $n = 64$; Section 5.3). Most n^* lie at 64–512, so the background costs data on balance on these suites.

F.5 Tests scaled to the residual

`labelnoise` replaces s by 1 in every test, so the tests are scaled to the labels' noise instead of the residual's: the character thresholds and the spectral edge, which are proportional to s, rise by the factor $1/s$, and the statistics of the blocks and of the linear block, which are divided by s^2, fall by the factor s^2. (The subsampled searches keep their subsample's root mean square, and a block's second test, on the projected residual, keeps that residual's variance.) In round 0 the residual is $y - \bar{y}$, with variance $1 - \bar{y}^2$, and after the first features fit it shrinks. By Proposition 2.4(c), characters admitted after round 0 enter 1.4–3.7 doublings below n_{glob} (class medians), mainly because the residual variance there is 3.2–19 times smaller (class medians; Proposition 2.4, "Checks").

Results [Numerical]. The mean ratio is +0.14 (15 higher, 5 lower). By family: formulas +0.33 (6 of 11 higher), the staircase +0.70, random formulas +0.19, mixtures +0.15 (5 of 21), compositions +0.11, symmetric targets +0.10, parities 0, halfspaces -0.17 (Table 26). The mean test error rises by 0.024 at $n = 128$ (18 targets higher by more than 0.02, 2 lower) and by 0.013 at 256, and by at most 0.004 in absolute value elsewhere. Two refits at $n = 128$ (seed 0) show the two ways the scaling matters.

- *Later rounds*, rand3. DFG alone admits $\chi_{\{5,6\}}$ in round 0 ($s = 1.00$), $\chi_{\{4,5,6\}}$ by the one-input extension in round 1 ($s = 0.84$), $\chi_{\{5\}}$ and $\chi_{\{4,5\}}$ in round 2 ($s = 0.77$; the local class scores both highest, and the global degree-1 test also passes $\chi_{\{5\}}$), and five modulated blocks and a spectral feature in rounds 3 and 4: error 0.049. With $s = 1$ the first round is the same, but nothing passes in round 1, and the fit stops: error 0.317.
- *Biased labels*, far9 $= [|x_1 + \cdots + x_9| \geq 5]$, whose round-0 residual has $s = 0.71$. DFG alone admits the folded block $|x_1 + \cdots + x_9|$ and a spectral feature in round 0: error 0.006. With $s = 1$ the character and spectral thresholds are 1.41 times higher, and the block search's statistic is divided by 1 instead of $s^2 = 0.51$, so the folded block falls from 1.52 to 0.77 times its threshold. Nothing passes, and the model is the constant and the background: error 0.181.

Reading [Heuristic]. The residual scaling is what makes the growth greedy in the sense of DFG's design: each round's tests are calibrated to the noise left after the previous round, so weaker features pass once stronger ones are in. Its cost on these targets is modest, because the features that bring most targets below 10% error enter in round 0 (Section 6.6). It is larger for formulas and staircases, whose later characters build on the first ones, and for targets with biased labels.

F.6 The core in detail

The core's code is the frozen file, with the constructor arguments `readout="ls"`, `sum_inner=False`, `sum_seeds=(0,3,2)`, `blocks=False`, `max_ext_bases=0`, `local_max=0`, `K=3`, `K_hi=3` and `background=False`. It ran on all 11 suites, with seeds 0 and 1 and the standard grid. Table 27 gives its results by suite, Table 28 by target family, and Table 29 the leave-one-in runs.

Against the networks, by target [Numerical; post hoc]. The core's mean against `nets_best` is −1.74 over the 122 of 126 targets both learn, clearly better on 111 and clearly worse on 9. DFG alone on the same seeds is at −1.88, clearly better on 117 and worse on 5. On `dlheld7` the core is at −1.30 (9 better, 1 worse) and DFG alone at −1.27 (10 and 1). The core is clearly worse than `nets_best` on par4, par5, rf17, thr30, thr20, h6_10, thrint, f7_01 and h6_00. DFG alone (same seeds) is clearly worse on thrint, f7_01, h6_00, h3rf0 and h3rf1; the core is better than the networks on the last two.

Compute and CPU time [Numerical; post hoc]. On the same 3036 fits, the core's median FLOP count is 0.094 of DFG alone's (paired). Its largest fit uses 0.150 of one network run, against DFG alone's 0.500. Its total CPU time is 2302 s against DFG alone's 9058 s, with median 0.30 s against 2.01 s per fit. Both ran four workers on four CPUs, so CPU times are rough.

Where the core loses, in full [Numerical; post hoc]. To attribute the core's largest differences, each removed part was put back on its own, on the targets where the core moved most (Table 29; the targets were chosen after reading the core's results). The runs cover 16 of the 26 targets on which the core needs clearly more data. Putting one part back, or for h6_10 and f7_09 one pair of parts, returns n^* to within 0.25 of DFG alone's on 13 of them and part of the way on 2 (thr30, h5t0); no part tried restores h5s2. The other 10 were not tested: h5rf1, h6_03, rf01, mux11, rf19, h6_07, rmaj9, thrsum, fd_00 and rand3 (+0.26 to +0.60; h6_07 is also moved by the paired null). The losses that were tested fall into three groups.

- *Parities of degree 4 and 5 with no lower-degree footprint.* These are par4 (+6.02), rf17 (+6.19; its gates compute a 4-parity), poly4 (+4.28; its term $x_7x_8x_9x_{10}$) and par5 (never learned). Putting the degree-4 to degree-6 searches back restores all four: 48, 49, 151 and 56, against DFG alone's 48, 49, 161 and 56. Putting the extension and local classes back restores none, since these targets have no lower-degree character to extend. In the core they are found only when the block search hits the right inputs by chance, as par4 under `K3` (Appendix F.3): par4 at $n = 512$ and from 2048 on in seed 0, and from 4096 on in seed 1. par5 is found only at $n = 16384$ in seed 0. In a refit of that cell, a product block whose two factors together cover its five inputs, next to nine irrelevant ones, passes in round 0 at 6.7 times its threshold, and the exact block $(x_1 + x_2 - x_3 + x_4 + x_5)$ follows in round 1 (error 0). At $n = 512$ the round-0 searches' best candidates stand at 0.34–0.37 of their thresholds, and the model is the constant.
- *Dense halfspaces.* thr30 (+1.98), thr20 (+1.76), maj15 (+0.76), maj21 (+0.67), maj9 (+0.39), f7_01 (+0.32) and thr10 (+0.27). Without the linear block a degree-1 character must pass its own test, which most of them cannot do at $n = 128$–512 on 20 or 30 inputs. Putting the linear and spectral blocks back restores maj9, maj15, maj21, thr10, thr20 and f7_01 to within 0.25 of DFG alone, and most of thr30 (192 against 158). The background restores maj9, thr10 and f7_01 and part of the others; the logistic readout restores only thr10. In a refit of thr30 at $n = 512$ (seed 0), the core's model holds the 5 degree-1 characters that pass their own tests,

a 12-input signed block and a modulated block, with error 0.108. DFG alone's holds the same 5 characters, the linear block's other 25, $\chi_{\{3,19,22\}}$ and modulated blocks, with error 0.034, and 0.188 without the linear block's characters.

- *Mixtures with an inner term, and a few single steps.* h6_10 (+1.60) and f7_09 (+0.68) come back only with the weighted blocks and the eigenvector seeds together (207 and 234, against 215 and 222), not with either alone (494 and 649; 355 and 355). In DFG alone, removing either one already cost h6_10 about a doubling (Table 6). In a refit of h6_10 at $n = 256$ (seed 0), DFG alone's model is the weighted folded block $|x_1 + \cdots + x_{12} + 3x_{13}|$ (term s.d. 0.85, 0.96 of the model's standard deviation; error 0.017), and the core's is the two pair characters $\chi_{\{1,13\}}$ and $\chi_{\{3,13\}}$ (error 0.329). DFG alone's block has the wrong relative sign: the target is $[|s - 3x_{13}| > 1]$ with $s = x_1 + \cdots + x_{12}$. No function of $|s + 3x_{13}|$ has error below $156/8192 = 0.019$ (exact enumeration), and on the test inputs the best such function has DFG alone's error, 0.017. sym8 (+0.88) comes back with the background (54). At $n = 64$ (seed 0), DFG alone keeps the forced variant, whose forced block $|x_1 + \cdots + x_8|$ is exact (held-out errors 23 against 11; error 0). The core keeps the default variant (16 against 12) and predicts the constant (0.281): the background's noise had made the default variant look worse in the folds. sym8 is also one of the 8 targets the paired null moves (+0.69): its n^* hinges on that one choice at $n = 64$. h5t0 (+0.72) comes back in part with the background (146 against 112). h5s2 (+0.88; one grid step later on both seeds) comes back with none of the parts tried.

Where the core gains [Numerical; post hoc]. The core needs less data than DFG alone on 28 targets. The 12 examined are 12 of the 13 largest gains (the 11th, hrf0 at −0.75, was not run): 8 small formulas, and3 and three mixtures (f7_11, fd_02, fd_11), which the core learns at $n^* = 16$–63, except h5rf0 (164). They are h3rf1 (−2.13), h3rf0 (−1.98), h4rf1 (−1.84), and3, f7_11, rf06, fd_02, dlist8, h5rf0, fd_11, rand0 and rand1 (−1.09 to −0.72). Putting the background back removes the gain, to within 0.25, on rand0, rand1, rf06, f7_11 and fd_11, and part of it on h3rf1, h4rf1 and fd_02. With the linear block as well, it removes all of h4rf1's gain (104 against 103). The linear block alone removes the gain on and3, and $K = 4$ with the degree-4 to degree-6 searches removes it on h5rf0 (292 against 293) and rand1 (88 against 105). On h3rf0 and dlist8 no part tried removes the gain, alone or paired. In both learners their test error stays near 10% for several doublings (h3rf0: 0.07–0.12 at $n = 32$–256 in the core, 0.07–0.20 in DFG alone), so small differences move the 10% point by one or two doublings.

suite	status for DFG v1.3	targets	core vs DFG alone	newly never	core vs `nets_best`	DFG alone vs `nets_best`
`designed`	development	21	+0.11	–	−2.03	−2.14
			(8/5) [21]		(20/1) [21]	(20/0) [21]
`random`	development	20	+0.26	–	−1.78	−2.04
			(3/3) [20]		(19/1) [20]	(20/0) [20]
`dlheld`	development	12	+0.15	par5	−1.57	−2.17
			(2/3) [11]		(8/2) [11]	(11/0) [12]
`dlheld2`	held out for	7	+0.46	–	−1.15	−1.62
	v1.1 only		(0/3) [7]		(6/1) [7]	(7/0) [7]
`dlheld3`	held out for	10	−0.08	–	−1.30	−1.22
	v1.1 only		(4/2) [10]		(8/1) [10]	(7/3) [10]
`dlsymdev`	development	8	+0.05	–	−2.09	−2.14
			(0/1) [8]		(8/0) [8]	(8/0) [8]
`dlheld4`	exposed	12	−0.20	–	−1.90	−1.70
	to v1.2		(2/0) [12]		(12/0) [12]	(12/0) [12]
`dlheld5`	exposed	12	+0.12	–	−2.63	−2.75
	to v1.2		(1/3) [11]		(11/0) [11]	(11/0) [11]
`dlheld6`	development	12	+0.06	–	−1.22	−1.28
			(3/3) [11]		(10/2) [11]	(11/1) [11]
`dlfamdev`	development	12	−0.19	–	no networks	–
			(3/1) [11]			
`dlheld7`	**held out**	12	−0.03	–	−1.30	−1.27
			(2/2) [11]		(9/1) [11]	(10/1) [11]
pooled		138	+0.07	1	−1.74	−1.88
			(28/26) [133]		(111/9) [122]	(117/5) [123]

Table 27: The core DFG (post hoc) [Numerical]. Per suite and pooled, the core against DFG alone on the same seeds (0 and 1) and training sets, and the core and DFG alone (same seeds) against `nets_best`. Cells as in Table 5: mean $\log_2$ ratio over the targets both learn, below it the targets clearly lower / clearly higher (here: lower = the first-named learner needs clearly less data) and in brackets the number both learn. "Newly never": learned by DFG alone on these seeds but never by the core. By target family in Table 28. Sources: `study/part11/RESULTS.json` (`q5b_core`) and `study/part11/interp/ablations/ablations.json` (`core`).

	parities (4)	stair-cases (2)	multi-plexers (2)	half-spaces (13)	compo-sitions (13)	formulas (15)	random formulas (35)	sym-metric (25)	mixtures (29)
core vs DFG alone	+2.01	−0.06	+0.21	+0.32	+0.17	−0.08	−0.07	+0.06	−0.04
	(0/2) [3]	(0/0) [2]	(0/1) [2]	(3/7) [13]	(4/2) [13]	(4/2) [15]	(10/5) [35]	(0/2) [22]	(7/5) [28]
core vs `nets_best`	−1.97	−2.98	−0.66	−0.15	−2.92	−0.77	−1.89	−2.07	−1.85
	(2/2) [3]	(2/0) [2]	(2/0) [2]	(6/4) [12]	(12/0) [13]	(12/0) [12]	(34/1) [35]	(21/1) [21]	(20/1) [22]
DFG alone vs `nets_best`	−4.73	−2.92	−0.87	−0.49	−3.09	−0.67	−1.82	−2.13	−1.90
	(4/0) [4]	(2/0) [2]	(2/0) [2]	(8/2) [12]	(13/0) [13]	(12/0) [12]	(33/2) [35]	(21/1) [21]	(22/0) [22]

Table 28: The core DFG (post hoc) by target family over all 138 targets [Numerical; families post hoc]. Cells as in Table 27: the core against DFG alone on the same seeds (0 and 1), mean $\log_2$ ratio over the targets both learn, with the targets clearly lower / clearly higher; group sizes in the header. Sources: `study/part11/RESULTS.json` (`q5b_core`) and `study/part11/interp/ablations/ablations.json` (`core`).

target	DFG alone	core	core vs DFG alone	degrees 4–6	compo-sition	linear, spectral	logistic readout	back-ground	weighted blocks	eigen. seeds	weighted + eigen.	backgr. + linear
the core needs more data												
par5	56	never	$+\infty$	**56**	never							
rf17	49	3550	+6.19	**49**	3550							
par4	48	3124	+6.02	**48**	3124							
poly4	161	3127	+4.28	**151**	3127							
thr30	158	623	+1.98			192	483	223				
thr20	134	455	+1.76			**140**	414	249				
h6_10	215	649	+1.60					668	494	649	**207**	
h5s2	367	678	+0.88					713	678	678	678	
sym8	54	100	+0.88			100		**54**	100	100		
maj15	122	206	+0.76			**122**	207	174				
h5t0	112	185	+0.72					146	185	185	187	
f7_09	222	355	+0.68					357	355	355	**234**	
maj21	109	174	+0.67			**109**	174	136				
maj9	84	110	+0.39			**84**	110	**94**				
f7_01	156	195	+0.32			**132**	195	**161**				
thr10	128	154	+0.27			**128**	**136**	**123**				
the core needs less data												
h3rf1	135	31	−2.13	31	31	31	31	52				52
h3rf0	120	30	−1.98	30	30	30	30	32				32
h4rf1	103	29	−1.84	29	29	29	29	59				**104**
and3	78	37	−1.09	37		**78**	37	44				
f7_11	63	30	−1.07	30		30	30	**54**				
rf06	31	16	−0.97	16		16	16	**27**				
fd_02	58	31	−0.93	31	31	31	31	42				42
dlist8	55	30	−0.86	30	30	30	30	31				31
h5rf0	293	164	−0.84	**292**	186	164	164	166				166
fd_11	109	61	−0.84	62		68	61	**125**				
rand0	27	16	−0.73	16		16	16	**26**				
rand1	105	63	−0.72	**88**		68	63	**102**				

Table 29: Leave one in (post hoc). For 16 of the 26 targets on which the core needs clearly more data (the other 10 were not run) and 12 of its 13 largest gains (hrf0 was not run), n^* (seeds 0 and 1) of DFG alone, of the core, and of the core with one part put back: the degree-4 to degree-6 searches ($K = 4$, $K_{\text{hi}} = 6$), the extension and local classes, the linear and spectral blocks, the logistic readout, the kernel background, the weighted blocks, the eigenvector seeds; the last two columns put two parts back together (the weighted blocks with the eigenvector seeds; the background with the linear and spectral blocks). Empty: not run (the targets for each part were chosen after reading the core's results). Bold: putting this part, or pair of parts, back returns n^* to within 0.25 of DFG alone's. Same training sets, column orders and folds throughout. Source: `study/part11/interp/ablations/leavein.py`, results in `study/part11/interp/ablations/results/leavein/`, summarized in `study/part11/interp/ablations/ablations.json` (`leavein`).

G What the models contain: the detail

This appendix holds the detail behind Section 6: the identity checks of the analyses and DFG's small-n choices (Appendix G.1), the models family by family (Appendix G.2), the spurious terms (Appendix G.3), the comparison with v3's DFG member (Appendix G.4), false admissions (Appendix G.5), h5s1's refits (Appendix G.6), when structure appears (Appendix G.7) and the order of acquisition (Appendix G.8). Most of it is seed 0, from `study/part11/interp/ATLAS.md`, `study/part11/interp/calib/CALIB.md`, `study/part11/interp/theory/THEORY.md`, `study/part11/interp/order/ORDER.md` and `study/part11/interp/ablations/ABLATIONS.md`, each with its verification.

G.1 The analyses' fits, and DFG's choices at small n

The wrapper [Numerical]. The analyses refit DFG through one wrapper (`study/part11/interp/dfg_hooks.py`). It loads the frozen file (md5 checked), builds the training rows, the fit's random stream and the column permutation exactly as the evaluation does, and records every call of DFG's fit routine without changing it: at $n \leq 512$ the $2F$ fold fits of the cross-validation, then the final fit on all n rows, whose model predicts. 517 seed-0 cells (47 targets of all 11 suites, every grid n) reproduce the test error and the FLOP count of the result files bit for bit, as do 66 cells of the runs at $d = 15$, 60 and 120 (`study/part11/interp/tables/identity.md`). The result files store no models, so the identity is of errors and FLOP counts, a strong fingerprint of the same code path. The other analyses pass the same check:

- *The atlas*: 517 cells at $d = 30$ (47 targets of all 11 suites, every n, seed 0) and 66 cells at $d = 15$, 60 and 120, equal in test error and FLOP count (`study/part11/interp/tables/identity.md`).
- *The order analysis*: 363 fits (11 `designed` targets, every n, seeds 0–2), equal in both; the residual rebuilt from the recorded weights matches DFG's own round log in all of them (`study/part11/interp/order/ORDER.md`).
- *The theory checks*: 330 new seed-0 cells of `designed` and `random`, equal in both, and the 110 fits on null labels, whose round-0 decisions an independent replica reproduces in all 110 (`study/part11/interp/theory/THEORY.md`).
- *The false-admission and compute analysis* runs an instrumented copy of the frozen file with 64 inserted lines that only read. Its verifier rebuilt the copy byte for byte and found that removing the inserted statements leaves a syntax tree identical to the frozen file's. Every fit checked against the frozen module or the result files is equal (88 on noise, 451 on the designed and random targets, 35 profiled cells), and the unchecked fits are equal by that construction (`study/part11/interp/calib/VERIFY_CALIB.md`).
- *This paper's check* of h5s1 (Section 6.4): 6 refits, equal in test error (`paper_dfg/check_h5s1.json`).

The ablation analysis: All 38 refits of the analysis, and the 42 made when its verification was applied, reproduce their run's result file exactly in test error and FLOP count (`study/part11/interp/ablations/results/refits.json`, `refits_verif.json`). In four cells the same fit through the evaluation's own loader of the frozen file gives the same model, term by term (`study/part11/interp/ablations/results/refit_crosscheck.json`).

Terms, term s.d. and categories, in full. A model is read as terms in the original input coordinates, with the column permutation undone: the constant, each character χ_S, each spectral feature, each sum block with all its level indicators as one term, and the kernel background. A term's *s.d.* is the standard deviation of its contribution to the model's output over the 4096 test inputs (an absolute standard deviation, not a fraction of the output's); a group's *share* is that of its summed contribution, as a fraction of the output's. A term is a *target term* (T) if it is a character in the target's Fourier support or in one of its compact representations, or a block with a true class's input set and signs. It is *relevant* (R) if it reads only relevant inputs but is not a target term, and *spurious* (S) if it reads an irrelevant one. S-lin marks the irrelevant degree-1 characters that entered with the linear block, which adds every input not yet in the model.

The target's own representations. For each target the atlas writes down the compact representations it has in DFG's dictionary (`study/part11/interp/atlas_truth.py`). They are the sign of a few characters (parities, staircases, majorities of parities, halfspaces, DNFs, CNFs, decision lists), the exact Fourier expansion (multiplexers, random and read-once formulas), and sum blocks on the target's signed-symmetry classes, additive, modulated, weighted or as a product. Character representations are verified on all 2^k assignments of the k relevant inputs. Block representations are checked only as "f is a function of the block statistics"; their accept sets are not checked.

Complete, exact, and what the features can express. A representation is *complete* in a model when all its characters are present, or when every true block is a factor of a sum term with the true statistic (the true inner term or modulator, if any), or the factors of a product partition it. A model is *exactly* the target's Fourier expansion when it holds all the target's Fourier characters and nothing else but the constant and the background. "Complete" checks the dictionary, not the weights. What the learned features can express whatever DFG's weights is measured by the *oracle error*: the test error of a lightly regularized logistic fit of f on the model's features, fitted on 2048 test inputs and scored on the other 2048, without the kernel background. The categories depend on the representations written down. A model can be exact in a form not listed, and its terms then count as R. Five representations (the folded pair, the multiplexers' address blocks, the clause blocks, f7_06's and the auxiliary symmetry-class blocks) were added after the first fits had been looked at, and every cell was then reclassified.

DFG's own choice of variant [Numerical]. At $n \leq 512$ the paired cross-validation (Section 2.4) kept the default variant in 204 of the atlas's 222 cells and chose the forced one in 18; in 12 of these 18 the forced model is the target's first fit (par2 at 16, andxor and majpar at 64, stair5, parmaj and hrf0 at 128, and six more at 256–512). Refitting the variant DFG did not choose on the same rows (`study/part11/interp/cv_counterfactual.py`), the two differ in test error in 140 cells, and the choice made is the better one in 89 of them. The forced variant would have been better in 48 of the 204 default cells, most of all par4 at $n = 32$ (0 against 0.488): its forced character is right on all 32 rows, but the 10-fold fits on 28–29 rows do not search degree 4. The mean test error over the 222 cells is 0.2257 for DFG's choice, 0.2319 for the default always, 0.2256 for the forced variant always and 0.2111 for the better of the two in each cell. *Reading* [Heuristic]. The paired test is conservative where the folds are too small for the forced guess to be right, which is where the guess helps most. There v3's stacker, which kept both variants, did better (Section 6.2 and Appendix G.4).

Below the structure [Numerical]. In the 198 cells (36 targets) below the first n at which a compact representation is complete, 187 models are the default variant and 11 the forced one. 108

are the constant alone, with the background; 87 hold at least one true character; the median test error is 0.31 (`study/part11/interp/tables/atlas_below.csv`).

The kernel background [Numerical]. The background is added after the weights are fitted, so removing it from a model gives exactly DFG's prediction without it. Removed, the test error is lower in 85 of the 407 cells, higher in 54 and unchanged in 268. It helps most at $n = 16$–32, where the model holds little else and the target has a strong degree-1 part (maj9, thrint, thr30, majmaj5, symand, f7_04, f7_10, rf06): without it the error would be 0.19–0.27 higher (maj9 at $n = 32$: 0.491 against 0.283). It hurts most on the residue classes at $n = 16$–32 (h6_00 at 16: 0.488 with it, 0.343 without; the exception is f7_05 at 16, where it helps: 0.517 with it, 0.676 without), and at the first fits of andxor and majpar at $n = 64$ (0.092 and 0.067 against 0). *Reading* [Heuristic]: there the residual's degree-1 correlations are noise.

Structure lost as n grows [Numerical]. On one training set per n the models are not monotone in n. A compact representation present at one n is missing at a larger one, with the test error at or near 0, in stair5 (missing its degree-5 prefix character at $n = 256$ only), stair6 (missing its degree-6 prefix character from 512 on), thrint (at 512), dlist8 (at 256 and 16384), f7_07 and f7_09 (complete only at 256), f7_03 (only at 512) and f7_00 (from 4096 on). h5s7 loses its whole block at 4096 (error 0.368) and recovers it at 8192. After the first fit the test error rises with n in nine targets, by at most 0.013 (f7_02: 0 at $n = 256$, 0.013 at 512). In the order analysis, DFG drops 1139 of the 2157 character-seeds it carried at a smaller n (945 of them on dnf3 and 159 on rmaj9; 35 of 396 on the other nine targets), because each fit stops once every training sign is right (Section 6.6).

G.2 Family by family: the detail

The paragraphs below are the atlas's account of each family in full (Section 6.2 summarizes them).

Parities: one character [Numerical]. From the first fit on (par2 at $n = 16$, par4 and par5 at 64), every model is the constant plus the parity's character, with term s.d. 0.98–1.00, and nothing else up to $n = 16384$. Before it the models are the constant and the background. par2 at $n = 16$ needs the forced variant: no character passes a union-bound test on 16 rows, and the cross-validation's held-out errors are 7 (default) against 0 (forced). par5 is found at $n = 64$ by the final fit's degree-5 search, which runs on all 64 rows there and which the fold fits skip; their held-out errors are high and equal in both variants (21 and 21 of 64 at $n = 64$), so the cross-validation cannot see what the final fit will find. *Reading* [Heuristic]. For parities the dictionary is the target class, and DFG either has the character or nothing. The size at which it has it is set by the union bound (Proposition 2.2, Figure 3).

Staircases, multiplexers and formulas: characters, for the latter after block encodings [Numerical].

- *Staircases.* The prefix characters x_1, x_1x_2, ... are complete at the first fit, with error 0 (stair5 at $n = 128$, forced; stair6 at 256), next to one further Fourier character each. With more data the rest of the expansion fills in: at $n = 16384$ stair5's model is exactly its 16 Fourier characters, and stair6's holds 57 of its 63 and two relevant blocks. stair6 loses its degree-6 prefix character from $n = 512$ on, at error ≤ 0.0007.
- *Multiplexers.* The first fits mix characters and blocks. mux11 at $n = 512$ (error 0) holds 4 of its 8 modulated address blocks, such as $x_5 \cdot (-x_1 - x_2 + x_3)$, weighted blocks on address and

				first n with test error ≤ 0.1					$n = 16384$		
target	suite	n_0^* (n_3^*)	complete	n	error	var.	size	T / R / S	error	size	T / R / S
Parities											
par2	designed	16 (16)	16	16	0	f	2	1.00 / 0.00 / 0.00	0	2	1.00 / 0.00 / 0.00
par4	designed	56 (52)	64	64	0	d	2	1.00 / 0.00 / 0.00	0	2	1.00 / 0.00 / 0.00
par5	dlheld	56 (56)	64	64	0	d	2	1.00 / 0.00 / 0.00	0	2	1.00 / 0.00 / 0.00
Staircases											
stair5	designed	108 (111)	128 (512)	128	0	f	7	0.99 / 0.00 / 0.00	0	17	1.00 / 0.00 / 0.00
stair6	dlheld	210 (207)	256 (–)	256	0	d	8	0.97 / 0.00 / 0.00	0	81	0.96 / 0.21 / 0.00
Majorities of parities; andxor											
majpar	designed	57 (61)	64	64	0.067	f	4	0.92 / 0.00 / 0.00	0	11	0.92 / 0.22 / 0.00
majpar5	dlheld	182 (128)	256	256	0.011	d	6	0.98 / 0.00 / 0.00	0	29	0.99 / 0.11 / 0.00
andxor	designed	63 (52)	256	64	0.092	f	3	0.85 / 0.00 / 0.00	0	4	1.00 / 0.00 / 0.00
Multiplexers											
mux6	dlheld	213 (197)	1024	256	0.002	f	43	0.89 / 0.63 / 0.16	0	17	1.00 / 0.00 / 0.00
mux11	dlheld3	430 (509)	4096	512	0	d	76	0.83 / 0.56 / 0.02	0	65	1.00 / 0.00 / 0.00
DNF, CNF, decision lists, read-once formulas											
cnf3	dlheld3	187 (189)	1024	256	0.004	f	48	0.85 / 0.48 / 0.09	0	67	0.92 / 0.10 / 0.07
f7_02	dlheld7	188 (190)	2048	256	0	f	43	0.91 / 0.30 / 0.12	0	69	0.98 / 0.04 / 0.00
dlist8	dlheld3	71 (39)	64 (–)	128	0.049	d	40	0.89 / 0.02 / 0.36	0	149	0.91 / 0.04 / 0.10
f7_04	dlheld7	99 (97)	4096	128	0.021	d	33	0.96 / 0.00 / 0.24	0	181	0.92 / 0.09 / 0.05
f7_06	dlheld7	80 (82)	2048	128	0.024	d	42	0.87 / 0.50 / 0.28	0	84	0.93 / 0.10 / 0.09
Random formulas											
rf06	random	46 (31)	–	64	0.045	d	2	0.96 / 0.00 / 0.00	0	66	0.98 / 0.00 / 0.03
hrf0	dlheld	75 (80)	16384	128	0	f	8	0.97 / 0.37 / 0.00	0	32	1.00 / 0.00 / 0.00
Majorities and halfspaces											
maj9	designed	85 (81)	64	128	0	d	37	1.00 / 0.00 / 0.01	0	54	1.00 / 0.00 / 0.00
thrint	dlheld3	163 (186)	64 (1024)	256	0.071	d	34	1.08 / 0.39 / 0.22	0	43	0.85 / 0.28 / 0.00
thr30	dlheld	187 (158)	64	256	0.058	d	31	1.00 / 0.00 / 0.00	0.009	149	0.91 / 0.15 / 0.00
f7_01	dlheld7	177 (119)	64	256	0	d	56	0.75 / 0.00 / 0.27	0	57	0.92 / 0.30 / 0.00
Parities of two majorities											
parmaj	designed	111 (111)	128	128	0	f	37	1.00 / 0.00 / 0.00	0	19	1.00 / 0.00 / 0.00
majxor	dlheld	223 (225)	256	256	0.0005	d	9	0.96 / 0.00 / 0.10	0	137	1.00 / 0.00 / 0.00
Majority of majorities											
majmaj5	dlheld	348 (351)	–	512	0.023	d	49	0.91 / 0.69 / 0.11	0	256	0.90 / 0.20 / 0.00
Symmetric and signed sums											
sym9	dlheld3	111 (111)	128	128	0	d	4	1.00 / 0.00 / 0.00	0	75	0.88 / 0.00 / 0.21
band11	dlsymdev	891 (828)	1024	1024	0	d	5	1.00 / 0.00 / 0.00	0	62	0.98 / 0.12 / 0.00
f7_08	dlheld7	448 (644)	512	512	0.012	f	52	0.96 / 0.00 / 0.20	0	60	1.05 / 0.18 / 0.01
h5s7	dlheld5	1705 (6186)	2048 (8192)	2048	0.0007	d	164	0.79 / 0.43 / 0.07	0.0005	255	0.59 / 0.45 / 0.05
Signed-sum mixtures											
f7_07	dlheld7	205 (167)	256 (–)	256	0.006	d	21	1.00 / 0.00 / 0.00	0	87	1.00 / 0.00 / 0.00
f7_09	dlheld7	222 (222)	256 (–)	256	0.0005	f	15	1.00 / 0.00 / 0.00	0	35	1.00 / 0.00 / 0.00
symand	dlheld4	173 (149)	–	256	0.012	d	36	1.00 / 0.00 / 0.19	0	80	1.00 / 0.00 / 0.00
f7_10	dlheld7	90 (84)	256	128	0	d	33	1.18 / 0.64 / 0.12	0	27	1.00 / 0.00 / 0.00
f7_00	dlheld7	221 (211)	256 (–)	256	0	f	21	1.00 / 0.00 / 0.00	0	101	0.98 / 0.04 / 0.00
f7_03	dlheld7	352 (352)	512 (–)	512	0.008	d	68	0.85 / 0.45 / 0.01	0	110	1.00 / 0.00 / 0.00
f7_11	dlheld7	72 (51)	–	128	0.016	d	8	0.97 / 0.00 / 0.17	0	115	0.97 / 0.11 / 0.06
Residue classes mod 3											
f7_05	dlheld7	never (never)	–				never		0.3245	1	0.00 / 0.00 / 0.00
h6_00	dlheld6	never (never)	–				never		0.3430	1	0.00 / 0.00 / 0.00

Table 30: Every target of the structure atlas: DFG alone, seed 0 [Numerical]. n_0^* (n_3^*): the 10% test-error point of the seed-0 fits (of the seed mean in the result file; 2 seeds for rf06), interpolated in $\log n$. "Complete": the first grid n at which some compact representation of the target is complete in the model and, in brackets where it differs, the n from which it stays complete (–: it does not stay complete to $n = 16384$). Variant: d = default, f = forced, as DFG's cross-validation chose it. Size: features, the constant included. T / R / S: the shares of target, relevant non-target and spurious terms in the model's output, each the standard deviation of the terms' summed contribution over that of the output; parts that partly cancel can exceed 1. Families as in Table 7. Source: `study/part11/interp/tables/atlas_family.json` and `atlas_targets.csv`, via `paper_dfg/make_interp.py`; the leading terms of every model are in `study/part11/interp/tables/atlas_summary.md`.

data bits, and 17 of its 64 Fourier characters. mux6 at 256 (forced, error 0.002) holds 10 of its 16 characters and two additive blocks on two address bits and a data bit. From $n = 1024$ (mux6) and 4096 (mux11) on, the model is the complete Fourier expansion and nothing else.

- *DNF, CNF, decision lists, read-once formulas.* At the first fit the models are characters of degree 1–3 and blocks that encode a clause, a term or a rule. cnf3 at $n = 256$ (forced, error 0.004) holds one clause block exactly, $(-x_7 + x_8 - x_9)$; a second clause's block with two of the first clause's literals as inner term, $(-x_4 + x_5 + x_6 - 3\,\mathrm{AND}(\neg x_1, \neg x_2))$, term s.d. 0.49; and 10 of its 21 clause characters. f7_02 at 256 holds the block of its 4-literal term. The character representations are complete later: cnf3 at 1024, f7_02 at 2048 and f7_04 at 4096; f7_06's block-plus-characters form, its 5-literal block as a factor of a relevant product, is complete at 2048. The decision lists start as their degree-1 characters, through the linear block.
- *Random formulas.* rf06 and hrf0 are dominated by one input (Fourier coefficients -0.94 and -0.81), and their first models are that character ($n = 32$–64). Error 0 follows at $n = 128$ for hrf0, with a relevant block on the other inputs, and at 256 for rf06. At $n = 16384$, hrf0's model is exactly its 31-character expansion.

At $n = 16384$, 8 of the 37 models are exactly the target's Fourier expansion: par2, par4, par5, stair5, andxor, mux6, mux11 and hrf0. *Reading* [Heuristic]. Characters enter in order of how detectable they are (Section 6.3). Blocks, often weighted with an AND of two literals as inner term, encode a whole clause, term or branch before its characters are all detectable, and give way to them as they become detectable.

Majorities and halfspaces: the linear block, then a block [Numerical]. In maj9, thrint, thr30 and f7_01 the linear block enters at $n = 64$ and puts all 30 degree-1 characters in the model. The degree-1 representation is then complete and the oracle error is at most 0.007, but DFG's own error is 0.17–0.20, and the 17–21 irrelevant characters carry an S-lin share of 0.45–0.56 (thr30 has no irrelevant input). The error falls when a block arrives: maj9's 9-input block at $n = 128$ (error 0, S-lin 0.01) and f7_01's 13-input block at 256 (error 0). thrint, which has no single exact block, reaches error 0 at 512 with a product of its two blocks and two modulated folded blocks, all relevant non-target terms. thr30, with Gaussian weights on all 30 inputs, is never fitted exactly (0.009 at 16384). At $n = 16384$ the degree-1 characters carry 0.76–0.91 of the output's standard deviation. *Reading* [Heuristic]. For a threshold function the dictionary is complete almost at once, and what remains is to learn the weights. The logistic readout of 30 noisy degree-1 coefficients learns them slowly; one block, whose levels encode the threshold, does it at once.

Parities of two majorities: two folded blocks [Numerical]. parmaj and majxor ($\mathrm{MAJ}_3 \oplus \mathrm{MAJ}_3$, $\mathrm{MAJ}_5 \oplus \mathrm{MAJ}_5$) are first fitted, at $n = 128$ (forced, error 0) and 256 (error 0.0005), by two folded blocks over the union of the two majorities' inputs, $|t_A + t_B|$ and $|t_A - t_B|$, with term s.d. 0.56–0.77 and next to nothing else. They represent the target because $\mathrm{sign}(|t_A + t_B|^2 - |t_A - t_B|^2) = \mathrm{sign}(t_A t_B)$. With more data the cross-pair characters come in, all 9 of parmaj's at $n = 1024$ and all 25 of majxor's at 2048. The product block of the two majorities never appears, and the spectral pair once (majxor at 512). The majority of three majorities, majmaj5, is fitted without error from $n = 2048$ on without ever holding one of its compact forms, and at 16384 its model sits at DFG's cap of 256 features. *Reading* [Heuristic]. The folded pair is the block analogue of the spectral pair the quadratic feature was built for.

Symmetric and signed sums: the exact signed block [Numerical]. sym9 ($|s| \leq 1$ on 9 inputs) is its folded block alone at $n = 128$ (term s.d. 0.98, 4 features, error 0), and band11 at 1024 (term s.d. 0.99, 5 features). f7_08, an arbitrary accept set of an 11-input count, is its unfolded block at 512 (term s.d. 0.92), next to copies contaminated by irrelevant inputs. h5s7, an arbitrary set of a signed 13-input count, is constant to $n = 1024$; at 2048 it is a product of two factors that partition its class with its signs (term s.d. 0.72, error 0.0007), at 4096 the constant again (error 0.368), and from 8192 on the exact block. What varies across these targets is not the form but when the search finds it (Sections 6.3 and 6.4).

Mixtures: partner characters, then weighted blocks [Numerical]. In the seven signed-sum mixtures of the atlas, a sum combined with a parity, a gate, a selector or a second sum:

- *The partner comes first* in 5 of the 6 mixtures that have one: the characters x_{13} (f7_07), x_{11} (f7_03, f7_11), x_7 (symand) and x_6 (f7_10) enter one or more doublings before the block. In f7_09 the parity $x_{11}x_{12}$ enters at $n = 256$ together with its modulated block.
- *At the first fit* the compact form is complete in 4 of the 7: f7_07 and f7_09 (the block modulated by the parity, at 256), f7_00 (the product of its two sums, 256) and f7_03 (512).
- *With more data* the compact forms give way to weighted blocks with the partner as inner term, and to characters. At $n = 16384$ the models hold $\pm s - 3x_{13}$ (f7_07), $s \pm 3x_{11}x_{12}$ (f7_09), $s - 3\,\mathrm{AND}(\neg x_7, \neg x_8)$ and $s + 2\,\mathrm{AND}(\neg x_7, \neg x_8)$ (symand), and $s_1 - 3x_{11}$ and $s_2 - 3x_{11}$ (f7_03). Only f7_10 still holds its compact form there. All seven have test error 0, and their features span f with $R^2 \geq 0.998$ in five of them (0.96 and 0.89 for f7_03 and f7_00).

Reading [Heuristic]. DFG treats a partner variable as a heavy input of the block rather than as a modulator. A weighted block with the partner as its inner literal is easy to find, because the partner is itself a strong single input or pair. Two such blocks in opposite orientations, or one beside characters, represent the target as well as the modulated block does.

Dictionary or readout [Numerical]. Of the 148 cells with test error above 0.1, the learned features cannot express the target (oracle error above 0.1) in 133. In 9 they can (oracle error at most 0.02), and only DFG's own weights fall short: maj9, thrint, thr30 and f7_01 at $n = 64$ (thrint, thr30 and f7_01 also at 128), and dlist8 and f7_10 at 64, all just after the linear block entered. The other 6 lie in between. The oracle leaves out the kernel background, so "cannot express" concerns the learned features only: in 27 of the 133 cells DFG's own error is more than 0.05 below the oracle's, because the background carries the signal there. At the first fit, DFG's error is within 0.0005 of the oracle's for the median target. The largest gaps are andxor at $n = 64$ (0.092) and thrint at 256 (0.071), then majpar at 64 (0.067). andxor's and majpar's vanish without the background; thrint's is a readout gap, 0.071 with or without it, on a complete degree-1 dictionary (oracle error 0). *Reading* [Heuristic]. Outside threshold functions, DFG's data cost is the cost of getting the right features into the model. Once they are in, its least-squares and logistic readout is close to the best weights for them.

G.3 Spurious terms

Spurious terms in the atlas [Numerical]. They are of three kinds.

- *The linear block's irrelevant degree-1 characters.* The block entered in 83 of the 407 cells. Its irrelevant characters carry an S-lin share of 0.40–0.78 at $n = 64$ in the 8 targets with irrelevant inputs where it entered then, and at most 0.035 at $n = 16384$.

- *Products of a true block with a factor of irrelevant inputs*, such as $|x_1 + \cdots + x_9| \times |x_{12} - x_{23}|$ in sym9: 9 cells of 4 targets, term s.d. 0.07–0.41, test error at most 0.0083.
- *Blocks contaminated by irrelevant inputs* at the first n at which a block appears (f7_08 at 512, symand at 256, dlist8, f7_04).

At the first fit the spurious share is at most 0.1 for 22 of 35 targets (median 0.01, largest 0.36 for dlist8), and removing the spurious terms changes the test error by at most 0.035. At $n = 16384$ it is at most 0.1 for 36 of the 37 (the exception is sym9's spurious product, 0.21), and removing them leaves every error unchanged. In 5 cells the same block appears twice. *Reading* [Heuristic, from the code]: a block's key keeps the order of its inputs, so a block found again with its inputs in another order is not recognized as present.

Spurious features on the designed and random targets [Numerical]. On the 21 designed and 20 random targets (seed 0, every grid n; 451 fits, Table 31), 1047 of the 4499 admitted terms read an irrelevant input. 961 of them are the linear block's degree-1 characters, which come in with the relevant ones whenever the block passes; the other spurious terms are 86 of 3364 (0.026). The share of spurious terms falls from 0.69 at $n = 64$ to 0.08 at 16384 (without the linear block's characters, from 0.044 to 0.005), and their share of the output from at most 0.86 at $n = 64$ to at most 0.086 from $n = 2048$ on, with median 0 at every n. Removing them lowers the mean test error by 0.020 at $n = 64$ and changes it by at most 0.0002 from $n = 128$ on. Only 2 characters admitted by their own round-0 test read an irrelevant input, both wholly irrelevant, against 2.2 expected under an independent-Gaussian approximation and at most 29.8 by the argument of P1 applied to the irrelevant characters. Once a spurious term is in the model, the residual correlates with characters on its irrelevant inputs, and later rounds can admit them (rand2 at $n = 1024$). From $n = 2048$ on, 11 of the 14 spurious terms other than the linear block's characters are products of two blocks, one of which reads irrelevant inputs, as in the atlas.

			terms reading an irrelevant input			spurious share of the output		test error
n	fits with error ≤ 0.1	admitted terms	all (share)	linear-block characters	others (share of the other terms)	mean	max	with / without them (mean)
16	2	5	1 (0.200)	0	1/5 (0.200)	0.024	1.000	0.3713 / 0.3654
32	7	18	3 (0.167)	0	3/18 (0.167)	0.033	0.752	0.2818 / 0.2800
64	22	362	250 (0.691)	247	3/68 (0.044)	0.136	0.863	0.1158 / 0.0958
128	38	399	204 (0.511)	185	19/164 (0.116)	0.063	0.493	0.0190 / 0.0191
256	41	405	174 (0.430)	159	15/212 (0.071)	0.025	0.170	0.0027 / 0.0028
512	41	403	103 (0.256)	92	11/300 (0.037)	0.016	0.111	0.0007 / 0.0006
1024	41	408	96 (0.235)	76	20/323 (0.062)	0.012	0.148	0.0002 / 0.0002
2048	41	519	53 (0.102)	50	3/464 (0.006)	0.003	0.086	0 / 0
4096	41	681	56 (0.082)	50	6/625 (0.010)	0.006	0.067	0 / 0
8192	41	608	52 (0.086)	50	2/553 (0.004)	0.003	0.055	0 / 0
16384	41	691	55 (0.080)	52	3/632 (0.005)	0.003	0.060	0 / 0
all		4499	1047 (0.233)	961	86/3364 (0.026)			

Table 31: Spurious terms on the 21 `designed` and 20 `random` targets, seed 0, 41 fits per n [Numerical]. A term is spurious if it reads an input on which the target does not depend; a sum block counts as one term. "Linear-block characters": irrelevant degree-1 characters that entered with the linear block. Spurious share: the standard deviation of the spurious terms' summed contribution over that of the model's output. Test error with and without the spurious terms' contributions, mean over the 41 fits; from $n = 128$ on the two differ by at most 0.0002 (`study/part11/interp/calib/VERIFY_CALIB.md`). Source: `study/part11/interp/calib/calib.json` (`A2.by_n`), via `paper_dfg/make_interp.py`.

G.4 DFG alone and v3's DFG member

DFG alone and v3's DFG member [Numerical]. Part 10 analysed the same code as a member of v3, on the same targets and training rows ([12]; `study/part10/interp/TOY_INTERP.md`, with the corrections in `study/part10/interp/VERIFICATION.json`). Over the 255 seed-0 cells the two analyses share, the DFG test error is identical in 141: in all 15 cells at $n \geq 4096$, in 28 of 32 at $n = 1024$–2048, and in 30 of the 110 at $n \leq 64$. At $n \leq 64$, half of the 80 differing cells differ through the variant (v3's stacker kept the forced member, DFG's own paired test the default), and most of the rest are cells where the stacker gave DFG no weight, so that Part 10 described an out-of-fold fit. Part 10's first n with the exact signed blocks is the same for all 15 targets where both analyses have one. Part 10's findings on v3's member therefore carry over to DFG alone: the exact signed blocks of the symmetric targets, parities and small juntas as characters, halfspaces through the linear block, partners as characters and inner terms, spurious products, and the residue classes mod 3 never learned, a failure that Part 10 traced to the search, not the test. Part 11 adds the models of DFG alone itself; every target family at every grid n (Part 10 described the 70 non-sum targets only at their largest analysed n), with each target's own representations and the oracle error; the calibration on noise; and exact low-degree weights for the failures (Section 6.4).

G.5 False admissions: the detail

What is proved [Proved] (P1, Proposition 2.1). Let the labels be independent of uniform inputs, with any law. In round 0 of any fit call, some character of degree k passes the global test with probability at most $\delta/(2K) = 0.0125$, for any d and n. The linear block's test over all d inputs passes with probability at most δ, so a false linear block in round 0 has probability at most $\delta + \delta/(2K)$. If round 0 admits nothing, the default variant's model is the constant plus the kernel background. The proof does not cover the spectral block, the sum blocks, the forced variant, the cross-validation's choice between the variants, or the rounds after an admission.

The character tests sit about ten times inside their bound [Numerical]. In round 0 of the final fits on noise, the global character classes passed in 2 of 2400 class searches at $d = 30$ and 1 of 840 at $d = 60$: two degree-1 characters on biased labels at $d = 30$ and one degree-2 character on balanced labels at $d = 60$, all within the classes' level. The null rate of the degree-1 and degree-2 classes is about 0.1% per test, and all six classes are compatible with it. That is about a tenth of the bound: by THEORY.md the bound exceeds the independent-Gaussian rate by 10.9–14.3 at $d = 30$, close to the Mills-ratio factor of a Gaussian tail, so the union over characters is nearly tight and Hoeffding's inequality is what is loose [Heuristic]. On balanced labels none of the 220 final models at $d = 30$ holds a feature that passed a test (95% upper limit 0.017). At $d = 60$, 5 of 110 do: the linear block 3 times, a degree-2 character once, and once the product of the forced variant's two characters (`study/part11/interp/calib/CALIB.md`, short answer). In THEORY.md's 80 full fits on balanced labels at $n = 64$–4096, 78 final models are the constant plus the background. One holds a pair that passed its round-0 test at $z = 1.06$, and one is a forced model that the cross-validation preferred.

Most false admissions are the dense block tests' [Numerical].

- *The linear block* passes at about 1%, close to the χ^2_d tail at its threshold and a tenth of its nominal $\delta = 0.1$. Its statistic, a sum of squared Rademacher sums, has a lighter tail than χ^2_d. When it passes on noise it adds all d inputs: the two false linear blocks on biased labels at $d = 30$ raise the test error to 0.336 and 0.283, against 0.195 for a constant prediction.

- *The spectral block* has no proved level. On balanced labels it never passed in the fits, and in at most 3 of 20 000 null draws. On biased labels it passes often at $n = 4d$, the smallest n at which it runs: in 5.4% of null draws at $d = 30$ and 47% at $d = 60$ ($P(y{=}1) = 0.8$), 99% at $d = 120$, and 39% at $d = 30$ with $P(y{=}1) = 0.9$. The fits agree: 2 of 20 at $d = 30$, $n = 128$, and 5 of 10 at $d = 60$, $n = 256$; and in THEORY.md's fits with $P(y{=}1) = 0.85$ at $n = 128$, 7 of 30 final models end with spurious spectral features. *Reading* [Heuristic]. Its edge $2s\sqrt{d/n}$ is the semicircle edge of a matrix with many comparable weights. On biased labels the minority rows carry residual weights of about -1.6 against $+0.4$, and the Marchenko–Pastur edge of their own Gram matrix reaches 0.99 and 1.08 of DFG's edge at $d = 30$, $n = 128$ and $d = 60$, $n = 256$. At $n = 4d$ that edge is fixed while DFG's margin factor $1 + \sqrt{2\ln(1/\delta)/d}$ shrinks with d, so the false admissions grow with d. The low rates measured at larger n therefore need not carry over to strongly biased targets, such as and3 with $P(y{=}1) = 1/8$, or to the designed suite at $d = 120$; the 15 spurious spectral features that DFG admits on the designed and random targets at $n = 128$–256 are plausibly the same effect.
- *The sum blocks* have no proved level either. They never passed in the 660 final fits on noise (the largest statistic was 0.93 of its threshold), but they did in round 0 of 2 default-variant fold fits at $d = 30$ with $P(y{=}1) = 0.8$, and in 1 of THEORY.md's 30 fits at $P(y{=}1) = 0.85$.

The forced variant is not calibrated after its guesses [Numerical]. When nothing passes, the forced variant adds the best candidate as a guess, at most twice. After two forced characters χ_A and χ_B, the local class, which tests every subset of the inputs in use, can admit their product $\chi_A\chi_B$ by its own test. On biased noise at $d = 30$, 229 of the 900 forced fold fits hold a test-passed feature after a forced one, mostly such a product. In fold-sized refits the local test passes after the two forced characters in 244 of 334 cases, and after two random characters of the same degrees in none of 334. *Reading* [Heuristic]. The selection does it: the characters that best match a biased residual are nearly constant on its minority rows, and so is their product, which least squares on $\{1, \chi_A, \chi_B\}$ leaves in the residual. DFG's paired cross-validation protects the final fit: it chose the forced variant in none of the 180 biased fits. On balanced labels it chose it in 5 of 120 fits at $d = 30$, with test errors 0.491–0.500, and in 2 of 60 at $d = 60$, one of which then admitted the product.

On noise the background, not the features, costs error [Numerical]. With $P(y{=}1) = 0.8$ the test error falls from 0.339 at $n = 16$ to the test set's minority rate, 0.195, from $n = 1024$ on, although no fit at $n \le 32$ holds a feature. The excess is the kernel background's random direction: with q the share of test points predicted -1 and a the minority rate, $a + q(1 - 2a)$ reproduces the mean error at every n to within 0.001. On such data the background carries no information and costs up to 0.144.

G.6 Where it fails: the detail

The paragraphs below are Section 6.4 in full, with this paper's refits of h5s1.

The residue classes mod 3: detectable, but out of the search's reach [Numerical]. Let B count the k signed inputs of a block that equal $+1$. f7_05 = $[B \bmod 3 \neq 0]$ on 14 inputs, h6_00 = $[B \bmod 3 \neq 1]$ on 13 and fd_10 = $[B \bmod 3 = 0]$ on 10 are three of the four sum-structured targets that DFG alone never learns; the fourth is h5s1, below. v3 does not learn them either (Section 4.2). DFG alone's model is the constant plus the kernel background at every n (f7_05 and h6_00 in the atlas, fd_10 in the identity sample), and at $n = 16384$ its test error is the test

test	proved under the null (P1)	null pass rate of its statistic	admitted in DFG's own fits
global characters, degrees 1–6, round 0	at most $\delta/(2K) = 0.0125$ per degree, for any d, n and law of the labels	0.11% (degree 1) and 0.07% (degree 2) per test at $d = 30$	2 of 2400 round-0 class searches at $d = 30$, 1 of 840 at $d = 60$
linear block, round 0	at most $\delta = 0.1$ when it tests all inputs; a false linear block at most $\delta + \delta/(2K)$	0.84% ($d = 30$) and 1.04% ($d = 60$), close to the χ^2_d tail at its threshold (0.92%, 1.05%)	2 of 360 round-0 tests at $d = 30$, 4 of 160 at $d = 60$; each adds all d inputs
spectral block	nothing	balanced labels: at most 3 in 20 000 draws. $P(y{=}1) = 0.8$, at $n = 4d$, the smallest n at which it runs: 5.4% of 122 000 draws ($d = 30$), 47% of 121 000 ($d = 60$), 99% of 4 000 ($d = 120$); $P(y{=}1) = 0.9$: 39% of 20 000 ($d = 30$)	biased labels: 2 of 20 fits ($d = 30$, $n = 128$), 5 of 10 ($d = 60$, $n = 256$); at $P(y{=}1) = 0.85$, $n = 128$: 7 of 30 fits; balanced: none
sum blocks	nothing	–	none in the final fits (largest statistic 0.93 of its threshold); in round 0 of 2 default-variant fold fits ($d = 30$, $P(y{=}1) = 0.8$); at $P(y{=}1) = 0.85$, $n = 128$: 1 of 30 fits
forced variant, then the local class	nothing: the variant adds a guess by design	–	a test-passed feature after a forced one in 229 of 900 forced fold fits ($d = 30$, $P(y{=}1) = 0.8$), mostly the product of the two forced characters; in fold-sized refits the local test passes after the two forced characters in 244 of 334, after two random ones in 0 of 334. The cross-validation chose the forced variant in 0 of 180 biased fits
final model	the constant and the kernel background, if round 0 admits nothing	–	a test-passed feature in 0 of 220 balanced and 6 of 220 biased fits at $d = 30$, 5 of 110 and 6 of 110 at $d = 60$

Table 32: False admissions on labels independent of uniform inputs [Proved, Numerical]. "Proved": P1, Proposition 2.1 (`study/part11/interp/theory/THEORY.md`, with the wording of its verifier, `VERIFY_THEORY.md`). "Null pass rate": the share of null draws in which the test's round-0 statistic, computed as DFG computes it, passes; 80 000 draws for the characters and the linear block, and for the spectral block 20 000 on balanced labels and the draws given in its row on biased labels (CALIB.md's runs pooled, after its correction 6). "Admitted": DFG alone's own fits on labels drawn independently of the inputs, with $P(y{=}1) = 0.5$ (balanced) or 0.8 (biased): 20 seeds per $(P(y{=}1), n)$ at $d = 30$ and 10 at $d = 60$, $n = 16, \ldots, 16384$, 660 fits in all; the fits at $P(y{=}1) = 0.85$ are THEORY.md's 30 at $n = 128$. "Test-passed": a feature admitted because it passed a test, not as a forced guess. Sources: `study/part11/interp/calib/calib.json` and `CALIB.md`, with the corrections of `study/part11/interp/calib/VERIFY_CALIB.md` (the fold-fit counts, the scope of the sum-block row, the $d = 120$ and $P(y{=}1) = 0.9$ rates) and the spectral rates as CALIB.md recomputed them after that verification (its correction 6; `calib.json`, `spectral_pooled` and `mc_spectral`); `study/part11/interp/theory/check_theory.json`. Made by `paper_dfg/make_interp.py`.

set's minority rate in every seed. v3's member was not constant at small n: it held one or two characters, and for h6_00 a sum block at $n = 256$–512. Table 8 sets the test against the search, and Proposition 2.5 proves the exact statements.

- *The test would accept the block.* One block, with one group per value of its signed sum, represents each target with at most $k+1$ parameters, where the target's Fourier expansion has all $2^k - 1$ non-constant characters [Proved]. DFG's grouping merges the rarest values, which costs 0.1–0.2% error [Numerical]. Part 10 found the true block passing DFG's test from $n = 128$ on the same training rows, and at 134–155 times the threshold at $n = 16384$ [Numerical].
- *The search's seeds see nothing.* The targets' Fourier weight at degrees 1 and 2 is exactly $1.6 \cdot 10^{-6}$, $5.4 \cdot 10^{-6}$ and $2.1 \cdot 10^{-4}$, and it peaks at degrees 11, 9 and 7 [Numerical, exact]. Every starting point of the block search is computed from statistics of one or two inputs, whose expectations in round 0 are the target's degree-1 and degree-2 coefficients [Proved, Proposition 2.5(f)]: here 2^{-13}, 2^{-12} and 2^{-9} in absolute value [Numerical, exact].
- *How far out of reach* [Heuristic]. At $n = 16384$ one entry of the pair matrix inside the block has a signal-to-noise ratio of 0.017, 0.033 and 0.27. Its top eigenvector would point at the block only from $n \approx 1.1 \cdot 10^7$, $3.1 \cdot 10^6$ and $8.6 \cdot 10^4$, and a ranking of the inputs by their degree-1 coefficients only from $n \approx 4.1 \cdot 10^8$, $1.0 \cdot 10^8$ and $1.6 \cdot 10^6$. Part 10 measured the reach on v3's member: at $n = 16384$ the eigenvector orders put 0.50–0.57 of the block's inputs among their first k (`study/part10/interp/TOY_INTERP.md`, §4), against 0.40–0.52 for the better of two random orders (the chance level of Part 10's verifier, `study/part10/interp/VERIFICATION.json`) [Numerical].

Reading [Heuristic]. A residue class mod 3 is detectable once proposed and invisible from every foothold the search has; more data within any practical range would not help the seeds. Only the most regularized network, `nets_cewd1.0`, learns h6_00 ($n^* = 12304$), the one sum-structured target on which a network beats DFG alone (Section 4.2). How it gets there was not analysed.

h5s1: a modulator below its test's reach [Numerical]. h5s1 is an arbitrary accept set of a signed 9-input sum s, multiplied by the parity $x_{10}x_{11}$, and no learner's seed-mean error reaches 10% on it by $n = 16384$ (DFG alone fits it exactly in 2 of its 3 seeds there). Its Fourier weight is 92% on the single degree-11 character of all its inputs, and the plain block carries no signal: at $n = 16384$ its statistic is 0.04 of the threshold (Part 10). The block modulated by $x_{10}x_{11}$ would pass DFG's test from $n = 128$, but DFG builds a modulated block only on a character already in the model. The pair's coefficient is exactly $-1/32$, the mean of the sum's condition, so its round-0 test admits it in expectation only from $n_{\text{glob}} = 2.3 \cdot 10^4$ (Proposition 2.4), above the grid [Heuristic]. Refits of DFG alone at $n = 8192$ and 16384, made for this paper, show the mechanism (`paper_dfg/check_h5s1.json`; all six refitted test errors equal the result file's):

- at $n = 16384$ the pair passes its round-0 test in seeds 0 and 2 ($z = 1.14$ and 1.23), the modulated block $x_{10}x_{11} \cdot s$ enters in round 1, and the test error is 0. In seed 1 the pair misses ($z = 0.98$), and the model is the constant;
- at $n = 8192$ the pair passes only in seed 2 ($z = 1.04$), where the block does not follow and the error stays at 0.48;
- at every $n \leq 8192$ every seed's test error is 0.48–0.51.

Reading [Heuristic]. h5s1's bottleneck is a low-degree statistic like the residue classes', but one that becomes visible within the grid: the target is learned when the pair's test fluctuates above its threshold.

Arbitrary accept sets come last [Numerical]. The six signed-sum targets that DFG alone learns last are all pure arbitrary accept sets: h5s6, h5s2, f7_08, band11, h5s3 and h5s7, with n^* from 597 to 6186 (the median over the 50 sum-structured targets it learns is 186). Their share of variance at Fourier degrees 1–2, $(W_1 + W_2)/(1 - W_0)$, is 0.008–0.081, against a median of 0.34 over the 54 (Part 10's `study/part10/interp/tables/toy_timing.csv`). The models hold the exact block once it arrives (Section 6.2), and Table 33 shows that for f7_08, band11 and h5s7 it arrives two to six doublings after the test would admit it, by Part 10's rule (one block on each whole signed-symmetry class). By that rule h5s7's arrives at 8192 (+6), but its exact split product, two factors that partition the class, already fits seed 0 at 2048 (+4). h5s7 also shows how much one training set matters: its seed-0 model is exact at $n = 2048$, empty at 4096 and exact again from 8192 on. Its seed-0 fits reach 10% error at $n_0^* = 1705$, the seed-mean error only at $n^* = 6186$.

G.7 When structure appears: the detail

The paragraphs below are Section 6.3 in full, except for the paired null, which is in Section 5.

Characters: round 0 admits by detectability [Proved, then Numerical]. In round 0 the residual is the labels minus their mean. A character χ_S of degree k searched on all n rows passes its test exactly when

$$n > 2c^2 L_k \, s^2 / v_S^2, \qquad L_k = \ln(4K\tbinom{d}{k}/\delta),$$

where v_S is the mean of the residual times χ_S and s^2 the residual's mean square (Proposition 2.4). At $d = 30$ degrees 1 and 2 are searched on all rows at every grid size, and degrees 3 and 4 from $n = 32$ on. Replacing v_S by the target's Fourier coefficient $\hat{f}(S)$ and s^2 by $1 - \hat{f}(\emptyset)^2$ gives the *detection size* of Equation (7),

$$n_{\text{glob}}(S) = 2c^2 L_{|S|} \, (1 - \hat{f}(\emptyset)^2)/\hat{f}(S)^2,$$

a first-order prediction [Heuristic]. Degree enters only through the logarithm. With $K = 4$, $\delta = 0.1$ and $c = 1$, L_k is 8.5, 11.2, 13.4 and 15.3 nats for $k = 1$–4 at $d = 30$. Size enters through $1/\hat{f}(S)^2$, so a coefficient $\sqrt{15.3/8.5} = 1.34$ times larger at degree 4 than at degree 1 already has the smaller n_{glob}.

Figure 5a shows how closely DFG's round-0 decisions follow n_{glob} on the 41 targets of `designed` and `random`, seed 0, at every grid size [Numerical]. These are 451 fits, each equal to the evaluation's in test error and FLOP count; in each, every target character's test was replayed with DFG's own residual (`study/part11/interp/theory/THEORY.md`, checked by an independent verifier).

- *The decisions.* Over the 8681 (character, n) pairs with $|S| \le 4$, DFG's round-0 decision agrees with the rule "admit when $n > n_{\text{glob}}$" in 8409 (96.9%). The disagreements lie near n_{glob}: 100 of the 104 pairs above it that fail lie within one doubling of it, and 139 of the 168 pairs below it that pass. The share admitted is 6% and 12% at 1–1.5 and 0.5–1 doublings below n_{glob}, and 94% and 98% at 0.5–1 and 1–1.5 doublings above. A normal approximation for v_S gives the share to within 0.051 in every band.
- *First entries.* The 249 characters that first enter the model through this test do so at a median $2^{0.26} \, n_{\text{glob}}$. The order analysis finds the same +0.26 doublings over 151 character-seeds of the Part 5 targets (Section 6.6).

Later rounds make the rest of a junta cheaper [Proved identity, then Numerical]. In round j a class with N_{cls} candidates admits χ_S when $n > 2c^2 \ln(4KN_{\text{cls}}/\delta) \, s_j^2 / v_{S,j}^2$. So n_{glob} divided by this

size is the product of three factors (Proposition 2.4c): a smaller union bound, $L_{\text{glob}}/L_{\text{cls}}$; a smaller residual, $(1-\hat{f}(\emptyset)^2)/s_j^2$; and the residual's coefficient on S, $v_{S,j}^2/\hat{f}(S)^2$. On the same 41 targets [Numerical]:

- *Against the prediction.* For 427 characters the predicted grid size (the first at which the rows searched exceed n_{glob}) lies inside the grid. 105 of them first appear at the predicted grid size, 266 earlier, 54 one doubling later and 2 never. Each of the 54 had $z < 1$ at the predicted size (at most 0.994), a fluctuation of the test. Another 321 characters appear although no grid size was predicted for them, 262 of them through the local class.
- *By class* (Table 33a). The classes of the later rounds admit characters a median 1.4 (the global test in rounds ≥ 1) to 3.7 doublings (the local class, every subset of the inputs in use) below n_{glob}. In every class the residual is the main factor: its median is 3.2 to 19 times smaller than the labels' variance. The smaller union bound gains a median factor of 1.0 to 1.7 per class (at most 0.8 doublings), though single characters of the local class gain up to 3.3.

Reading [Heuristic]. Round 0 ranks characters by how detectable they are against the labels' variance. Once the large terms are fitted, the residual is small, and characters on the inputs already in use pass the tests of small classes. This resembles two of Part 5's findings for the networks [11], junta completion and the faster learning of products of learned features. A class label says which test admitted a character, not what would have happened without that class; the ablations ask that (Section 5).

Parities: no gap [Proved, then Numerical]. For a k-parity χ_S passes the round-0 test exactly when $n(1-\bar{y}^2) > n_{\text{thr}}(k,d)$, with $\bar{y}$ the mean label (Proposition 2.2). In all 250 parity cells at $n \geq 32$ (par2 and par3 at $d = 15$–120, par4 at $d = 15$–60) the test error is 0 exactly where the criterion holds; at $n = 16$, where no test passes, the forced variant can put the parity into the model earlier. For a parity no search stands between the test and the model.

Sum blocks: detection is cheap, discovery lags [Numerical]. Part 10 measured both sizes for v3's DFG member on the 54 sum-structured targets [12].

- *Definitions.* n_{detect} is the first grid size at which DFG's block test would admit the target's *true* block if it were proposed in round 0 on the seed-0 rows; it is an oracle. n_{struct} is the first grid size at which the model holds every signed-symmetry class of size ≥ 3 exactly, as a block, a factor of a product block or a modulator.
- *Part 10's result* (`study/part10/interp/TOY_INTERP.md`). Part 10's verifier corrected the detection points of the gate and multiplexer mixtures, testing the modulated block on a residual already fitted on its modulator, as DFG tests it. With these points, 39 of the 54 targets are detectable at $n = 128$. Over the 49 found, n_{struct} is earlier than n_{detect} for 6, equal for 20, one doubling later for 9 and two or more doublings later for 14. On the 25 pure symmetric targets every true block passes at $n = 128$; 22 are found, 9 of them two or more doublings later. The delay grows as the target's share of variance at Fourier degrees 1–2 shrinks: Spearman -0.86 over the 25 pure symmetric targets, counting the 3 never found at $n_{\text{struct}} = 2^{15}$ (-0.80 over the 22 found), and -0.54 over all 54 with the corrected detection points [12].
- *Part 10's reading.* Part 10 read this as the search, not the test, setting the data cost. It labelled the reading Heuristic: nothing isolated the search, and the stacker's choice between DFG's two variants could also have contributed.

What Part 11 adds is threefold: the same timing for DFG alone, without the stacker; an intervention on the test (the next paragraph); and, for the residue classes mod 3, exact low-degree weights that put numbers on the search's missing foothold (Section 6.4). For DFG alone (Table 33b; seed 0): on the 15 atlas targets where both runs have an n_{struct}, DFG alone's equals v3's member's. On all 13 sum-structured targets of the atlas the two runs agree; three of them never reach it. n_{detect} depends only on the training rows and the test, so Part 10's detection points apply unchanged. On these 13 targets the delay is 0 for 4, one doubling for 3, and two to six doublings for f7_08, band11 and h5s7. The structure never appears for the two residue classes mod 3, nor for f7_11, whose two branch classes are never matched in one model. When the block does enter, its statistic is well above the threshold: in Part 10's runs, the median over the 22 pure symmetric targets found is 2.3 times the threshold at n_{struct} (range 0.89–74). The residue classes mod 3 are the extreme case, with a true block that passes the test from $n = 128$ but no foothold for the search.

Moving the threshold, in full [Numerical, then Heuristic]. The predictions:

- *What the tests would predict* [Heuristic]. n_{glob} is proportional to L_k, so `strict` multiplies it by $(L_k + 6.9)/L_k$: +0.54 (degree 4) to +0.86 doublings (degree 1). `lenient` moves it by −0.22 to −0.43. The block test of Equation (5) counts every block of a given size and sign pattern, so its log term is larger, 33.7 nats for an unmodulated 11-input block in round 0, and the same change of δ moves it less. At the true blocks' detection points of the 33 symmetric and mixture targets, `strict` moves the threshold by +0.20 to +0.30 doublings of n (median +0.26) and `lenient` by −0.07 to −0.11 (Part 10's block statistics, re-thresholded post hoc at its uncorrected round-0 detection points).

- *Parities: the test sets the cost exactly.* Proposition 2.2 with δ moved, applied to each training set's own $\bar{y}$, predicts the measured changes to within 0.03. `strict` moves $n_{\text{thr}}(3, 30)$ from 26.8 to 40.6, above the grid point 32, and par3's n^* doubles (+0.98 predicted, +1.00 measured). par4 moves by +0.20 against +0.21 under `strict`, and by −0.79 against −0.79 under `lenient`, which moves $n_{\text{thr}}(4, 30)$ from 30.6 to 26.2, below 32. par2 does not move: the forced variant fits it at $n = 16$.

- *Other character targets: one grid step where the test binds.* `strict` raises n^* by more than 0.25 on 8 of the 25 (class mean +0.17; 3 lower), mostly by about the predicted amount: par3 +1.00, stair5 +0.83, rand3 +0.72, rand2 +0.71, majpar +0.70, sparsepoly +0.65, h6_03 +0.54, h6_05 +0.30. The first fit moves one grid step: stair5's test error at $n = 128$ is 0 on both seeds with $\delta = 0.1$, and 0.309 and 0.184 with $\delta = 10^{-4}$. `lenient` is not the mirror image. It lowers n^* on 2 (par4, rand2 −0.69) and raises it on 8 (andxor +0.87, majpar +0.75, mux +0.59 and 5 more; mean +0.12). These are first fitted at $n = 32$–256, in 11 of their 16 seed cells by the forced variant. With $\delta = 0.9$ the linear block enters 8 of these 16 first fits where it did not with $\delta = 0.1$, and the cross-validation's choice of variant changes in 7 (Appendix F.2), so the first fit is lost: majpar's error at 64 is 0.341 and 0.277 on seeds 0 and 1, against 0.067 and 0.099.

- *Halfspaces: a stricter test helps* (−0.23; 4 lower of 7; −0.17 if f7_10, a weighted threshold that the analysis groups with the mixtures, is counted with them). In a refit of f7_01 (seed 1, $n = 128$), with $\delta = 0.1$ the model is $\chi[x_7]$, admitted by its own test, plus the linear block on the other 29 inputs (error 0.091). With $\delta = 10^{-4}$ no character passes on its own in round 0, and the exact 13-input block enters next to the linear block (error 0). A block must also pass on the residual projected off the round's admitted characters, so a degree-1 character that passes first takes part of the block's signal.

- *Symmetric and mixture targets: n^* stays.* Both runs learn 30 of the 33; the three residue classes are learned by neither. The class means are +0.04 under `strict` and +0.05 under `lenient` (5 and 4 higher, 2 and 1 lower). The ablation analysis compares n^* with Part 10's round-0 detection points, uncorrected for the gate and multiplexer mixtures (here f7_03, f7_11, fd_08, fd_11 and h6_07). With the corrected points of Table 33b, f7_03's n^* lies below its detection point, so 6 targets, not 7, lie a doubling or more beyond n_{detect} (band11, f7_08, h6_01, h6_02, h6_06, h6_08; ringed in Figure 5). On them `strict` changes n^* by at most 0.05 (on f7_03 by 0.00), although it moves their blocks' threshold by about a quarter of a doubling. Under `lenient` two of them move, h6_08 by −0.56 and h6_01 by −0.24; the paired null moves neither by more than 0.02, and its largest move on two seeds is 0.69. On the other 24, n^* lies less than a doubling beyond n_{detect}, and for 20 of them at or below it (23 and 18 with the uncorrected points), so the grid cannot separate the test from the search. For those 20 the 10% point comes at or before the first grid size at which the block passes, often through characters (fd_08: $n^* = 51$, against $n_{\text{detect}} = 2048$, 512 uncorrected) or through the forced variant's block (sym7); f7_10's comes at its n_{detect}, through its modulated block. The movers under `strict` among them (f7_09 +0.56, fd_11 +0.46, fd_08 +0.43, f7_10 +0.26; h6_09 −0.53, fd_05 −0.30) are all targets whose true block's uncorrected detection point does not move under `strict`. h6_07 (+0.33), whose uncorrected point does move, is the exception; its corrected point was not re-thresholded, and the paired null moves it too (+0.40).

Figure 5b,c in full. The threshold ablations `strict` and `lenient`: $\log_2(n^*_{\text{ablation}}/n^*_{\text{DFG alone}})$ per target (seeds 0 and 1, paired: the same training sets, column orders and folds), by the class the ablation analysis assigns to each target (character targets: parities, staircases, multiplexers, compositions, DNF, CNF, decision lists and read-once or random formulas; halfspaces; symmetric and mixture targets: functions of one signed sum, alone or with a parity, a gate, a second sum or an inner term) [Numerical]. The three residue classes mod 3, which neither run learns, are not drawn. Orange: the move of n_{glob} for one character of degree 1–4, which is the move of n^* if a round-0 character test set it [Heuristic]. Green: the move of the true block's threshold at its detection point, from the smallest to the largest over the 33 symmetric and mixture targets (Part 10's block statistics re-thresholded, at its uncorrected round-0 detection points) [Numerical]. Rings: the 6 targets whose n^* lies a doubling or more beyond the true block's detection point, with the detection points Part 10's verifier corrected for the gate and multiplexer mixtures (7 with the uncorrected points that the ablation analysis uses: f7_03 as well). Sources: `study/part11/interp/theory/check_theory.json` (`P3.test0`), `study/part11/RESULTS.json` (`q5_ablations`), `study/part11/interp/ablations/families.py` and `study/part11/interp/ablations/ablations.json` (`block_thr_shift`, `block_gap`), the numbers of `study/part11/interp/ablations/ABLATIONS.md`; the corrected detection points from `study/part10/interp/tables/toy_timing.csv` and `study/part10/interp/VERIFICATION.json` (both sets of counts in `paper_dfg/tables/interp_numbers.json`, `fig_dyn_when.gap_split`).

G.8 The order of acquisition: the detail

- *The fits.* There are 363 fits: 11 targets, 11 grid sizes, 3 seeds. Each equals the evaluation's fit in test error and FLOP count.
- *The replay.* Every decision DFG made about a target character agrees with a replay of its test statistic, apart from 13 forced picks and 17 characters cut by one round's limit of 64 admissions.

(a) Characters by the class that first admits them: 41 targets, seed 0, every grid n

class	characters	below n_{glob} (doublings)	class gain (doublings)	union bound $L_{\text{glob}}/L_{\text{cls}}$	residual $(1-\hat{f}(\emptyset)^2)/s_j^2$	coefficient $v_{S,j}^2/\hat{f}(S)^2$
global test, round 0	249	−0.26	–	1	1	1
global test, rounds ≥ 1	37	+1.4	1.7	1.00	3.3	0.85
extension by one input	33	+1.7	2.3	1.32	3.2	0.86
extension by two inputs	19	+1.9	2.6	1.20	5.0	1.06
local (subsets of the inputs in use)	332 (277)	+3.7	4.5	1.71	19.2	0.80

(b) The 13 sum-structured targets of the structure atlas: DFG alone, seed 0

target	form	k	low-degree share	n_{detect}	n_{struct} (alone = v3)	delay (doublings)	n_0^*
f7_01	threshold	13	0.688	128	256	+1	177
f7_08	arbitrary	11	0.081	128	512	+2	448
band11	arbitrary	11	0.047	128	1024	+3	891
h5s7	arbitrary	13	0.017	128	8192	+6	1705
f7_05	mod	14	0.000	128	never	–	never
h6_00	mod	13	0.000	128	never	–	never
f7_10	threshold; weighted	5	0.703	128	128	0	90
f7_00	band/interval; two sums	7	0.267	256	256	0	221
f7_07	interval; parity	12	0.658	128	256	+1	205
f7_09	threshold; parity	10	0.061	128	256	+1	222
symand	band; gate	6	0.777	256	256	0	173
f7_03	mod/band; mux	6	0.540	512†	512	0	352
f7_11	arbitrary/band; mux	6	0.828	1024†	never	–	72

Table 33: When characters and blocks enter (seed 0) [Numerical]. (a) Characters of the 41 `designed` and `random` targets, by the class whose test first admitted them into the model: the round-0 global test (249 characters), and the classes of later rounds (the characters that entered through them before their n_{glob}, or without a predicted grid size; every character that first entered through a later class except 3). "Below n_{glob}": the median of $\log_2(n_{\text{glob}}/n_{\text{first}})$ over the characters of degree ≤ 6 (their number in brackets where it is smaller); negative in the first row, where characters enter just after n_{glob}. "Class gain": the median of $\log_2(n_{\text{glob}}/n_{\text{cls}})$ at the round of entry; it exceeds the entry gain because a character enters at the first grid size at which its class passes. The last three columns are the medians of its three factors (Proposition 2.4c); single characters spread widely around them. (b) The 13 sum-structured targets of the structure atlas. Form: the accept set of the signed sum (mod: a residue class), then the mixture; k: the inputs of the tested true block; low-degree share: $(W_1+W_2)/(1-W_0)$, which for a mixture includes the partner's own weight. n_{detect}: Part 10's first grid size at which DFG's block test admits the true block in round 0 on the seed-0 rows; †: the verifier's correction for the multiplexers of two sums. n_{struct}: the first grid size at which the model matches every signed-symmetry class of size ≥ 3 exactly (Part 10's rule); DFG alone's equals v3's member's on all 13. Delay: $\log_2(n_{\text{struct}}/n_{\text{detect}})$. n_0^*: DFG alone's 10% point on seed 0. Sources: `study/part11/interp/theory/check_theory.json` (`P3.factors`, `P3.entry_vs_nglob`, `P3.entry_vs_nglob_all`), `study/part11/interp/tables/atlas_targets.csv`, `study/part10/interp/tables/toy_timing.csv` and `study/part10/interp/VERIFICATION.json`.

The orders, and what is like for like. A model's *learned fraction* of a character, φ_S, is the coefficient on χ_S of the model's output averaged over the irrelevant inputs, divided by the target's coefficient $\hat{f}(S)$.

- *The networks' orders.* The *time order* ranks characters by the training step at which the seed-mean φ_S first reaches $1/2$ and stays above 0.4, online. The *n order* ranks them by the smallest grid size n from which the network after 20000 steps has $\varphi_S \geq 1/2$, with $\varphi_S \geq 0.4$ at every larger n. It comes from a fixed-n study with the same networks and training sets, which recorded characters of degree ≤ 4 only.
- *DFG's orders.* DFG has three: the *round order* within one fit; the *round-0 priority*, the order of the characters' best round-0 z-scores; and the *n order*, by the networks' rule applied to the model's least-squares function.
- *Like for like.* Only the two n orders are like for like. They share the training sets, the grid and the rule, but not the precision of the estimate. The networks' φ_S on the n axis is a mean over the 4096 test inputs, whose sampling standard deviation on φ_S is about $0.0156/|\hat{f}(S)|$. That is 0.06 for $|\hat{f}(S)| = 0.25$, 0.25 for 0.0625 (rmaj9, dlist5), and 0.57 and 4.0 for dnf3's characters of size 0.027 and 0.0039. A network equal to the target would score φ from -9.9 to $+9.6$ on the latter, so the networks' n onsets of dnf3's small characters are mostly noise.
- *Ranks or analogies only.* The other comparisons are not like for like. A round is a test-and-refit step on a fixed sample, not time, so the round order and the time order are compared by ranks only. The round-0 priority and the n order are compared with the time order only as analogies.

At Part 5's sample sizes DFG's rounds collapse [Numerical] (Table 34a). Part 5 trained at $n = 512$, 2048 and 16384. At these sizes DFG admits every target character it will ever carry in a single round in 24, 30 and 32 of the 33 fits. All 99 fits then stop by the sign rule, because every training sign is right; the test error is 0 in 97 of them. DFG's round axis is empty where the networks' time axis is richest. Fits with two or more entry rounds are most common at $n = 128$ (24 of 33).

At small n the round order agrees moderately [Numerical]. Over the fits with two or more entry rounds, the mean Kendall τ between the networks' online onset order and DFG's round order is 0.77 at $n = 32$ (2 fits), 0.44 at $n = 64$ (11 fits, 9 positive) and 0.44 at $n = 128$ (24 fits, all positive); it is 0.30 at $n = 256$ and 0.10–0.25 above. Per fit it is negative in 3 of the 69 multi-round fits: mux at $n = 64$ in seeds 2 and 1 (-0.48 and -0.05), and rmaj9 at $n = 512$ in seed 0 (-0.01); it is 0.00 for sparsepoly at $n = 256$ in seed 1. At $n = 128$, where DFG's round order is richest, its own cross-validation picks the forced variant in 15 of the 33 fits.

Inside round 0 the priority agrees on every target [Numerical]. DFG's round-0 priority is positively rank-correlated with the networks' onset order on all 11 targets, in each of four conditions: at $n = 16384$, 2048 and 512, network seed s against DFG seed s on the same training set; and online, against DFG at $n = 16384$. All 44 target-condition means are positive, and 131 of the 132 per-seed values; the other is 0.00 (mux, $n = 2048$, seed 2). At $n = 16384$ the means range from 0.22 (dnf3) to 0.81 (sparsepoly), median 0.54 (Table 34b). The online networks see fresh data at every step and have no DFG counterpart, so that row is not seed-matched; seed-matched, all its 33 per-seed values are positive.

(a) DFG's rounds: 11 targets × 3 seeds = 33 fits per n

n	16	32	64	128	256	512	1024	2048	4096	8192	16384
fits whose characters enter in ≥ 2 rounds	0	2	11	24	12	9	2	3	4	1	1
mean τ(network time, DFG round)	–	0.77	0.44	0.44	0.30	0.25	0.18	0.15	0.22	0.12	0.10
of which positive	–	2	9	24	11	8	2	3	4	1	1

(b) Per target: Kendall τ_b over its Fourier characters (positive: same order)

		order against order					against degree / $1/\lvert\hat{f}(S)\rvert$	
target	chars	net time, DFG z_0	net time, DFG n	pairs c / t / i	net n, DFG n ($\lvert S\rvert \leq 4$)	net time, net n	net time	DFG n
sparsepoly	7	+0.81	+0.68	13 / 1 / 1	+0.67 (5)	+0.95	+0.88 / +0.53	+0.85 / +0.55
stair5	16	+0.52	+0.31	45 / 18 / 18	+0.39 (15)	+0.73	+0.73 / +0.16	+0.13 / +0.79
mux	8	+0.31	+0.09	7 / 10 / 4	+0.21 (8)	+0.91	+0.89 / −0.50	+0.11 / +0.38
majpar	4	+0.67	+0.71	3 / 0 / 0	tied (3)	+0.82	+0.71 / tied	+1.00 / tied
rmaj9	76	+0.56	+0.68	764 / 0 / 2	+0.37 (39)	+0.55	+0.67 / +0.41	+0.80 / +0.61
andxor	3	+0.33	tied	0 / 2 / 0	tied (3)	+0.82	+0.82 / tied	tied / tied
dlist5	31	+0.54	+0.40	180 / 117 / 45	+0.50 (30)	+0.75	+0.77 / +0.47	+0.35 / +0.77
parmaj	16	+0.54	tied	0 / 69 / 0	tied (15)	+0.72	+0.63 / tied	tied / tied
dnf3	511	+0.22	+0.22	34171 / 8989 / 13265	+0.17 (255)	+0.19	+0.25 / +0.27	+0.22 / +0.47
rand1	15	+0.65	+0.44	51 / 6 / 14	+0.49 (14)	+0.83	+0.89 / +0.61	+0.43 / +0.82
rand3	32	+0.58	+0.24	107 / 136 / 35	+0.41 (26)	+0.62	+0.69 / +0.39	+0.52 / +0.33
dnf3*	21				+0.56 (21)	+0.87		

Table 34: DFG alone's order against the Part 5 networks' [Numerical]. (a) Per grid size: the fits (of 33) in which the target's characters enter DFG's model over two or more rounds; the mean Kendall τ between the networks' online onset order and DFG's round order over those fits (a character carried only by a sum block counts half a round later); how many of them are positive. (b) Per target, Kendall τ_b over its Fourier characters ("tied": one of the two orders is a single tie). The columns are: the networks' onset order in Part 5's run on the same 16384 training rows against DFG's round-0 priority there (mean of 3 seeds, seed s against seed s); the networks' online onset order against DFG's n order (median seed); the pairs the networks order (onsets more than 1.35 times apart) that DFG's n order keeps, ties or inverts; the networks' n order against DFG's ($\lvert S\rvert \leq 4$; the characters in brackets); the networks' time order against their n order. The last two columns give the networks' time order and DFG's n order against degree and against $1/\lvert\hat{f}(S)\rvert$; positive means that higher degree, or a smaller coefficient, comes later. dnf3*: dnf3's 21 characters of size 0.19, which the networks' n-axis estimate can resolve. Source: `study/part11/interp/order/tables/order_numbers.json`.

On the n axis the agreement is moderate [Numerical] (Figure 7, Table 34b).

- *Against the networks' time order.* τ is 0.09 (mux) to 0.71 (majpar), median 0.40, on the 9 targets whose DFG n order is not a single tie. On andxor and parmaj every character enters DFG at the same n.
- *Like for like*, against the networks' own n order ($\lvert S\rvert \leq 4$). τ is 0.17 to 0.67, median 0.40, on the 8 targets where DFG's order is not a single tie.
- *The networks against themselves.* The networks' time and n orders agree at 0.19 to 0.95, median 0.75: they agree with each other more than either agrees with DFG.
- *dnf3.* Its two n-axis values (0.17 and 0.19) rest on characters the networks' estimate cannot resolve. On its 21 characters of size 0.19 they are 0.56 and 0.87. With those values the two ranges become 0.21–0.67 (median 0.45) and 0.55–0.95 (median 0.82).

The networks follow construction, DFG follows detectability [Numerical, then Heuristic].

- *Degree or size.* On the 8 targets whose coefficient sizes differ, the networks' time order follows degree more than coefficient size on 7. dnf3 is a near-tie that slightly favours size (0.25 against 0.27). DFG's n order follows $1/|\hat{f}(S)|$ more than degree on 5 of the 8 (stair5, mux, dlist5, dnf3, rand1) and degree on the other 3 (sparsepoly, rmaj9, rand3) (Table 34b).
- *Mux, the clearest case.* Its largest term $x_1x_2x_3x_4$ ($\hat{f}(S) = 0.75$, degree 4) is the networks' last character: online at 452 steps, against 60–66 for the linear terms, 169–176 for the pairs x_ix_4 and 430 for $x_1x_2x_3$; in the fixed-n study at $n = 2048$. It is DFG's first character by round-0 priority: it has the highest round-0 z-score of mux's 8 characters in all 27 fits with $n \geq 64$, and in 2 of the 3 at $n = 32$. On the n axis it is tied first, at a median onset of 64, with x_2, x_3 and x_2x_4. It is in the model from $n = 64$ in all 3 seeds: admitted by the global degree-4 test (seed 1), carried by a block whose levels span it (seed 0), or picked by the forced variant (seed 2). Its n_{glob} is 54, against 271 for the linear terms. The coefficients differ by a factor 3 and the z-scores by 2.2.
- *Data cost, like for like.* For the 64 characters of degree ≤ 4 with $|\hat{f}(S)| \geq 0.25$, DFG's median n onset is 4 times smaller than the networks' (median $\log_2$ ratio -2.0). DFG needs less data for 46, the same for 11 and more for 7. For the 349 smaller characters the median ratio is 1 (147 less, 64 equal, 138 more or never). 255 of these are dnf3's, whose network onsets are mostly estimator noise; without dnf3 the split is 47, 15 and 32 of 94, and the median ratio is still 1.
- *Bundles, cut by detectability.* DFG also admits characters in bundles, but they are sets of near-equal detectability. It merges a whole target whose coefficients are equal: parmaj's 16 characters all enter at $n = 128$ (median seed), carried mostly by two folded blocks. It splits network bundles that mix sizes: the networks learn stair5's x_1x_3, x_1x_4, x_1x_5 ($\hat{f}(S) = -0.125$) with x_1x_2 (0.375) at 188–234 steps, while DFG takes them at $n = 512$–2048 against 128 for x_1x_2.
- *Above degree 4* DFG's global search reaches only subsamples at degrees 5–6, and there DFG does build. Of the 490 character-seeds of degree ≥ 5 that enter for good, 473 are carried by sum blocks, 11 enter through the local class, 3 by extension and 3 through the degree-5 subsample search. They come after the lower-degree characters they build on: majpar's degree-6 term follows its pairs, and stair5's top term enters by extending its lower terms.
- *Linear stage and stopping.* On all 8 targets with linear terms the networks learn every linear term before any other. In DFG's n order the linear terms come strictly first in 6 of the 24 target-seed cases, tie in 12 and do not come first in 6. At $n = 512$ the networks have learned 36% of a target's characters ($\varphi \geq 1/2$; mean over the 33 target-seed cells, targets weighted equally) and hold 30% partially ($0.25 \leq \varphi < 0.75$, a range that overlaps the first); DFG has learned 80% and holds 7% partially. At $n = 16384$ the networks have learned all of them and DFG 92%. DFG stops once every training sign is right, so its model is sign-sufficient rather than an estimate of the labels' conditional mean. On the n axis it drops characters it carried at a smaller n: 1139 of 2157 character-seeds, 945 of them on dnf3.

Reading [Heuristic]. The two cost laws are close to opposite, although they price different resources. For DFG a character costs data roughly logarithmically in its degree, through the union bound, and quadratically in $1/\hat{f}(S)$. Part 5's model of the networks charges training time, not data: as e^2 in the number e of new inputs a move coordinates, and only as $\rho^{-1/2}$ in its share ρ of the residual. So the comparison is of the orders the two laws imply. The orders therefore agree where a target's

large coefficients sit at low degree (sparsepoly, majpar, rmaj9: $\tau = 0.68$–0.71 against the time order). They invert where a large coefficient sits at high degree (mux), and they disagree where a stage mixes sizes (stair5, dnf3, rand3). DFG's foothold is chosen by size. The composition steps that resemble the networks' come after it, through the later rounds' small classes (Section 6.3) and through sum blocks. *Caveats.* Three seeds, one network architecture and $d = 30$ only. The 11 targets come from a few designed families, so no p-values are quoted. DFG's φ is that of its least-squares function; the logistic refit can distort single coefficients.

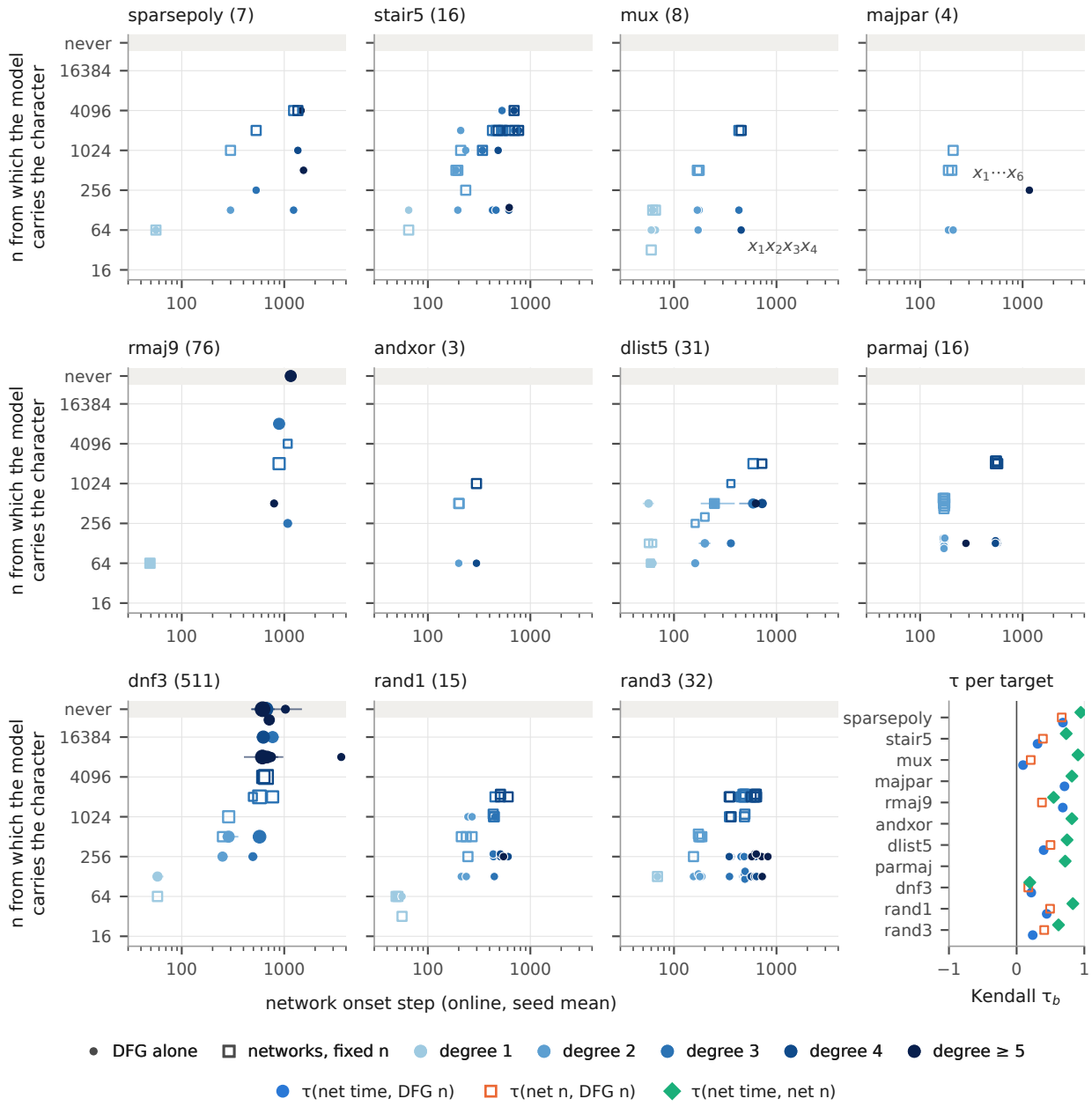


Figure 7: The order of acquisition, DFG alone against the Part 5 networks [Numerical]. One panel per target, with its number of non-constant Fourier characters on the relevant inputs; each point is a character. x: the networks' onset step online (the seed-mean learned fraction reaches 1/2 for good, log-interpolated). y: the smallest grid size from which the final model carries the character (learned fraction $\geq 1/2$, and ≥ 0.4 at every larger n). Filled: DFG alone's least-squares function, median of 3 seeds. Open: the networks of the fixed-n study, median of 3 seeds, $|S| \leq 4$ only. Grey band: never within $n \leq 16384$. Colour: degree. For rmaj9, dlist5 and dnf3 there is one marker per (degree, $|\hat{f}(S)|$) group, at the group's median, with area $\propto \sqrt{\text{count}}$ and a bar over the 10–90% range of the network onsets. Coinciding points are offset vertically by up to ±0.26 doublings. dnf3's open squares for $|\hat{f}(S)| \leq 0.027$ are dominated by the noise of the networks' estimate. Last panel: the three Kendall τ_b of Table 34b per target; ties leave andxor and parmaj without a τ against DFG's n order, and majpar without the like-for-like one. Re-plotted from `study/part11/interp/order/tables/order_chars.csv` and `order_numbers.json`; the analysis' own figure is `study/part11/interp/order/figures/fig_order.pdf`.

H How Part 11 was run

Part 11 and this paper were produced by an AI system, Claude (Anthropic), on 2 and 3 November 2026, in one orchestrating session that delegated parts of the work to separate agents. Kaarel Hänni posed the request (Appendix H.1). No human has reviewed the code, the results or this text. Times are UTC, and the commits named are on the working branch `claude/nifty-mccarthy-j0lw2a`. The records are:

- the protocol `study/part11/PROTOCOL.md`, with its post-hoc addendum;
- the runners `study/part11/eval_dfg.py` (toy suites) and `study/part11/dfg_real.py` (tabular data), the shard merger `study/part11/merge.py`, and the comparison script `study/part11/compare_dfg.py` with its outputs `study/part11/RESULTS.md` and `study/part11/RESULTS.json`;
- the five analyses in `study/part11/interp/`, each with its scripts, tables and result files, and their verification records (Table 35);
- this paper's generators and check scripts in `paper_dfg/`.

Organisation. The orchestrating session chose the object, wrote the protocol and the comparison script, and ran the queue of pre-registered runs and the core. It delegated three kinds of work to separate agents, each with its own instructions: five analysts (the structure atlas, the order of acquisition, the theory notes, false admissions and compute, and the ablations); one independent verifier for each analysis; and writing agents for the sections of this paper. Each writing agent drafted from the records it names. The abstract, the introduction, the discussion and this appendix were written last, from the drafted sections and the records.

H.1 The request and the choice of object

From its sixth part on, this study has asked whether a learner without continuous feature parameters can do at least as well as gradient-trained networks, with the same training data and no more FLOPs. Its tenth part answered with v3, a stacked portfolio whose input type decides which of its members carry a fit [12]. After the Part 10 paper, Kaarel Hänni's request was: "can you please suggest a discrete method that is more like 'one thing', less an ensemble of different things. eg maybe one of the components of your current proposal would do? the aim behind this last question is to get a specific interesting object worth studying. then write an analogous paper but specifically about that". This is that paper. The orchestrating session chose DFG v1.3 run alone, for the reasons of Section 1. Three alternatives were considered and set aside. v3's image boosters are gradient-boosted trees on a fixed, data-derived feature bank; what is new in them is the bank, and the learner is LightGBM. The portfolio itself is the ensemble the request asked to avoid. A single DFG for real-valued data would be a new design, with a dictionary derived from the data, not a frozen object that can be studied as it is; it is left for future work (Section 7.6). The session wrote the choice, its reasons and its domain limit into the protocol before any Part 11 run. The choice was made after every Part 10 result had been read, v3's on `dlheld7` included, and the protocol says so (Section 3.4).

H.2 The protocol and the runs

The object is the file study/part10/hyb_portfolio3/dfg13.py, frozen by Part 10's toy builder (the Part 10 agent that built v1.3) at 19:44 UTC on 27 October 2026, with md5 `69cabe5c50345cb24079198ed52b562f`; it is identical to study/part10/p10_sum/dfg13.py.

The protocol was written on 2 November and committed at 22:15:39 together with the runners and the merge script (`aca1f3c8`), before the first run. It fixed the object, the seven questions with their runs, suites and seeds, the rules of Parts 8–10, the rule for crashes, and what is held out (Section 3.4). Its pre-registered part has not changed since. The addendum that defines the core run was committed at 02:25:54 on 3 November (`12eae4e0`; its heading says 02:30), after every pre-registered run had finished and its results had been read, and two seconds before the core's first job started.

The queue. The pre-registered runs were 282 jobs in one file:

- 48 shards of DFG alone on the 11 toy suites, 3 to 7 per suite;
- 11 `lowdeg` jobs, one per suite;
- 18 jobs at $d = 15$, 60 and 120, one per target and d;
- one job for the two tabular datasets;
- 204 ablation shards, 17 for each of the 12 ablations over the five ablation suites.

Four worker processes started at 22:15:41. Each worker took the next unclaimed job in file order (claiming it by creating a directory, which is atomic), ran it at nice 10 with single-threaded BLAS, and marked it done or failed. A merger joined the shards of each run and suite with `merge.py` once all of them were done, and the orchestrator committed the merged files. No job failed. The first results, DFG alone on the held-out `dlheld7`, were committed at 22:23 (`5bd788c4`). DFG alone had finished on all 11 suites by 23:31, and `lowdeg`, the runs at other d and the tabular runs by 23:41. The ablations' results were committed from 23:42 on, and at 02:24 all 282 jobs, 87 result files, were complete (`28d64a85`).

The container restart at about 01:09 on 3 November stopped every process. Four ablation shards were running: `noblocks` on `dlheld6` (shards 2 and 3) and on `dlheld7` (shards 1 and 2). A shard writes its result file only when it finishes, so these four had written nothing. Their claims were removed, three workers were restarted at 01:10:54, and the four shards ran again from the start, with the same code, seeds and random streams. The 158 jobs already finished kept their files, and the merger skips any run and suite merged before a restart. So no finished result was lost or computed twice, and no result file mixes two runs; the worker logs show 286 starts for the 282 jobs. After the restart the queue ran on three workers instead of four. The worker start and restart times, the statement that no job failed, the four interrupted shards, the 286 starts and the core's shard times below rest on the queue file and the worker logs, which are not in the repository (Appendix H.5); every commit time quoted here can be checked in git. The calibration analysis's record writer was also stopped, at 8 of the 220 noise fits at $d = 60$ and 32 of the 105 profile records. It was resumed from its records, which skip every key already written, so no cell was fitted twice. The commit `ba81503c` (01:10:57) kept the analysts' partial edits.

The post-hoc runs.

- *The core* (Section 5.3): 36 shards on four workers, from 02:25:56 to 02:38:32; its 11 result files were committed by 02:39 (`22e45319`). With them, the 98 result files of Part 11 are all complete (`study/part11/RESULTS.md`, "Run status").
- *The paired null* (`study/part11/interp/ablations/perm_null.py`) on all 11 suites, seeds 0 and 1, after the core had finished and until 03:36; *the leave-one-in runs* (`study/part11/interp/ablations/leavein.py`), until 03:24; and 38 refits of single cells for the ablation analysis (`study/part11/interp/ablations/refit.py`), with 42 more when its verification was applied (`study/part11/interp/ablations/results/refits_verif.json`). Every refit reproduces its run's result file in test error and FLOP count. The leave-one-in targets were chosen after reading the core's results, and five of its configurations after reading the first leave-one-in results (they are marked in `leavein.py`).
- *This paper's checks*: `paper_dfg/check_parity_variant.py` (43 refits of the parity cells at small n) and `paper_dfg/check_h5s1.py` (6 refits of h5s1). Every refitted test error equals the result file's.

All of these ran single-threaded, as separate processes outside the queue.

H.3 The analyses and their verification

The analyses ran from about 22:30 on 2 November (their first scripts and fits were committed at 22:33, `f3abc88a`), in parallel with the queue, on the result files as they landed (Section 6). Each fit they analyse is an instrumented rerun of an evaluation fit, and each analysis checks its reruns against the result files in test error and FLOP count (Appendix G). The theory notes' proofs are about the frozen code, line by line. The protocol fixed Q7's questions, not the analyses' methods: each analysis chose its targets, definitions and groupings itself, and the atlas revised some definitions after its first fits had been looked at (Section 6.1). The paired null, the leave-one-in runs, the refits and the target families of the ablation analysis are post hoc (Section 5).

report	verification	what it covers	claims checked	confirmed	corrected	omissions listed
`ATLAS.md`	`VERIFY_ATLAS`	what the models contain: 37 targets, 407 fits	61	39	22	9
`ORDER.md`	`VERIFY_ORDER`	the order of acquisition, against the Part 5 networks	60	41	19	9
`THEORY.md`	`VERIFY_THEORY`	the exact statements P1–P5 and their checks	54	47	7	4
`CALIB.md`	`VERIFY_CALIB`	false admissions; where the compute goes	28	16	12	8
`ABLATIONS.md`	`VERIFY_ABLATIONS`	the ablations and the core DFG	75	49	24 (+2 unsupported)	10

Table 35: The independent verifications of the Part 11 analyses, counted from the verifiers' own records [Numerical]. "Corrected": the verifier changed a number, narrowed a statement or rejected a reading; "unsupported": a claim the records do not support, counted apart from the corrections; "omissions": further problems it listed that were not tied to one claim. The reports are in `study/part11/interp/`, each next to its verification record (`.md` and `.json`). Generated by `paper_dfg/make_tables.py` (`tab_verif`) from the `VERIFY_*.json` files.

The verifications. Each verifier was a separate agent with its own instructions. It read the report, the code and the stored files, recomputed the numbers with its own scripts, refitted cells through its own code path, and listed every claim as confirmed or corrected, with the evidence. The analyst then checked each correction against the stored files, applied it or gave its reason for declining, and listed every change in the report's Verification section. Table 35 counts the verdicts. The paper uses the corrected numbers and wording throughout.

- *The structure atlas* (`ATLAS.md`; verified at about 00:36). The verifier refitted 11 cells and 6 counterfactual variants and recomputed every count. All 22 corrections were applied and none was declined. Among them: the 37 targets come from 9 of the 11 suites; block representations are checked only as functions of the block statistics; thrint is first fitted without error at $n = 512$ by a product block, not at 1024 by weighted blocks; and DFG alone's model of the residue classes is constant at every n, whereas v3's member was not at small n.
- *The order of acquisition* (`ORDER.md`; about 00:34). The verifier made 12 refits through a separate code path and recomputed all 23,727 round-0 z-scores. All its corrections and omissions were applied, two of them only in part, with the stored evidence for the rest, and the analyst's own re-check found eight further errors (for example 27 fits with $n \geq 64$, not 30). Among them: DFG's round order is negative in 3 of the 69 multi-round fits; the n-axis comparison is like for like in data and rule, but not in the precision of the networks' estimate for small coefficients; and DFG's bundles are cut by detectability, not by coefficient size alone.
- *The theory notes* (`THEORY.md`; about 02:59). The verifier re-derived every proof against the cited lines of the code, refitted 615 cells, and gave the verdict "sound". All 7 corrections and 4 omissions were applied. No proof or measured value changed. The corrections narrowed wording that overstated what is proved (passing a test against being admitted to the model; n_{thr}'s exact decomposition) and fixed one count, and the omissions added P1a′, the bound for other input laws (Section 2.5).
- *False admissions and compute* (`CALIB.md`; about 02:43). The verifier rebuilt the instrumented copy byte for byte, showed that its syntax tree minus the hooks equals the frozen file's, and made 17 independent refits. It corrected 12 claims, among them the identity claim (only the checked fits are checked; the others equal the frozen learner by construction), the linear block's null rate (close to the χ^2_d tail, not at it), sum-block admissions in the fold fits, a count (229, not 238) and one step of a proof. Its important omission is the spectral block's null pass rate on biased labels, which grows with d to 99% at $d = 120$. Section 6.5 reports the corrected rates.
- *The ablations* (`ABLATIONS.md`; about 04:22). The verifier recomputed every cell of the ablation tables, both nulls, the parity predictions and every leave-one-in n^*, re-ran 33 paired-null cells with the frozen module, and made 94 refits through its own instrumented copy, all equal to the result files. It corrected 24 claims and found 2 unsupported; all were applied, and none was declined (one value of the verifier, a floor of 0.0195, is 0.0190 by exact enumeration). Among them: like for like, the unpaired null overstates the noise 20–30 times, not tenfold; K does not enter the linear and spectral block tests; under `labelnoise` the block statistics fall by s^2, not s; the targets that `lenient` slows are first fitted at $n = 32$–256, mostly by the forced variant; and the leave-one-in runs restore 13 of the core's 26 clearly-higher targets, leave h5s2 unexplained and did not test 10. The analyst added 42 refits, all equal to the result files. Section 5 reports the corrected analysis.

H.4 Disclosures

- *The timing check.* Before the protocol was written, DFG alone was fitted on two `dlheld7` targets, f7_03 and f7_08 (seed 0; $n = 64$, 512, 4096 and 16384), to time it, and the 8 test errors were printed. The protocol records them, and the queue's runs reproduce all 8 to the three decimals recorded (`study/part11/RESULTS.md`, "Consistency checks"). The code had been frozen in Part 10 and nothing is tuned in Part 11, so no setting could respond to them.
- *Smoke tests* of the ablations and of `lowdeg` used three `designed` targets: maj9, stair5 and par4.
- *A replaced setting.* The first choice for `lenient`, $\delta = 10$, crashed: the linear block's threshold takes the square root of a multiple of $\ln(1/\delta)$, which is negative for $\delta > 1$. It was replaced by $\delta = 0.9$ before any run, and the protocol records both.
- *The object was chosen after v3's results* on every suite had been read, `dlheld7`'s included, where DFG is v3's toy member (Appendix H.1). DFG alone's results there were new, the timing check aside, but they are not independent of v3's, which were known; the choice is post hoc, the code is not.
- *Post-hoc choices.* The core was defined after every pre-registered result had been read, and the leave-one-in targets after the core's (Section 5.3). The target families and classes were assigned by hand after the results were in.
- *Load.* The queue ran four workers at nice 10 on a four-CPU machine, three after the restart, next to the analyses, so the CPU times are rough (Section 4.6).
- *Written and checked by AI agents.* The analyses, their verifications and this paper were produced by AI agents (Claude), each with its own instructions. No human has reviewed them.

H.5 Reproducing the numbers

Which files stand for which learner.

- *DFG alone, its ablations, the core and `lowdeg`* on the toy suites: `toy__dfg13-perm__<suite>.json`, `toy__dfg13-<tag>-perm__<suite>.json`, `toy__dfg13-core-perm__<suite>.json` and `toy__lowdeg-perm__<suite>.json`, all in `study/part11/results/`. The paired null's files are in `study/part11/interp/ablations/results/`.
- *DFG alone on tabular data*: `dfg13__kr_vs_kp.json` and `dfg13__mushroom.json` in `study/standard_tasks/results/`.
- *v3 on toy inputs*: `p10_sum`'s files in `study/hybrid/results/`, which is v3's code there (Section 3.2). The networks and the other references are read from their own result files, as the docstring of `study/part11/compare_dfg.py` lists them.

Commands. The commands are run from the folder `gd-circuit-growth/`, with single-threaded BLAS (the environment variables `OMP_NUM_THREADS` and `OPENBLAS_NUM_THREADS` set to 1):

- the runs: `cd study/part11`, then `python3 eval_dfg.py dfg13 <suite>` (an ablation adds `--kw=<json> --name=<tag>`, with the arguments of Section 3.4 and, for the core, those of the protocol's addendum; shards add `--fns=` and `--part=`, and `python3 merge.py <run>`

<suite> joins them), python3 eval_dfg.py lowdeg <suite>, python3 eval_dfg.py dfg13 designed_d<d>, and python3 dfg_real.py kr_vs_kp,mushroom;

- the comparisons: python3 compare_dfg.py (writes RESULTS.md and RESULTS.json);
- the analyses: the scripts that each report names in interp/ (for example atlas_run.py and atlas_make.py; order/order_fit.py and order_analysis.py; theory/theory_fits.py and check_theory.py; calib/calib.py and calib_summary.py; ablations/perm_null.py, leavein.py, refit.py and ablations.py);
- this paper: cd ../../paper_dfg, then python3 make_tables.py, python3 make_figures.py, python3 make_interp.py, python3 check_parity_variant.py and python3 check_h5s1.py, and latexmk -pdf main.tex; dfg-paper.pdf is a copy of main.pdf.

Every table and figure is generated from these files. Derived numbers that the text quotes and no table shows are in paper_dfg/tables/numbers.json, paper_dfg/tables/interp_numbers.json and the figures' .json files, or in the analysis reports and their outputs that the text names: CALIB.md with calib.json and calib_tables.txt, ORDER.md with order_numbers.json, THEORY.md with check_theory.json, ATLAS.md with the atlas tables, and ABLATIONS.md with ablations.json.

What is not in the repository, and what stays on the working branch. The job queue, the worker and merger scripts and their logs stayed in the session scratch directory and are in neither branch; they only wrap the committed runners, whose commands are above. The networks' raw outputs also live in a scratch directory, as in Parts 1–10. On the working branch, the LaTeX build output in paper_dfg/ (main.pdf and the auxiliary files) and the run logs of the post-hoc runs (study/part11/interp/ablations/logs/) stay there; dfg-paper.pdf is the published copy of the paper.